Appunti di Introduction to Machine Learning

Alessio Novel

Introduction....2
Introduction to the course....2
Introduction to Machine Learning....4
Machine Learning Basics....21
Data....26
Types of learning....28
k-Nearest neighbor....62
Decision Boundaries, Hyper-parameters....70
Linear models....77
Online learning....83
Learning A Linear Model....85
Multiclass classification....91
From Binary to Multiclass Classification....91
OVA & AVA....94
Evaluation....102
Gradient Descent....104
Loss functions....107
Gradient descent....110
Regularization....117
Avoid Overfitting....117
Regularizers....119
Support Vector Machines....126
Soft Margin Classification....130
Non Linearly Separable Data....135
Decision Trees and Random Forests....143
Unsupervised Learning....161
Principal Component Analysis....163
k-Means Clustering....177
Clustering....182
Introduction to Neural Networks....194
From Perceptron to Deep Networks....196
Feedforward Networks: Basic Elements....208
Backpropagation....214
Neural Networks II....219
Training a Neural Network....219
Convolutional Neural Networks....226
Convolutional Neural Networks Architectures....230
Other Neural Networks....236

Deep Generative Models....242
Variational Autoencoders....245
Generative Adversarial Networks....255
Diffusion Models....264
Theory....270
Advanced Tricks....278
Reinforcement Learning....286
Reinforcement Learning Idea....286
Markov Decision Process....289
Value Based Methods....293
Policy Gradient Methods....300

Introduction

Introduction to the course

This course provides a **broad introduction** to machine learning and statistical pattern recognition, so the concepts will be treated at a very high level. The course will cover different topics ranging from **supervised learning** (parametric and non-parametric algorithms, kernel methods, feedforward neural networks), **unsupervised learning** (clustering, dimensionality reduction, autoencoders, deep generative models) and **reinforcement learning**.
During the course some **applications** of machine learning will be presented in the field of computer vision, robotics (the research field of the teacher is an intersection between computer vision and robotics) and natural language processing.
Theoretical discussion will be complemented with **labs in Python** using open-source libraries.

The goals of the course are to learn about:

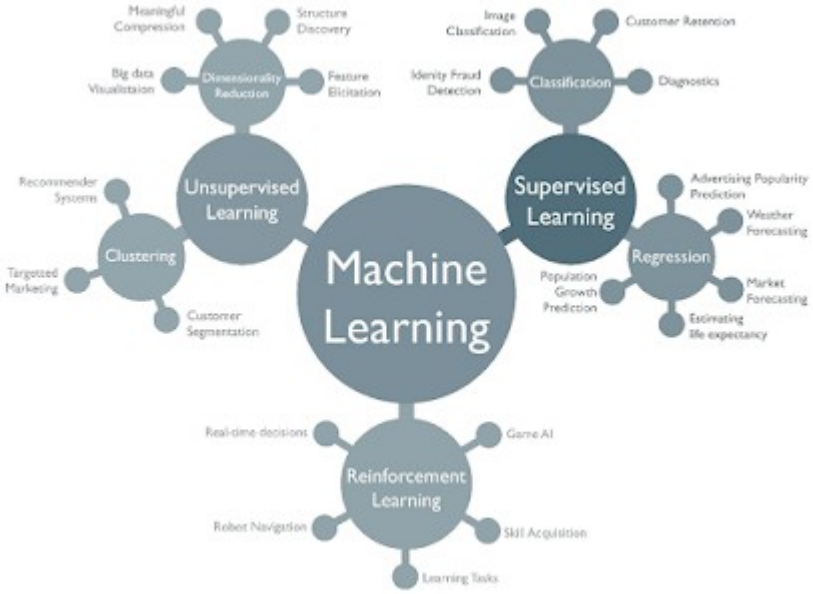
- The fact that **different** machine learning settings/**problems** require **different algorithms**
- **Common techniques and tools** used **to solve problems** (both theoretical understanding and practical implementation)
- **Proper experimentation and evaluation** (using programming tools)

The **contents of the course** are:

- Introduction to Machine Learning
- Supervised Learning
    - Nearest Neighbors
    - Decision Trees and Random Forests
    - Kernel Methods
    - Ensemble Methods
    - Deep Neural Networks
        - (MAYBE) Feedforward, convolutional and recurrent networks
- Unsupervised Learning

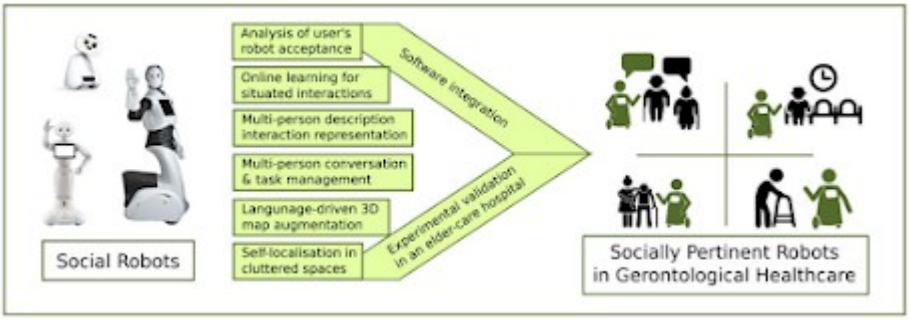
    - Clustering, dimensionality reduction
    - Deep Generative Models
- Reinforcement Learning

This is an overview of the ML subject:



We’re gonna start with the traditional algorithms of Machine Learning because it is impossible to do sophisticated things without knowing the easy ones.

An example of a complex application of Machine Learning in reality is SPRING.
SPRING is a project focused on the development of **Socially Assistive Robots** (SARs) with the capability of **performing multimodal multi-person interactions** and **engaging in open-domain dialogue**. The project addresses the limitations of current Human-Robot Interaction (HRI) technology, specifically in the context of social robots deployed in various public spaces, such as museums, airports, hospitals, and retirement homes.



SPRING's mission involves enhancing the cognitive interactions of social robots, allowing them to effectively communicate and fulfill social roles in complex and unstructured populated spaces. The research question of SPRING is centered on how to create robots that can move, see, hear, and communicate with multiple actors in such environments.

The overall objective of SPRING is to develop SARs with the ability to perform multi-person interactions and engage in open-domain dialogue. The strategic objectives include enabling robust robot perception, facilitating sensor-based and knowledge-based robot actions for multi-modal multi person interaction, and validating the technology based on the needs of gerontological healthcare.









To achieve these goals, SPRING outlines scientific objectives related to developing socially-aware robots, conceiving innovative methods and algorithms for computer vision, audio processing, sensor-based control, and spoken dialog systems. Technological objectives involve creating a new generation of flexible robots that can adapt to user needs. Experimental objectives focus on validating the technology through HRI experiments in gerontology hospitals and assessing its acceptability by patients and medical staff.

Introduction to Machine Learning

Let’s first see some definitions of Machine Learning from different sources.

> “***Machine learning*** *is the study of computer algorithms that improve automatically through experience. It is seen as a part of artificial intelligence.*”
>
> By Wikipedia about Machine Learning

Let's be specific about the distinction between Artificial Intelligence and Machine Learning. Artificial Intelligence (AI) and Machine Learning (ML) are related concepts but have distinct meanings. Here's an overview of their differences and relationship:

- **Artificial Intelligence (AI)**: It is a broad concept referring to the development of machines or systems that can perform tasks requiring human-like intelligence. It covers a wide range of approaches, including rule-based systems and expert systems, and can be categorized into narrow or general AI.
- **Machine Learning (ML)**: It is a subset of AI focused on the development of algorithms and models that enable machines to learn from data. ML systems improve their

performance on a task without being explicitly programmed, and it includes techniques such as supervised learning, unsupervised learning, and reinforcement learning.

In simpler terms, AI is the overarching goal of creating intelligent machines, while ML is a specific approach within AI that emphasizes learning from data. Many AI applications leverage machine learning techniques to achieve intelligent behavior.

"*Machine learning is the field of study that gives computers the ability to learn and improve from experience without being explicitly programmed*"

By ChatGPT about Machine Learning

In supervised learning, the algorithm learns from labeled data, where the inputs are paired with corresponding outputs. The goal is to learn a mapping from inputs to outputs, which can then be used to make predictions on new, unseen data.

In unsupervised learning, the algorithm learns from unlabeled data, where there are no predefined output labels. Instead, the algorithm seeks to find patterns, structure, or relationships in the data.

Semi-supervised learning combines elements of both supervised and unsupervised learning, using a small amount of labeled data along with a larger amount of unlabeled data.

Reinforcement learning involves training an algorithm to make sequences of decisions by rewarding it for good decisions and penalizing it for bad ones, based on feedback from its environment.

Machine learning has a wide range of applications, including but not limited to image and speech recognition, natural language processing, recommendation systems, medical diagnosis, and autonomous vehicles.

"*Machine learning is the science of getting computers to act without being explicitly programmed.*"

By A. Samuel (1959) about Machine Learning

Machine Learning changes the approach of tackling problems, in fact before Machine Learning, and this breakthrough of learning from experience, when you want to perform a task the procedure was:



With Machine Learning the procedure became:



With conventional programming, it is impossible to perform the task of object recognition in images because one would need to write a program that exactly describes the object. However, since an image consists of a set of pixels, this is deemed impossible.
Therefore, by reversing this framework, we can now tackle problems that were previously considered 'impossible', such as object recognition.

Another example which will help us of understanding the approach of Machine Learning is regarding walking robots: if we want a robot to be able to walk, we could **program the robot to learn to walk** (Machine Learning), or we could attempt to **directly write a program that species how to walk manually** (Conventional Programming). Also in this case the Conventional Programming isn't practicable.



So in summary Machine Learning allows computers to acquire some **knowledge**.
This knowledge is acquired through **algorithms** by learning and inferring from (prepared) **data** and it is represented by a **model**. Then the model is used on other/future data.



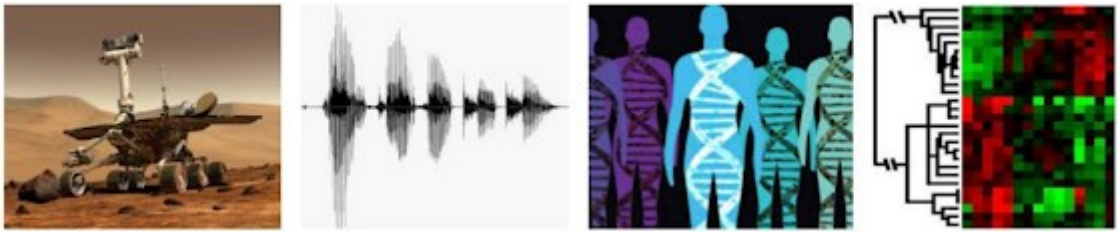
These are the **three fundamental ingredients** of Machine Learning.

The trained model can subsequently be applied to additional data, assuming it bears resemblance to the original dataset. In machine learning, the process involves training a model with historical data, enabling predictions for future outcomes using new data and the established model.

Machine Learning is applicable and useful in various scenarios:

- When human expertise is unavailable, such as navigating on Mars.
- When humans are unable to articulate their expertise, like in speech and object recognition tasks.

- When models need to be tailored to specific domains, as seen in personalized medicine applications.
- When models rely on extensive datasets, as in the field of genomics.

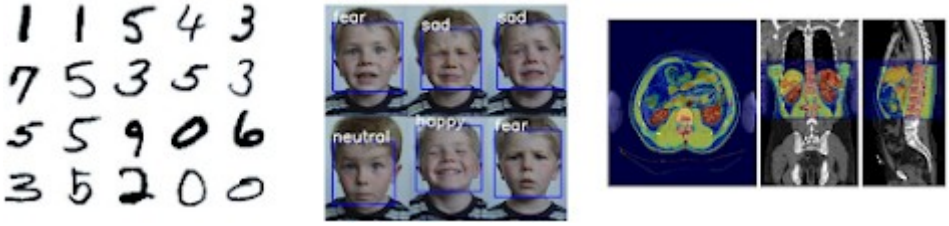


But **Machine Learning isn't always useful**, in fact in fields where we have strict rules, theorems, prior knowledge and etc. there isn't the need to learn anything (for example to calculate payroll).

Enhancing the ability to formalize problems (that is to move from something very abstract to a mathematical formalization) in the field of Machine Learning is of utmost significance.

Examples of real problems where we need Machine Learning are:

- **Recognizing patterns**
  Like handwritten digits, facial expressions/facial identities (now forbidden by the AI act) and medical images (different color for each type of tissue).



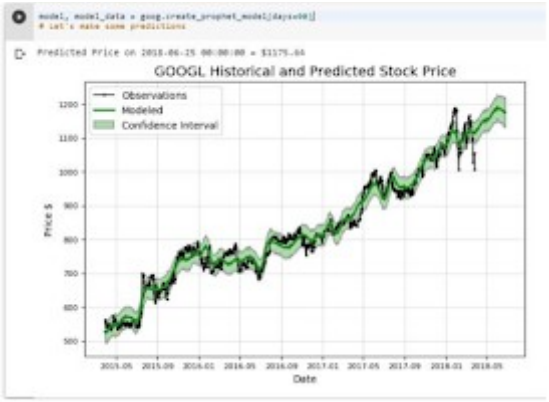
- **Generating patterns**
  Like generating images or motion sequences. Given a bunch of images of people's faces, with Machine Learning we can output a mathematical model of the distribution of these images. Then the model is able to sample new faces.

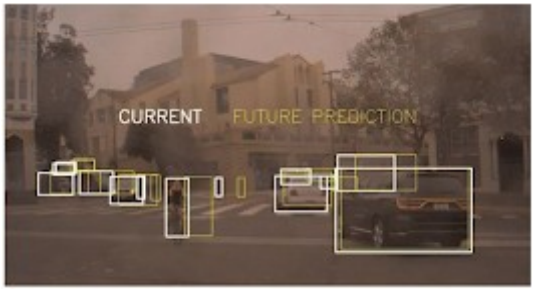


- **Recognizing anomalies**
  Like unusual credit card transactions or unusual patterns of sensor readings



- **Prediction**
  Like future stock prices, future currency exchange rates or autonomous driving





- **Predict best moves** (agents which can play video games)
  Machine learning empowers the development of agents capable of executing intricate tasks. It becomes indispensable in scenarios where the objective is to train agents, such as playing video games, like GO, against human opponents.
  If you want a system to be able to play GO, the system should be able to take into account all the possible configurations of the stones into the board.



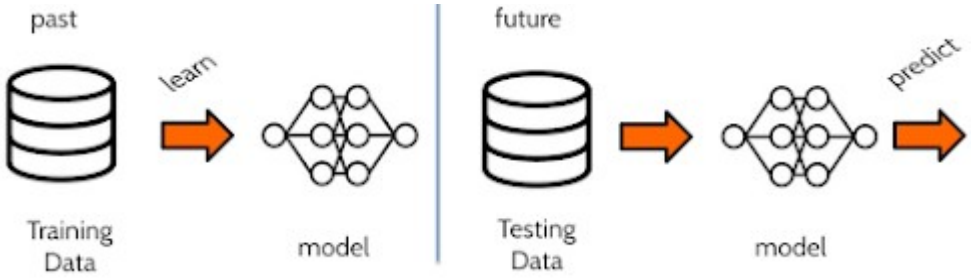
Let's see other definitions of Machine Learning.

"*It is concerned with the **automatic discovery of regularities in data** through the use of computer algorithms and with the use of these regularities to take actions.*"
By Christopher M. Bishop about Machine Learning

"*The goal of machine learning is to develop methods that can **automatically detect patterns in data**, and then to use the uncovered patterns to **predict future data** or other outcomes of interest.*"

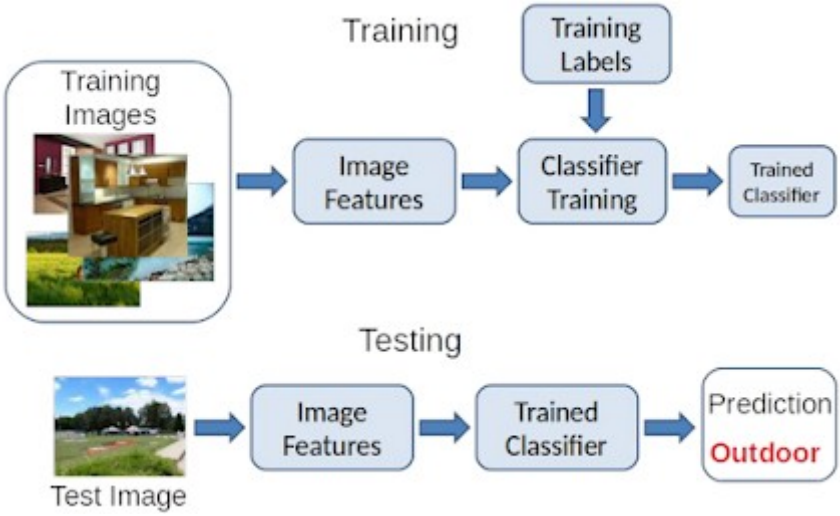
By Kevin P. Murphy about Machine Learning

The simplest illustration of how Machine Learning works is:



The left part is called **training time** and the right one is called **inference/testing time**.
We will see that all of this can work only if there exists a relationship between the training data and the testing data, meaning that they have to be in the same "format"
In the image it isn't represented but, in order to give your data to the algorithms, sometimes preprocessing/preparation is required.
This is the more general illustration, but in each type of Machine Learning there are additional components.

Let's see a practical example of this workflow:



From this example we can understand that we can't feed our model directly with the data, but we need to map them somehow (into features).

"*A computer program is said to learn from **experience E** with respect to some class of **tasks T** and **performance measure P**, if its performance at tasks in T, as measured by P, improves with experience E.*"

By T. Mitchell (1970) about Machine Learning

So, whatever the machine learning problem is, it's always necessary to specify these three things: Experience, Tasks and Measure.

In other words, Machine Learning is the study of algorithms that:

- improve their performance P (metrics which tells if the model perform well or bad)
- at some task T
- with experience E (the data)

So a well-defined learning task is given by a **triplet** $<T, P, E>$. Every learning task we've seen before can be formalized as a triplet of this type.

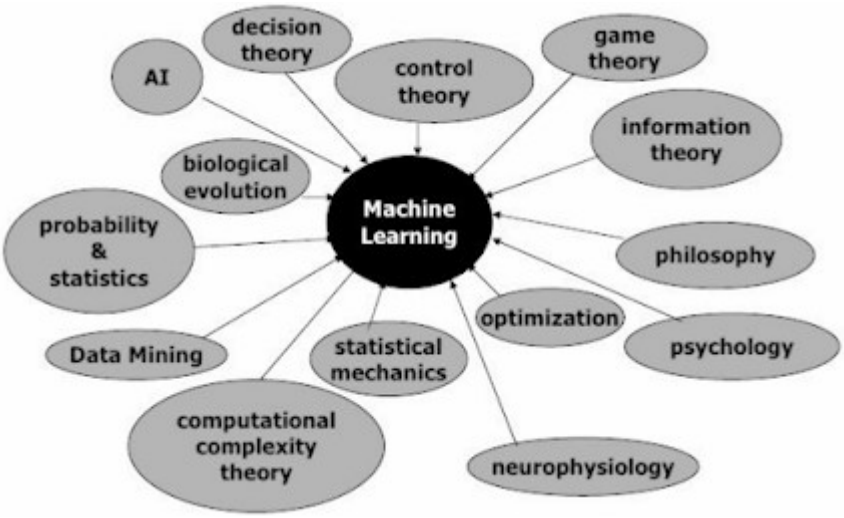
Some examples of learning tasks defined by this triplet are:

- **Handwritten Word Recognition**
  - T: Recognizing handwritten words
  - P: Percentage of words correctly classified
  - E: Database of human-labeled images of handwritten words
- **Vision-Based Highway Driving**
  - T: Driving on four-lane highways using vision sensors
  - P: Average distance traveled before a human-judged error
  - E: A sequence of images and steering commands recorded while observing a human driver
- **Email Spam Classification**
  - T: Categorize email messages as spam or legitimate
  - P: Percentage of email messages correctly classified
  - E: Database of emails, some with human-given labels

It isn't always simple to formalize the performance of a Machine Learning model, for example it was very difficult to find a metric which measures the performance of the generation of people's faces.

Some disciplines which are strictly related to Machine Learning are:

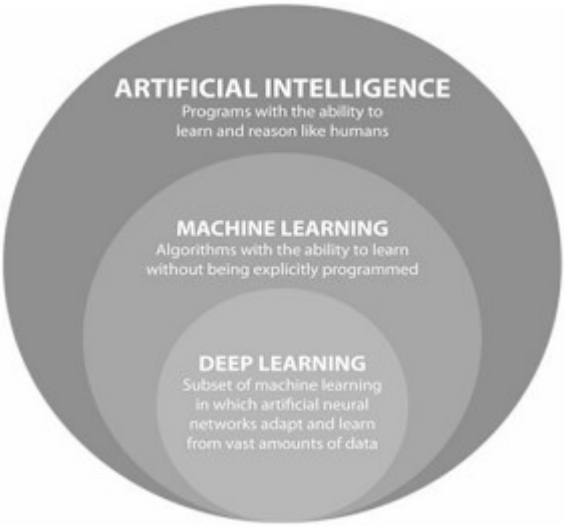
- **data mining**: data analysis, not prediction, though often involves some shared techniques
- **inference** and/or **estimation** in statistics
- **pattern recognition** in engineering
- **signal processing** in electrical engineering
- **optimization**

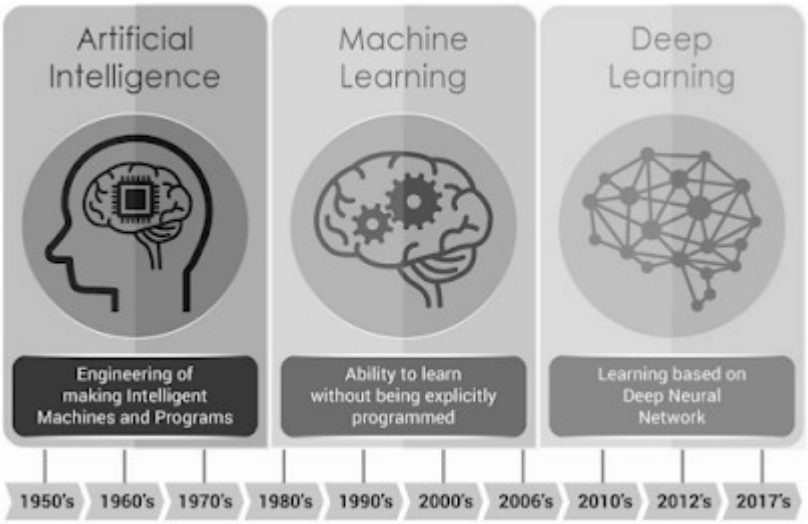


It is apparent that now Machine Learning is central to several disciplines.

It is important to understand the relationship between Artificial Intelligence, Machine Learning and Deep Learning:

- **Artificial Intelligence**
  Programs with the ability to learn and reason like humans, which don't evolve.
- **Machine Learning**
  Algorithms with the ability to learn without being explicitly programmed, knowledge and experience are fundamental.
- **Deep Learning**
  Subarea of machine learning in which artificial neural networks (like human brain) adapt and learn from vast amounts of data





"*Our ultimate objective is to make programs that learn from their experience as effectively **as humans do**.*"

By J. McCarthy (1958) about Artificial Intelligence

"***Machine learning** is the science of getting computers to act without being explicitly programmed.*"

By A. Samuel (1959) about Machine Learning

Some Machine Learning success stories are:

- **Face detection** (2002)



- **Pedestrian detection** (2005)



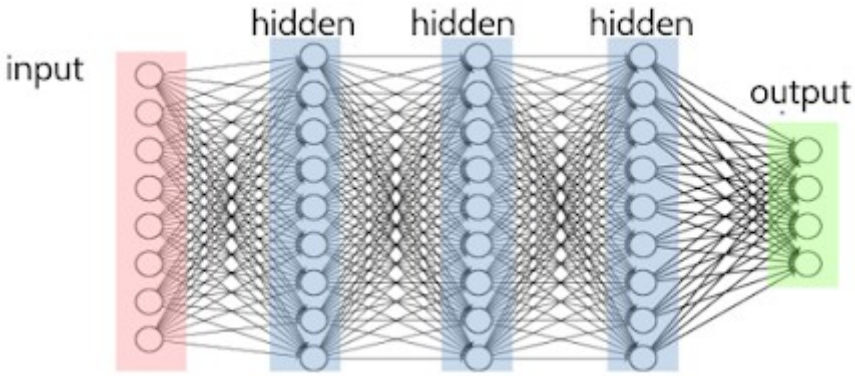
- **Body tracking, RGB-D** (2006?)



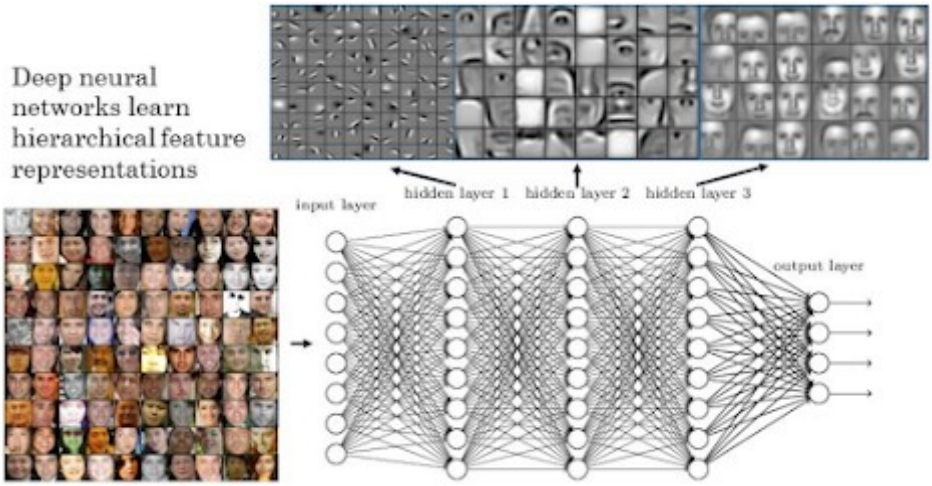
It is interesting to note that despite these applications being highly performant and widely used today thanks to deep learning, they originated and functioned with classical machine learning already.

"*Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction*"
https://www.nature.com/articles/nature14539
By Nature (2015) about Deep Learning

So, **Deep Learning** means using a neural network with several layers of (densely connected) nodes between input and output.
Deep learning revolutionizes the process of learning because you don't need to extract features from data before training the model, but it produces an output directly from the data. This mapping from the data through the output is implemented by a neural network.



The series of layers between input and output compute relevant features automatically in a series of stages, just as our brains seem to. These neural networks learn some good representation of the data.

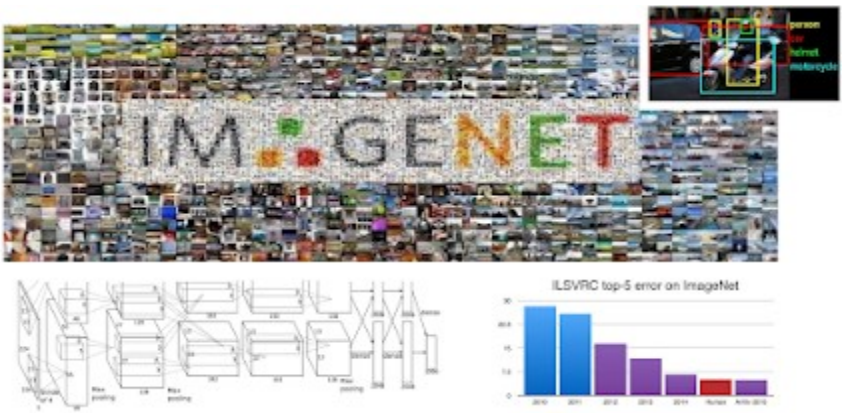


Networks like the one above are very old, because now we're using huge models which digest data from multiple modalities (images, texts, speeches) bringing the explainability to another level.

The idea that created Deep Learning was "Maybe the data are more important than the algorithms".

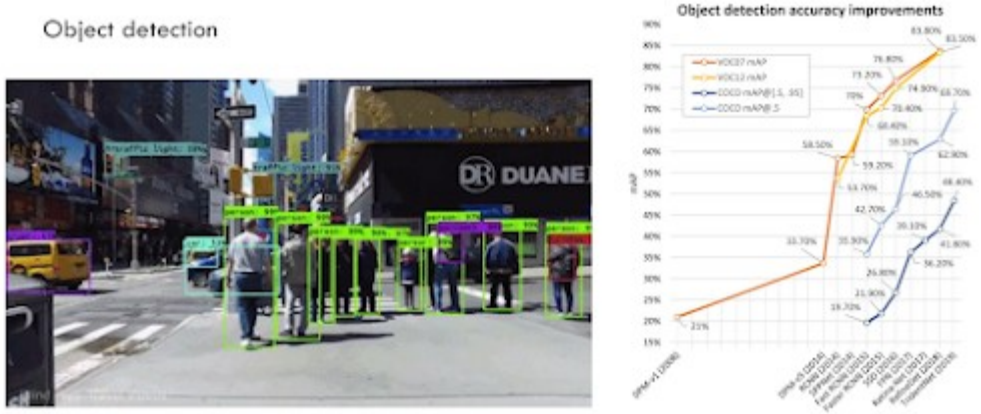
Some Deep Learning success stories are:

- **Object recognition** (2012)
  In this case we can see the huge effort done to prepare the data



In the plot, the first two blue bars indicate that the winner of the challenge were people using traditional Machine Learning techniques. Purple bars represent Deep Learning. We can observe how much Deep Learning improves the performance of tasks like this.

- **Object detection**



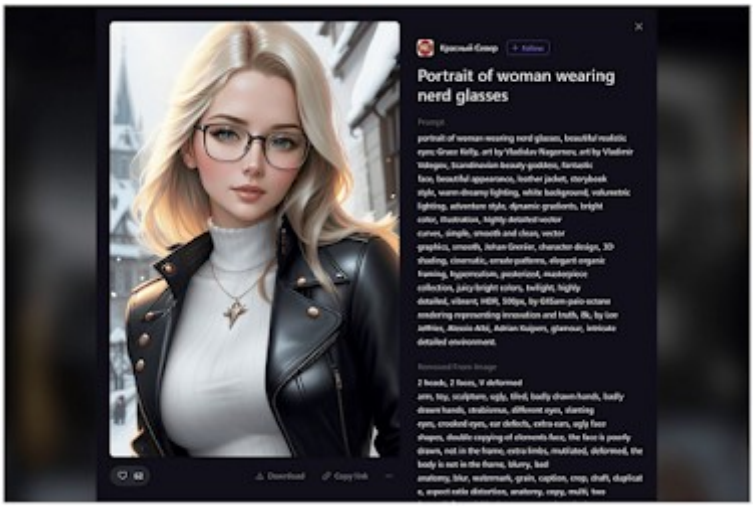
- **Image captioning** (2015)



- **Image synthesis** (2016)



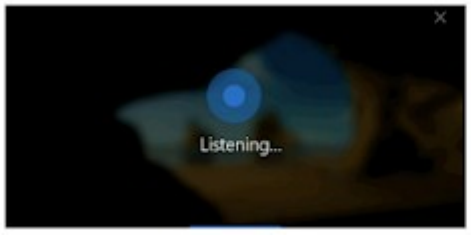
- **Text to Image synthesis** (2022)

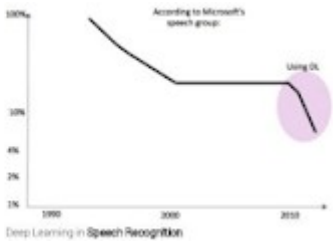


- **Text to Video synthesis** (2024)

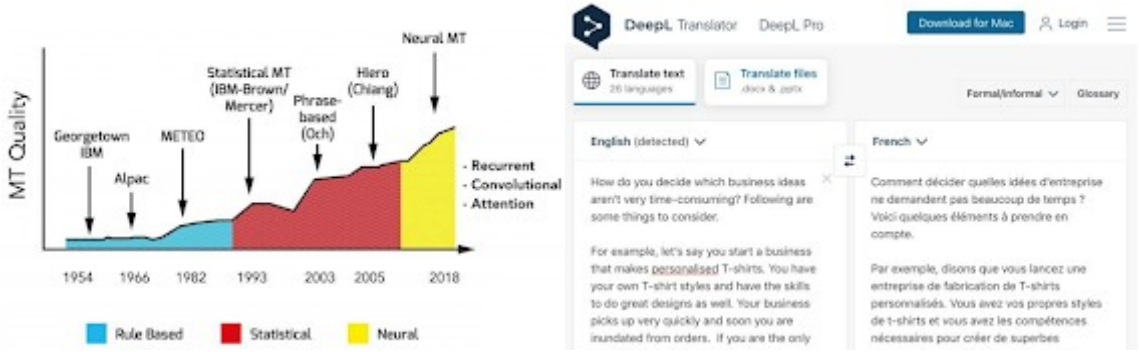


- **Speech Recognition** (2009)

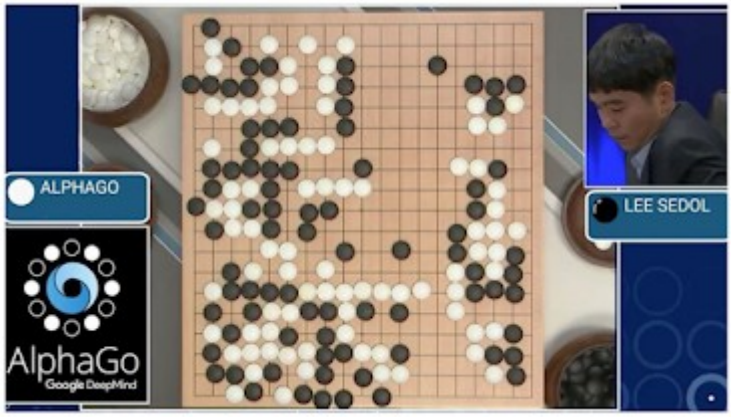




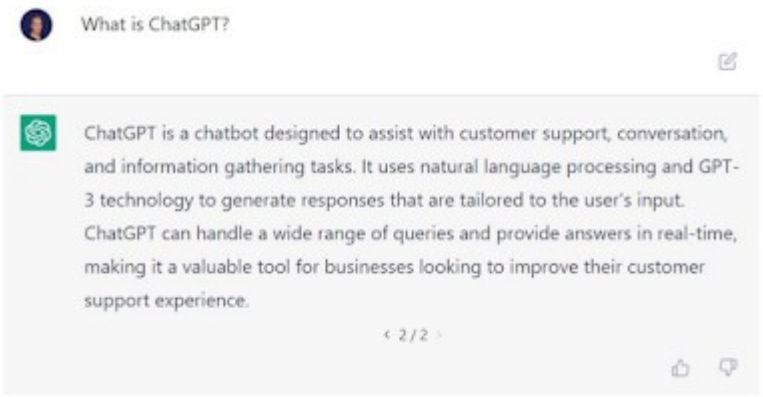
- **Neural Machine Translation** (2014)



- **AlphaGO** (2015)



- **ChatGPT** (2022)



The factors contributing to the deep learning revolution and making it happen now are:

- A **flood of available data**
- **Increased computational power**
- **A growing number of machine learning algorithms and theories developed by researchers**
- **Increased support from industry** (in the AI field, there isn't a clear distinction between academic and industrial research). In fact, the most recent crucial AI innovations were made by researchers working in industries.

Studying Machine Learning is very important because nowadays it is everywhere.



AI is really pervasive also in applications where AI traditionally was not used (like in design field)



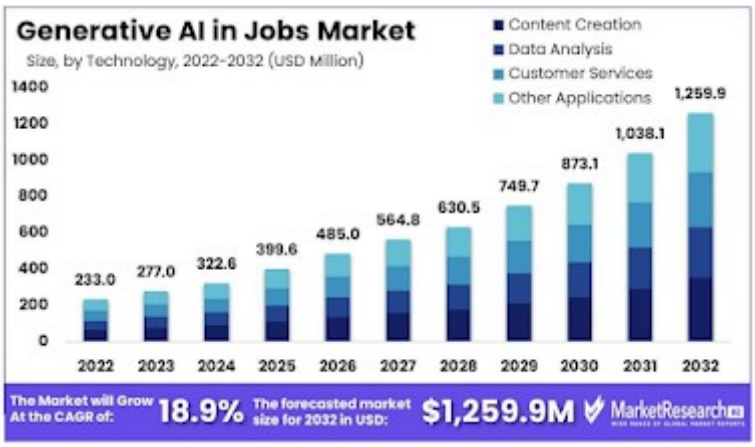
Machine Learning is a highly sought-after skill in today's market. In fact, the investment trend in this field is as follows:



Notes: Data shown is using current exchange rates and reflects market impacts of the Russia-Ukraine war.

Most recent updateL Aug 2023

Source: Statista Market Insights



The major benefits of Machine Learning include:

- **Efficiency**:
  Automates repetitive tasks, enhancing efficiency and productivity (although this may result in job displacement for some individuals).
- **Decision Support**:
  Analyzes vast amounts of data to offer valuable insights for decision-making. It can also automate decision-making processes.
- **Accuracy**:
  Machine Learning systems execute complex tasks with precision and accuracy. For instance, recognizing individuals' faces to ascertain if they have visited Israel in the past.
- **Innovation**:
  Fosters the development of innovative solutions and technologies.

But with Machine Learning we have also risks:

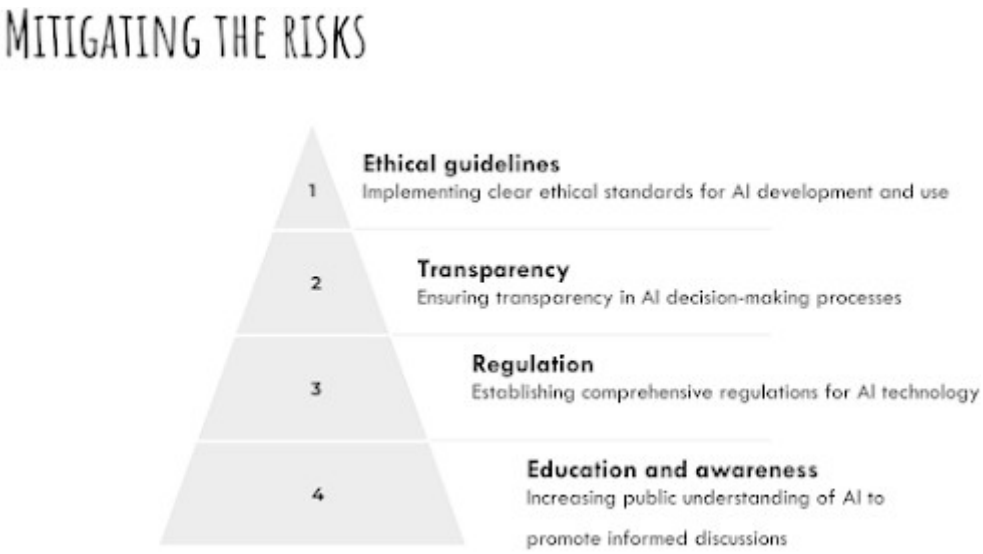
- **Algorithmic Bias**
  Artificial Intelligence can perpetuate biases in decision-making, impacting fairness.
  Bias example: When asking for an image of a doctor, the AI tends to generate more images of men than women (because it was learned with images of men).
- **Privacy Concerns**
  Artificial Intelligence technologies raise privacy issues through the collection and analysis of massive amounts of personal data.
  Utilizing data protected by copyright or containing personal information is prohibited in the training of AI models. There exists a practice known as "unlearning" which allows for the removal of personal or copyrighted information from the model to comply with these restrictions.
- **Bias in Facial Recognition**
  Facial recognition systems have shown biases against certain demographics, leading to discriminatory outcomes in law enforcement and surveillance.
  For example, in a study conducted by researchers from MIT and Stanford University, it was found that three commercially released facial-analysis programs from major technology companies exhibit biases related to skin type and gender. The error rates in determining the gender of light-skinned men were consistently low, never exceeding 0.8

percent. However, for darker-skinned women, the error rates were significantly higher, reaching more than 20 percent in one case and over 34 percent in the other two. The research raises concerns about biases in today's neural networks and questions the training and evaluation methods used. The study emphasizes the need for further investigation into disparities in various applications beyond facial analysis. The primary researcher, Joy Buolamwini, underscores the impact of data sets on model performance and challenges existing benchmarks. The research was presented at the Conference on Fairness, Accountability, and Transparency.

- **Misinformation by Chatbots**
  AI-powered chatbots have spread misinformation and fake news, leading to widespread confusion and mistrust in online information sources.
- **Job displacement**
  Automation thanks to ML may lead to widespread job loss, particularly in industries relying on routine tasks.



It is very important to mitigate risks associated with the development and use of artificial intelligence.



Let's break down each point:

1. **Ethical guidelines**:
   This refers to the need for establishing clear and explicit ethical standards for the development and use of AI. Ethical guidelines help ensure that AI systems are designed and deployed in a manner that aligns with moral principles and societal values. These guidelines may cover aspects such as fairness, accountability, transparency, and considerations for the impact of AI on various stakeholders.
2. **Transparency**:
   Transparency in AI decision-making processes involves making the operations and decisions of AI systems understandable and interpretable. It is important for users and stakeholders to have insights into how AI systems arrive at their conclusions. Transparent AI systems are more accountable, and users can better trust and comprehend the outcomes, helping to avoid unintended consequences or biases.
3. **Regulation**:
   This point emphasizes the need for comprehensive regulations governing AI technology. Regulations can provide a legal framework to ensure responsible and ethical development, deployment, and use of AI systems. Regulations may address issues such as data privacy, security, fairness, and the ethical treatment of AI in various industries. A real example of regulation is the “AI act” in Europe.
4. **Education and awareness**:
   Increasing public understanding of AI is crucial for promoting informed discussions and decisions about its use. Education and awareness initiatives aim to equip the public, policymakers, and other stakeholders with the knowledge needed to navigate the ethical and societal implications of AI. This can lead to more informed decision-making and the establishment of a consensus on how AI should be used and regulated.

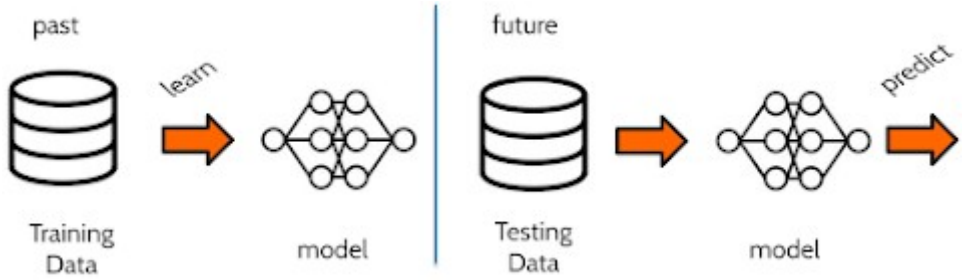
Machine Learning Basics

Let's reconsider the definition below

"*The goal of machine learning is to develop methods that can **automatically detect patterns in data**, and then to use the uncovered patterns to **predict future data** or other outcomes of interest.*"

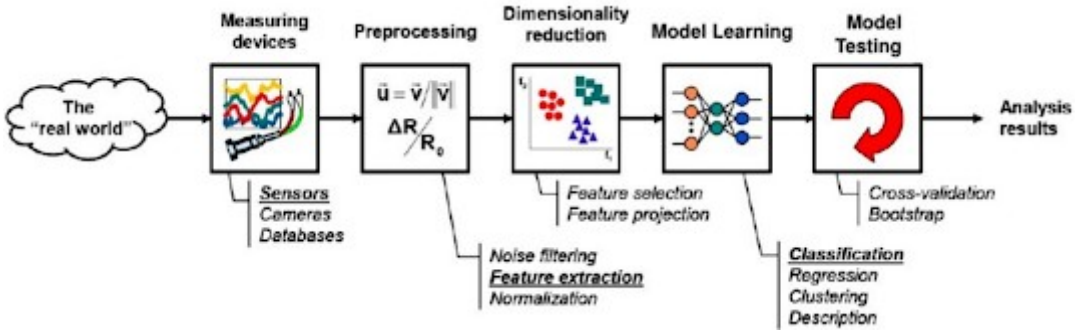
By Kevin P. Murphy about Machine Learning

This definition is the one which describes better the following image:



Training data are employed during the training phase to **learn** an (arbitrary) **model**. Subsequently, the model is applied to a set of data called **testing data** (distinct from the training data) to generate **predictions**. It is crucial that the data are in some way compatible for the model to function effectively.

The image above outlines the process in broad strokes, but the learning process involves a bunch of other phases; let's now delve into the **learning process** in more detail.



Let's explain each phase of the process:

- **Data Acquisition**
  This phase involves acquiring data from the real world using various measuring devices, such as sensors or instruments. The goal is to collect a large collection of relevant information that will be used in the subsequent steps of the learning process.
- **Data Preprocessing**
  In the preprocessing phase, the raw data collected from measuring devices undergoes cleaning and transformation/mapping (typically into vectors). This may include handling missing values, removing noise, feature extraction and standardizing or normalizing the data to prepare it for further analysis.
- **Dimensionality Reduction**
  Dimensionality reduction aims to simplify the dataset by reducing the number of features or variables while retaining essential information. Techniques like Principal Component

Analysis (PCA) or feature selection/projection methods are applied to streamline the data and improve computational efficiency.

- **Model Learning**
  This is the core phase where a machine learning model is trained on the preprocessed data. The model learns patterns, relationships, and dependencies within the data to make predictions or classifications. The choice of the learning algorithm depends on the nature of the problem (e.g., classification, regression, clustering, description) and the characteristics of the data.
- **Model Testing**
  Once the model has been trained, it needs to be tested to assess its performance on new, unseen data. Testing involves feeding the model with a separate set of data (not used during training) to evaluate its accuracy, precision, recall, or other relevant metrics. This step helps ensure that the model generalizes well to new instances and provides reliable predictions.
  Additionally, two common techniques used in model testing are:
    - **Cross-Validation**:
      Cross-validation is a technique used to assess the model's performance across different subsets of the data. By partitioning the dataset into multiple folds and systematically rotating them for training and testing, cross-validation provides a more robust estimate of the model's generalization capabilities, reducing the impact of dataset variability.
    - **Bootstrap**:
      Bootstrap is a resampling method that addresses issues related to limited dataset size. It involves creating multiple random samples (with replacement) from the original dataset, allowing for the assessment of a model's stability and uncertainty. Bootstrap provides insights into how the model performs under various sampling scenarios, making it valuable when dealing with smaller datasets.

Throughout the pipeline, it's important to **iterate and refine** each step based on the insights gained from the evaluation phase. This iterative process helps improve the performance of the model and ensure its effectiveness in real-world applications.



Let's delve deeper into each phase.

Data Acquisition

This phase involves collecting **relevant data** for the problem at hand. This data can come from various sources such as databases or sensors. Ensuring the quality and relevance of the data is crucial at this stage; in fact, it can happen that, whatever model is chosen, the system does not work due to low-quality data.





Data Preprocessing

Once the data is acquired, it often needs to be cleaned and prepared for analysis. This step involves handling missing values, dealing with outliers, and transforming the data into a suitable format for further analysis. Common preprocessing techniques include normalization, feature scaling and handling categorical variables.

a. Employees

name	*age*	*salary*
Paul	1978	NULL
Paul	NULL	29,000
Paul	1979	NULL
Melanie	1990	NULL
Bob	NULL	37,000
Bob	1977	NULL
Charlie	1978	32,000

b. Employees

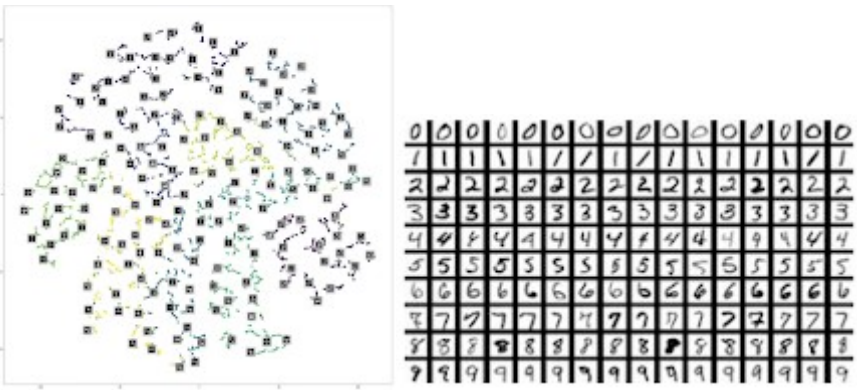
name	*age*	*salary*
Paul	?	29,000
Melanie	1990	NULL
Bob	1977	37,000
Charlie	1978	32,000

Paul.age = 1978 OR 1979 ?

Dimensionality Reduction

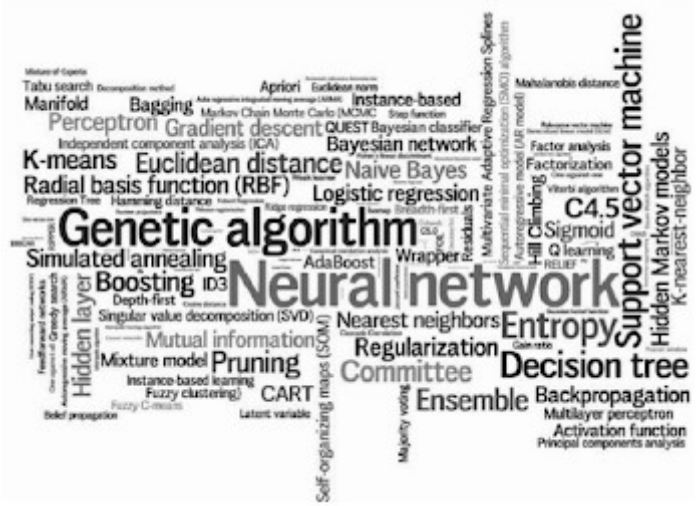
Datasets may contain a large number of features, which can lead to overfitting and increased computational complexity. Dimensionality reduction techniques or feature selection methods can be applied to reduce the number of features while **preserving the most important information**.

An example of dimensionality reduction is when you simply transform an image represented by a 32x32 pixel matrix (a black and white image) into an elongated vector, which means that for each image there will be 1024 features, so we need a tool which converts this vector into something smaller preserving the relative similarities between images.



Model Learning

This is the core of the machine learning pipeline, where a model is trained on the preprocessed data to learn patterns and relationships. The choice of model depends on the nature of the problem (e.g., classification, regression) and the characteristics of the data. Common machine learning algorithms include linear regression, decision trees, support vector machines, and neural networks. It is very important to understand the pros and cons of every algorithm we will see.



Model Testing

Once the model is trained, it needs to be evaluated to assess its performance and **generalization capabilities**. This is typically done using a separate dataset called the test set, which was not used during the training phase. **Performance metrics** such as accuracy, precision, recall, F1-score, or mean squared error are computed to quantify the model's performance. For example counting the number of “right” choices made by the model (but it isn’t always possible testing in this way, like in the video generation field where humans can do it but it is a lot more expensive than a machine).

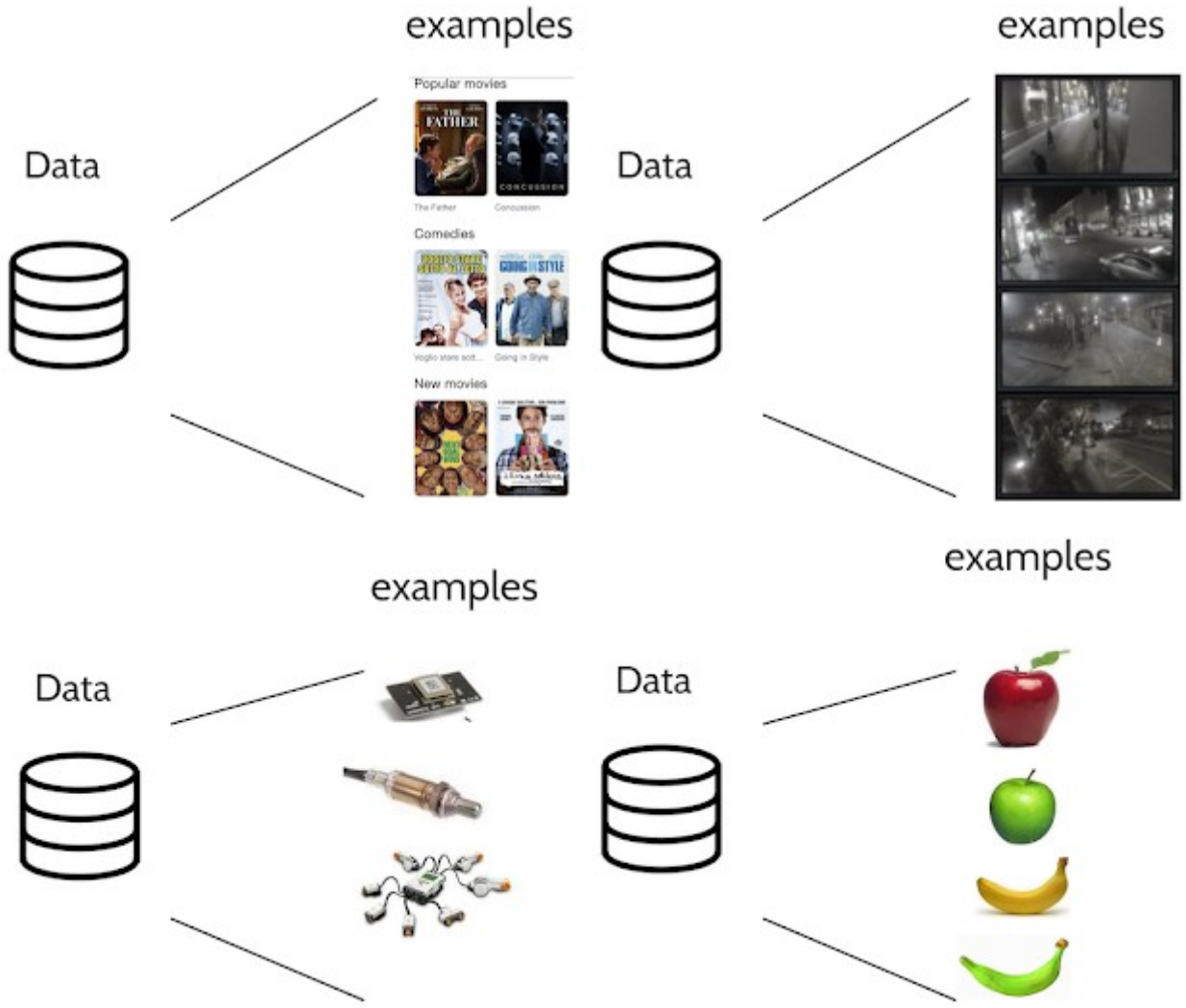
Additionally, techniques like cross-validation can be used to assess the model's robustness.

In conclusion, it is important that the model performs well on the training data, but it is fundamental that the model performs well also on other data, and this is **generalization**. Testing is very important for the algorithms on which we have less control, like neural networks.

		Predicted	
		Has Cancer	Doesn't Have Cancer
Ground Truth	Has Cancer	TP	FN
	Doesn't Have Cancer	FP	TN

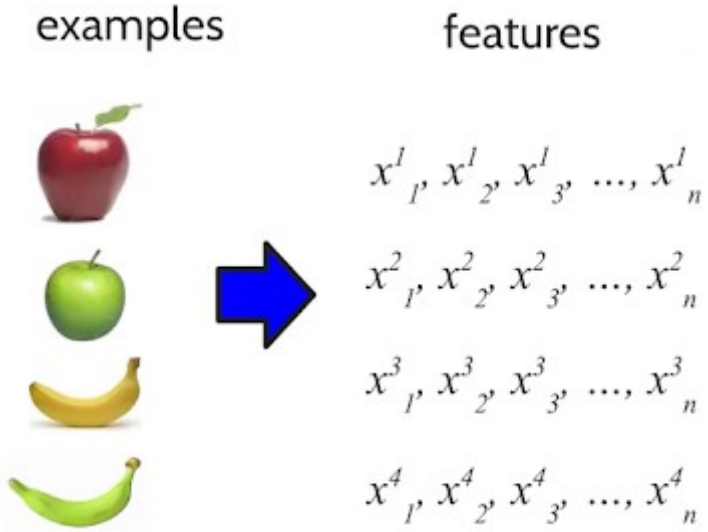
Data

So the Learning Process starts with **data** (which are real world things). Let's see some examples of which type of data we can take into consideration.

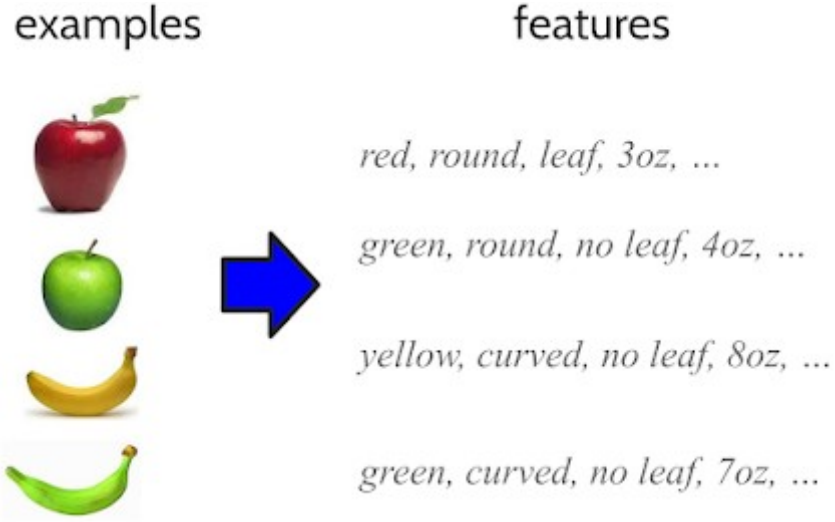


The examples should be presented in a way that the computer can read and process them. A typical choice of representation is **feature vectors**, but there exists other ways of representation. We have to decide how many features we want to take in consideration, for

example n, and if we have m examples then the training set will be a $m \times n$ matrix. Every column is a different feature.
The decision of how to map the data into the features vector is crucial (the blue arrow).



So the features are how our algorithms actually “view” the data. In general features are the **questions** we can ask about the examples.
For example, taking into consideration fruit, we can have a feature for the color, one for the shape, etc.



Obviously in this process of mapping there are some problems:

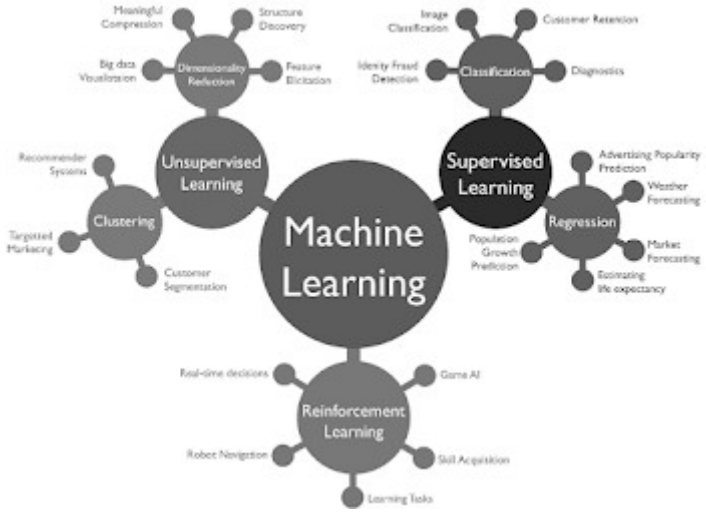
- **Loss of information**
  Mapping data to features often implies a loss of information because features are typically derived or selected representations of the original data that capture certain aspects deemed relevant for a particular task or analysis. This process involves **simplification or abstraction**, which can lead to a reduction in the amount of information available compared to the raw data.
- **Impact of the decision of the feature**
  The choice of features significantly influences model performance. Inadequate or

irrelevant features may lead to overfitting or poor generalization. Proper feature selection enhances model interpretability and reduces computational complexity. Unbiased feature selection ensures the model focuses on relevant patterns, improving predictive accuracy. Consideration of domain knowledge is crucial to identify and prioritize features for optimal model outcomes.

Types of learning

The field of **Machine Learning** can be **divided** into three major subsets:

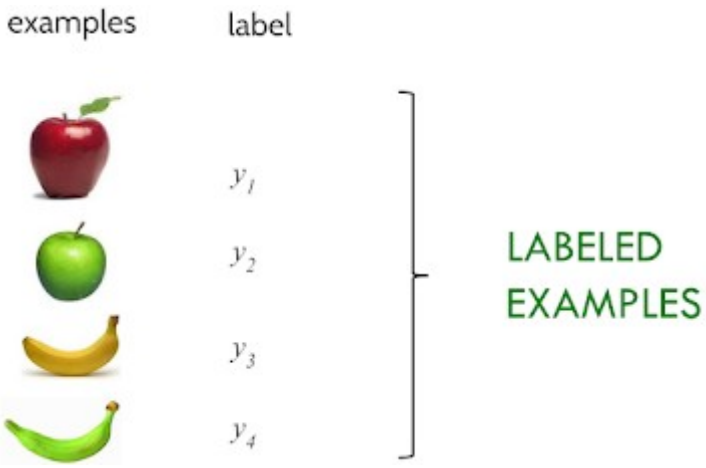
- **Supervised Learning**
  In supervised learning, the algorithm is trained on a **labeled dataset**, where each input has a corresponding desired output. The goal is to learn a mapping from inputs to outputs, enabling the model to make predictions on new, unseen data.
- **Unsupervised Learning**
  Unsupervised learning involves training models on **unlabeled data**, aiming to identify patterns, relationships, or structures within the dataset. Common techniques include clustering and dimensionality reduction, providing insights into the inherent structure of the data.
- **Reinforcement Learning**
  Reinforcement learning focuses on training models to make **sequences of decisions** by interacting with an environment. The model receives feedback in the form of rewards or penalties, allowing it to learn optimal strategies over time. This subset is often employed in scenarios where an agent must learn to navigate and make decisions in dynamic environments.



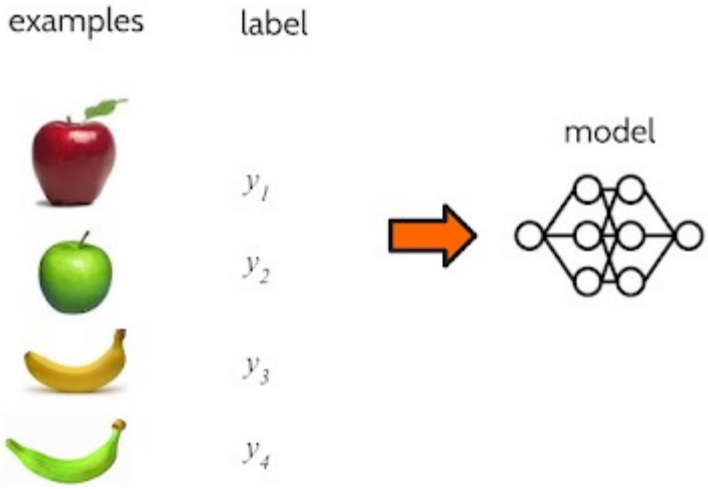
Supervised learning

In supervised learning each example has been associated with a **label** in training time. So in addition to the features matrix we have a label vector (one label for each example).
The process is as follows:

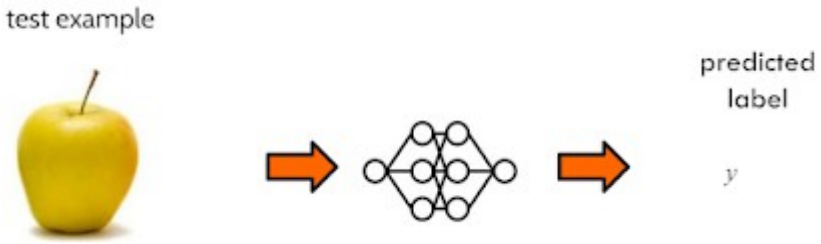
- Given labeled examples



- Build a model



- Train the model to predict the label associated with other examples



Three fundamental techniques within supervised learning are:

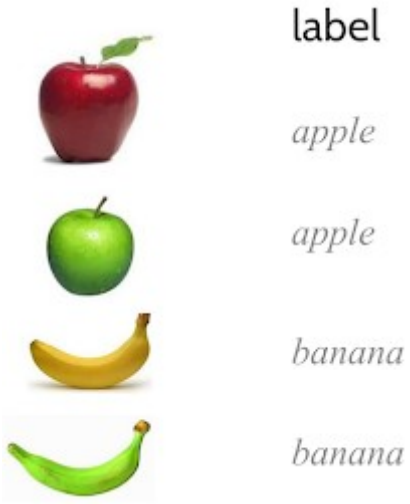
- **Classification**
  In classification, the goal is to assign predefined categories or labels to input data. The algorithm learns from labeled examples in the training dataset to categorize new instances accurately. This technique is widely used for tasks such as spam detection, image recognition, and sentiment analysis, where the model classifies inputs into distinct classes.
- **Regression**
  Regression, on the other hand, deals with predicting a continuous outcome or numerical

value. The algorithm learns the relationship between input features and a target variable, allowing it to make predictions within a range. Regression is commonly applied in scenarios like stock price prediction, housing price estimation, and forecasting, where the goal is to predict a specific numeric value.

- **Ranking**
  Ranking involves the arrangement of items in a specific order based on their relevance or importance. Supervised learning algorithms for ranking learn from pairs of items, understanding the preference or priority between them. This technique is employed in search engine result ranking, recommendation systems, and information retrieval, where the model determines the most relevant items for a given query.

Classification

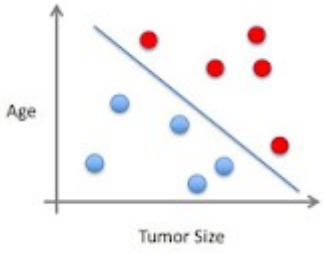
In Classification there is a **well-defined finite set of labels**.



The scope of Classification is, given in input couples of the type $(example, label)$ like $T = \{(x_1, y_1), \ldots, (x_m, y_m)\}$, to learn a function to predict y given x.

x is generally multidimensional (that is multiple features).

$$f : \mathbb{R}^d \to \{1, 2, \ldots, k\}$$



In this example we have k categories/labels.

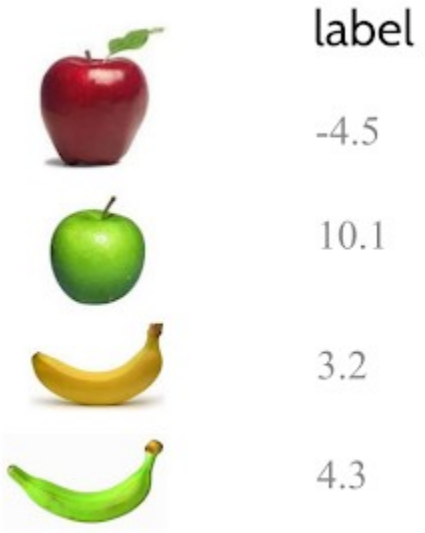
Some real useful applications of Classification are:

- Face recognition (in this case k is very high)
- Character recognition
- Spam detection ($k = 2$)
- Medical diagnosis: from symptoms to illnesses

- Biometrics: Recognition/authentication using physical and/or behavioral characteristics: Face, iris, signature, etc

Regression

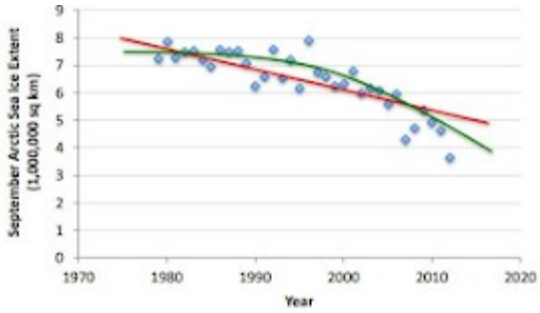
In Regression the **labels** are **real-valued** (that is non-finite).



The scope of Regression is, given in input couple of the type $(example, label)$ like $T = \{(x_1, y_1), \ldots, (x_m, y_m)\}$, to learn a function to predict y given x.

y is real-valued.

$$f : \mathbb{R}^d \rightarrow \mathbb{R}$$

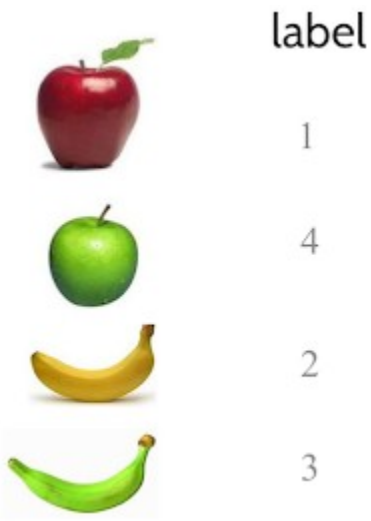


Some real useful application of Regression are:

- Economics/Finance: predict the value of a stock
- Car navigation: angle of the steering wheel, acceleration, …
- Temporal trends: weather over time

Ranking

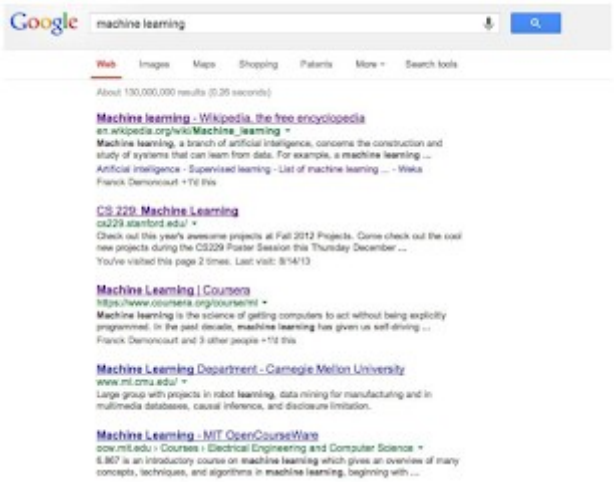
In Ranking **labels** are **ranks**.

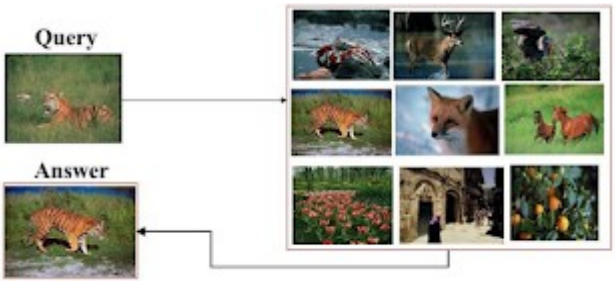


The scope of Ranking is, given in input couple of the type $(example, label)$ like $T = \{(x_1, y_1), \ldots, (x_m, y_m)\}$, to learn a function to predict y given x.
y is a rank.

Some examples of Ranking are:

- Given a query and a set of web pages, rank them according to relevance
- Given a query image, find the most visually similar images (with an order) in the database





Some real useful application of Ranking are:

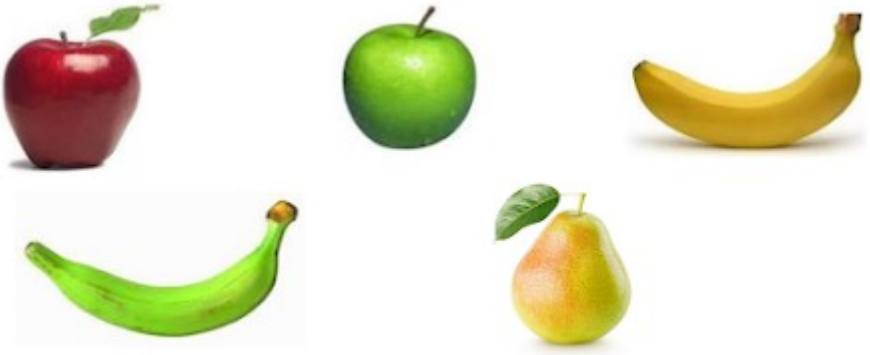
- User preference, e.g. Netflix movie ranking
- Image retrieval
- Flight search (search in general)

So, in summary, Supervised Learning are divided into:

- **Classification**
  Classification categorizes input data into predefined classes or categories.

- **Regression**
  Regression predicts continuous numerical values based on input features.
- **Ranking**
  Ranking orders a set of items based on their relevance or preference.

Unsupervised learning
As we've already mentioned, in unsupervised learning the given **data is unlabeled** and **there aren't a set of labels**.

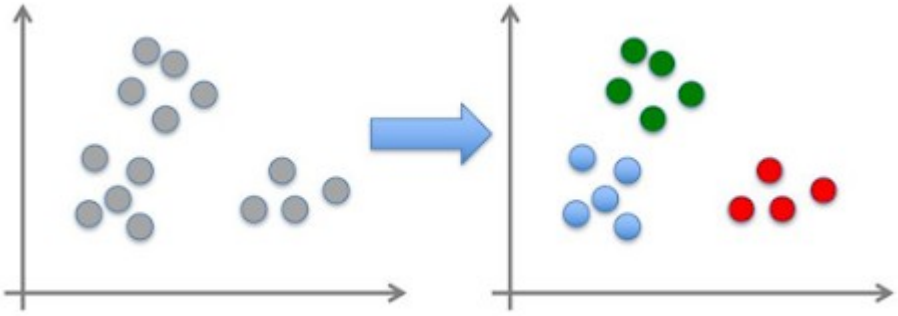


Two fundamental techniques within unsupervised learning are:

- **Clustering**
  In clustering, the goal is to group similar data points together based on inherent patterns or similarities without explicit category labels. The algorithm identifies natural clusters within the dataset, allowing for the discovery of underlying structures or relationships. Common applications of clustering include customer segmentation, anomaly detection, and image segmentation. K-means clustering, hierarchical clustering, and DBSCAN are examples of clustering algorithms.
- **Dimensionality Reduction**
  Dimensionality reduction aims to reduce the number of input features while retaining essential information. This helps in simplifying the dataset, alleviating the "curse of dimensionality," and improving the efficiency of subsequent analysis. Principal Component Analysis (PCA), t-Distributed Stochastic Neighbor Embedding (t-SNE), and autoencoders are popular techniques for dimensionality reduction. Applications include visualization of high-dimensional data, feature selection, and noise reduction in data preprocessing.

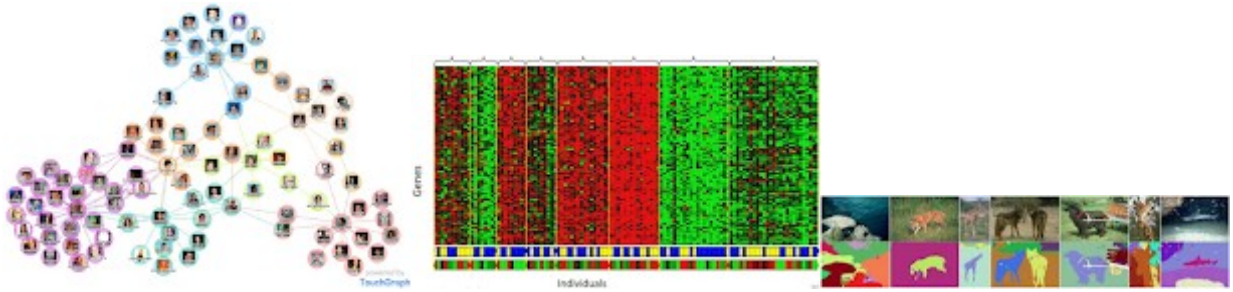
These unsupervised learning techniques play a crucial role in extracting meaningful insights from data without relying on predefined labels, making them valuable tools in exploratory data analysis and pattern discovery.

Clustering
The scope of Clustering is, given in input the examples $T = \{x_1, \ldots, x_m\}$ (without labels), to output hidden structure behind the x's, that is clusters.



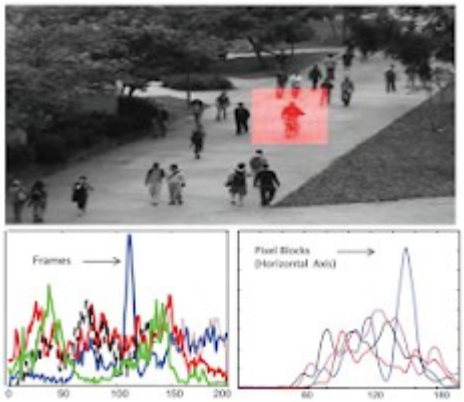
Some real useful application of Clustering are:

- Social Network Analysis
- Genomics: group individuals by genetic similarity
- Image segmentation



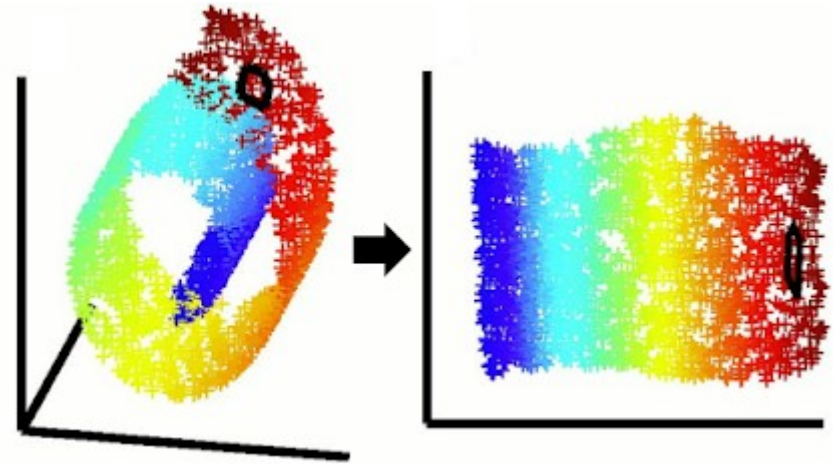
An application of clustering is Anomaly Detection, which consists in analyzing a set of events or objects and identifying some as unusual or atypical. In anomaly detection, the algorithm flags instances that deviate from the expected patterns or norms within the data.
This method finds applications in various fields, such as credit card fraud detection and video surveillance, where the goal is to pinpoint irregularities or suspicious activities in a dataset.





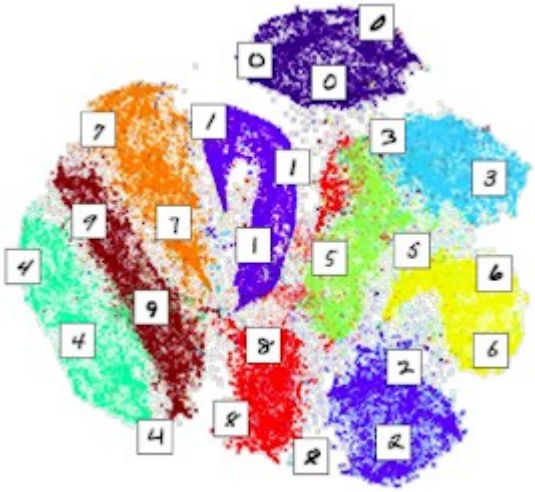
Dimensionality Reduction

The scope of Dimensionality Reduction is to reduce the number of features under consideration by mapping data into another low dimensional space.



Some real useful application of Dimensionality Reduction are:

- **Inspecting your classification algorithm**
  Dimensionality reduction reduces the number of variables in a dataset while retaining most of the original information. This facilitates the inspection of classification algorithms by making it easier to visualize and understand their performance, identify issues, and enhance efficiency.



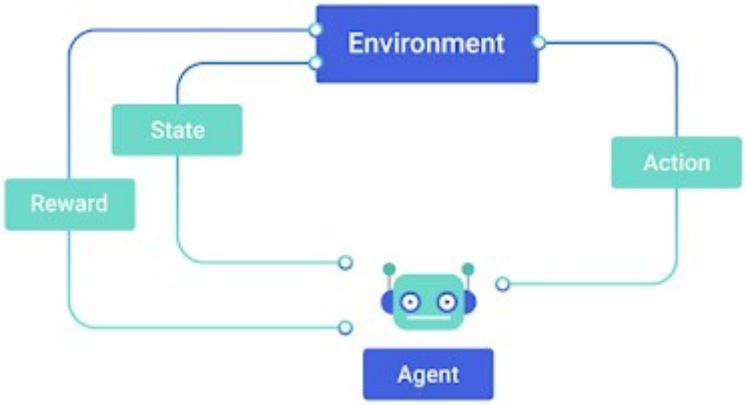
So, in summary, Unsupervised Learning are divided into:

- **Clustering**
  Clustering groups similar data points into clusters or groups based on their inherent characteristics or features.
- **Dimensionality Reduction**
  Dimensionality reduction reduces the number of input variables or features in a dataset while preserving the most important information.

Reinforcement learning

Reinforcement learning is considered the most challenging paradigm in machine learning due to its interactive nature, where agents learn by making decisions, taking actions in an environment, and adapting to complex and dynamic consequences over time.
In reinforcement learning there is no concept of offline learning, but it is all interactive.

The idea is: an **agent** learns from the **environment** by interacting with it and receiving **rewards** for performing **actions**. The agent will change its actions based on the rewards. The agent can also see the state of the environment.



Some examples of Reinforcement Learning are:

- Backgammon: Given sequences of moves and whether or not the player won at the end, learn to make good moves
- Atari breakout



In these cases:

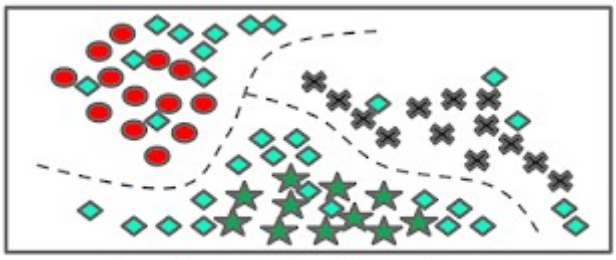
- Actions represent the moves or decisions that the agent can take in the game
- States refer to the different situations or configurations of the game at any given moment
- Rewards are numerical values that the agent receives as feedback based on its actions and the resulting states

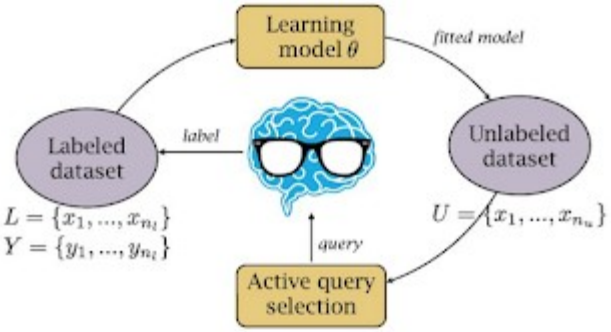
So the agent will receive indications whether the moves are good ones or bad ones.

The landscape of machine learning extends beyond the conventional paradigms of supervised, unsupervised, and reinforcement learning, incorporating more intricate approaches to address the complexities of real-world scenarios.

One notable example is **semi-supervised learning**, which blends elements of both supervised and unsupervised learning methodologies, allowing models to benefit from labeled data while simultaneously exploring and leveraging the inherent structures within unlabeled data. These models take in input some data which are labeled and others which aren't.

Additionally, **active learning** represents another dynamic approach, wherein the learning process commences with a small set of labeled data, and the algorithm actively selects the most informative instances for annotation, optimizing the utilization of resources and refining the model's performance iteratively. Then the model receives another set of unlabeled data and instead of making the inference without further help, there is someone who actively provides the labels.


Semi-supervised learning

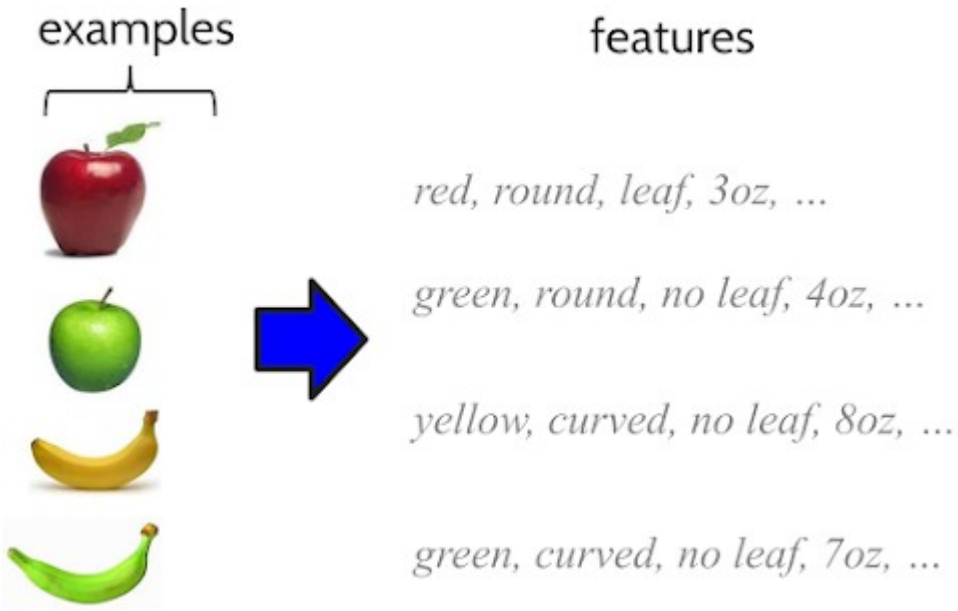


There exists also another macro-differentiation into two type of learning: online and offline (batch) learning
The difference between online and batch (offline) learning lies in how the learning process occurs and **how the model is updated with new data**:

- **Batch (Offline) Learning**:
  The model learns from the entire dataset in one go and is updated only after processing all the data. Once the model is trained, it remains static and does not change unless retrained with the entire dataset.
  Typically used when the dataset fits into memory and can be processed efficiently as a whole.
- **Online Learning**:
  The model learns incrementally from each new data point or small batches of data.
  Allows the model to adapt to changes in the data distribution over time.
  Suitable for scenarios where data isn't all available in the beginning but arrives sequentially and needs to be processed in real-time or where computational resources are limited.

Offline learning is often considered preferable due to its capacity to train models on a complete dataset in a single pass, allowing for efficient and robust computation and resource utilization. However, the feasibility of offline learning is contingent upon the availability of sufficient memory resources, and in cases where memory constraints arise, online learning becomes a pragmatic alternative.

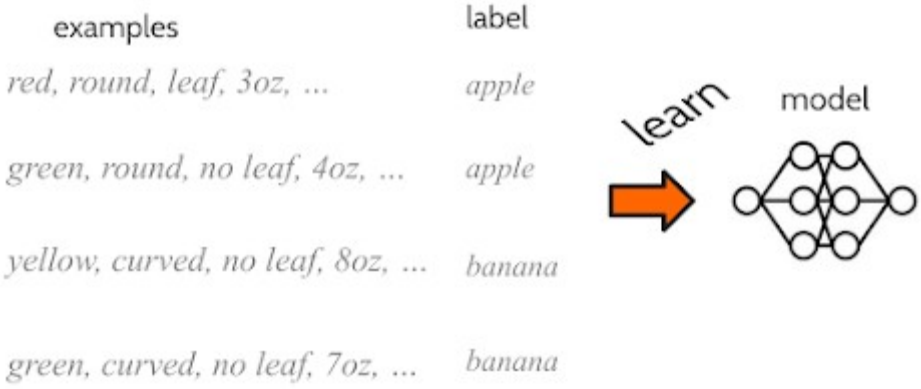
Let’s go back to data and features.



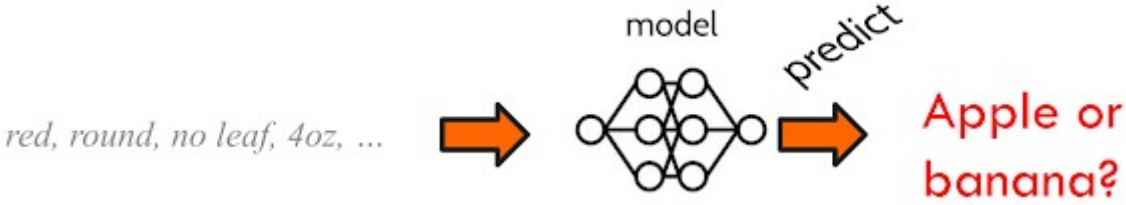
A key step in Machine Learning is the blue arrow, that is the transition from real examples into features. Obviously this function has to be efficient and complete, meaning that it has to include all the data useful for the final goal.
It is the machine learning engineer's task to put a lot of effort into designing the best algorithm in order to compute the features.

The question is: How can the model predict?
Let's see how this works in Classification:

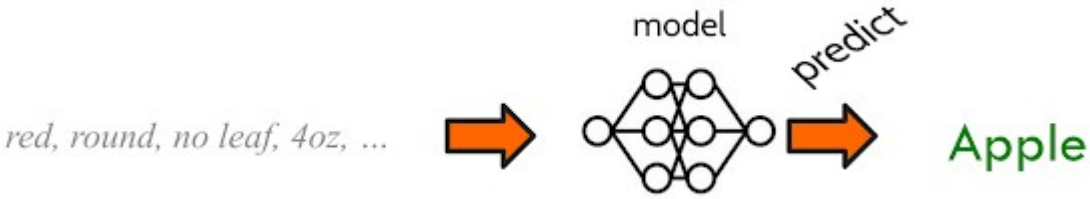
- **The model is trained**
  During training, learn a model of what distinguishes apples and bananas **based on the features**



- **The new example is described by features**



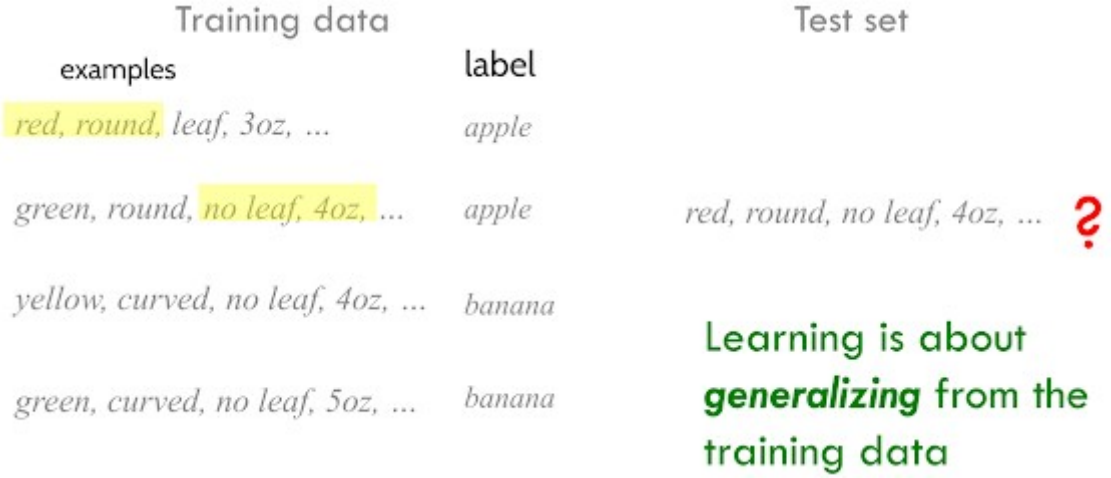
- **The model can then classify a new example based on the features**



But how is the prediction done? Let's examine a practical example:



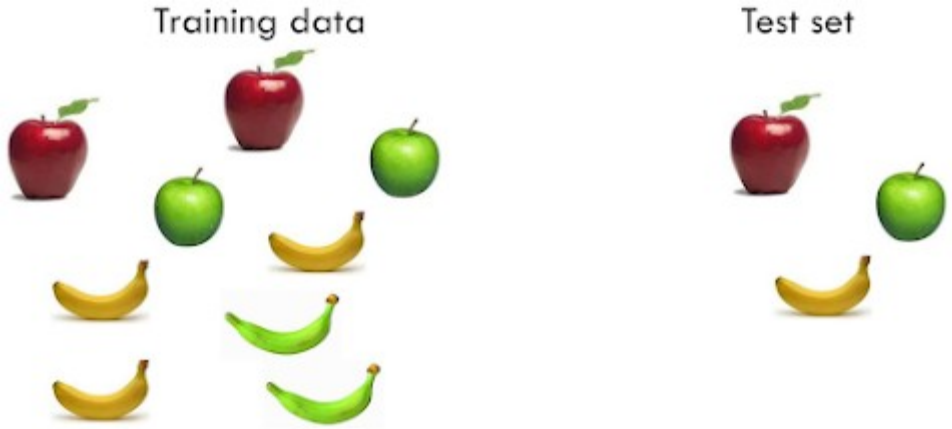
The fact that the training set and the test set have some relationship is crucial, otherwise it won't be able to label examples in the test set.
The keyword is **generalization**, in fact learning is about generalizing from the training data.



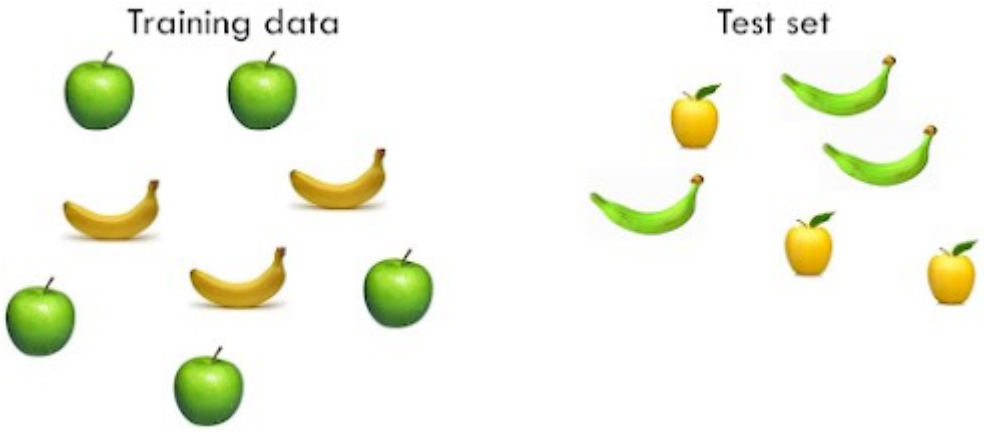
The underlying idea is that, throughout the training process, the model grasps that these features correspond to those of an apple.

Generalization in machine learning refers to the ability of a trained model to perform well on new, unseen data that was not used during the training process. A model generalizes well when it can accurately make predictions or produce outputs for data it has not encountered before.

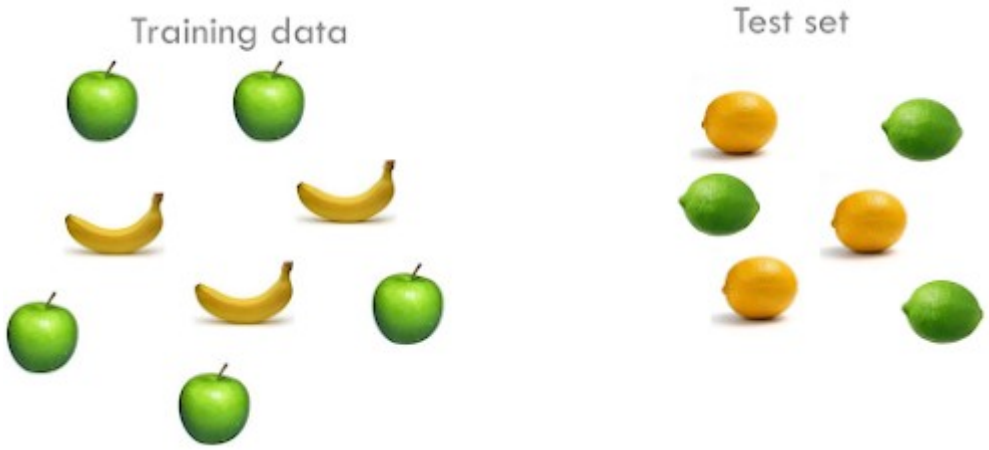
Obviously this assumes that there is a relationship between the training data and the test data, like in this case:



But this isn't always the case, and in the other cases the learning and the prediction aren't possible:



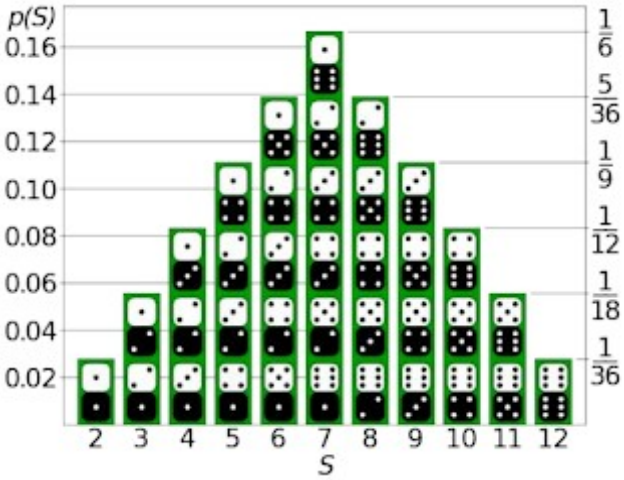
In this case, the model won't be able to identify green bananas as bananas if color is selected as a feature. However, if you omit color as a feature, the model will likely classify bananas correctly, but it may incorrectly classify lemons as apples.



More technically speaking, we will use a **probabilistic learning model**. There exists a probability distribution over example/label pairs, known as the **data-generating distribution**. **Both the training data and the test set** are derived from this distribution, which is unknown.

A probability distribution describes how likely (i.e. probable) certain events are:

- Considers probabilities for all possible events
- Probabilities are between 0 and 1 (inclusive)
- Sum of probabilities over all events is 1

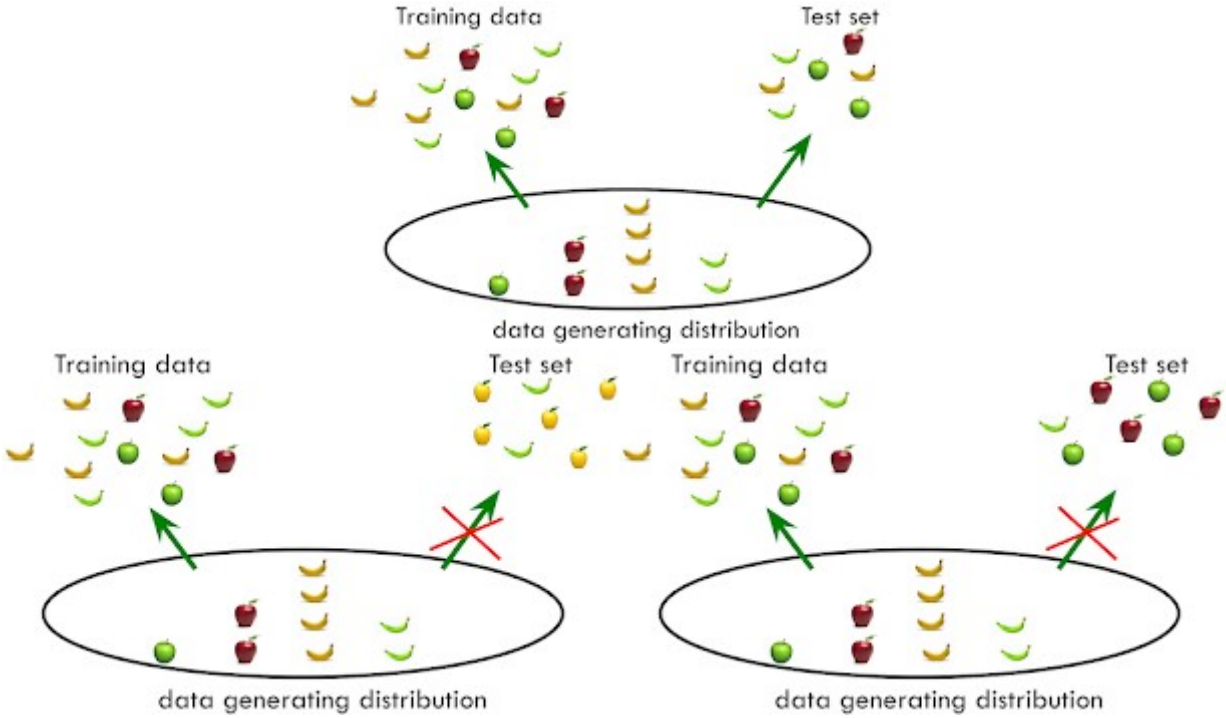


More precisely, a **probability distribution** is a mathematical function that describes the likelihood of different outcomes or events occurring within a certain set of possible outcomes. It provides a way to represent and quantify uncertainty in various situations.
In simpler terms, a probability distribution tells us **how probable it is for different events** to occur. It assigns probabilities to each possible outcome, with the sum of all probabilities equaling 1.

The probability comes in our example of the fruits because if I look at the training set we have something which comes with high probability and something with low probability.



As we've already mentioned, we assume that the test examples follow the same distribution as the training examples. This is achieved by sampling both datasets from the same distribution.



For the two examples above, the left one is impossible (there are objects which aren't in the distribution), the red one instead is possible despite being unlikely.

A **data generating distribution** refers to the underlying probability distribution that generates the observed data points in a dataset. The data generating distribution captures the inherent patterns, structure, and noise present in the data. Understanding this distribution is crucial for building accurate and generalizable machine learning models because **it enables us to make informed assumptions about the data and to make predictions** or decisions based on probabilistic reasoning.
The model can perform well only if both the training set and the test set are drawn from the same distribution. However, the challenge lies in the fact that we never know the distribution explicitly; we only have an implicit representation through the training set.

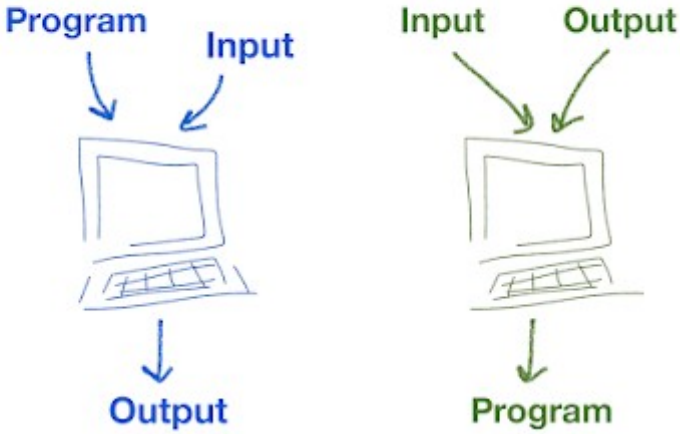
Let’s reprise some concepts we’ve already seen.



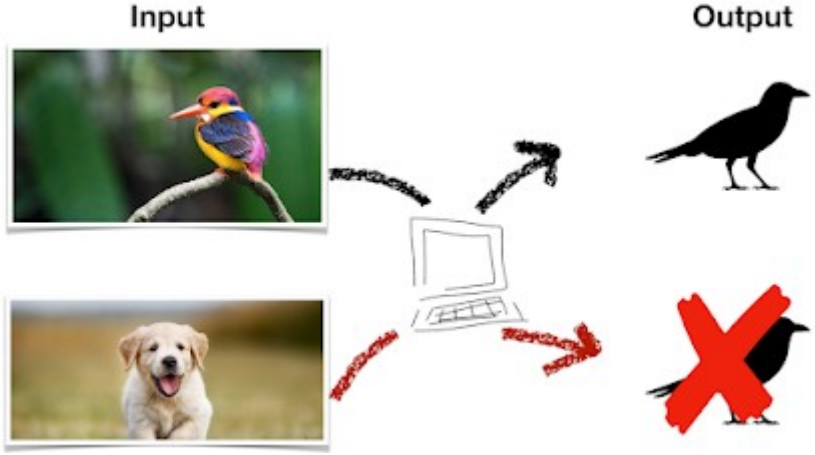
Machine learning is the science of getting computers to act without being explicitly programmed.

A. Samuel (1959)

Programming VS Machine Learning

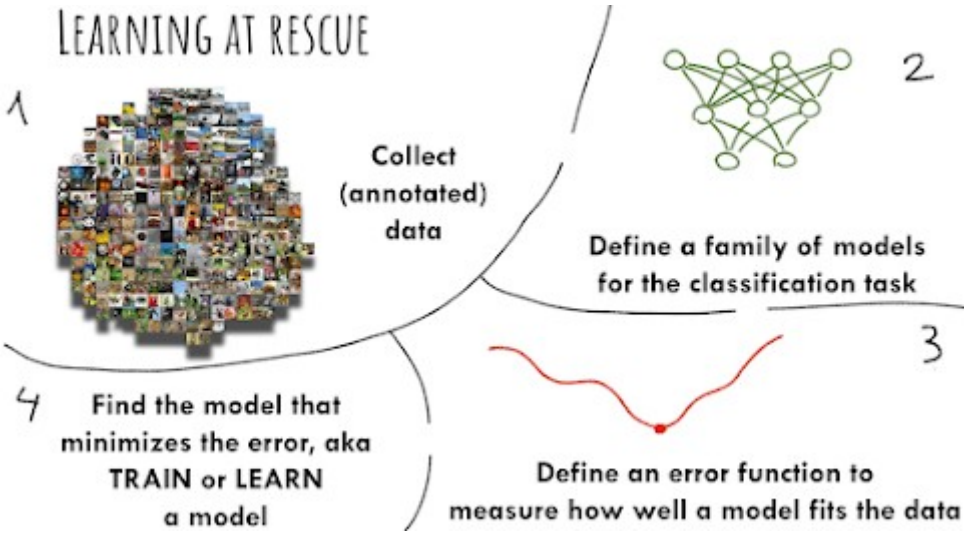


Classification can recognize objects and individuals in pictures



Solving this task with conventional programming is almost impossible.

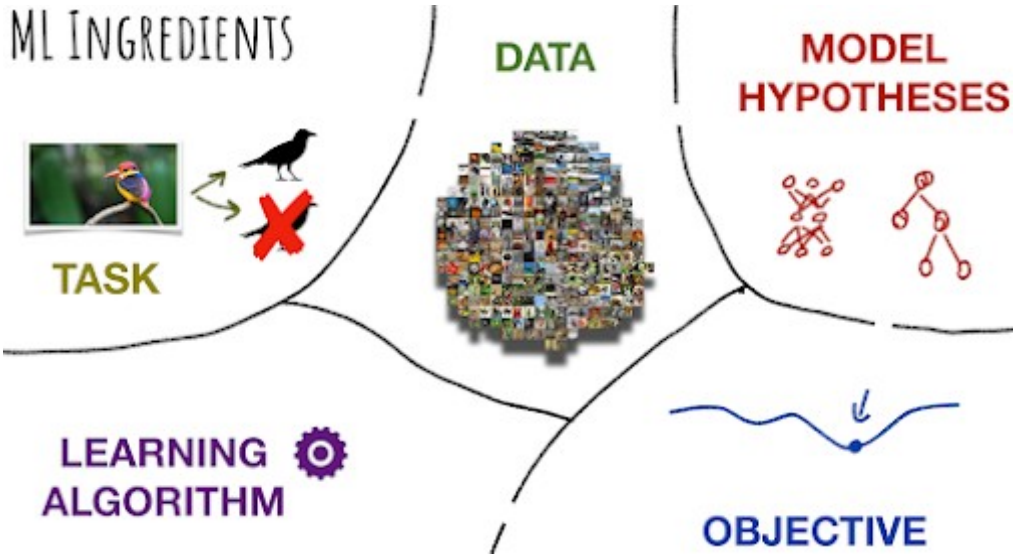
When I do have to a Classification pipeline, I care about four things:



Let's break down each step:

1. **Collect (Annotated) Data**:
   The first step involves gathering a dataset that is annotated or labeled. Annotated data means that each example in the dataset is paired with its corresponding correct classification or label. This labeled data serves as the foundation for training a machine learning model.
2. **Define a Family of Models for the Classification Task**:
   A family of models refers to a set of potential algorithms or architectures that could be employed for the given classification task. Different models may have varying complexities and capabilities, and choosing the right family of models is a crucial aspect of the machine learning process.
3. **Define an Error Function to Measure Model Fit**:
   The next step is to establish an error function, also known as a loss or cost function. This function quantifies how well a particular model aligns with the annotated data. The goal is to minimize this error, ensuring that the model accurately predicts the labels of the given examples.
4. **Find the Model that Minimizes the Error (Train or Learn a Model)**:
   The final step is to find the specific model within the chosen family that minimizes the error function. This process involves adjusting the model's parameters or weights iteratively based on the training data, with the objective of reducing the error and enhancing the model's predictive performance.

Let’s now look at the key components and ingredients involved in a machine learning (ML) process in general.



Let's delve into each element:

- **Task**:
  The task is the specific problem or objective that the machine learning system aims to address or solve. It defines what the model is supposed to learn from the data. Tasks can vary widely, ranging from image classification and speech recognition to natural language processing and regression.
- **Data**:
  Data is the raw information that the machine learning algorithm utilizes for training, testing, and making predictions. The quality, quantity, and relevance of the data play a crucial role in determining the performance of the machine learning model. Data can include features or attributes that describe the examples, as well as the corresponding labels or outcomes for supervised learning.
- **Model hypotheses**:
  Model hypotheses refer to the assumptions or expectations about the patterns and relationships present in the data. These hypotheses guide the design and selection of the machine learning model. For example, in a classification task, the model hypotheses might include assumptions about how certain features contribute to the prediction of different classes.
- **Objective**:
  The objective articulates the goal or measure of success for the machine learning task. It defines what the model is trying to achieve. In a supervised learning scenario, the objective might be to minimize prediction errors or maximize accuracy. The objective guides the learning process by providing a clear goal for the model.
- **Learning algorithm**:
  The learning algorithm is the computational method or procedure used to train the machine learning model. It is responsible for adjusting the model's parameters based on the training data, with the aim of optimizing performance according to the defined objective. Learning algorithms can vary based on the type of task (e.g., supervised, unsupervised, reinforcement learning) and the specific model used.

These are the ingredients for any type of Machine Learning task.

Task

A **task** represents the type of prediction being made to solve a problem on some data. We can identify a task with the set of mathematical functions that can potentially solve it.

In general, i.e. it consists of functions assigning each input $x \in X$ an output $y \in Y$

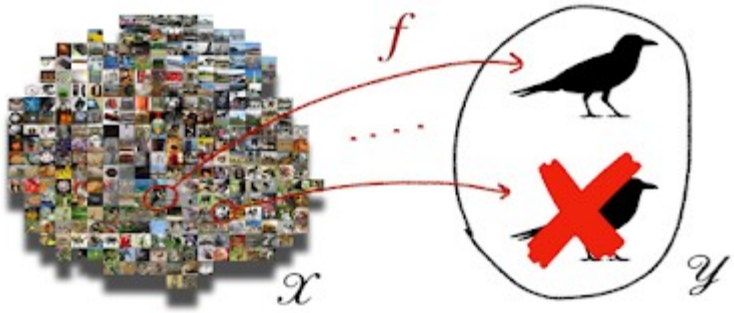
$$f : X \to Y \qquad\qquad F_{task} \subset Y^X$$

The nature of X, Y and F_{task} depends on the type of task. The function F_{task} is a subset of the cartesian product.

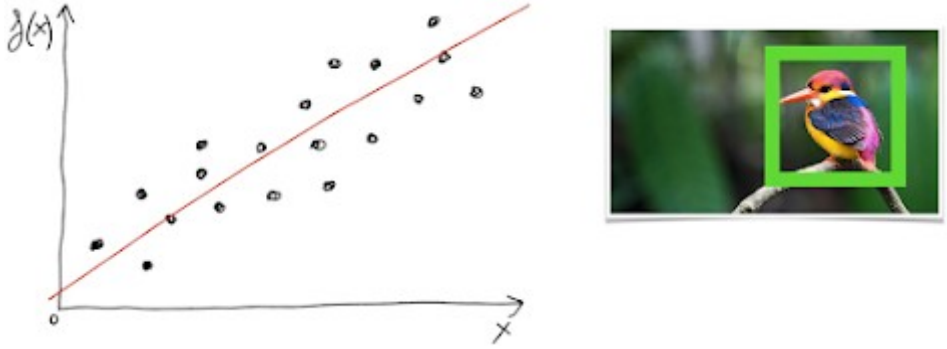


In **classification** (supervised learning) the task is to find a function $f \in Y^X$ assigning each input $x \in X$ a **discrete** label

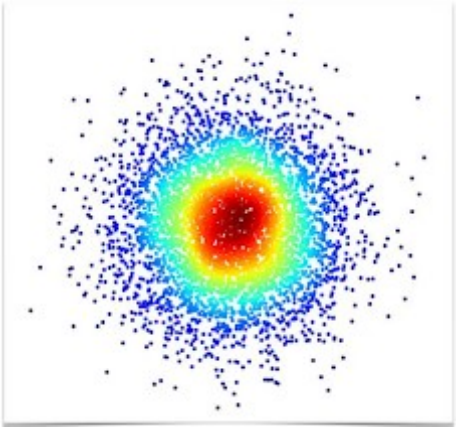
$$f(x) \in Y = \{c_1, \ldots, c_k\}$$



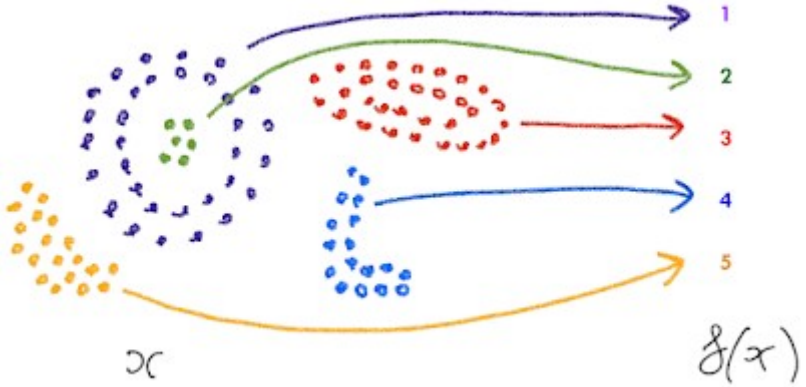
In **regression** (supervised learning) the task is to find a function $f(x) \in Y$ assigning each input $x \in X$ a **continuous** label



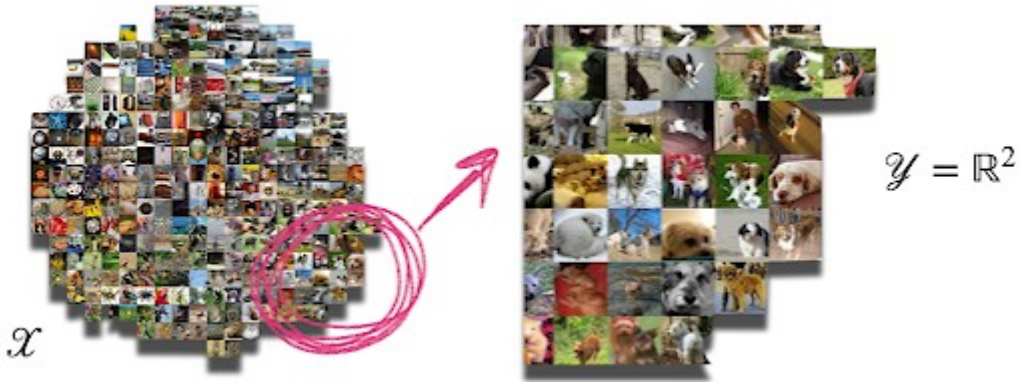
In **density estimation** (unsupervised learning) the task is to find a probability distribution $f \in \Delta(X)$ that fits the data $x \in X$ (Δ indicates all the possible set of probabilities)



In **clustering** (unsupervised learning) the task is to find a function $f \in N^X$ that assigns each input $x \in X$ a cluster index $f(x) \in N$. All points mapped to the same index form a cluster.



In **dimensionality reduction** (unsupervised learning) the task is to find a function $f \in Y^X$ mapping each (high dimensional) input $x \in X$ to a lower dimensional embedding $f(x) \in Y$, where $dim(Y) < dim(X)$. Dimensionality reduction can be used to inspect a classification algorithm.



Data

Data means information about the problem to solve, and it is the unique information that I have at my disposal. Data is in the form of a probability distribution p_{data}.
In classification and regression (supervised learning) this distribution takes in account both input space and output space:

$$p_{data} \in \Delta(X \times Y)$$

In density estimation, clustering and dimensionality reduction (unsupervised learning) this distribution takes in account only the input space:

$$p_{data} \in \Delta(X)$$

Δ indicates all the possible data distributions.

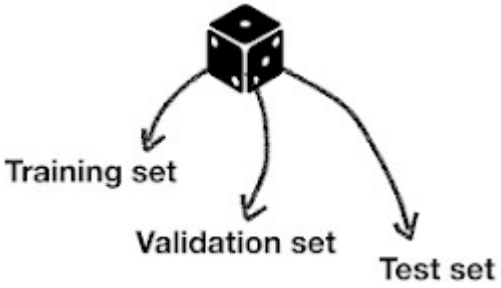


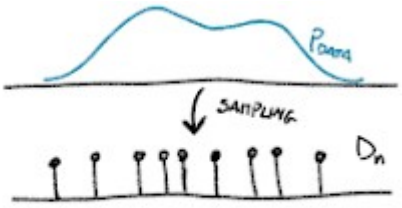
The data distribution is typically **unknown**, it is only represented by our training set. We can assume that the observations that we have about this probability distribution are obtained by **sampling** from the distribution.

The observations are divided into three sets:

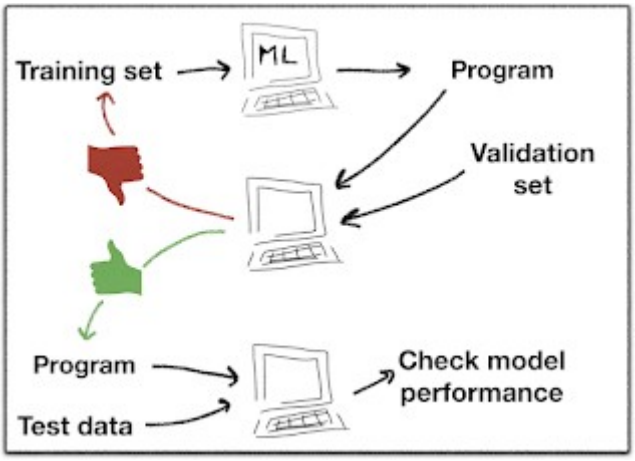
- **Training Set**: This is the data that we use to train our machine learning (ML) model. The training set is used to adjust the weights on the ML model, which is how it learns.
- **Validation Set**: After the model has been trained, we need to tune its hyperparameters to optimize its performance. This is where the validation set comes in. The validation set is used to prevent overfitting, which is when the model learns the training data too well and performs poorly on unseen data.
- **Test Set**: Once the model has been trained and validated, we need to test its performance on unseen data. This is done using the test set. The test set gives us an unbiased estimate of the model's performance when it's deployed in the real world.

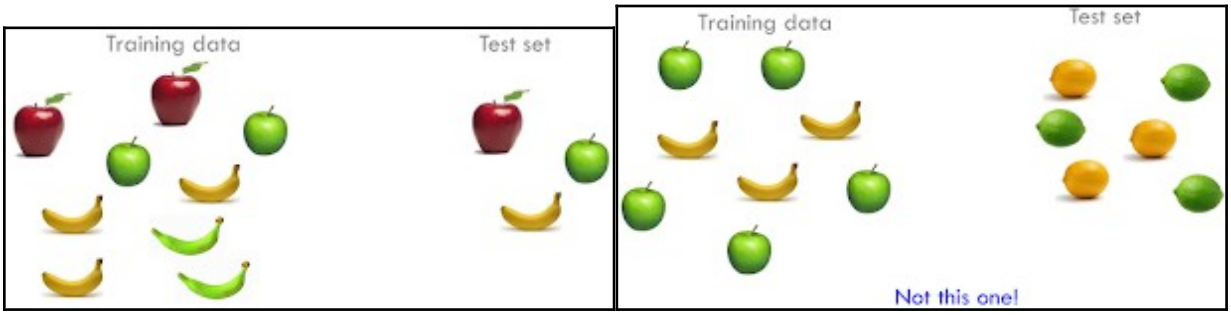
All three sets are assumed to be drawn from the same distribution, which is a necessary assumption for the system to function effectively.



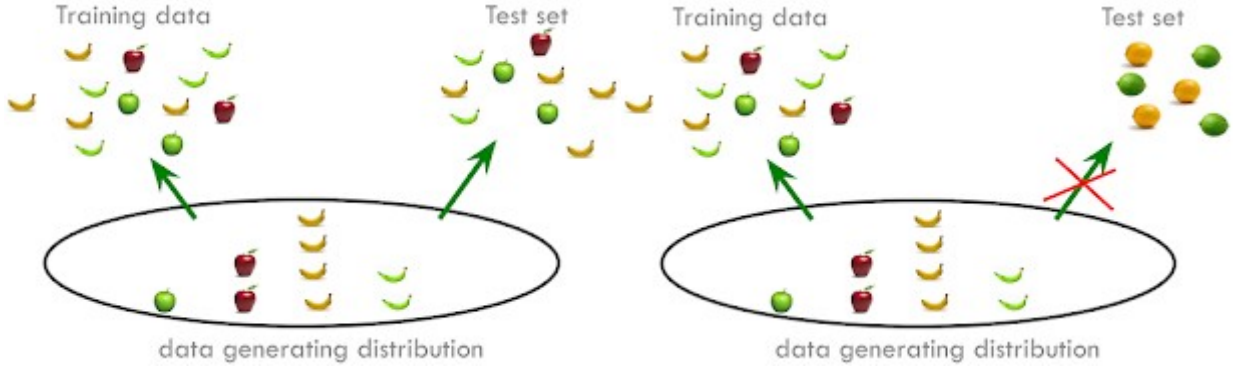


Let’s see the differences in the usage of training set, validation set and test set in the Machine Learning pipeline:





We use a **probabilistic model** of learning. There is some probability distribution over example/label pairs called the **data generating distribution**.
Both the training data and the test set are generated based on this distribution.

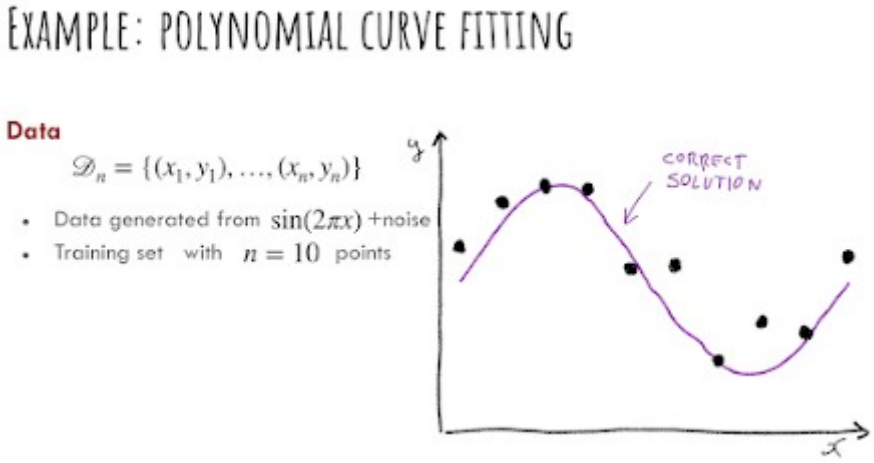


A crucial aspect of the design of the training set is that the failure of a machine learning algorithm is frequently attributable to poorly chosen training samples.
The problem arises when we inadvertently introduce undesired correlations, leading the algorithm to draw incorrect conclusions.

"The US Army trained a program to differentiate American tanks from Russian tanks with 100% accuracy. Only later did analysts realized that the American tanks had been photographed on a sunny day and the Russian tanks had been photographed on a cloudy day. The computer had learned to detect brightness." [probably a legend]

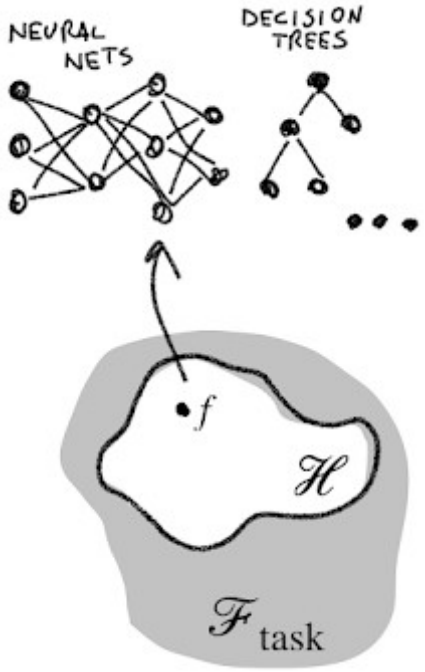


Let's introduce an example that we will use later. We're going to see a regression task.
The training set is made of pairs (input and output), there is only one feature (the x-axis) and the output (the labels) are on the y-axis.

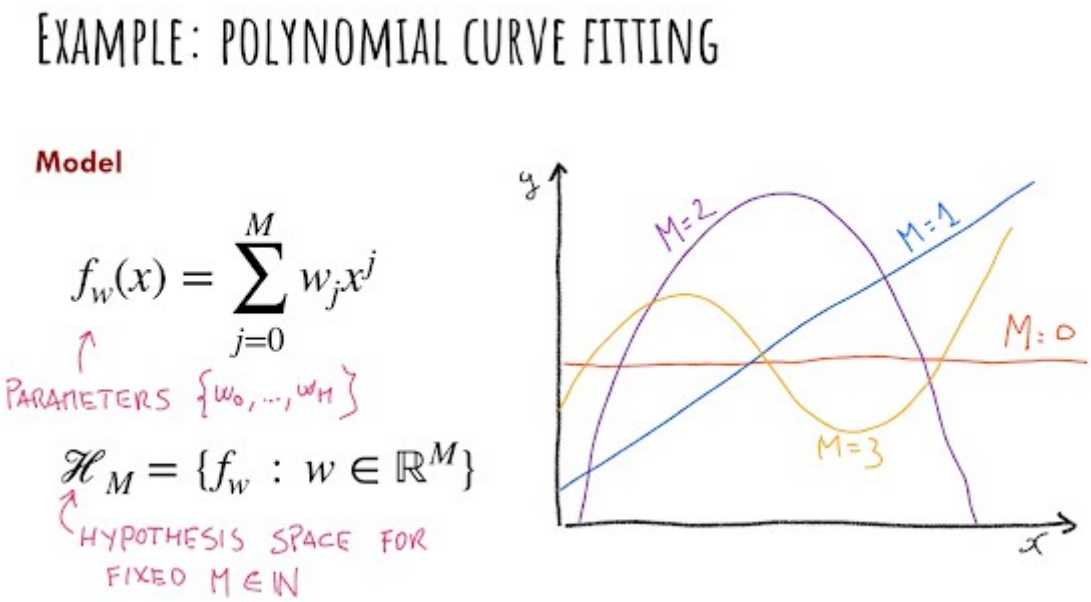


The data was generated by sinusoidal function (plus some noise)

Model and Hypothesis space
Once I have taken care of my data, I move to step 2: the (learning of) model.
A **model** is like a "program" to solve the problem. It is the implementation of a (mapping) function $f \in F_{task}$ that can be tractably computed.
A set of models forms an **hypothesis space** $H \subset F_{task}$, this is a subset of all the possible mappings to which I restrict my attention.
The learning algorithm seeks a solution within the hypothesis space



Retaking in consideration the previous example of Polynomial Curve Fitting, I can take a model that is a polynomial function (my choice).

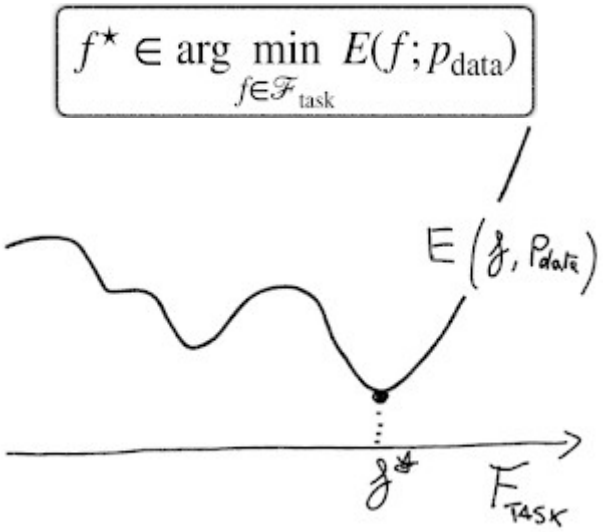


So I restrict my hypothesis space to all the polynomial functions of a certain degree M.
That means that computing my model means computing the values of the parameters w in that function up there.
This is done because I can't search in all the possible functions of F_{task}. In conclusion, I always need to restrict my space into an hypothesis space which I can compute.

Objective

In machine learning we are interested in minimizing the so-called **generalization error function**.
Formally, we have a (generalization) error function $E(f\,;p_{data})$ that we want to minimize. This error function determines how well a solution $f \in F_{task}$ fits some given data p_{data}.

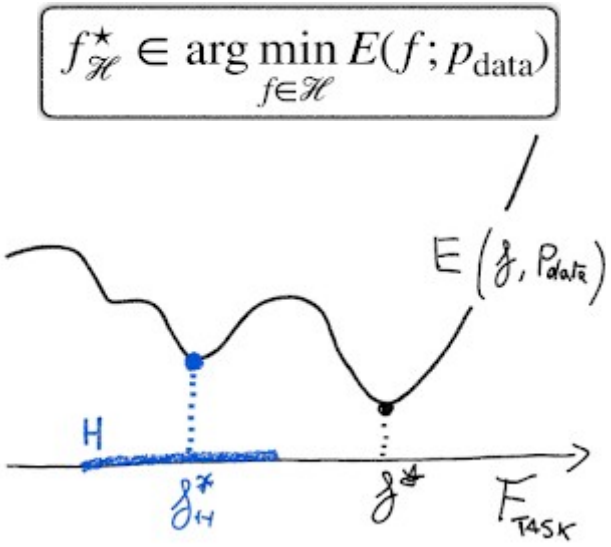
So the **ideal target** is searched in all possible mappings F_{task}



But, despite this being optimal in theory, in practice this is undoable because we have a too large search space. So this is ideal but impractical.

We need to restrict the focus on finding functions that can be implemented and evaluated in a tractable way. As we've already seen before we define a **model hypothesis space** $H \subset F_{task}$ and seek a solution within that space.

So the **feasible target** is searched in the hypothesis space H



So I'm not gonna search the function in all the possible set of functions but I'm gonna search only in the hypothesis space. This results in a local optimization and not in a global one.
In the example of the hypothesis space of polynomial function we will obtain the best polynomial, but not the best function.

But there's another issue, we don't have the exact probability distribution p_{data}, so we can't apply the function to the real p_{data}.
I can have access only to a proxy of the real distribution, which is the distribution of the training set. So we work on a data sample $D_n = \{z_1, \dots, z_n\}$ where

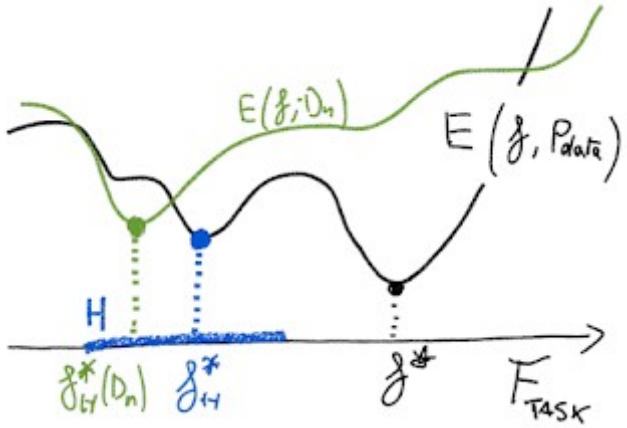
$$z_i = (x_i, y_i) \in X \times Y$$

$$z_i \sim p_{data}$$

So the **actual target** is searched using the distribution of the training set

$$f^\star_{\mathcal{H}}(\mathcal{D}_n) \in \arg\min_{f \in \mathcal{H}} E(f; \mathcal{D}_n)$$

TRAINING ERROR



If the training set is large enough, the green curve will be very similar to the black one.
In conclusion, in Machine Learning, unfortunately, we do not minimize the ideal target (the generalization error), but we minimize the empirical/training error.

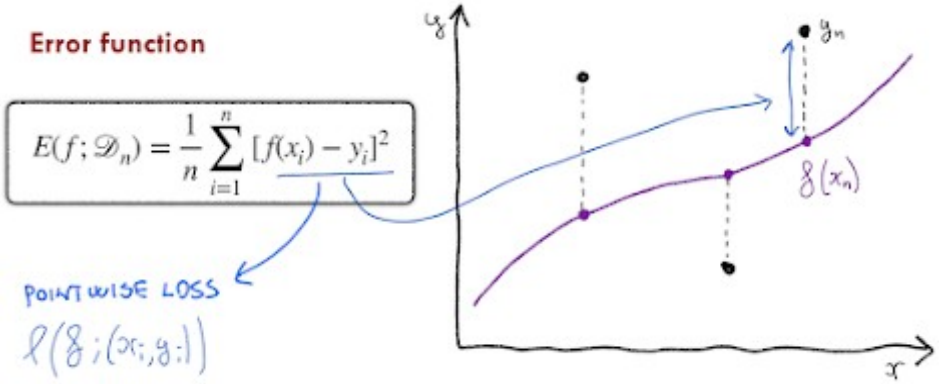
Typically the generalization and training **error functions** can be written in terms of a **pointwise loss** $l(f; z)$ measuring the error incurred by f on the training example z.

$$E(f; p_{\text{data}}) = \mathbb{E}_{z \sim p_{\text{data}}}[\ell(f; z)]$$

$$E(f; \mathcal{D}_n) = \frac{1}{n} \sum_{i=1}^{n} \ell(f; z_i)$$

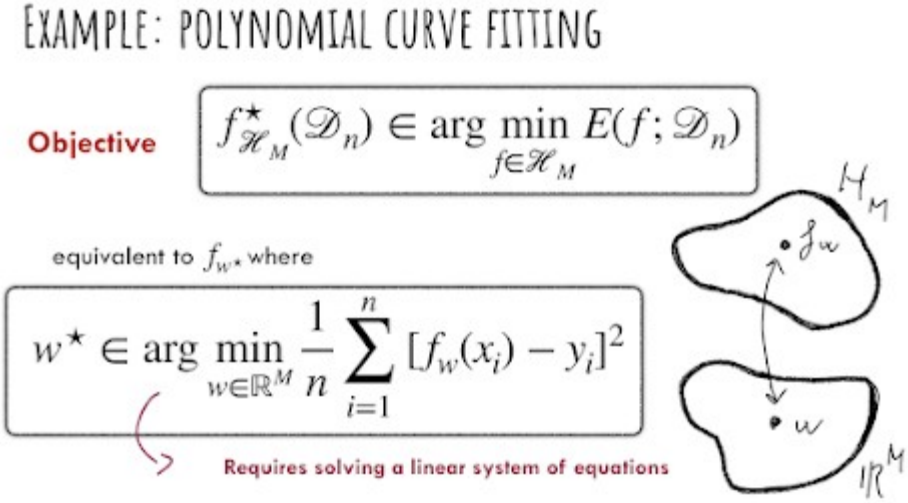
l is a function which measure the discrepancy between your prediction f and your ground truth that is D.

In the example before the **error function** could be calculated in this way

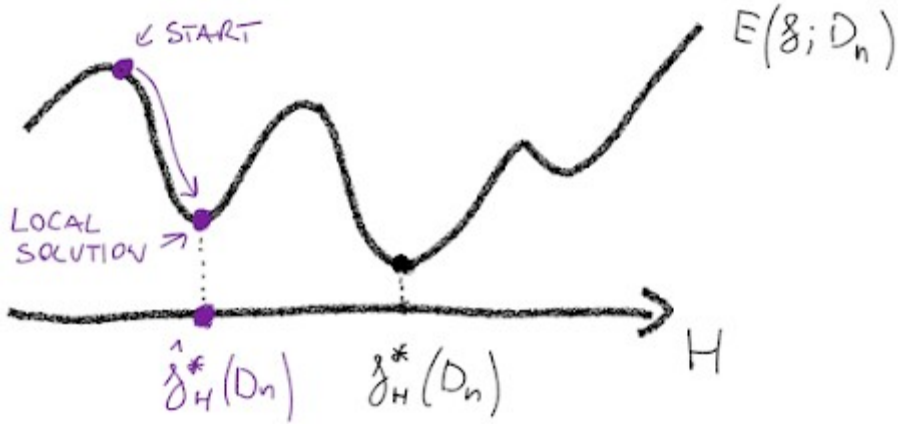


So in this case the error function (the green line) is computed by the difference between the prediction $f(x_i)$ and the ground truth y_i, and you repeat this n times because you have n samples.
This is a practical implementation of the empirical error.

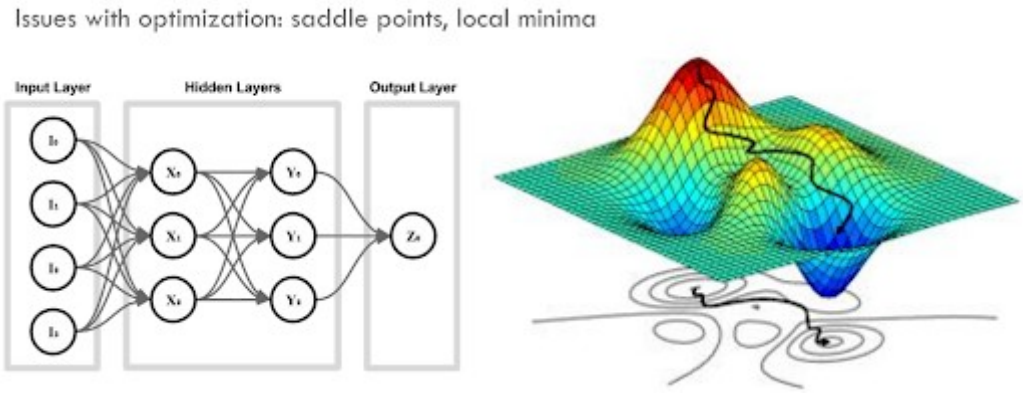
So putting things together, for our example of polynomial curve fitting we obtain this:



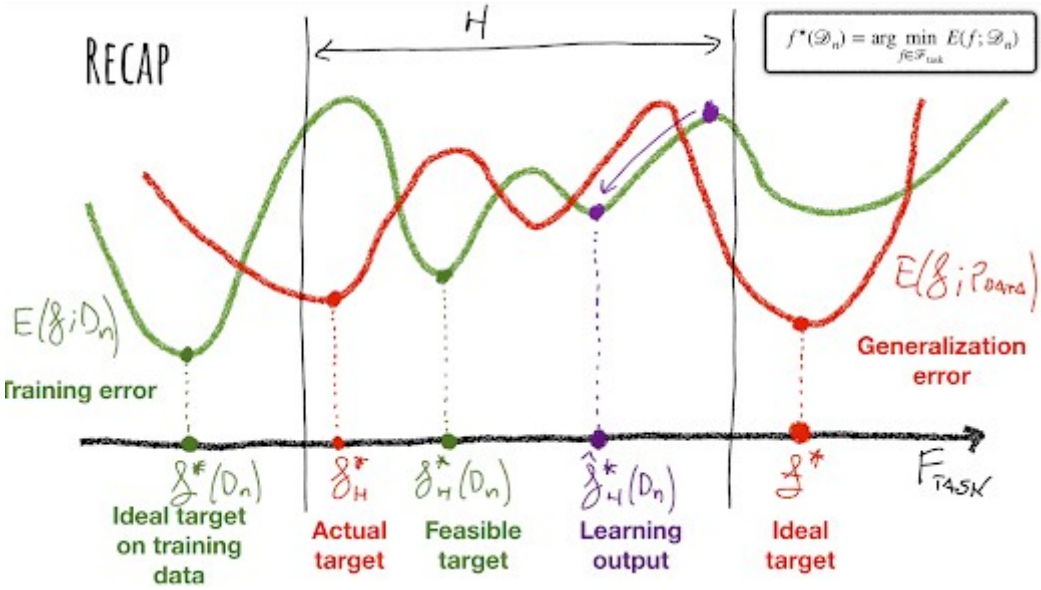
Another additional thing that complicates the process is that even if in theory it is possible to end up to something, there will be some obstacles at the optimization level that might make the process end up in a different result (local solution but not global).



Just to give an example of this with neural networks optimization.



Let's make a small recap:



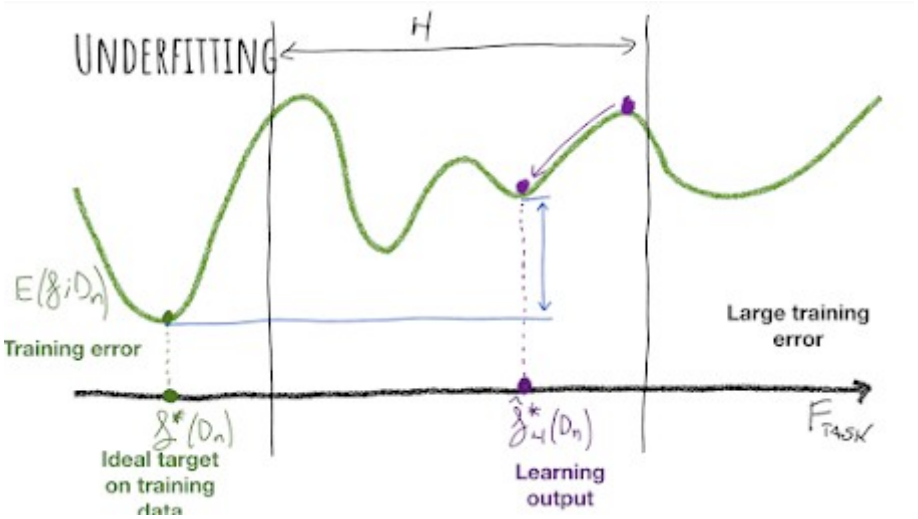
So:

- In Machine Learning I'm interested to minimizing the generalization error, finding the function f^* which is the optimal mapping
- However I can't search everywhere, so I'm searching only into the hypothesis space H
- But I don't have access to the real data distribution, so I must use the training set distribution D_n, therefore I'm not operating on the red curve, but only on the green one
- Inside the green one I have to take in account that if the choice of my hypothesis space is of complicated functions I may also have optimization issues.

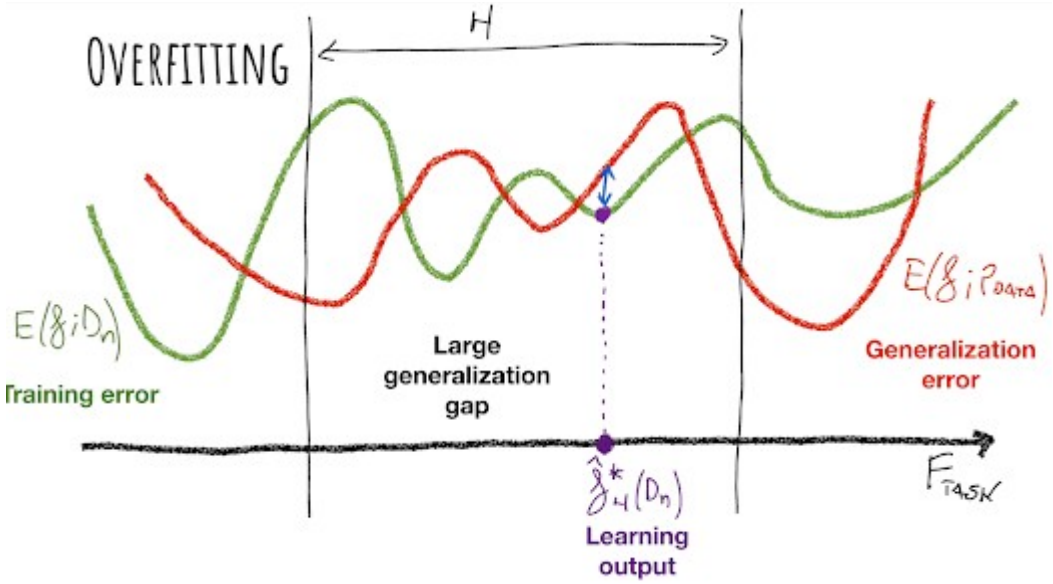
Let's now talk about two very important concepts:

- Underfitting
- Overfitting

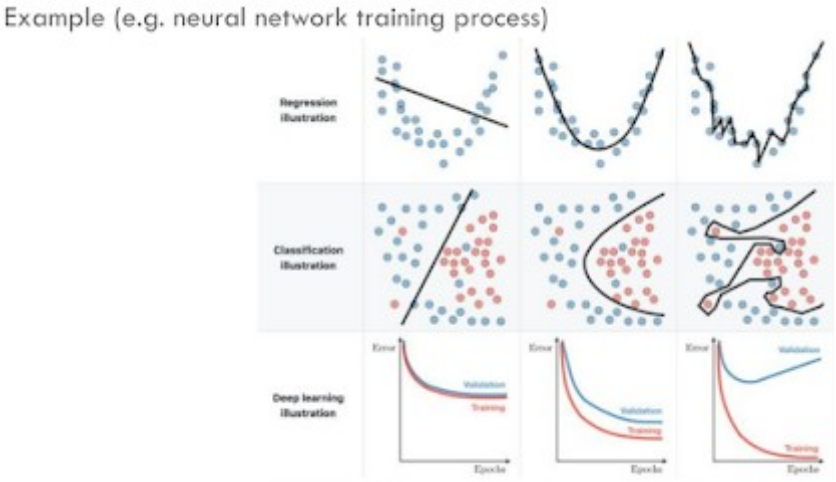
In **Underfitting** the problem is that you are not able to minimize the error at training time as you would like to do. That means that you are not able to train the model properly and it won't work much. So the problem is that we have a large **training (empirical) error**



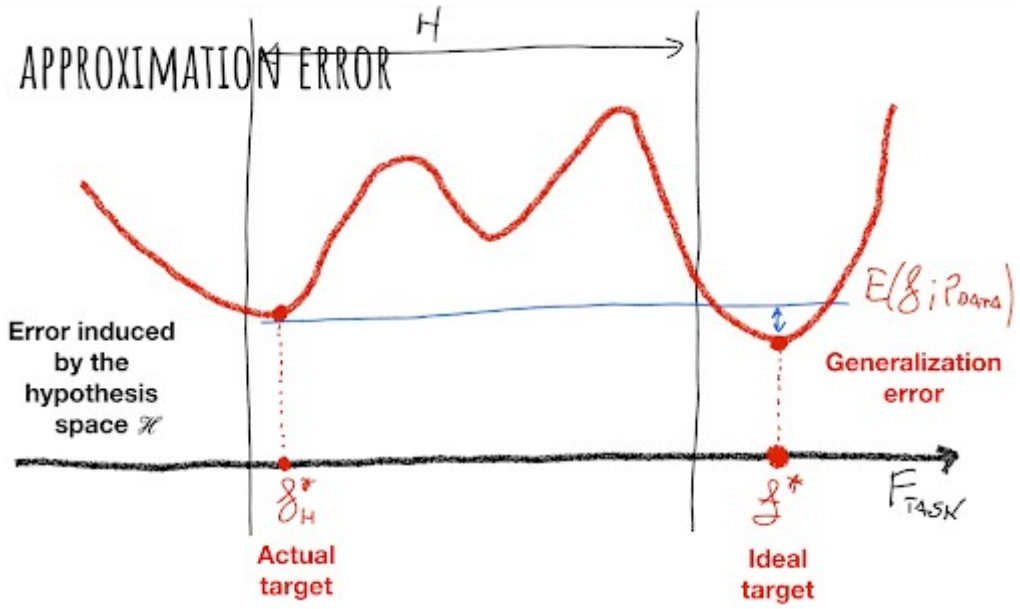
In **Overfitting** there is a large generalization gap between what I get in the training and what I get in the test. In practice the model works very well on the training data, but it isn't able to generalize to other data. So the problem is that we have a large **generalization error**.



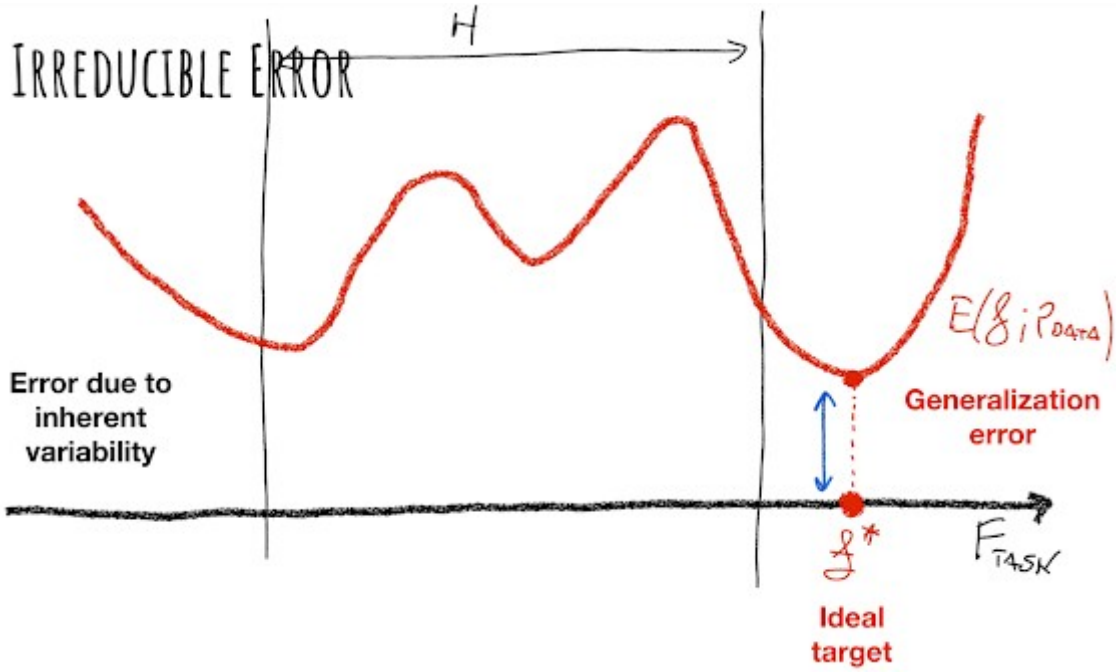
Let’s see in practice why these situations aren’t good (left underfitting, center ok, right overfitting):



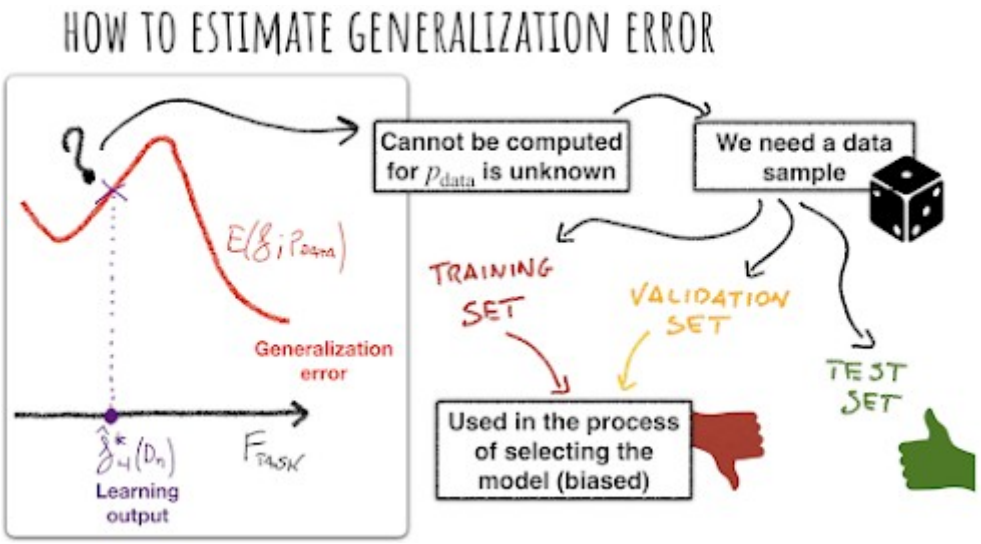
There are other things to mention about errors.
Obviously the choice of the hypothesis space has an impact on the solution I’m gonna get. This is called **approximation error**.



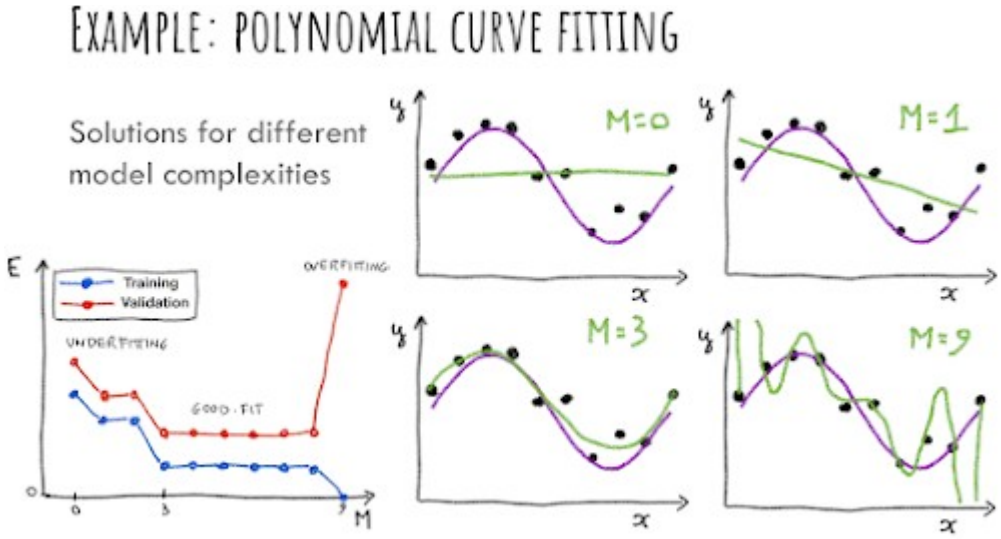
Another error is the **irreducible error** which is an error that we cannot reduce by choosing a better model. It is due to randomness or natural variability in a system (like noise). These are deterministic factors that you don't observe, but which could be interpreted as inherent variability.



Let’s see a recap:



Taking into consideration the example of polynomial curve fitting. We can have different models with different complexities (given by the degree of the polynomial). The choice of this complexity has an impact on the underfitting and overfitting.



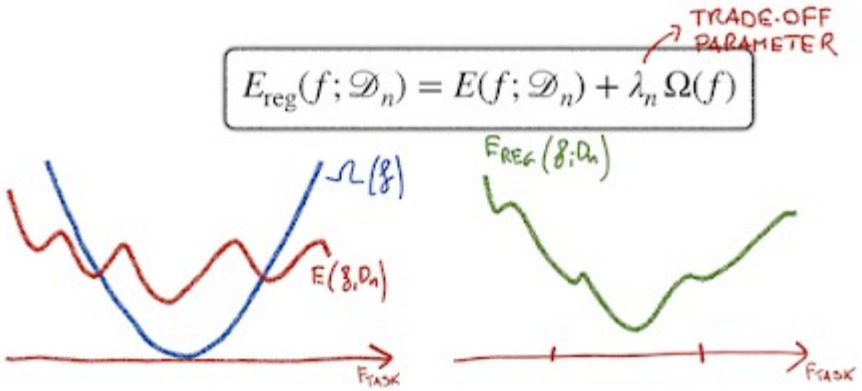
So a low degree polynomial will imply an underfitting situation and a too high degree polynomial will imply an overfitting situation.

There are some tips to improve generalization:

- Avoid attaining the minimum on training error (due to the risk of overfitting)
- Reduce model capacity
- Change the objective with a regularization term (we will see this later)
- Inject noise in the learning algorithm
- Stop the learning algorithm before convergence

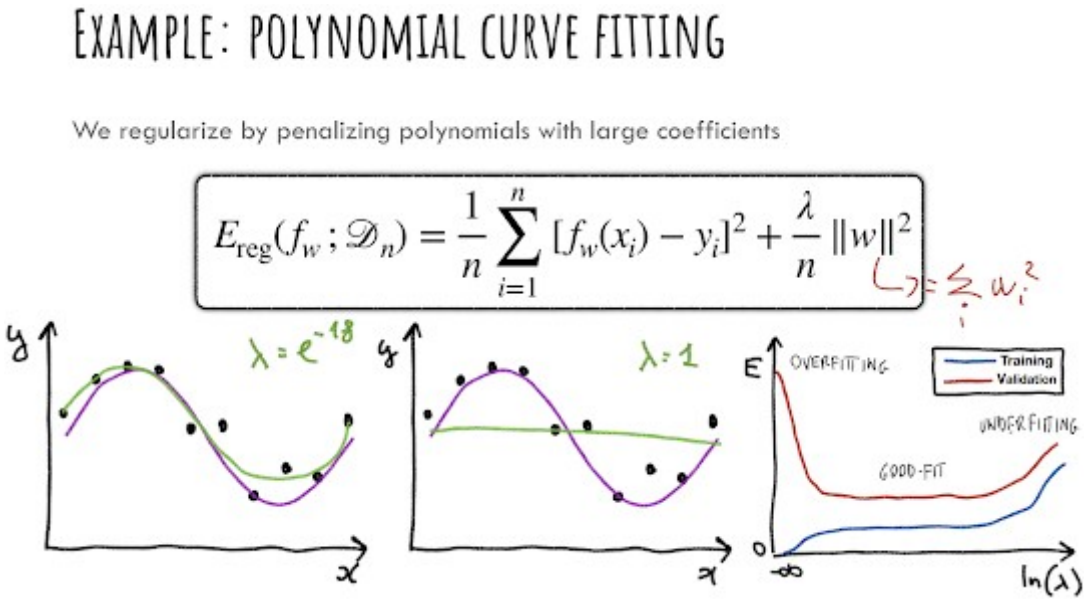
Let's talk about the **regularization term**.

In practice this consists into modifying the training error function with a term that typically penalizes complex solutions



The regularization function is only a function of f and not of the training data. In the case of the polynomial this means that it will be a function of the weights.
λ is a parameter, and it is a scalar that I set a priori. λ has the role to weigh the influence of the regularization term. It is also called **trade-off parameter**.
This is done in order to avoid overfitting, because complex solutions favor overfitting.
Lambda is chosen based on the validation set, in the sense that you repeat the training multiple times with different lambas and you pick the lambda which gives the minimum error.

Here's an example:

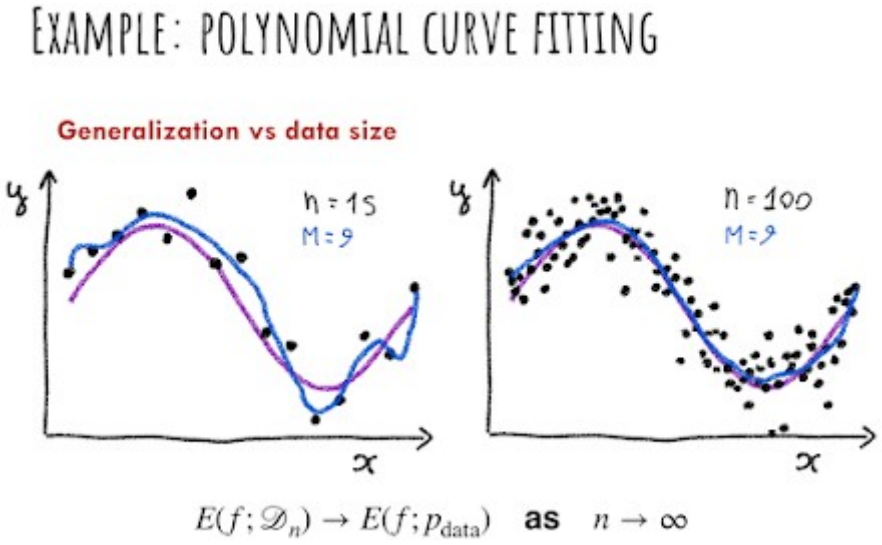


In this case, the regularization term is the square of the norm of the polynomial's weights.

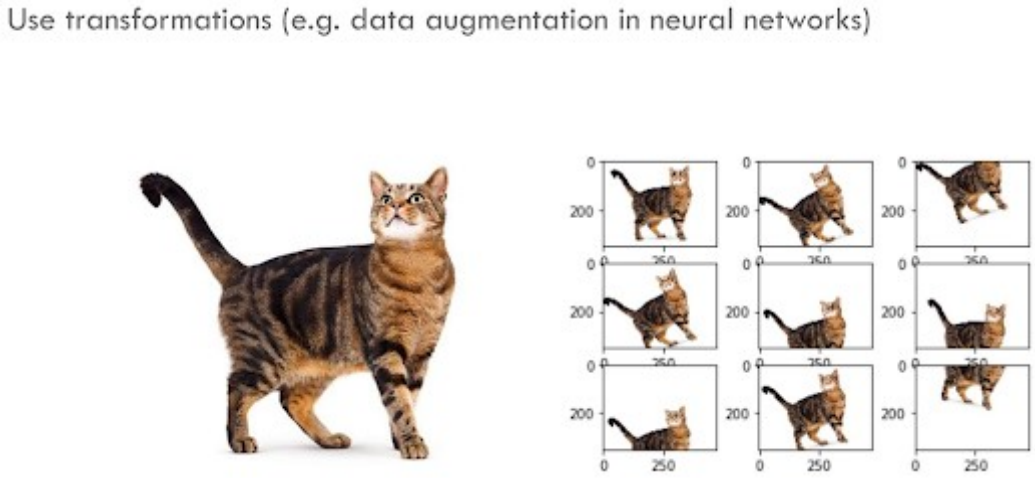
Other tips which improve generalization:

- Increase the amount of data (sometimes it is very expensive or it isn't possible)
- Adding more training samples
- Augmenting the training set with transformations (or with new artificial data)
- Combine predictions from multiple, decorrelated models (ensembling)

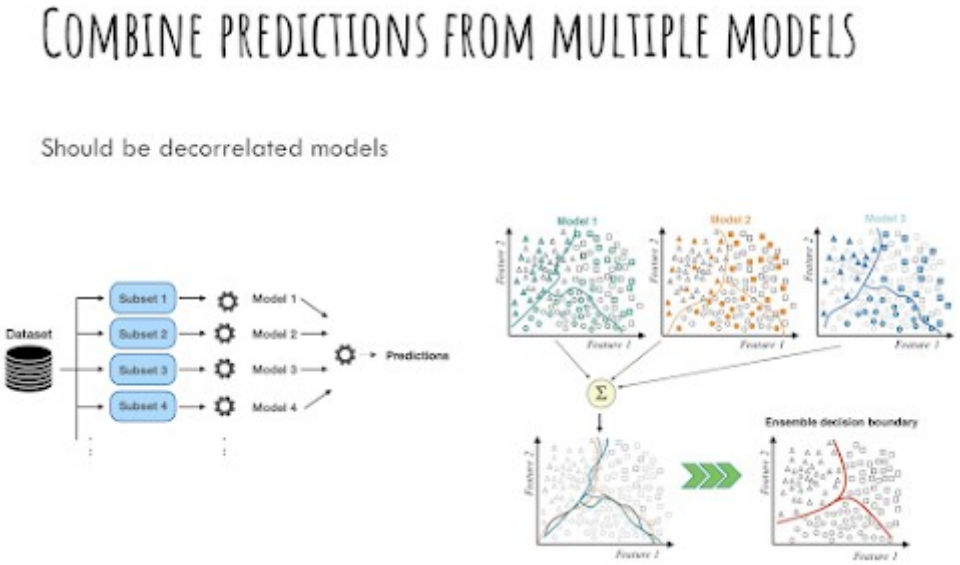
In fact, with more data the empirical error approaches the generalization error:



Due to the fact that sometimes it isn't possible to increase the training data, it is useful to augment the training data with transformations of existing data. In this example the transformation used is random crop:

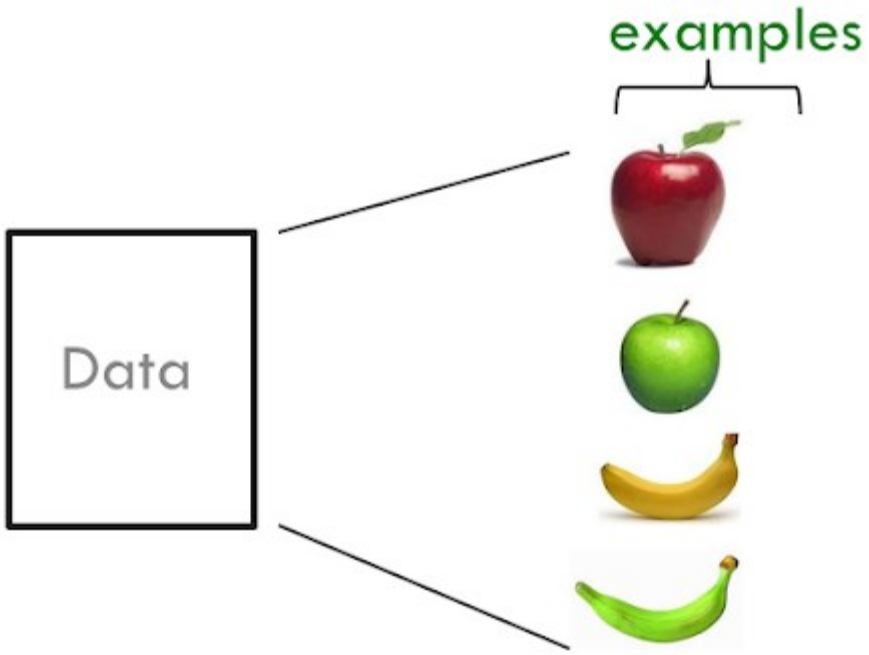


Another method to combat overfitting is by combining multiple models. For instance, you can extract various subsets from your dataset and train different models with these subsets. Then, you can amalgamate all the models to generate predictions.
This approach aligns well with the principles of **ensemble learning**. While the models may not be highly accurate individually, their combination results in a stronger model.

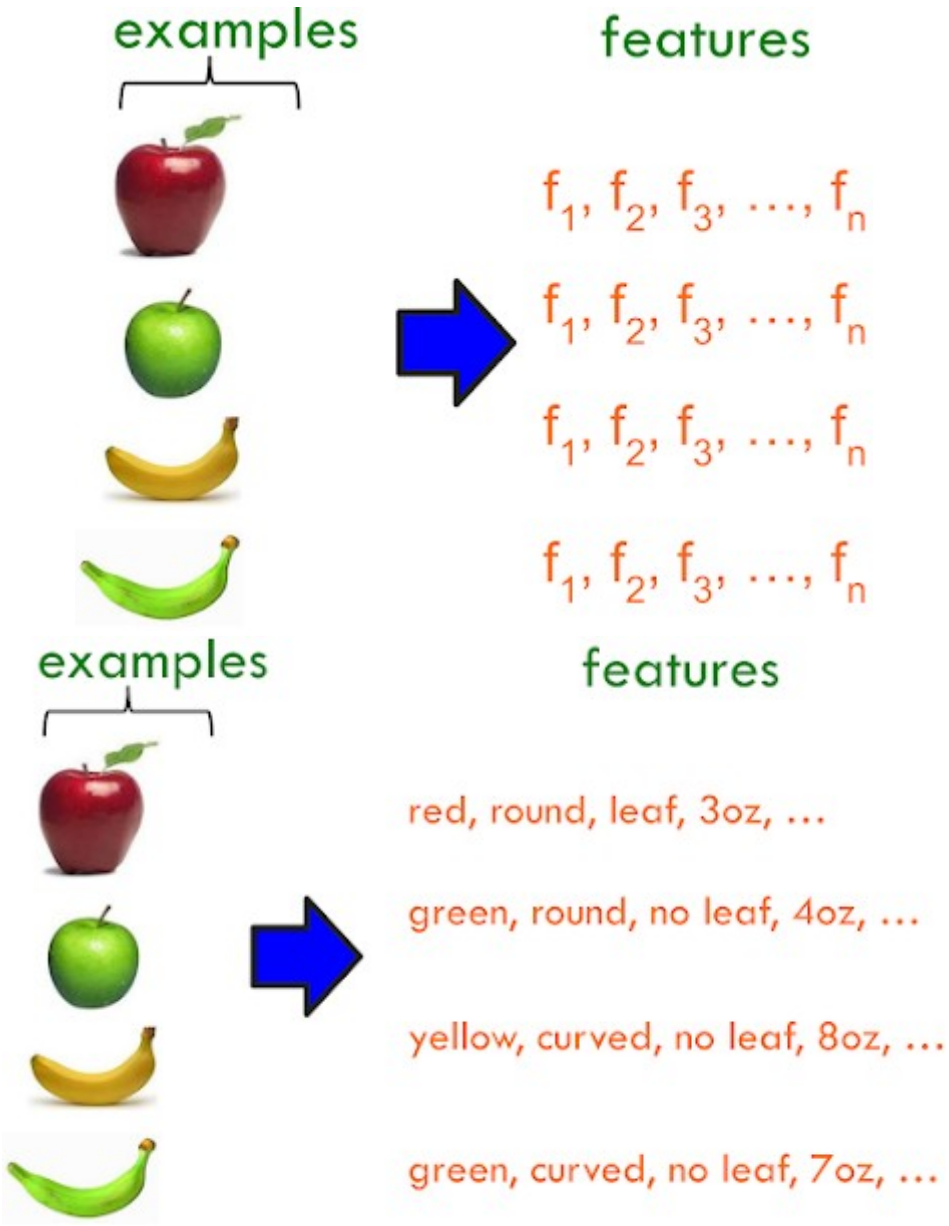


k-Nearest neighbor

Let's do a brief recap.
Let's reconsider the example of the fruit:



This data is typically represented in the form of feature vectors.



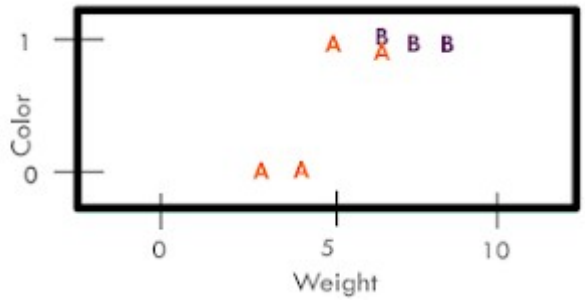
It's important to remember that features are how our algorithms actually 'view' the data; they represent the questions we can ask about the examples.

To delve into the intuition behind the k-Nearest Neighbor, we can consider visualizing the data in some way.
For instance, let's take this dataset with its corresponding labels:

Weight	Color	Label
4	Red	Apple
5	Yellow	Apple
6	Yellow	Banana
3	Red	Apple
7	Yellow	Banana
8	Yellow	Banana
6	Yellow	Apple

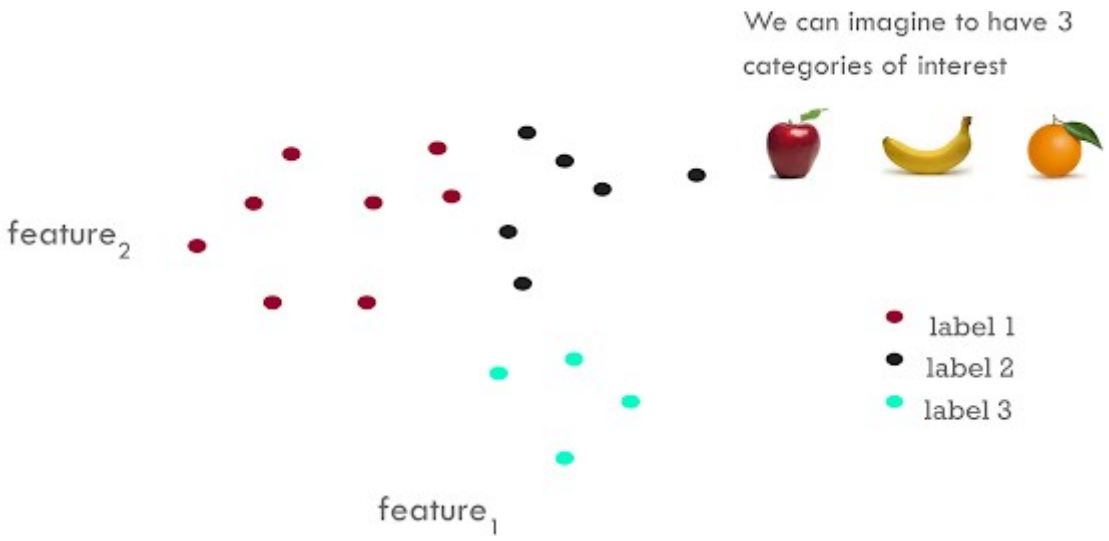
We can convert the features into numerical values (weight already is, but color is not) to visualize the data in an *n*-dimensional space, where *n* represents the number of features. Examples can then be depicted as points in this *n*-dimensional space:

Weight	Color	Label
4	0	Apple
5	1	Apple
6	1	Banana
3	0	Apple
7	1	Banana
8	1	Banana
6	1	Apple



In this manner, I can analyze the positions of these points in the *n*-dimensional space to establish an initial approximation of a classifier.
Generally, I have various points (which represent the examples) in the feature space.

Considering the classification problem with multiple classes: Apple, Banana, and Orange, let's assume we're dealing with a 2-dimensional space due to having 2 features.
Here's the training set with points colored according to their labels:

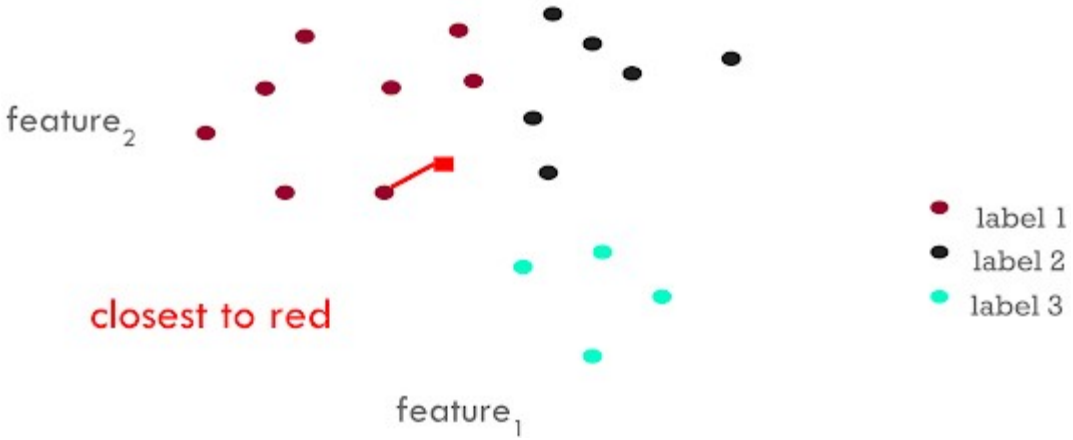


We can then include the point of a test example:



And without conducting any training, we can simply identify the neighbors of this point to determine its label.

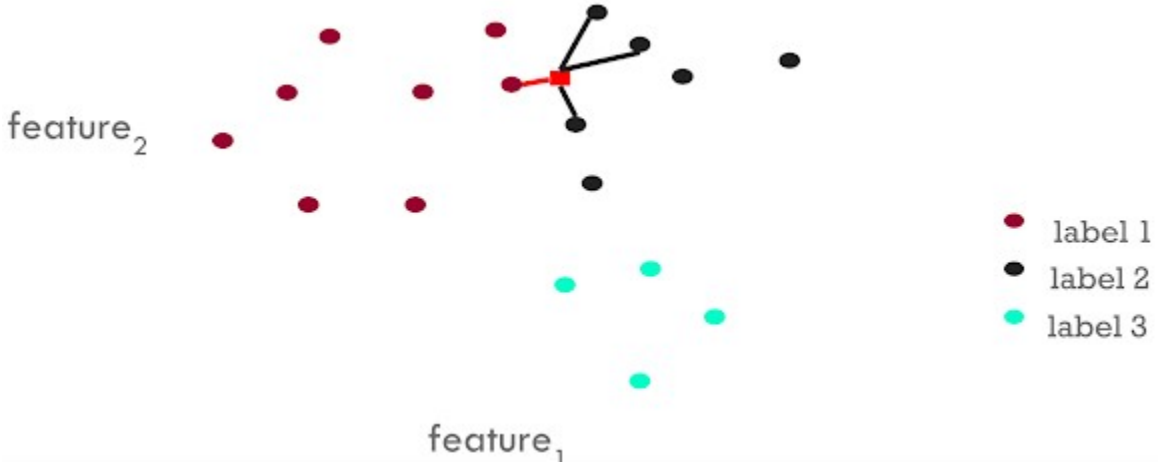
The simplest approach is to select the label of the nearest neighbor:



So, to classify an example d, we can simply assign d the label of the closest example to d in the training set.

Let's make an observation: if we had chosen different features to represent the examples, the point could be closest to an example with a different label.

However, this classification method poses a problem. In fact, in can happen the test example is closest to a red point, but most of the subsequent closest points are black:



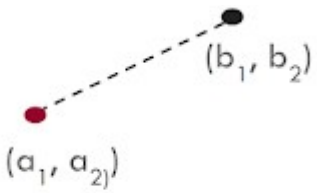
This implies that relying solely on one neighbor may not be the optimal choice, as that point could be an **outlier**. Therefore, it's advisable to consider multiple neighbors.

So, the simplest algorithm that can be used for classification **without requiring any training** is called **k-Nearest Neighbor** (k-NN). To classify an example d, this algorithm finds the k nearest neighbors of d and assigns d the **majority label** among these k nearest neighbors.
Compared to the previous method, this algorithm is more robust to noisy data by considering multiple neighbors, thereby reducing the impact of outliers.
One way to further improve this algorithm is to weight the distances of the neighbors, making the closest ones more influential in determining the label.

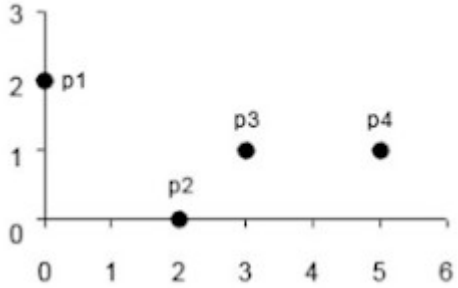
k is a hyperparameter, and like all hyperparameters, its value needs to be set using a validation set.

In the previous examples, we have assumed that "nearest" is determined in terms of Euclidean distance, but it is possible to make more sophisticated choices.

In two dimensions, the Euclidean distance is computed as follows:



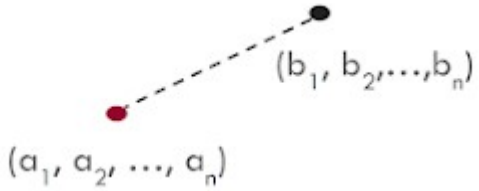
$$D(a,b) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}$$



point	x	y
p1	0	2
p2	2	0
p3	3	1
p4	5	1

	p1	p2	p3	p4
p1	0	2.828	3.162	5.099
p2	2.828	0	1.414	3.162
p3	3.162	1.414	0	2
p4	5.099	3.162	2	0

In *n* dimensions, the Euclidean distance is computed as follows:



$$D(a,b) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \ldots + (a_n - b_n)^2}$$

The point is that if I have several features, the value of the Euclidean distance may become less meaningful.

So, Euclidean distance is a common choice for computing distance in k-Nearest Neighbor. However, this approach only makes sense when the features are all comparable (e.g., each is a variable measured in the same units). For instance, if the measurements are different, such as length and weight, the Euclidean distance may not be meaningful.

To address this, let's introduce the first type of preprocessing on the data: **standardization**. When features are not comparable, we can standardize them by dividing each feature by its corresponding standard deviation. This ensures that all features are equally important.

In practice, **standardization or Z-score normalization** involves rescaling the data so that the mean is 0 and the standard deviation is 1. The formula to achieve this is:

$$x_{norm} = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation of the data.

This process is applied to each feature, replacing the original value with its standardized value.

Another method to "balance" the features is by computing **Min-Max scaling**, which involves scaling the data to a fixed range, typically between 0 and 1:

$$x_{morm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

where x_{min} and x_{max} are the minimum and maximum values of the feature, respectively.

Absolutely, measuring distance or similarity is a domain-specific problem, and there are numerous variations:

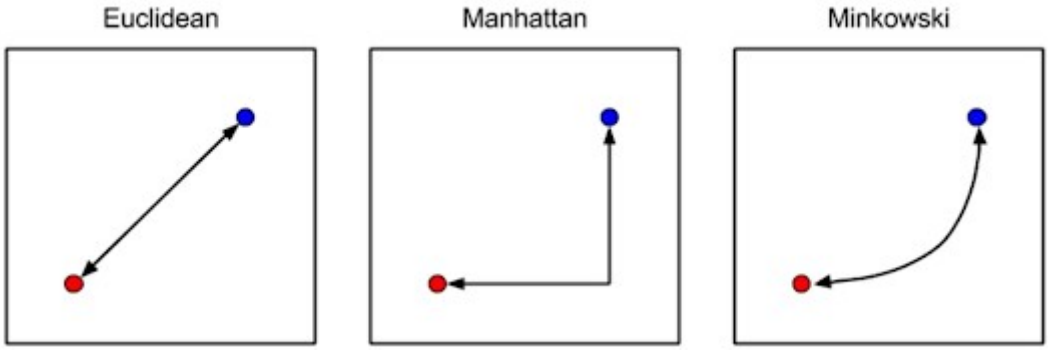
- **Similarity**: It's a numerical measure of how alike two data objects are, typically ranging between 0 and 1. Higher values indicate greater similarity, implying that objects are more alike.
- **Distance**: It's a numerical measure of how different two data objects are, usually with a lower value indicating greater similarity. The minimum dissimilarity is often 0, while the upper limit can vary depending on the context.

There is a family of distances called **Minkowski Distance**, which is a generalization of the Euclidean Distance. Suppose I have two vectors a and b to compare, each with coordinates ranging from a_1 and b_1 to a_p and b_p (where p is the number of dimensions).
The Minkowski Distance between a and b is defined as:

$$D(a, b) = \sqrt[r]{\sum_{k=1}^{p} |a_k - b_k|^r}$$

where r is a parameter:

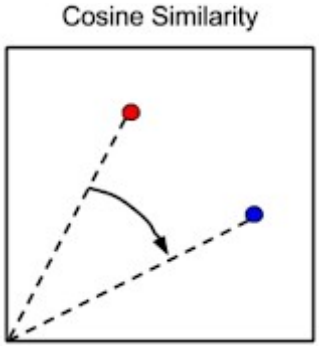
- $r = 1$: City block (Manhattan, L1 norm) distance, which is the sum of the absolute differences.
- $r = 2$: Euclidean distance, which uses the Pythagorean theorem.
- $r \to \infty$: "Supremum" (L_∞ norm) distance, which is the maximum difference between any component of the vectors.



It's becoming increasingly common in Machine Learning to utilize **cosine similarity**. Given two document vectors, cosine similarity is calculated as:

$$\cos(d_1, d_2) = \frac{(d_1 \cdot d_2)}{\|d_1\| \ \|d_2\|}$$

where $\cdot$ denotes the vector dot product, and $\|d\|$ represents the length of vector d.



The reason why cosine similarity is interesting is because it measures the difference between the angles of the vectors.

Example:

$d_1 = 3\ 2\ 0\ 5\ 0\ 0\ 0\ 2\ 0\ 0$
$d_2 = 1\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 0\ 2$

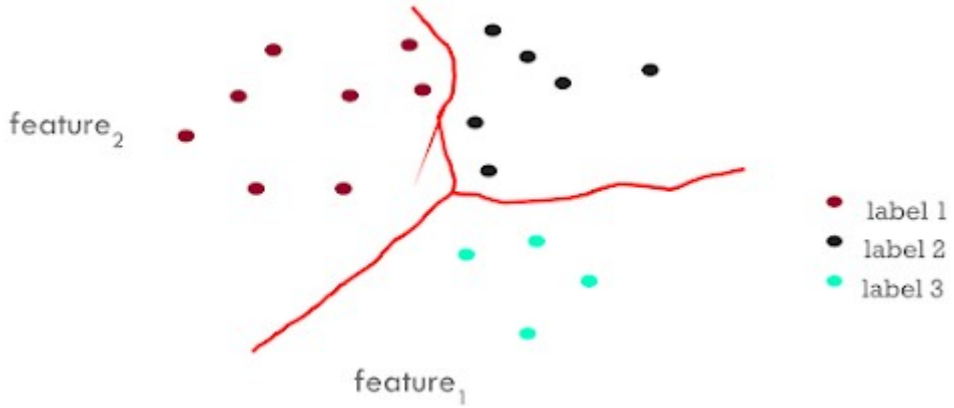
$d_1 \cdot d_2 = 3*1 + 2*0 + 0*0 + 5*0 + 0*0 + 0*0 + 0*0 + 2*1 + 0*0 + 0*2 = 5$
$||d_1|| = (3*3+2*2+0*0+5*5+0*0+0*0+0*0+2*2+0*0+0*0)\mathbf{0.5} = (42)\text{^}\mathbf{0.5} = 6.481$
$||d_1|| = (1*1+0*0+0*0+0*0+0*0+0*0+0*0+1*1+0*0+2*2)\mathbf{0.5} = (6)\text{^}\mathbf{0.5} = 2.245$
$\cos(d1, d2) = .3150$

Since this is a measure of similarity, to compute the distance, one can use $1 - cosine\ \ similarity$.

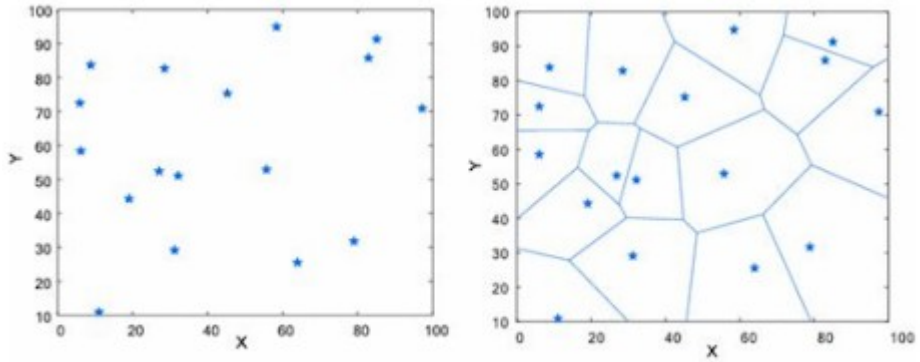
Cosine similarity is particularly useful for comparing histograms.

Decision Boundaries, Hyper-parameters

Decision boundaries are locations in the feature space where the classification of a point or example changes. In the case of k-NN:



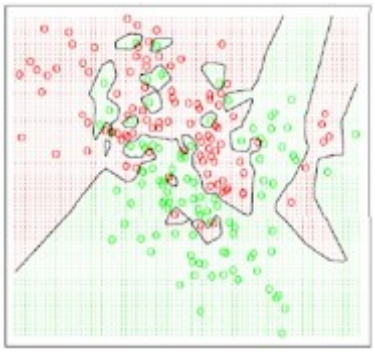
k-NN establishes **locally defined decision boundaries** between classes. These decision boundaries have a special structure known as the **Voronoi Diagram**.
The Voronoi Diagram describes the regions that are closest to any given point within a dataset. Each line segment in the Voronoi Diagram is equidistant between two points, delineating the boundaries between neighboring regions.
So given the initial distribution of points, you can create the Voronoi Diagram, which delineates regions of the feature space based on proximity to each point.



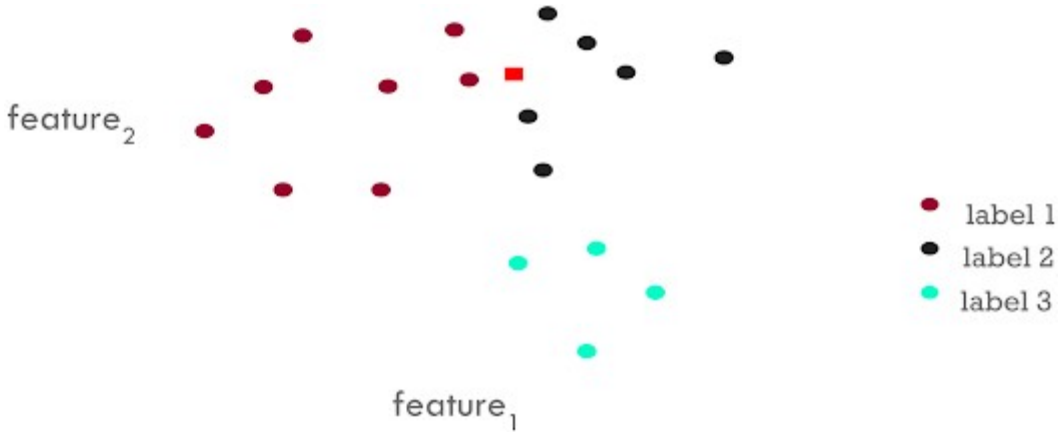
(a) Initial points distribution **(b)** Voronoi diagram by initial points

This approach can be naively applied to multiclass classification, as **no special modifications are required for the algorithm to handle more classes**. Each class would be represented by its own set of points, and the decision boundaries would be determined based on the Voronoi Diagram constructed from these points.

Therefore, as we've already mentioned, the k-NN algorithm does not explicitly compute decision boundaries; instead, they are implicitly induced.
The decision boundaries constitute a subset of the Voronoi diagram for the training data. Clearly, the greater the number of examples stored, the more intricate the decision boundaries can become.



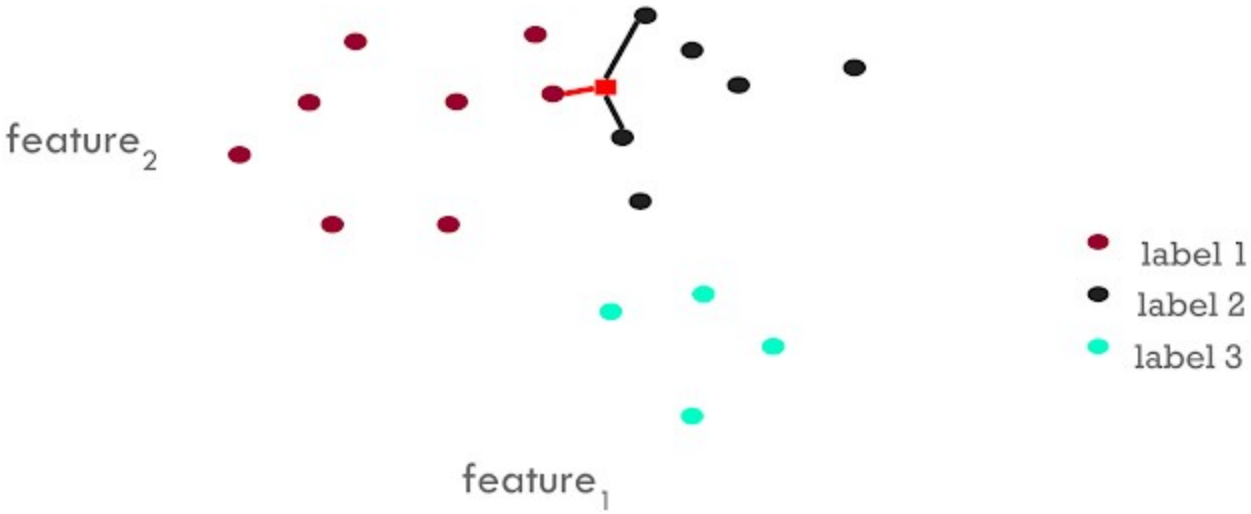
We've already seen that a critical issue with the k-NN algorithm is the choice of the value of k, as different values lead to different decisions.
Consider the following training examples:



If we set $k=1$, we will choose red as the label for the test example:

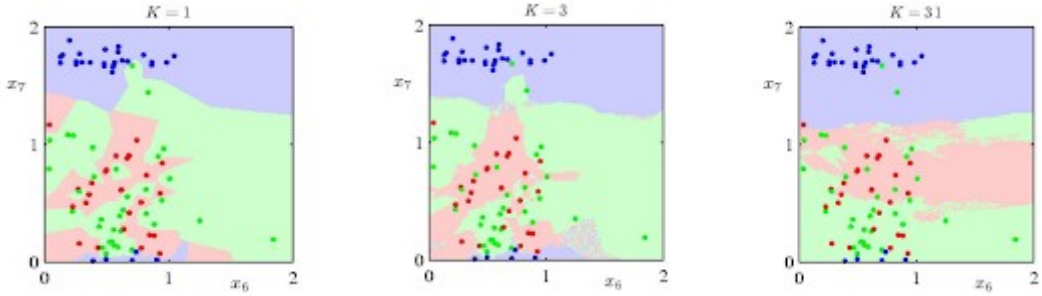


If we set $k=3$, we will choose black:



It's important to note that the choice of k needs to be sensible. For instance in our example, selecting $k = 20$ would always result in choosing the label of the class with the highest number of data points, which is nonsensical. Therefore, k should be a number much smaller than the cardinality of the training set.

Indeed, the choice of k has a significant impact on the decision boundaries:

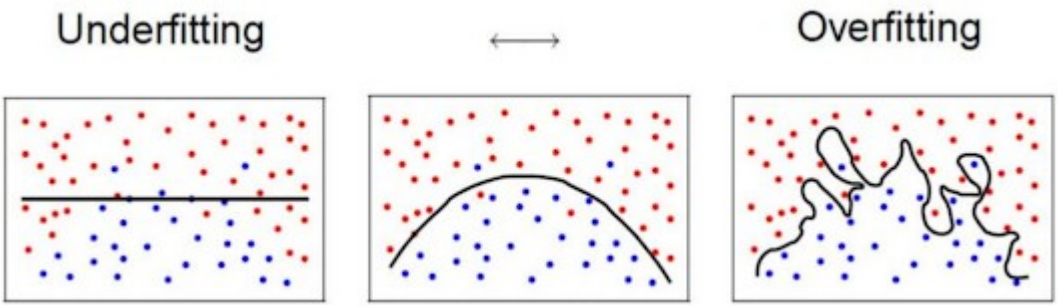


Larger values of k result in a smoother boundary effect. When $k = N$, where N is the total number of training examples, the model always predicts the majority class.
Moreover, the choice of k also affects the balance between underfitting and overfitting. The optimal k value is one that neither induces underfitting nor overfitting.

Let's review **underfitting and overfitting**, which are critical factors determining the performance of a machine learning algorithm:

- **Underfitting**:
  Occurs when the model fails to capture the underlying structure of the data.
  Characterized by a high training error, indicating that the model cannot achieve a sufficiently low error value on the training set.
- **Overfitting**:
  Occurs when the model captures noise or random fluctuations in the training data.
  Characterized by a significant gap between the training error and test error, indicating that the model performs well on the training set but fails to generalize to unseen data.

An illustration of underfitting and overfitting in classification is as follows:



In k-NN, some tips for choosing k include:

- Common heuristics:
    - choose an odd number to avoid ties (common choices are 3, 5, 7).
- Use a validation set.
- Employ cross-validation, which involves using multiple validation sets.
- Rule of thumb: $k < \sqrt{n}$, where n is the number of training examples.

A variation of the k-NN algorithm that can enhance performance is the **weighted k-NN**. In weighted k-NN, instead of treating all examples equally, you assign weights to the examples so that closer examples have more influence. This often involves using some form of exponential decay.

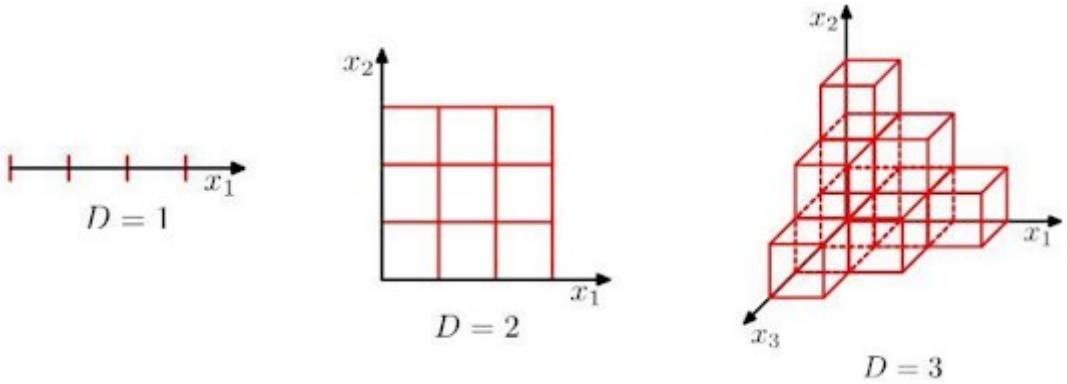
Another variation is to consider **all examples within a fixed distance** instead of the k nearest neighbors. This approach counts the majority from all examples within a specified distance.

Let's now explore the concepts of Lazy Learner and Eager Learner.

- **Lazy learning**: This approach simply stores training data (or performs limited processing) and operates when it is given a test example, with the processing happening during the inference phase.
  It requires less time during training but more time during prediction.
- **Eager learning** (e.g., Decision Trees, SVMs): In contrast to lazy learning, eager learning constructs a classification model using the entire training set before receiving new test data to classify.
  It spends more time during training but less time during prediction.

k-NN belongs to the class of lazy learning algorithms.

Another important concept we haven't discussed yet is the **curse of dimensionality**.
The curse of dimensionality refers to the phenomenon where, in high-dimensional spaces, almost all points are far away from each other. This is attributed to the exponential growth of the data space with the number of dimensions. Consequently, maintaining the same density requires the data set size to grow exponentially as well.



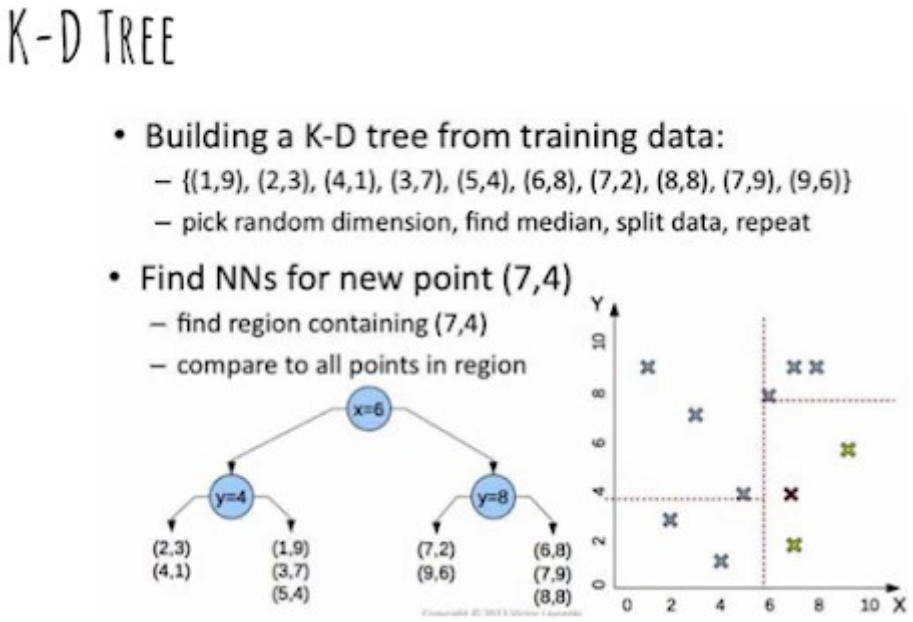
The curse of dimensionality poses a significant challenge for k-NN algorithms implemented with Euclidean distance. Relying solely on the closest points for class assignment can lead to decreased effectiveness. The success of k-NN is heavily dependent on having a **dense data set**.
While all machine learning algorithms benefit from dense datasets for accurate prediction, the curse of dimensionality is particularly severe for k-NN. This is because k-NN requires points to be close in **every single dimension**, meaning all features are equally important.
Some algorithms (which we haven't covered yet) can create models based on single dimensions and only require points to be close along that axis. These models are more robust to the curse of dimensionality.
However, k-NN doesn't work in this manner. It necessitates all points to be close along every axis in the data space.

In summary, the curse of dimensionality arises from the insufficient data available relative to the number of dimensions. As the number of dimensions increases, the data space expands, necessitating a corresponding increase in the amount of data to maintain density. Without significant increases in the size of the data set, k-NN loses its predictive power.

Let's delve into the computational cost of the k-NN algorithm. Since we haven't done any preprocessing, it operates linearly. Here's a breakdown:

- Compute distances for all N data points.
  Complexity of distance computation: $O(kN)$
- No additional space is needed.

However, it's very common to **pre-compute** certain **data structures** that enable quicker computations during the inference phase.
In the case of the k-NN algorithm, k-d trees are often utilized:



Another possible way to differentiate between algorithms is by categorizing them as **parametric or non-parametric models**.
In parametric models, there is a finite number of parameters. Examples include linear regression, logistic regression, and linear Support Vector Machines (SVMs).
Non-parametric models, on the other hand, may have a potentially infinite number of parameters. The complexity of these models grows with the size of the training data. For instance, k-NN, decision trees, and RBF kernel SVMs are considered non-parametric learning algorithms because the number of parameters increases with the size of the training set.

In summary, the k-NN algorithm is indeed a non-parametric model.
It's particularly suitable when instances can be naturally mapped to points in R^N, when there are few features per instance (e.g., less than 20), and when there's a large amount of training data available.
Advantages include its simplicity to program, swift training (no explicit training or optimization required), capability to learn complex target functions, and potential for high classification accuracy, often surpassing more intricate models.
However, drawbacks include its sluggishness during query time and susceptibility to being misled by irrelevant attributes.
Key considerations and challenges with this algorithm involve:

- Choosing the appropriate distance measure (often Euclidean).
- Determining the value of k.
- Dealing with the curse of dimensionality.
- Being mindful of its memory-based nature, as it necessitates a pass through the data for each classification, which can become prohibitive for large datasets.

We can observe from the following example that k-NN, in a specialized version, remains highly competitive despite its simplicity.

Decent performance when large training set



- Yann LeCunn – MNIST Digit Recognition
  - Handwritten digits
  - 28x28 pixel images: $d = 784$
  - 60,000 training samples
  - 10,000 test samples
- Nearest neighbour is competitive

	Test Error Rate (%)
Linear classifier (1-layer NN)	12.0
K-nearest-neighbors, Euclidean	5.0
K-nearest-neighbors, Euclidean, deskewed	2.4
K-NN, Tangent Distance, 16x16	1.1
K-NN, shape context matching	0.67
1000 RBF + linear classifier	3.6
SVM deg 4 polynomial	1.1
2-layer NN, 300 hidden units	4.7
2-layer NN, 300 HU, [deskewing]	1.6
LeNet-5, [distortions]	0.8
Boosted LeNet-4, [distortions]	0.7

In today's landscape, amidst the emergence of new Machine Learning and Deep Learning techniques, k-NN deserves serious consideration due to its effectiveness, especially considering the complexities involved in training other algorithms.

A practical application showcasing the efficacy of the k-NN algorithm is im2gps (image to GPS), where given an image, the algorithm can discern its geographical location.



This task presents challenges because while some images exhibit unique features with corresponding data points worldwide (as seen in the first example), others are remarkably similar across various locations (as depicted in the second and third examples).

Researchers discovered that by employing k-NN alongside a robust feature extractor, they could achieve satisfactory results.
Their approach involved acquiring 6 million images from Flickr, each tagged with GPS information, which were densely sampled across the globe. They then represented each image with meaningful features and applied k-NN (preferring a large k, in this case, $k = 120$) for the task.



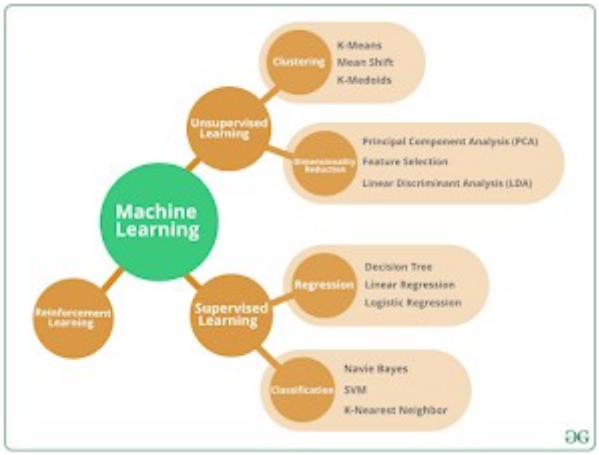
http://graphics.cs.cmu.edu/projects/im2gps/
http://graphics.cs.cmu.edu/projects/im2gps/results.html

https://www.youtube.com/watch?v=dgif39IKT9A

Linear models

We've already seen the three different settings:

- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning



The most popular setting, according to real-world applications, is **supervised learning**, particularly **classification learning**. We've also mentioned that the task of classification consists of learning a map from the feature space to the label space.

The first and only algorithm we've seen so far is k-NN, which is a very simple algorithm but has many possible variations.

In this chapter, we'll explore what linear models are in classification, specifically discussing the **perceptron**.

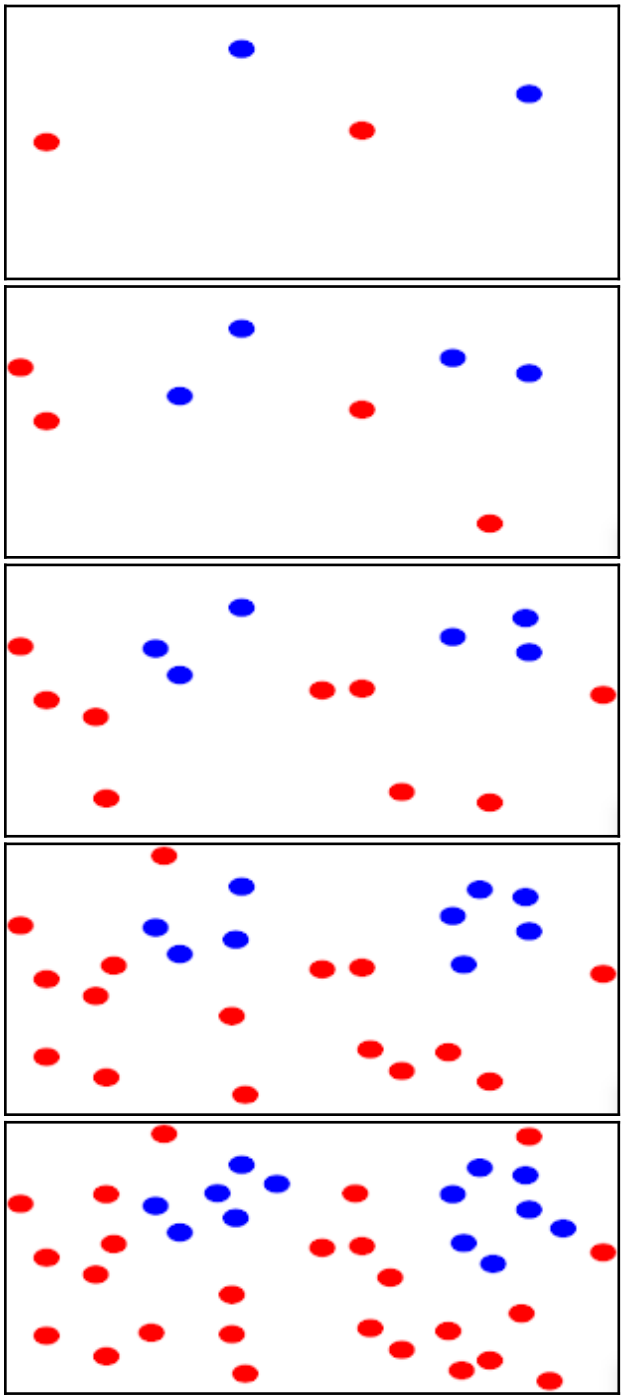
Now, let's introduce a concept that will contrast the k-NN algorithm with linear models. In Machine Learning, there are two types of models:

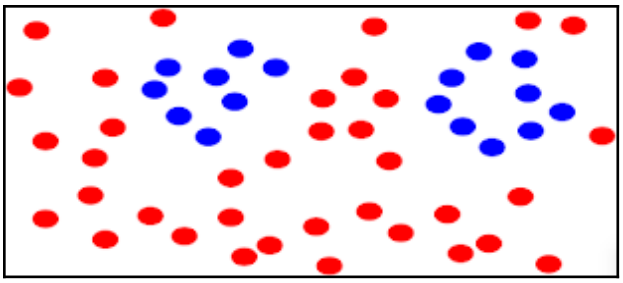
- Approaches that **make strong assumptions about the data distribution** (like linear models).
  If these assumptions are true, the approach can often lead to better performance. However, if the assumptions are not true, it can fail miserably.
- Approaches that **do not make many assumptions about the data distribution** (like k-NN).
  These can learn from more varied data but are more prone to overfitting due to their flexibility and generally require more training data.

Sometimes I need to make certain assumptions because this can lead to models with better performance (for example, it can reduce the time needed to find a solution).

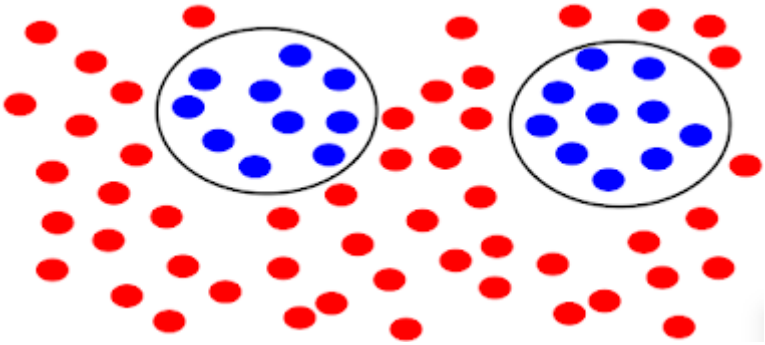
However, the problem with making strong assumptions is that if these assumptions are not accurate, the models will fail miserably in real-world applications.

To make this concept a bit more concrete, let's look at some examples.
We've already mentioned that the data points in the training set are generated from an underlying probability distribution. So, you start sampling from this probability distribution (the color of the points indicates the label):





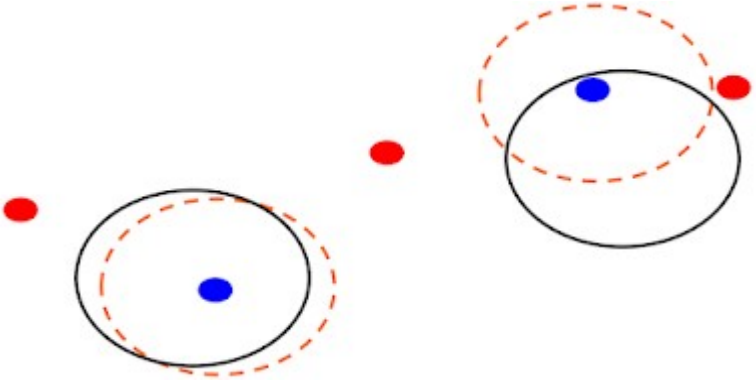
In the end, you'll get something like this:



This means that the model which separates the two classes (the red ones from the blue ones) might look like the one in the picture.
Another important consideration is how many samples we need to learn such a model.

If you don't have strong assumptions about the model, it can take a longer time to learn (in our example, we saw a lot of sampling before arriving at that conclusion).

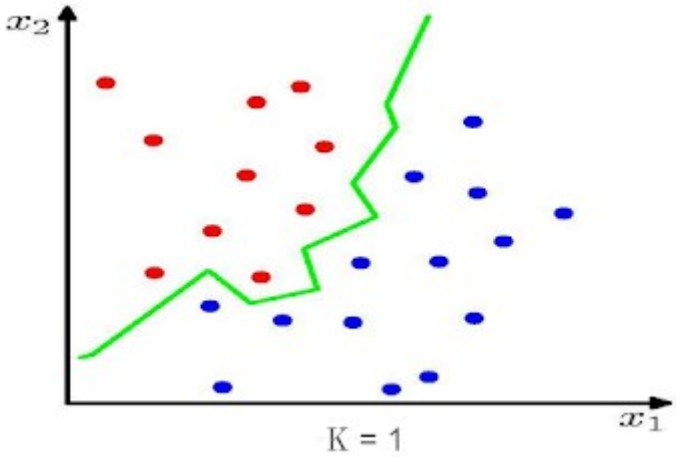
Assume now that our model for the blue class consists of two circles. In this case, I can arrive at the solution more quickly and don't need to sample a lot of data from the distribution.



So, if we have prior knowledge or an indication beforehand, we can drastically improve the learning process and reduce the number of training samples needed to learn the classifier.

However, as mentioned before, it's better to use prior knowledge about the problem only when you are confident that this knowledge is correct. Otherwise, if you make incorrect assumptions, the model will fail miserably.

The **k-NN algorithm doesn't make any model assumptions** about the data. In fact, this algorithm simply classifies the new point based on distance. The only principle it follows is that proximity is the sole criterion used to assign a point to a certain class.
Another important point to highlight is that the **k-NN algorithm could accurately learn from any dataset**, thus accommodating arbitrary decision boundaries.
However, as mentioned earlier, this algorithm requires a large training set cardinality to function effectively.



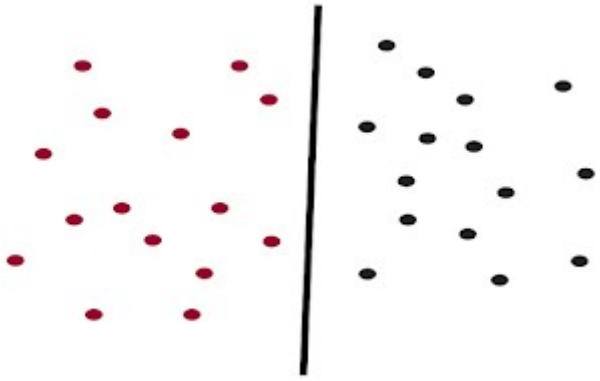
The **bias** of a model refers to the strength of the model assumptions:

- **Low-bias classifiers** make minimal (or no) assumptions about the data (examples include k-NN and Decision Trees, which are generally considered low-bias).
- **High-bias classifiers** make strong assumptions about the data (for instance, linear models are considered high-bias).

A strong high-bias assumption is **linear separability**, which means that:

- In two dimensions, it's possible to separate classes by a line.
- In higher dimensions, hyperplanes are needed to separate classes.

A **linear model assumes that the data is linearly separable**, meaning it can be separated by a straight line. Therefore, we are assuming a specific hypothesis space: linear functions.



Obviously, in this simple case, it's easy to make such assumptions. However, in practice, we typically deal with millions of data points in a very high-dimensional space, making it challenging to verify whether any assumption holds true.

In principle, having millions of data points means that assuming a linear classifier suffices, it's highly improbable that perfect separation of millions of data points in an N-dimensional space is practical. However, this is acceptable under the presumption that the number of misclassified points is relatively low. This acceptance stems from the understanding that learning a linear model is significantly more straightforward and efficient compared to learning a non-linear one.

We're dealing with a category of problems where we assume we'll learn a linearly separable model. While in straightforward cases, such as the example provided, it's feasible to separate two classes with a line, generally, we refer to hyperplanes (the generalization of lines in an N-dimensional space).

In a 2-dimensional space, defining a line through the origin requires 2 parameters or **weights**. Any pair of values (w_1, w_2) defines a line through the origin using the equation:

$$0 = w_1 f_1 + w_2 f_2$$

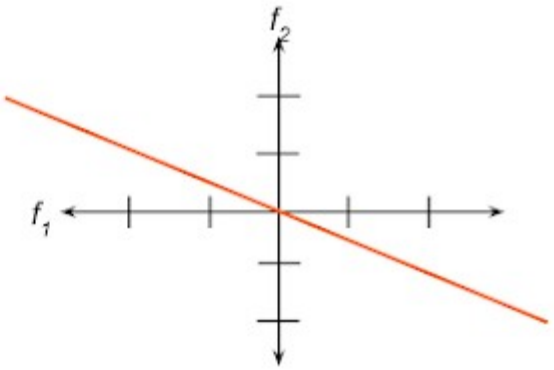
The number of weights corresponds to the number of features.

For example:

$$0 = w_1 f_1 + w_2 f_2$$

$$0 = 1 f_1 + 2 f_2$$

-2	1
-1	0.5
0	0
1	-0.5
2	-1



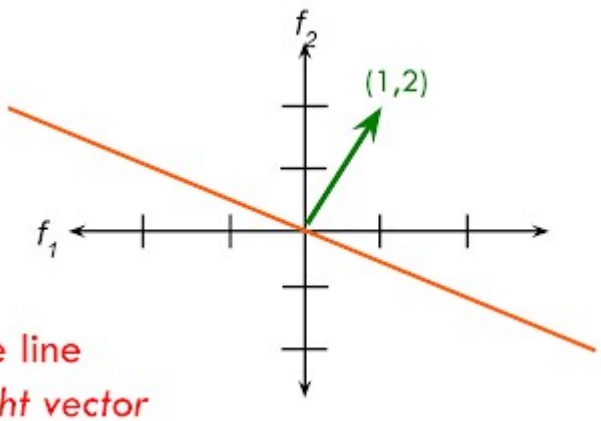
The points satisfying the equation (some of which are listed below the equation) represent the line under consideration once the weights are set.

This line can also be interpreted as perpendicular to the **weight vector**, which originates from the origin and points towards the coordinates of the weights.

$$0 = w_1 f_1 + w_2 f_2$$

$$0 = 1 f_1 + 2 f_2$$

w=(1,2)



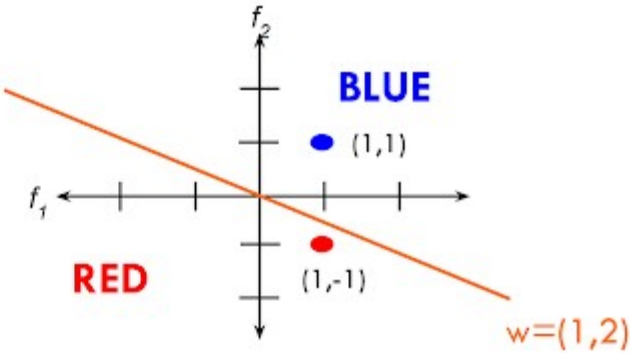
We can also view it as the line perpendicular to the *weight vector*

In machine learning, discussing weight vectors is common because they represent the parameters needed for learning. One method to learn this weight vector is the perceptron, which we'll explore in this chapter. Another method is SVM.

The weight vector shares the same dimensionality as the feature space.

The point now is how to mathematically classify points based on a line.
Reconsidering the previous example, we have parameterized our hyperplane with the weight vector $(1,2)$, and we have points that are either on one side of the line or the other.

$$0 = 1 f_1 + 2 f_2$$



Ideally, we have red points below the line and blue points above the hyperplane.
So, we utilize the formula of the line applied to the coordinates of the points to classify them based on the obtained value: if positive, then blue; otherwise, red. Indeed, the sign of the obtained value indicates which side of the line the point is on.

$$(1,1): \quad 1*1+2*1=3$$

$$(1,-1): \quad 1*1+2*-1=-1$$

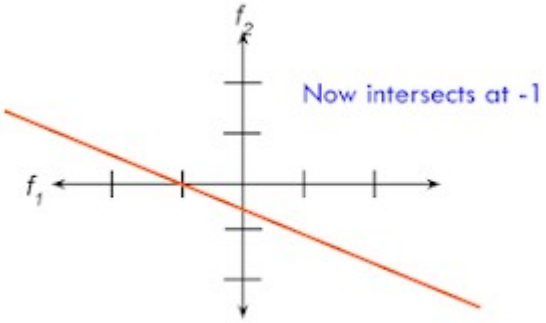
However, we're not restricted to having the hyperplane pass through the origin; sometimes, it's necessary to move the line away from the origin.
To achieve this, we adjust the **bias**, which is the value before the equals sign in the formula of the line, indicating the value at which the hyperplane intersects the horizontal axis:

$$a = w_1 f_1 + w_2 f_2$$

$a = w_1 f_1 + w_2 f_2$

$-1 = 1 f_1 + 2 f_2$

-2	0.5
-1	0
0	-0.5
1	-1
2	-1.5

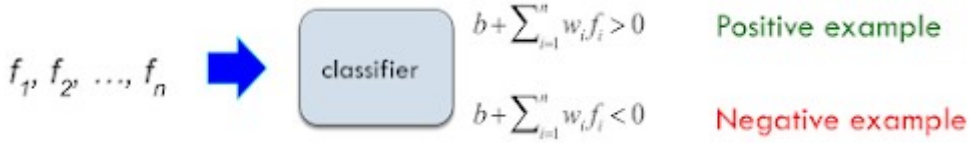


So, a linear model in an n-dimensional space (i.e., n features) is defined by $n+1$ weights:

- In two dimensions, we have a **line**:
  $0 = w_1 f_1 + w_2 f_2 + b$ where $b = -a$.
- In three dimensions, a **plane**:
  $0 = w_1 f_1 + w_2 f_2 + w_3 f_3 + b$.
- In n-dimensions, a **hyperplane**:

  $$0 = b + \sum_{i=1}^{n} w_i f_i.$$

In summary, we can classify with a linear model by checking the sign:



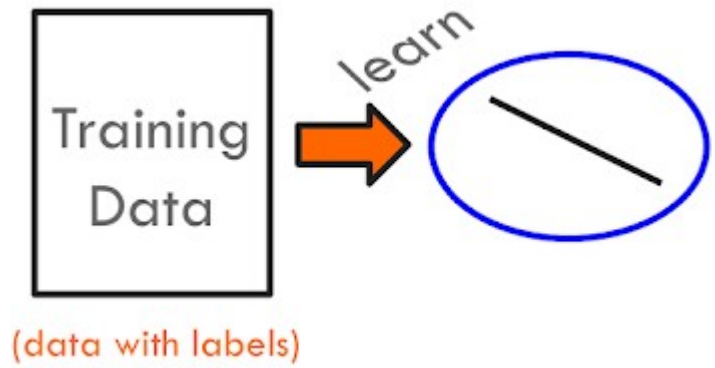
Thus, when utilizing a linear model as a classifier, inference is executed by calculating the dot product between the feature vector of the example and the weight vector, adding the bias term.

The weight vector "points" towards the positive side of the hyperplane.

Therefore, with a linear model (i.e., a set of weights w_i and b), we can classify examples.

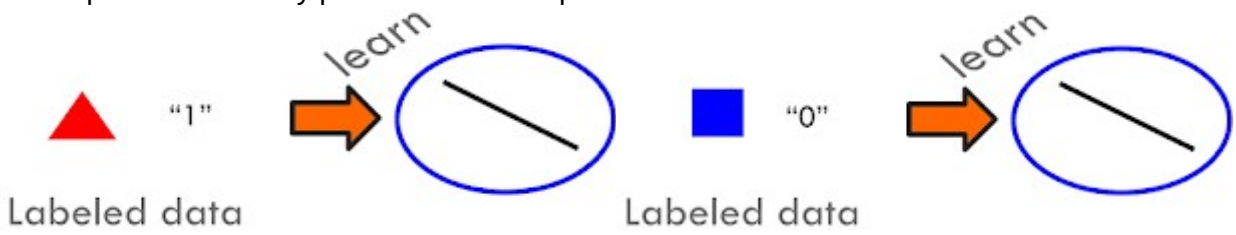
Online learning

Now, let's delve into learning a linear model, which involves defining the values of the weights in the formula.



In linear models, the learning setup differs from what we've seen before because we don't have the entire training set readily available beforehand. Instead, we receive the training set gradually over time, making linear models **online learning** models.

This implies that we only process one example at a time.



More generally, online learning is necessary when we encounter:

- **Data Streams**: In numerous real-world scenarios, data arrives continuously as streams, such as social media updates, sensor readings, or financial transactions. Online learning enables models to dynamically adapt to these data streams, ensuring they remain relevant and up-to-date.
- **Large-scale datasets**: Traditional batch learning methods can become computationally expensive and inefficient when dealing with large-scale datasets. Online learning provides a more scalable alternative by processing data incrementally, thereby significantly reducing computational costs and memory requirements.
- **Privacy-preserving applications**: In certain cases involving sensitive data, maintaining privacy is paramount. Online learning allows models to be trained directly on user devices or within secure environments, minimizing the necessity to share raw data across networks. This approach can help address privacy concerns while still facilitating effective model training and deployment.

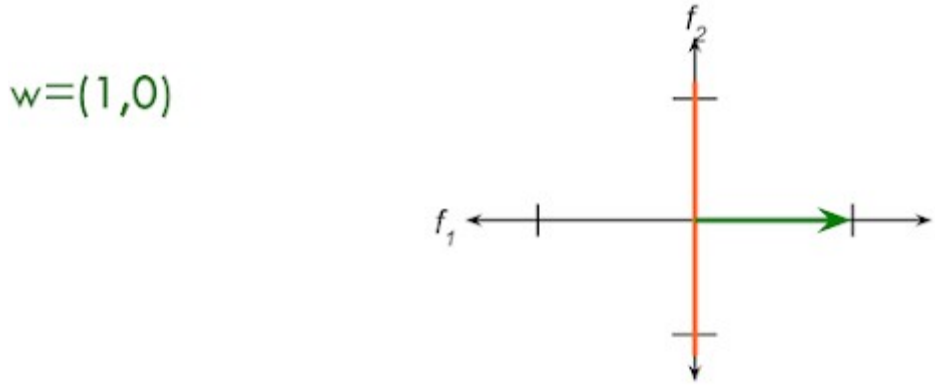
Formally, the differences between batch/offline learning and online learning are as follows:

- **Batch**: Given training data $\{(x_i, y_i) : 1 \le i \le n\}$, which is independent and identically distributed.
- **Online**: Data points arrive one by one
    - The algorithm receives an unlabeled example x_i.
    - The algorithm predicts a classification for this example.

- The algorithm is then provided with the correct answer y_i, and updates its model accordingly.

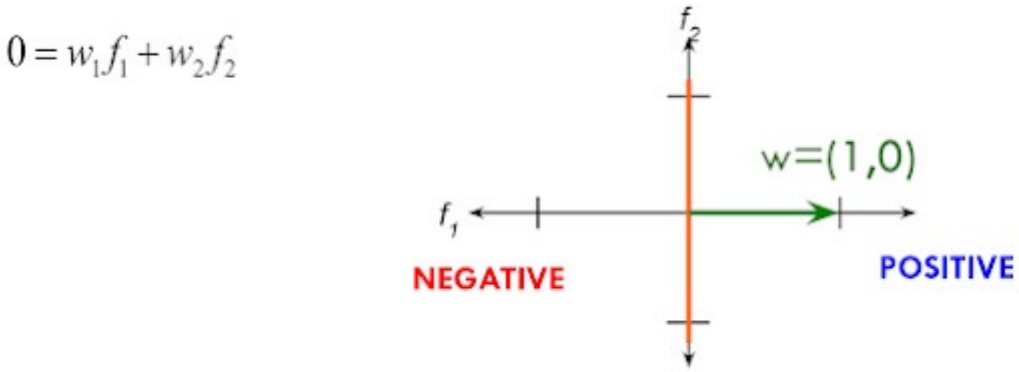
Learning A Linear Model

Let's suppose we start the learning in the following situation:

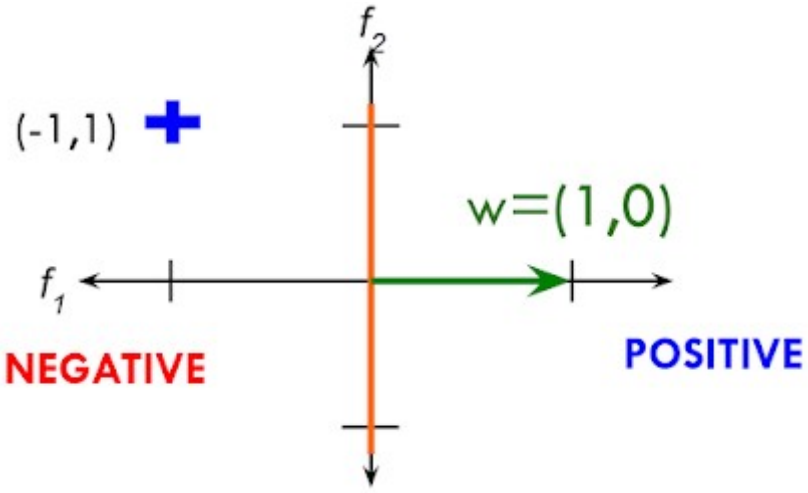


The initial hyperplane is the orange line, parameterized by the green vector w.

According to the rule we have seen before, the right side is the positive one, while the left side is the negative one.



Then a new sample arrives, which we can visualize in our 2-dimensional space as follows:



This sample is positively labeled but it is on the negative side of the hyperplane.

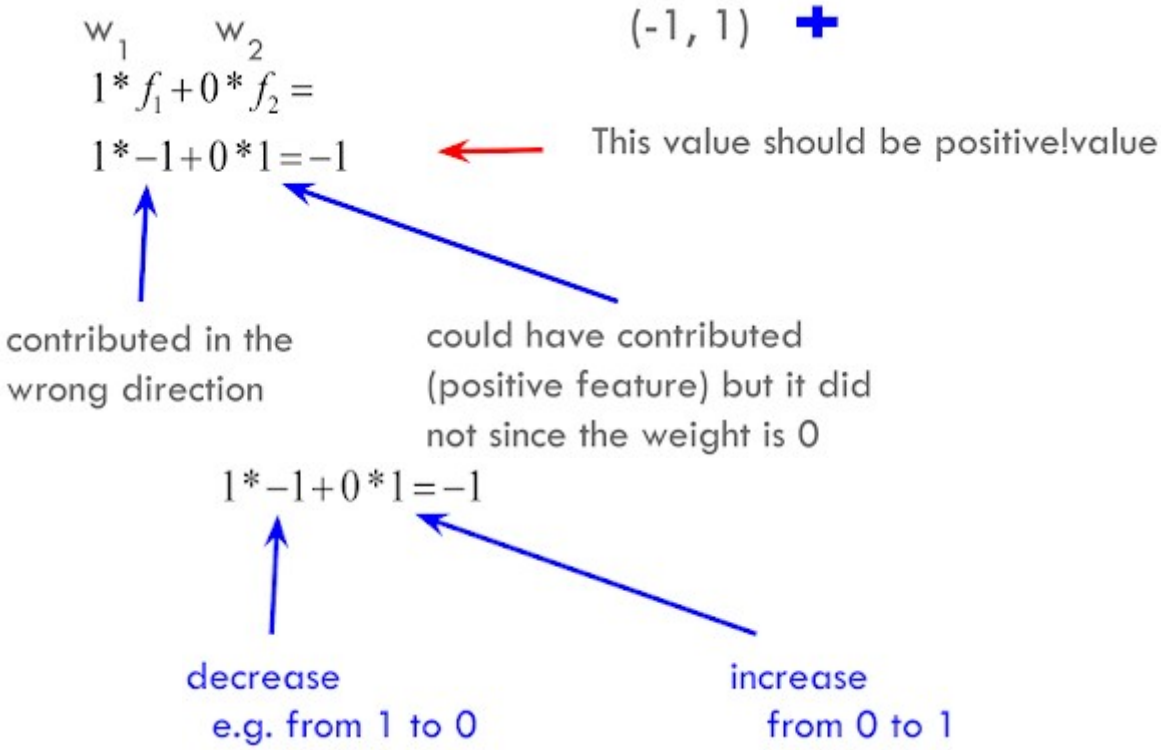
$$1 * f_1 + 0 * f_2 =$$

$$1 * -1 + 0 * 1 = -1$$

Negative, wrong!

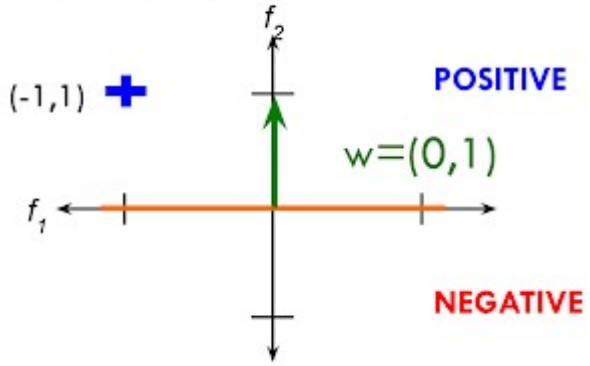
Therefore, the model must be updated.

Upon closer examination of why we got it wrong, we can see that to achieve a positive outcome in the formula, we need to adjust the weights by swapping them in this case.



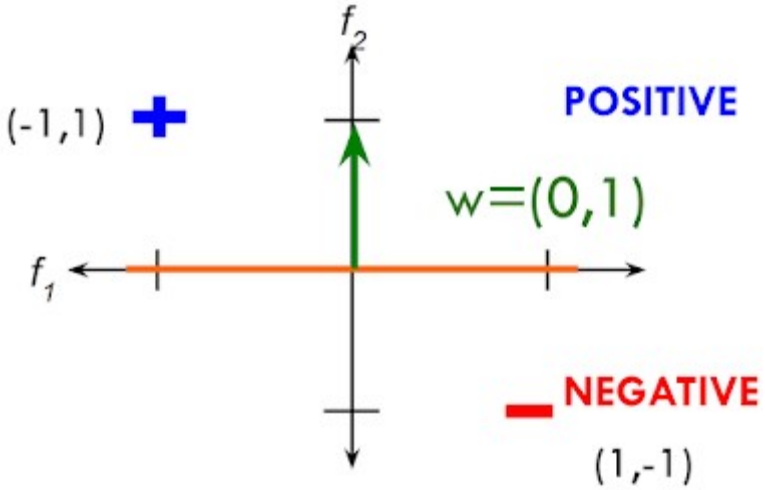
Now the model works for the sample:

Great! The model is successfully updated!



This is the intuition behind the algorithm we will see later: the **perceptron**.
The perceptron learning rule updates the weights based on the sign of the features.

Let's continue. Another sample arrives:

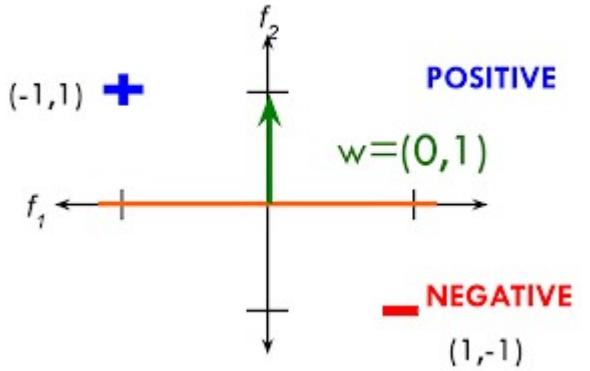


$$0 = w_1 f_1 + w_2 f_2$$

$$0 * f_1 + 1 * f_2 =$$

$$0 * 1 + 1 * -1 = -1$$

Is it correct? YES



In this case, the sample is correctly classified, so the model is already correct and we don't need to update it.

From this reasoning, we can derive the **perceptron learning algorithm**:

repeat until convergence (or for some # of iterations):
  for each training example (f_1, f_2, ..., f_n, label):
    check if it is correct based on the current model
    if not correct, update all the weights:
      for each w_i:
        $w_i = w_i + f_i$*label
      $b = b$ + label

We can observe the logic behind the updating of the weights:

- For each weight, we add the label (which is -1 or 1), proportional to the value of that feature.
- For the bias, we simply add the label.

It is like appending the bias to the weight vector and the label to the feature vector.

We can also observe that this algorithm works only for binary classification, so we need something more to support multiclass classification.

for each training example (f_1, f_2, ..., f_n, label): `label is -1/1`

The check is made as we've seen before:

check if it is correct based on the current model $prediction = b + \sum_{i=1}^{n} w_i f_i$

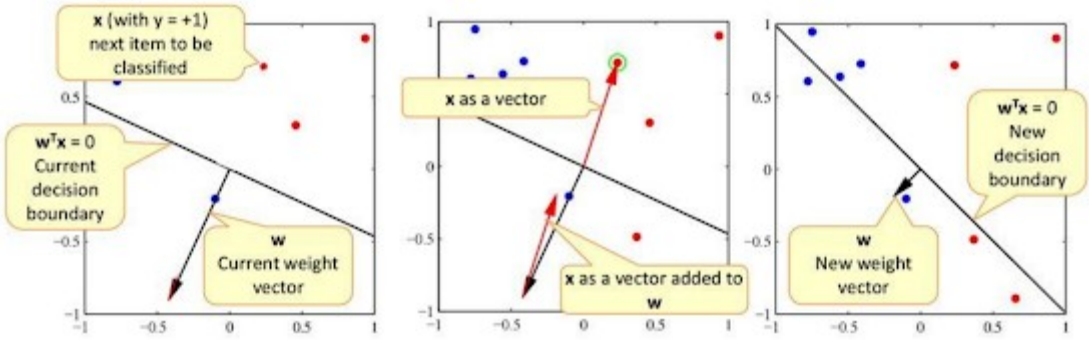
The model is incorrect if the prediction differs from the label:

if not correct, update all the weights: if *prediction is different from label*

And we obtain this:

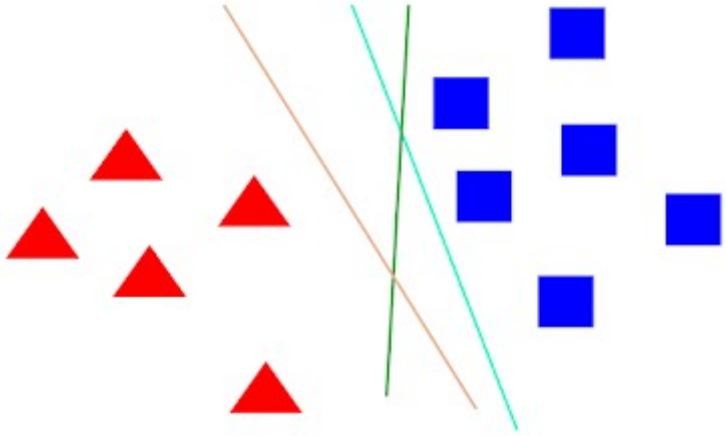
repeat until convergence (or for some # of iterations):
  for each training example (f_1, f_2, ..., f_n, label):
    $prediction = b + \sum_{i=1}^{n} w_i f_i$
    if *prediction is different from label*
      for each w_i:
        $w_i = w_i + f_i$*label
      $b = b$ + label

In practice, the perceptron learning algorithm updates the weights when a sample is misclassified by adding the vector of the new sample (the vector from the origin to the point defined by the sample's features) to the weights vector.



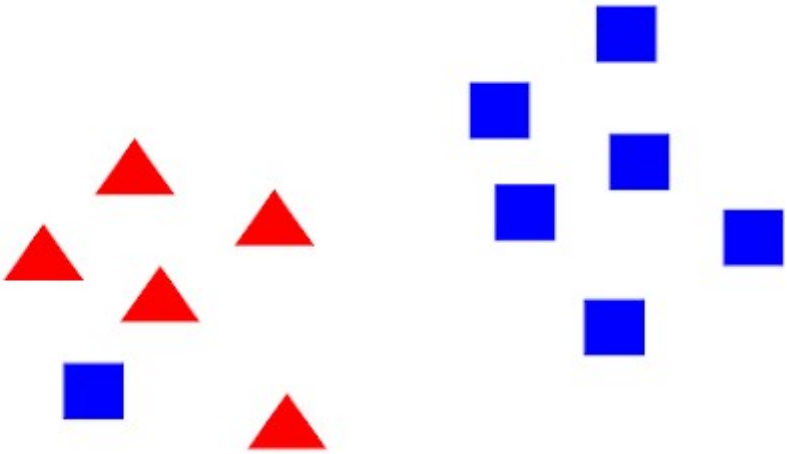
With this representation, we can also observe that after correcting the hyperplane, the point that caused the update may still be on the wrong side of the hyperplane. This is why the algorithm iterates multiple times.

The perceptron learning algorithm guarantees **finding some line** that separates the data (if the dataset is linearly separable), but it does not guarantee which line will be found. This is because the outcome depends on the order in which the data is presented to the perceptron.



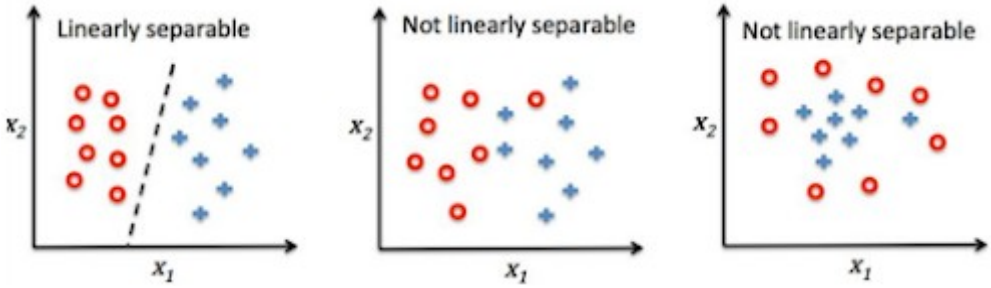
Thus, there is no concept of an "optimal" model with a perceptron, even though some models will generalize better than others. For example, if the line is too close to the red samples, the model may not generalize well.

Let’s discuss the **convergence** in the algorithm. The algorithm will converge only if the dataset is linearly separable. Otherwise, in order to make it able to also support non-linearly separable datasets, it will never converge so it’s needed to set a maximum number of iterations, accepting the solution the algorithm will reach.



There will be no convergence here!

The training instances are **linearly separable** if there exists a hyperplane that separates the two classes.



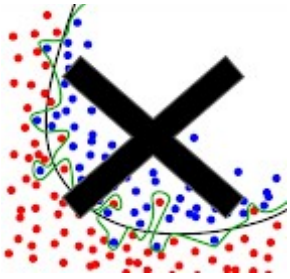
In some cases, I can apply a linear model to a dataset like the second one, accepting that it won't be 100% accurate because learning a linear model is simple and fast. To obtain more accurate results, you can train the model multiple times with different orderings of examples and measure performance on the validation set.

To ensure the model can produce different results in different instantiations, we randomize the order in which the samples are presented:

for each training example (f_1, f_2, ..., f_n, label): random sample one example (f_1, f_2, ..., f_n, label):

This randomization is very important.

Another reason to limit the number of iterations, in addition to the fact that convergence might be unreachable, is to limit overfitting.

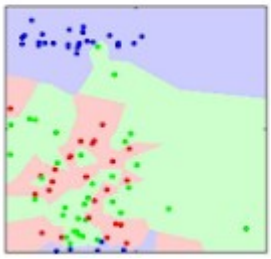


Multiclass classification

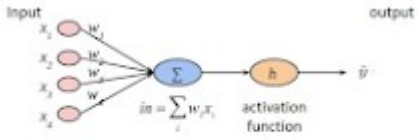
From Binary to Multiclass Classification

We've already mentioned that k-NN is more powerful than a linear model trained with a perceptron because k-NN can handle multiclass classification tasks.

K-NN



PERCEPTRON



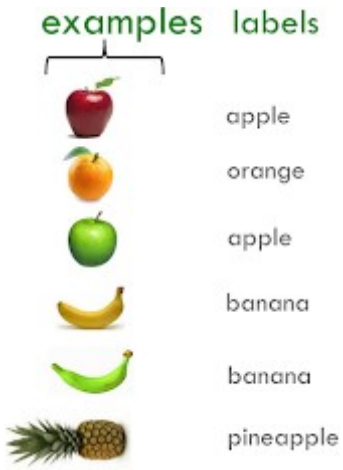
In this chapter, we're going to explore how we can perform multiclass classification with linear models. The linear classifier is traditionally used for binary classification problems, but we will see that it is possible to apply it to more complex problems, such as those involving multiple classes.

Before moving on to multiclass classification, let's be a bit more formal about binary classification.
Given:

- An input space X
- An unknown distribution Dis over $X \times \{-1, +1\}$
- A training set D sampled from Dis

Compute:
A function f that minimizes $E_{(x,y)\sim Dis}[f(x) \neq y]$ (the expectation of the times when the results of the function differ from the ground truth).

Multiclass classification is a natural extension of binary classification.
The goal is still to assign a **discrete label** to examples, but the difference is that you have $K > 2$ classes/labels to choose from.



We require multiclass classification because most real-world applications involve multiclass predictions.



document classification



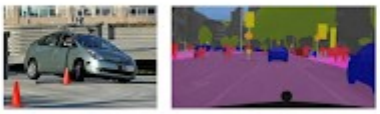
handwriting recognition



face recognition



sentiment analysis



autonomous vehicles



emotion recognition

Formally, multiclass classification consists of:
Given:

- An input space X and a number of classes K
- An unknown distribution Dis over $X \times \{1,2,\ldots,K\}$
- A training set D sampled from Dis

Compute:

- A function f that minimizes $E_{(x,y)\sim Dis}[f(x) \neq y]$ (the expectation of the times when the results of the function differ from the ground truth)

As we've already mentioned, k-NN works out of the box with multiclass problems, while perceptrons require small modifications.

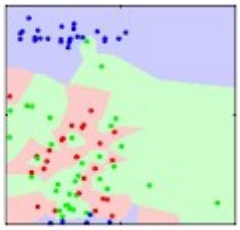


KNN PERCEPTRON

In fact, k-NN works in this way:

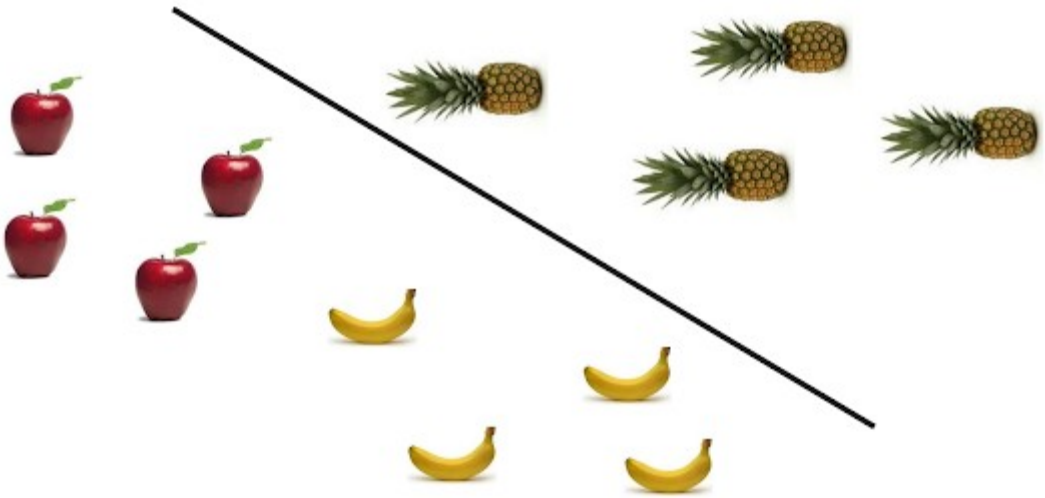
To classify an example ***d***:

- Find ***k*** nearest neighbors of ***d***
- Choose as the label the majority label within the ***k*** nearest neighbors

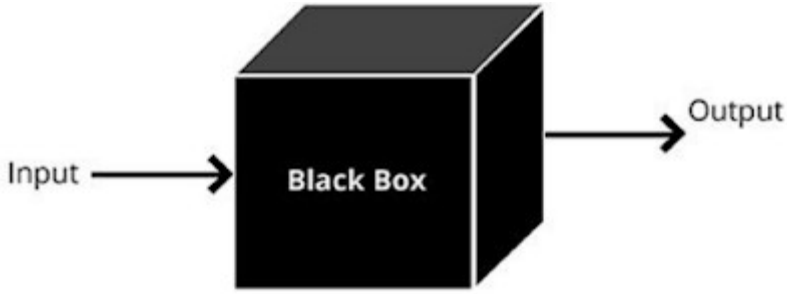


So no modifications are required in order to handle multiclass problems.

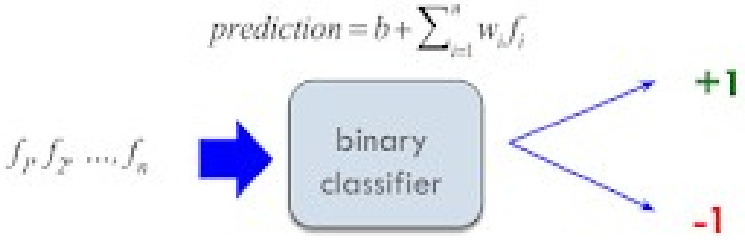
Indeed, with a perceptron, it is impossible to separate more than two classes with just one line/hyperplane.



To apply a perceptron to multiclass problems, we use a black box approach, meaning that we use an existing binary classifier algorithm to solve the multiclass classification problem.

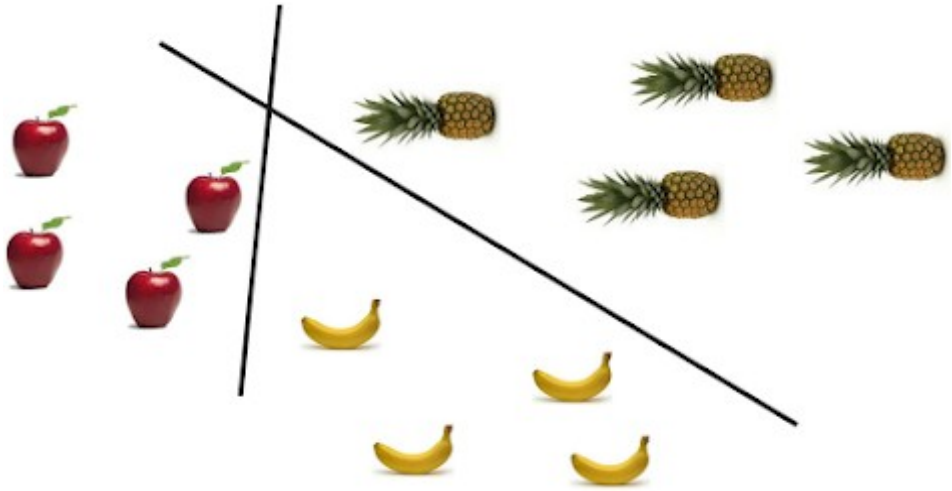


So given a generic binary classifier, we use it to solve the new problem



It is important to note that we can also output a confidence score of the classification, which indicates how high the value of the prediction is: the further the value is from zero, the more confident the prediction is.

We've said that one line does not suffice, but we can **combine multiple lines**.



OVA & AVA

There are two ways to address this problem:

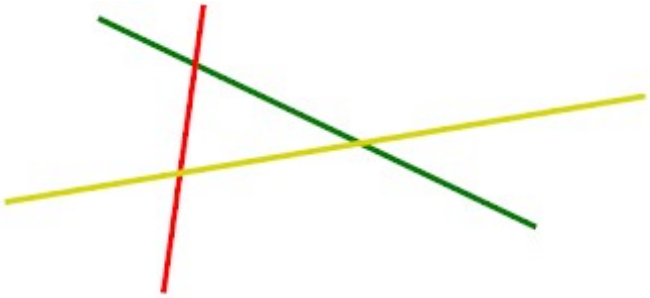
- **OVA**: One vs All
- **AVA**: All vs All

These methods are also used to solve the same problem as SVMs (Support Vector Machines), which are another linear model algorithm.

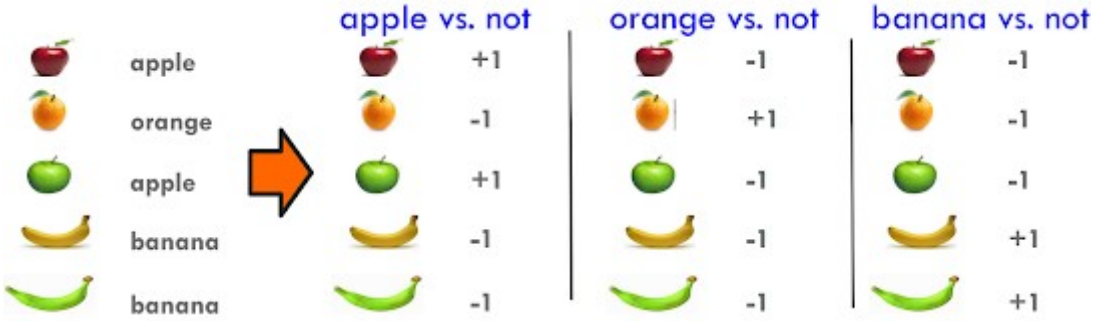
The first approach is **OVA** (One vs All). This approach consists of defining a binary problem for each label (class) L:

- All the examples with label L are positive.
- All other examples are negative.

In practice, K different binary classification models are trained and learned.

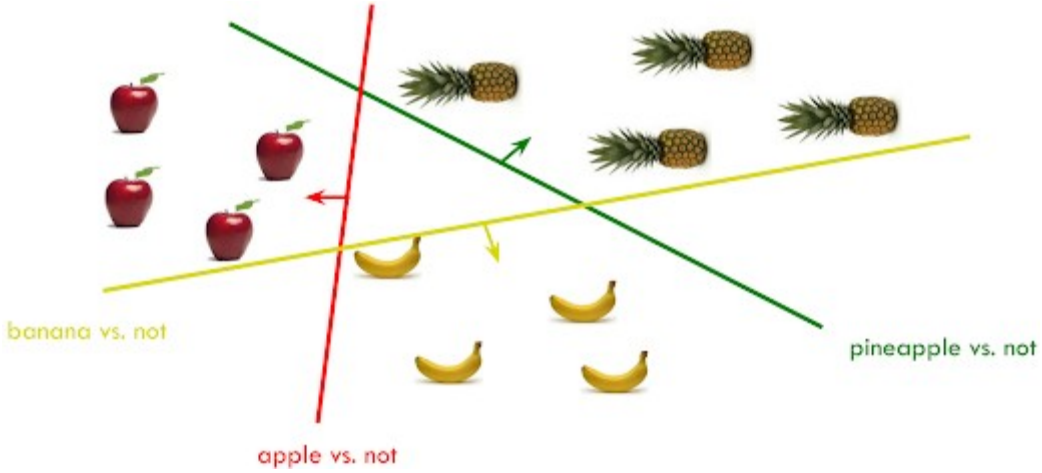


An example to better understand this concept is the following:



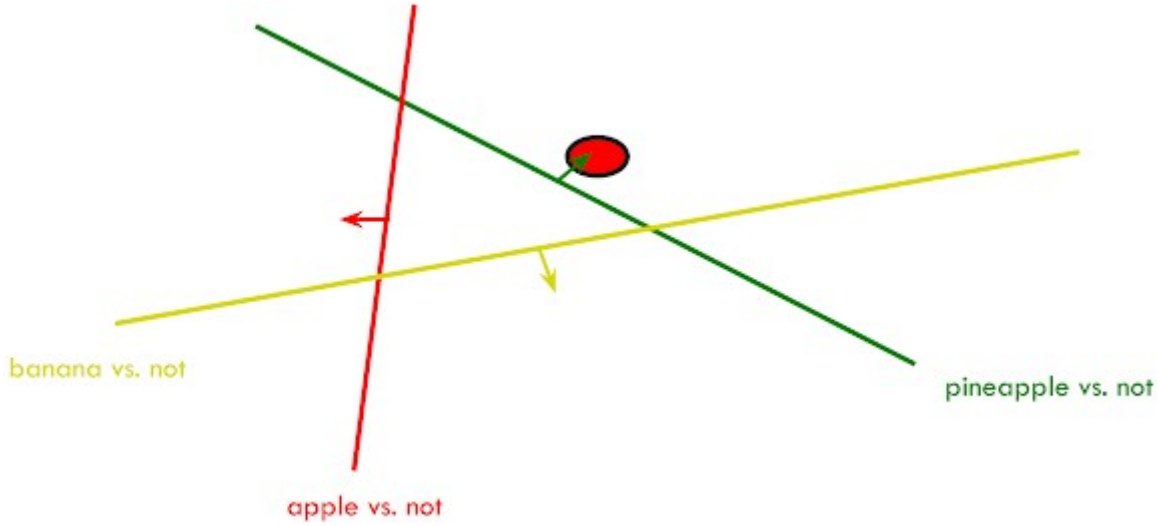
During learning, it is like considering K different binary classification problems with K different training sets (because the labels must be changed).

This process is visualized in the following picture:

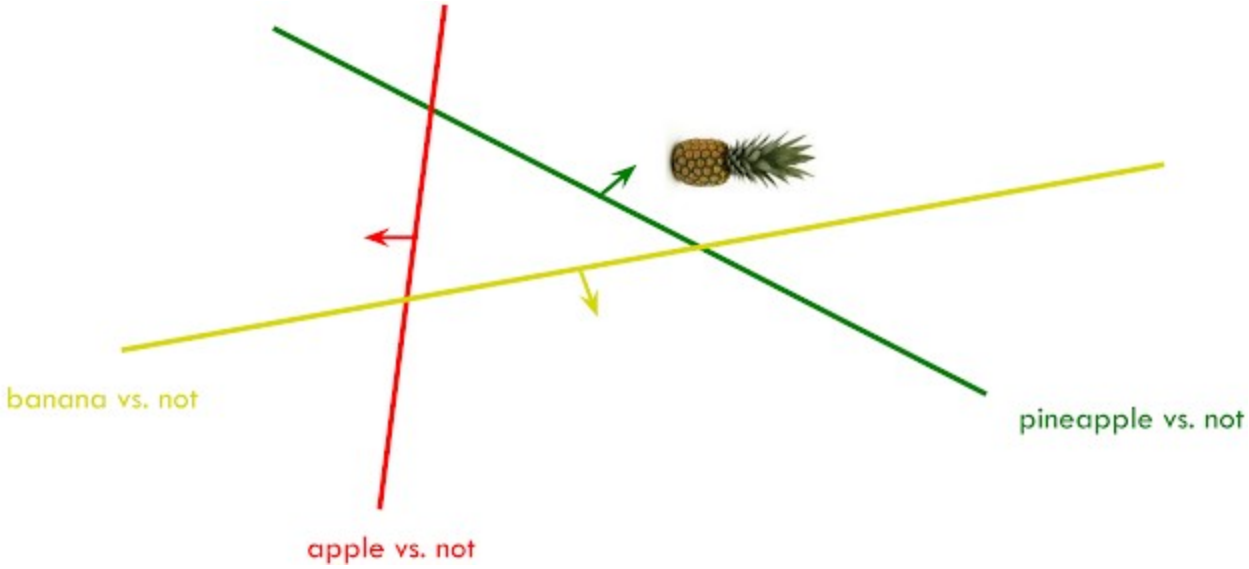


Now let's see how we can perform the inference phase in this situation.

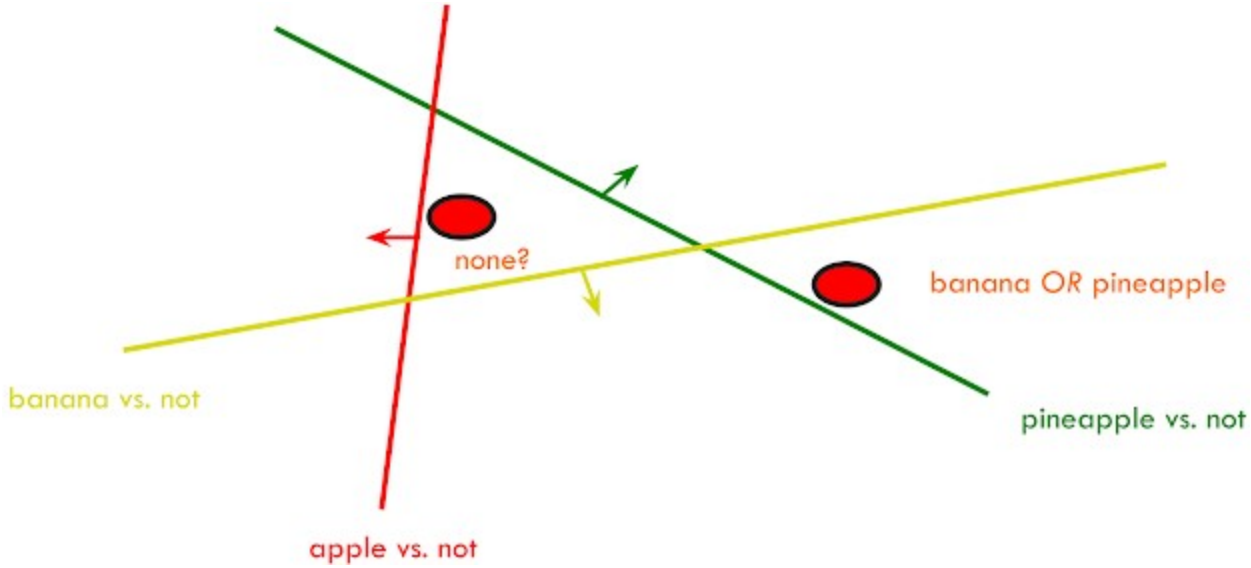
If, for example, this sample needs to be classified:



It is easy to understand that it will be classified as a pineapple:

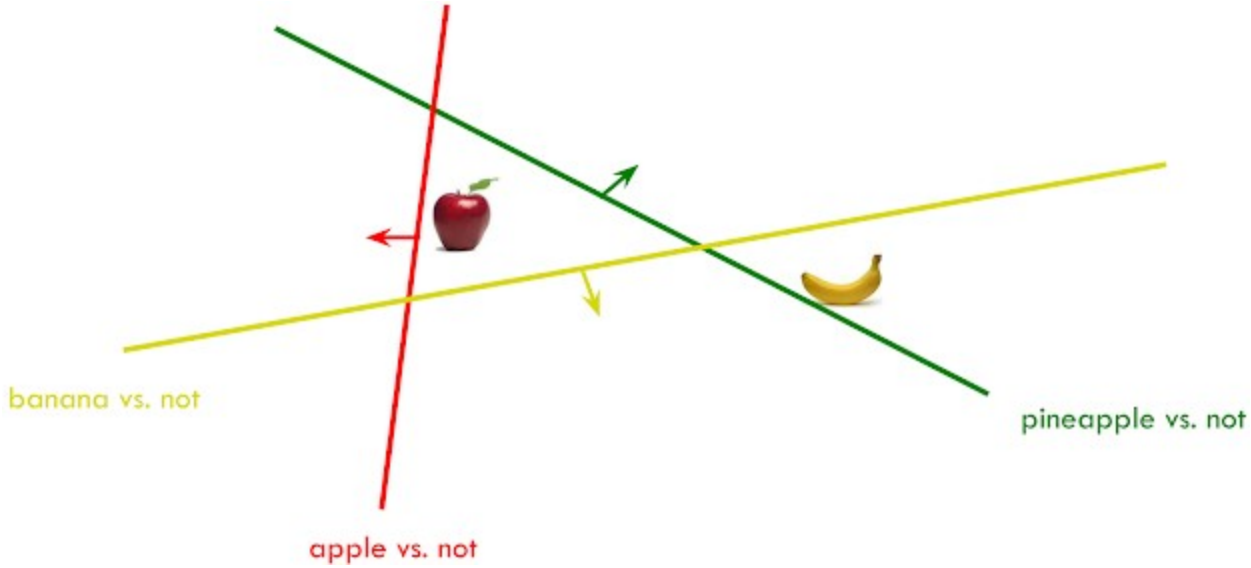


But what if the new samples are positioned in this way:



In this case, I have to use the **confidence** of the models, which is the distance with respect to the hyperplanes:

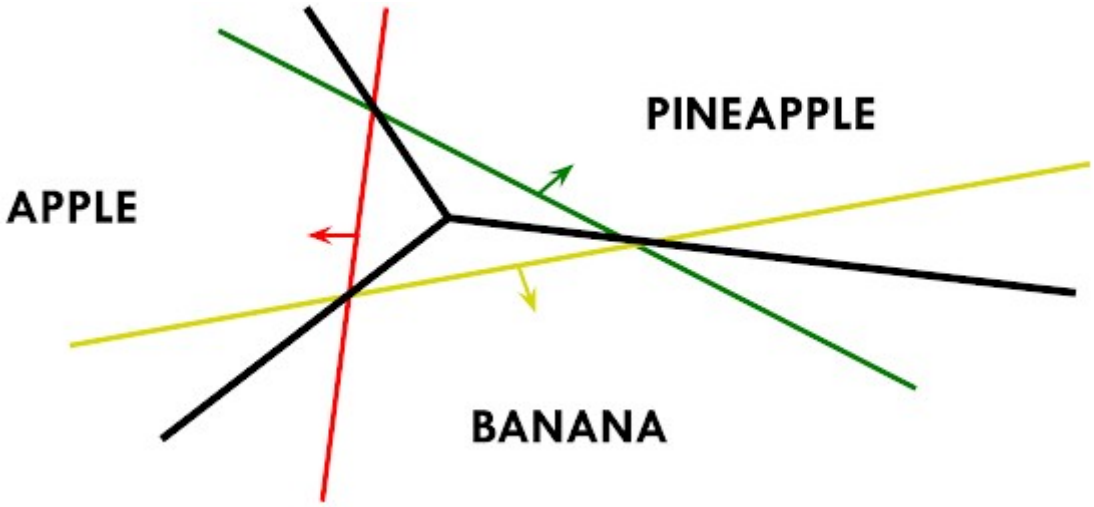
- If the sample results are negative for all the classifiers, I choose the label whose classifier prediction is the closest to zero.
- If the sample results are positive for different classifiers, I choose the label whose classifier prediction is the highest, i.e., most positive.



In summary:

- If the classifier does not provide confidence and there is ambiguity, pick one of the conflicting classes.
- However, in general, classifiers provide confidence. Then:
    - Pick the **most confident** positive.
    - If none vote positive, pick the least confident negative.

Reasoning with confidence, the decision boundaries look like this:



The confidence, as we've seen before, is the distance from the hyperplane and is given by this formula:

$$prediction = b + \sum_{i=1}^{n} w_i f_i$$

To formalize and summarize how OVA works, let's look at the functions for training the model and making the inference.

Algorithm 13 OneVersusAllTrain($\mathbf{D}^{multiclass}$, BinaryTrain)

```
for i = 1 to K do
    D^bin ← relabel D^multiclass so class i is positive and ¬i is negative
    f_i ← BinaryTrain(D^bin)
end for
return f_1, ..., f_K
```

Algorithm 14 OneVersusAllTest($f_1, \ldots, f_K, \hat{x}$)

```
score ← ⟨0, 0, ..., 0⟩                    // initialize K-many scores to zero
for i = 1 to K do
    y ← f_i(x̂)
    score_i ← score_i + y
end for
return argmax_k score_k
```

Obviously, since OVA is an algorithm that operates on binary classification, we must select a binary classification algorithm (in this case, a perceptron).

In the training function, which takes as input the multiclass training set and the binary classification training function, for each of the K classes, I relabel the multiclass training set, transforming it into binary training sets. Then, I use these binary training sets to learn the binary classification models. In the end, I will have K binary classification models (one for each class).

In the inference function, which takes in input the K binary classification models and a new sample, a score vector of size K is initialized to 0 and then for each K class the score (the confidence) is calculated. The class which has the higher score is the class the model inferences to that sample.

The second approach is **AVA** (All vs All), sometimes called **all pairs**. This approach consists of defining a binary problem for each pair of classes.
So, in practice, $\frac{K(K-1)}{2}$ different binary classification models are trained and learned, where F_{ij} , for $1 \leq i < j \leq K$, is the classifier that discriminates class i against class j. This classifier receives all the examples of class i as "positive" and all the examples of class j as "negative".

When a test point arrives, we evaluate it on all the F_{ij} classifiers:

- Every time F_{ij} predicts positive, class i gets a vote
- Otherwise, class j gets a vote.

It is also possible to penalize the "non-winning" class.
After running all $\frac{K(K-1)}{2}$ classifiers, the class with the most votes wins.

This approach handles the multiclass classification problem by decomposing it into binary classification problems, similar to **sport tournaments**:

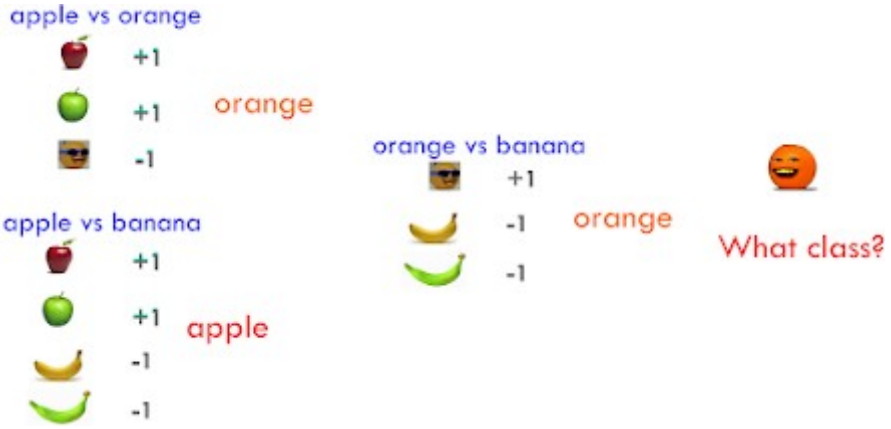
- You have K teams entering a tournament, but unfortunately, the sport they are playing only allows two to compete at a time (in our analogy, the teams are the classes, and you want to know which class is best).
- You want to set up a way of pairing the teams and having them compete so that you can figure out which team is best.
- In practice, every team competes against every other team.
- The team that wins the majority of its matches is the best.

An example to better understand the concept is the following:



During learning, it is like considering $\frac{K(K-1)}{2}$ different binary classification problems with $\frac{K(K-1)}{2}$ different (restricted) training sets (because I have to select only the examples which are of class i or j, and change the labels).
We can also encounter problems here if there is an unbalanced training set, particularly because if a classifier is learned for two classes that are not well-represented, then this classifier will totally fail.
This means that both OVA and AVA aren't the best choices when I have a very large multiclass classification problem (the same thing is valid for SVM).

As we've already mentioned, the inference phase is done by picking the class which "wins" more binary classifications:



The algorithm for training AVA involves training a classifier for each pair of labels to distinguish between them:

for i = 1 to number of labels:

for j = i+1 to number of labels:

train a classifier F_{ij} to distinguish between $label_j$ and $label_i$:

- create a dataset with all examples *with* $label_j$ labeled positive and all examples with $label_i$ labeled negative

- train classifier F_{ij} on this subset of the data

The algorithm for inference for a new example, after classifying the new example with each classifier F_{ij}, can work in two ways to choose the final class:

- Take a majority vote.
- Take a weighted vote based on confidence $y = F_{ij}(x)$:

- $score_i += y$
- $score_j -= y$

If y is positive, the classifier thought it was of type i, so raise the score for i and lower the score for j.
If y is negative, the classifier thought it was of type j, so lower the score for i and raise the score for j.

To formalize and summarize how AVA works, let's see the function for training the model and the function to make the inference.

Algorithm 15 AllVersusAllTrain($\mathbf{D}^{multiclass}$, BinaryTrain)

$f_{ij} \leftarrow \emptyset, \forall 1 \le i < j \le K$
for $i = 1$ **to** $K-1$ **do**
  $\mathbf{D}^{pos} \leftarrow$ all $\boldsymbol{x} \in \mathbf{D}^{multiclass}$ labeled i
  for $j = i+1$ **to** K **do**
    $\mathbf{D}^{neg} \leftarrow$ all $\boldsymbol{x} \in \mathbf{D}^{multiclass}$ labeled j
    $\mathbf{D}^{bin} \leftarrow \{(\boldsymbol{x}, +1) : \boldsymbol{x} \in \mathbf{D}^{pos}\} \cup \{(\boldsymbol{x}, -1) : \boldsymbol{x} \in \mathbf{D}^{neg}\}$
    $f_{ij} \leftarrow$ BinaryTrain($\mathbf{D}^{bin}$)
  end for
end for
return all f_{ij}s

Algorithm 16 AllVersusAllTest(all f_{ij}, $\hat{\boldsymbol{x}}$)

$score \leftarrow \langle 0, 0, \ldots, 0 \rangle$ // initialize K-many scores to zero
for $i = 1$ **to** $K-1$ **do**
  for $j = i+1$ **to** K **do**
    $y \leftarrow f_{ij}(\hat{\boldsymbol{x}})$
    $score_i \leftarrow score_i + y$
    $score_j \leftarrow score_j - y$
  end for
end for
return $\text{argmax}_k\ score_k$

The long notation in the training phase serves to relabel the subset of the dataset, making the labels of class i positive and the labels of class j negative.

Let's now compare OVA and AVA in terms of train/classify runtime and error probability.

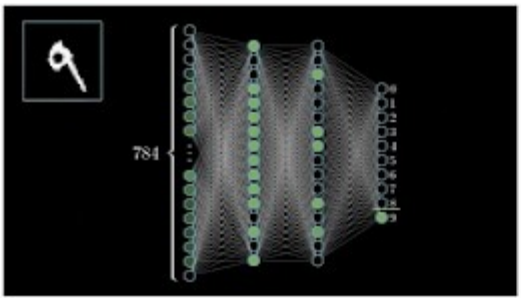
- Train time:
  AVA learns more classifiers than OVA; however, in AVA they are trained on much smaller datasets, which tends to make the learning faster than OVA if the labels are equally balanced.
- Test time:
  AVA has more classifiers, so it is often slower than OVA.
  In linear models, the test time isn't ever too high, but in AVA we have more models, and so more time is required to make the inference.
- Error:
  Both AVA and OVA require a balanced dataset to perform well.
  AVA tests with more classifiers and therefore has more chances for errors.

In general, it could be preferable to learn more models on a smaller training set than few models on a large training set (this is especially true for methods like SVM, because each training phase time depends quadratically on the size of the training set).

In practice, OVA is more popular because people are more interested in the inference time than the training time and because OVA is more intuitive.
So, if using a binary classifier, the most common thing to do is OVA.
Otherwise, use a classifier that allows for multiple labels:

- Decision Trees (DT) and k-NN work reasonably well.
- Other more sophisticated methods work better (we will see them later in the course).

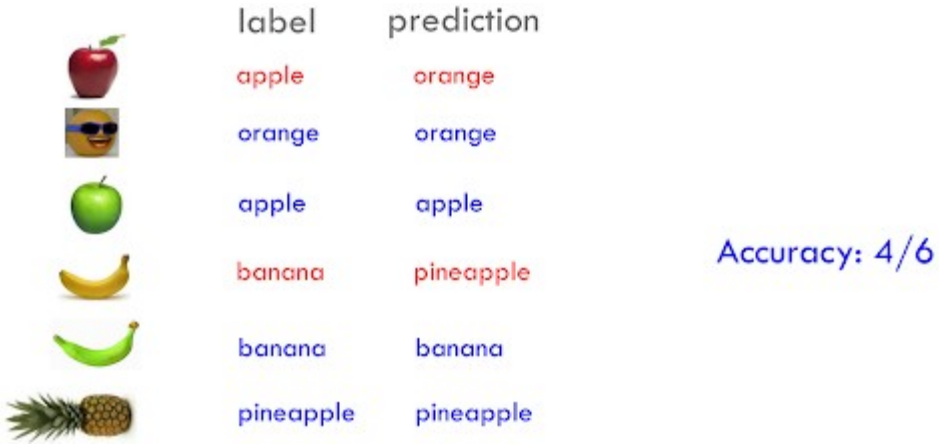
In 2024, we use Neural Networks for all multiclass classification problems, like number recognition.





Evaluation

Let's now see how we can **evaluate** the multiclass classification task.
The simplest way to evaluate multiclass classification is "general" accuracy:

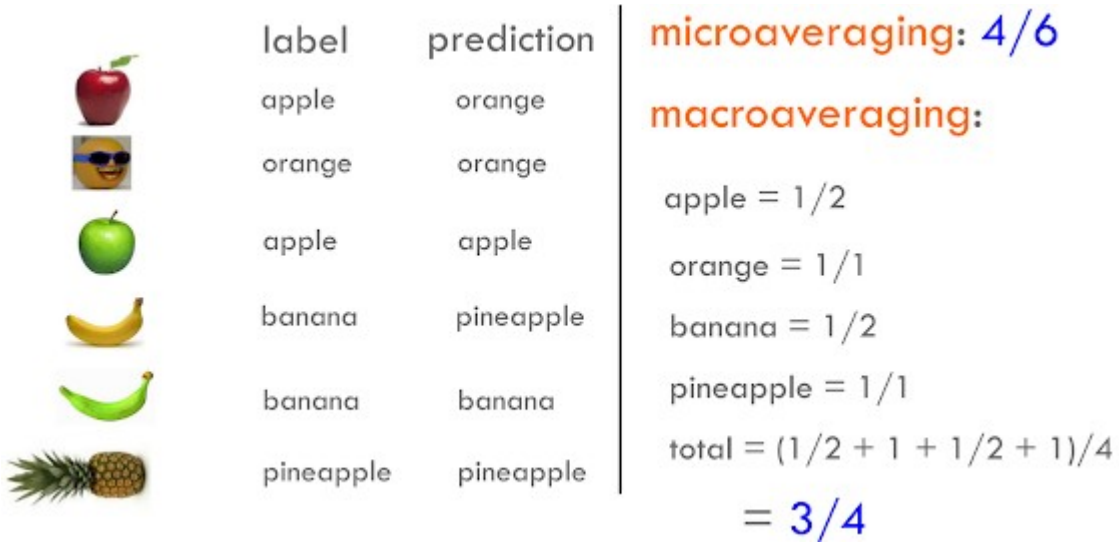


But it doesn't really consider the fact that there are multiple categories, and some categories may be more important than others. In fact, if we have unbalanced data, this (global) accuracy doesn't provide much insight.
So we need to consider a method of evaluation that observes which labels are the most problematic (i.e., inferred incorrectly).

When evaluating the performance of machine learning models, particularly in classification tasks, it's essential to consider different averaging techniques. Two common methods are Microaveraging and Macroaveraging:

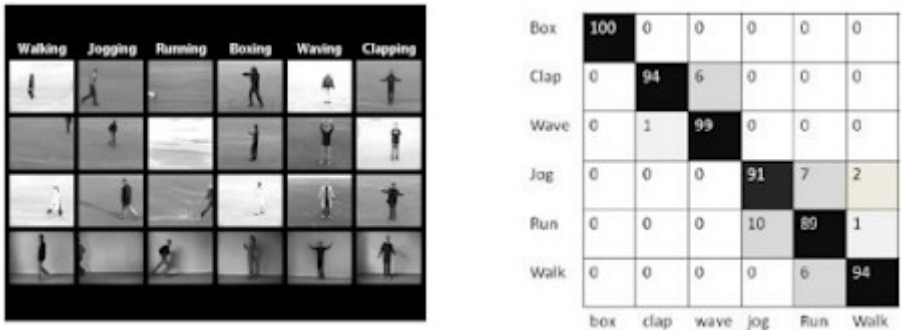
- **Microaveraging**: average over examples (this is the “normal” way of calculating and it is the way we’ve seen before).
- **Macroaveraging**: calculate evaluation score (e.g., accuracy) for each label, then average over labels (this is a more “meaningful” way to evaluate).

Let’s see an example to better understand the differences between the two methods:



Macroaveraging was introduced, as we’ve already said, to put more weight/emphasis on rarer labels and to allow another dimension of analysis.

It is very common when dealing with a multiclass classification problem to compute (in addition to macroaveraging and microaveraging) the **confusion matrix**.
The confusion matrix is a $K \times K$ matrix where the entry (i, j) represents the number of examples with true label i that were predicted to have label j (often in percentage):



This is very useful to understand which classes the model most frequently confuses or misclassifies and struggles with the most, allowing for the improvement of the training set with more examples from those classes (but in some cases, it could be an extremely hard task).

When there are a lot of classes, this helps to evaluate the classifier, analyzing how much the matrix is diagonal.

Gradient Descent

Resuming the idea of machine learning, which allows computers to acquire knowledge:

- Knowledge is acquired through **algorithms** by learning and inferring from **data**.
- Knowledge is represented by a **model**.
- The model is used on future data.



We can now have a more concrete idea of what a model can be. For example, it can be an equation, such as a set of parameters $w_1, \ldots, w_n$ and a bias b (in the case of linear models). The model is obtained from an algorithm. In the case of linear models, this could be the perceptron learning rule, and the data is data with associated labels.

One important thing to remember is that the perceptron won't necessarily find the optimal model; it is only guaranteed to find **some line** that separates the data.

The perceptron algorithm is one example of a linear classifier, but there exist many other algorithms which learn a line (i.e., a setting of a linear combination of weights).
So for the same model (a linear model), we can use different algorithms (like the perceptron) to learn it.

The goals now are to explore a number of linear training problems and to understand **why these algorithms work**.

Let's review the perceptron learning algorithm, specifying the step "if prediction is different from label" more concretely:

repeat until convergence (or for some # of iterations):

for each training example (f_1, f_2, ..., f_n, label):

$prediction = b + \sum_{i=1}^{n} w_i f_i$

if *prediction * label* ≤ 0: // they don't agree

for each w_i:

$w_i = w_i + f_i$*label

$b = b$ + label

Let's now explore the notion of **model-based** machine learning.
In the algorithms we've seen and in those we will examine, there is an underlying procedure we follow. Let's clarify this procedure:

1. **Pick a model** (e.g., a hyperplane, a decision tree, etc.)
   A model (in most cases) is defined by a collection of parameters:
   - Decision trees: The structure of the tree, which features each node splits on, the predictions at the leaves.
   - Linear models: The weights and the bias value.
2. **Pick a criterion to optimize** (also known as the **objective function**)
   - For example, the training error in linear models (because it is a proxy for the risk)
3. **Develop a learning algorithm**
   The algorithm should aim to minimize the criterion, sometimes in a heuristic way (i.e., non-optimally, as in decision trees), and sometimes exactly.

Picking a model essentially defines our hypothesis space.

Let's see a practical implementation of this procedure in linear models:
First, we have to pick a model. For example, by picking a linear model, we assume that we want to learn a hyperplane, which is a set of weights and a bias.

1. Pick a model

$$0 = b + \sum_{j=1}^{m} w_j f_j$$

These are the parameters we want to learn

Next, we have to choose a criterion to optimize.

To be a bit more formal, we need to introduce the **indicator function**: a convenient notation for turning True and False answers into numbers/counts:

$$1[x] = \left\{ \begin{array}{ll} 1 & if\ x = True \\ 0 & if\ x = False \end{array} \right\}$$

Typically, the indicator function is denoted by $1[x]$ or $I[x]$.
This function allows us to compare the predicted label with the ground truth label and then convert that comparison into a number.

Another thing we need is the **dot product**:

We use a vector notation

We represent an example $f_1, f_2, \ldots, f_m$ as a single vector, **x**

- j subscript will indicate feature indexing, i.e., x_j
- i subscript will indicate examples indexing over a dataset, i.e., x_i or sometimes x_i

Similarly, we can represent the weight $w_1, w_2, \ldots, w_m$ as a single vector, **w**
The dot-product between two vectors a and b is defined as:

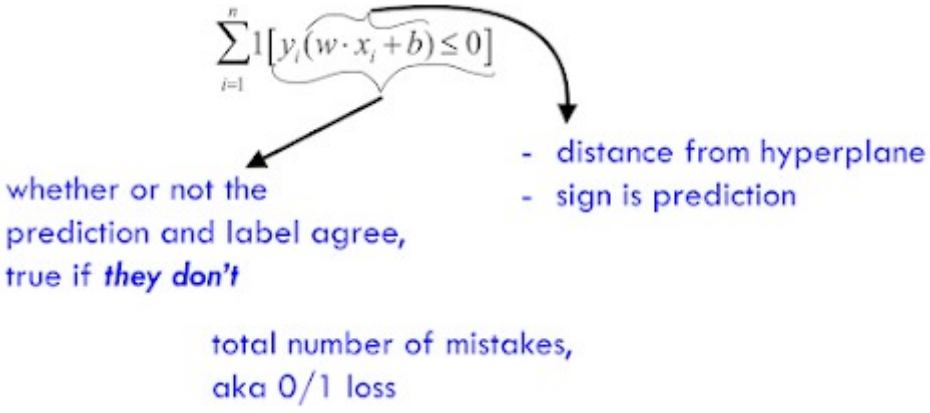
$$a \cdot b = \sum_{j=1}^{m} a_j b_j$$

An objective function for linear models, which uses the indicator function and the dot product, is:

2. Pick a criterion to optimize (aka objective function)

$$\sum_{i=1}^{n} 1[y_i(w \cdot x_i + b) \leq 0]$$

We compute the dot product between the weights and the features of an example, add the bias, multiply it by the ground truth, and if the product is less than or equal to 0, it means that the prediction is wrong, so the indicator function returns 1, increasing the sum. This is done for each example.
In this way, minimizing the objective function means minimizing the cases in which the prediction is wrong with respect to the ground truth.



This objective function is called **0/1 loss**, and it counts the number of mistakes.

Lastly, we have to develop a learning algorithm to find the weights and the bias that minimize the 0/1 loss (the training error):

3. Develop a learning algorithm

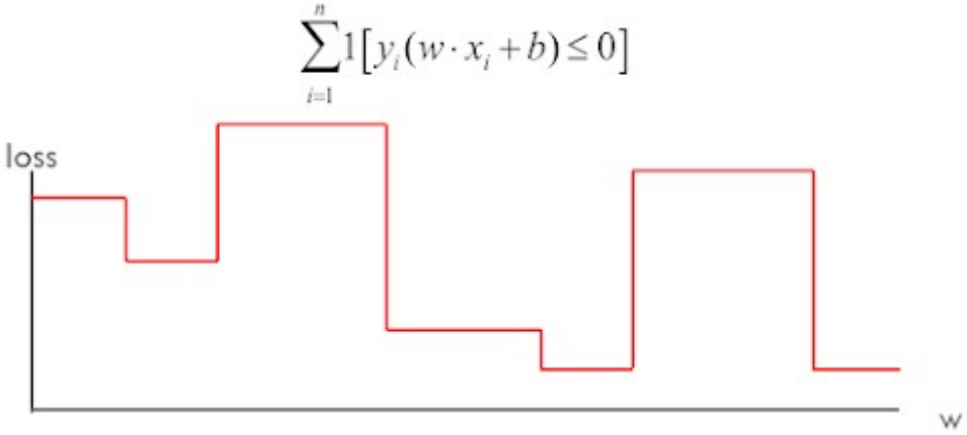
$$\text{argmin}_{w,b} \sum_{i=1}^{n} 1\left[y_i(w \cdot x_i + b) \leq 0\right]$$

The problem is that it is very difficult to develop an algorithm that minimizes the 0/1 loss exactly because 0/1 loss is a complicated function.
Therefore, we need a simpler function to work with.

Loss functions

Now we are going to see why we can't operate directly with the 0/1 loss and why we need a proxy for it.

Let's look at a graphical representation of the 0/1 loss in only one dimension (with only one feature):



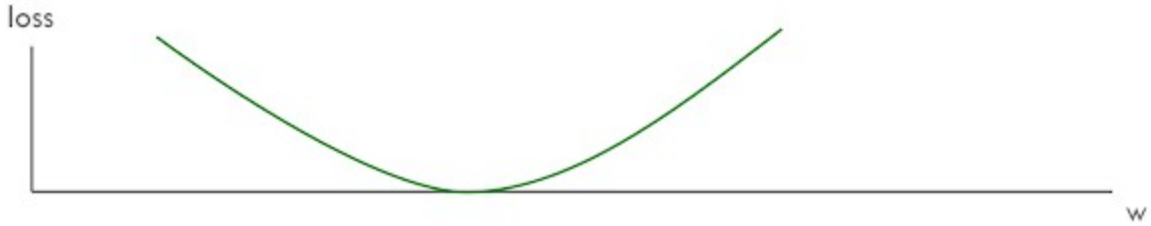
This function has a very erratic behavior. In fact, each time we make a small change in w such that the example is right/wrong, the loss will increase/decrease significantly.
Additionally, each new feature we add (i.e., weights) adds another dimension to this space!

Minimizing this function turns out to be challenging (in fact, it is an NP-HARD problem). The challenges include:

- Small changes in any w can result in large changes in the loss (the change isn't continuous).
- There can be many local minima.
- At any given point, we don't have much information to guide us towards any minima.

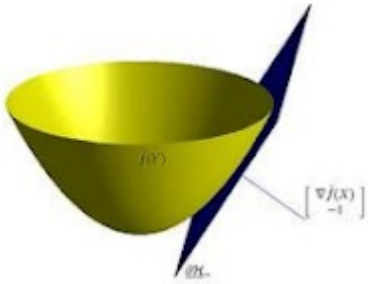
We desire **more manageable loss functions**, which ideally:

- Are continuous (i.e., differentiable) to provide direction for minimization.
- Have only one minimum.



Convex functions are those in which the line segment between any two points on the graph of the function lies above the graph itself:





In a convex function, we can compute the minimum very easily, and we are guaranteed to obtain the global minimum.
For many applications, we need to minimize the 0/1 loss.
A **surrogate loss function** is one that provides an upper bound on the ideal loss function (in this case, 0/1).
We strive to identify **convex surrogate loss functions** to simplify the minimization process.

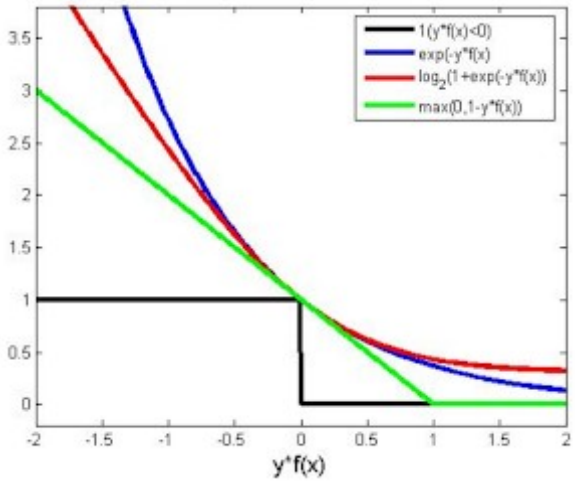
Key to a loss function: how it quantifies the difference between the actual label y and the predicted label y'.
Now, let's analyze different surrogate loss functions for the 0/1 loss, focusing on a single example (excluding the summation) to facilitate graphical visualization:

0/1 loss: $l(y,y') = 1[yy' \le 0]$

Hinge: $l(y,y') = \max(0, 1 - yy')$

Exponential: $l(y,y') = \exp(-yy')$

Squared loss: $l(y,y') = (y - y')^2$



The image displays logistic loss instead of squared loss because squared loss is more commonly used for regression problems rather than classification ones.

Each surrogate loss function has its advantages and disadvantages, and the choice depends on the task. Indeed, for complex problems, the design of the loss function is an area where researchers have invested years of effort.

In summary, optimizing a surrogate loss function is expected to lead to improvements in the primary loss function as well. This is because the surrogate is crafted to closely approximate the primary loss function (upper bound), and enhancements in the surrogate should directly translate to enhancements in the primary loss function.

Ultimately, instead of selecting 0/1 loss as the criteria to optimize, we opt for a convex surrogate loss function, such as the exponential:

1. pick a model

$$0 = b + \sum_{j=1}^{m} w_j f_j$$

2. pick a criteria to optimize (aka objective function)

$$\sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b))$$

use a convex surrogate loss function

3. develop a learning algorithm

$$\text{argmin}_{w,b} \sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b))$$

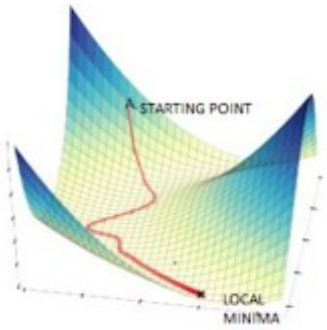
Find w and b that minimize the surrogate loss

This makes it a lot easier to develop an algorithm to compute the minimization of the objective function.

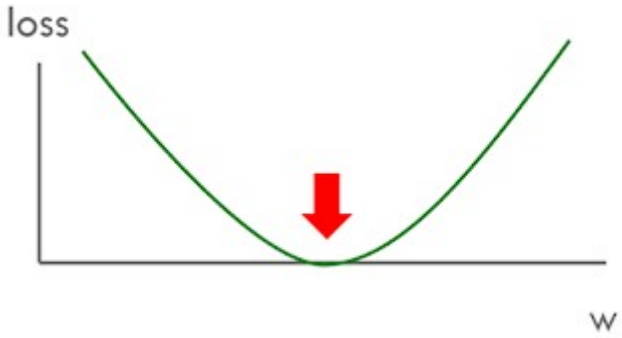
Gradient descent

An algorithm/procedure that can effectively compute the minimization of the surrogate loss function is **gradient descent**. It's a significant topic as it's also extensively utilized in neural networks.

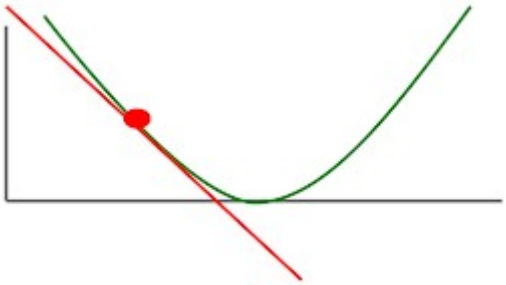
In essence, gradient descent is a procedure that starts with some initial values of weights and bias (typically chosen randomly) and iteratively guides you toward the parameter values that minimize the objective function.



Assuming we're dealing with a convex function, we can visualize it like this:

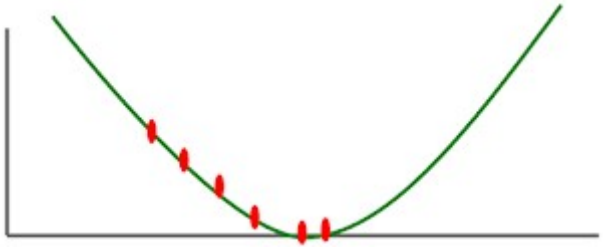


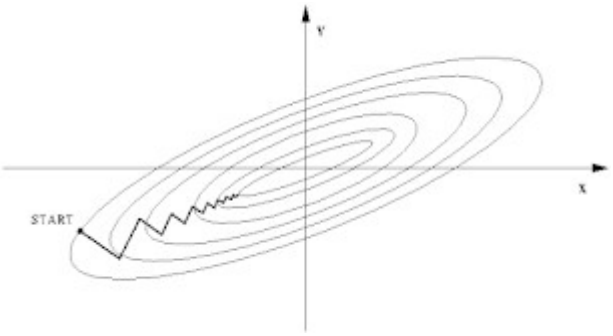
To find the minimum of such a function, gradient descent utilizes **partial derivatives**, which give us the slope (i.e., direction to move) in each dimension.



In more detail, the approach involves:

- Choosing a starting point (w)
- Iteratively:
    - Selecting a dimension w_j
    - Moving a small amount in that dimension towards decreasing loss (using the derivative)





The process of moving a small amount in a dimension towards decreasing loss is practically implemented as follows:

$$w_j = w_j - \frac{d}{d w_j} loss(w)$$

We'll later explore why the derivative is subtracted and not added.

We can also control how much we want to move in the direction of minimizing the error using the **learning rate** η:

$$w_j = w_j - \eta \frac{d}{d w_j} loss(w)$$

It's advisable to dynamically adjust this parameter over time because, initially, since the parameters are randomly initialized, you'll be quite distant from the solution. As you approach the minimum, it's logical to decrease this value to enhance precision.
The higher the learning rate, the larger the steps will be. The learning rate is an extremely sensitive hyperparameter; in large neural networks, an incorrect value can significantly disrupt the entire training process.

This mechanism is very general; as long as it's possible to compute the derivative of the loss with respect to all the parameters of the model, then this approach can be applied (in fact, it's used in neural networks).

Let’s focus on our loss, which is the exponential surrogate function and is computed across all examples:

$$\frac{d}{dw_j} loss = \frac{d}{dw_j} \sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b))$$

$$= \sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b)) \frac{d}{dw_j} - y_i(w \cdot x_i + b)$$

Let’s break it down:

- The derivative is applied inside the summation.

- The derivative of the exponential function is the exponential itself multiplied by the derivative of its argument.

Then, the yellow part is resolved as follows:

$$\frac{d}{d w_j} - y_i \left(w \cdot x_i + b \right)$$

$$= - \frac{d}{d w_j} y_i \left(w \cdot x_i + b \right)$$

$$= - \frac{d}{d w_j} y_i \left(\sum_{j=1}^{m} w_j \cdot x_{ij} + b \right)$$

$$= - \frac{d}{d w_j} y_i \left(w_1 \cdot x_{i1} + w_2 \cdot x_{i2} + \ldots + w_m \cdot x_{\mathrm{im}} + b \right)$$

$$= - \frac{d}{d w_j} \left(y_i w_1 \cdot x_{i1} + y_i w_2 \cdot x_{i2} + \ldots + y_i w_m \cdot x_{\mathrm{im}} + y_i b \right)$$

$$= - y_i x_{ij}$$

In other words, only the term of feature j will remain, and its derivative is $- y_i x_{ij}$.

So, we obtain:

$$\frac{d}{dw_j} loss = \frac{d}{dw_j} \sum_{i=1}^{n} \exp(-y_i (w \cdot x_i + b))$$

$$= \sum_{i=1}^{n} \exp(-y_i (w \cdot x_i + b)) \frac{d}{dw_j} - y_i (w \cdot x_i + b)$$

$$= \sum_{i=1}^{n} -y_i x_{ij} \exp(-y_i (w \cdot x_i + b))$$

For our chosen loss, we have:

$$w_j = w_j - \eta \frac{d}{dw_i} loss(w)$$

$$w_j = w_j + \eta \sum_{i=1}^{n} y_i x_{ij} \exp(-y_i (w \cdot x_i + b))$$

This is known as the **exponential update rule**.

So, for each example x_i:

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i (w \cdot x_i + b))$$

Upon observing this update rule, we notice its similarity to that of the perceptron, as the update depends on the label and the current features:

repeat until convergence (or for some # of iterations):
  for each training example (f_1, f_2, ..., f_n, label):
    $prediction = b + \sum_{i=1}^{n} w_i f_i$
    if *prediction * label* ≤ 0: // they don't agree
      for each w_i:
        $w_i = w_i + f_i$*label
      $b = b$ + label

The difference lies in the additional multiplication by a factor c:

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i(w \cdot x_i + b))$$

In practice $w_j = w_j + x_{ij} y_i c$ where $c = \eta \exp(-y_i(w \cdot x_i + b))$

In gradient descent, I have a bit more control compared to the perceptron because I have the parameter c. This parameter depends on the learning rate, the label, and the prediction:

$$c = \eta \exp(-y_i(w \cdot x_i + b))$$

learning rate label prediction

There are two major cases:

- If the label and the prediction have the same sign, as the prediction grows larger, the update becomes smaller.
- If the label and the prediction are different, the more different they are, the larger the update.

And that's why this approach is much more sophisticated than the perceptron.

An important distinction to highlight is that in the perceptron algorithm, we only update the weights if the prediction is wrong, whereas in gradient descent, we always update.

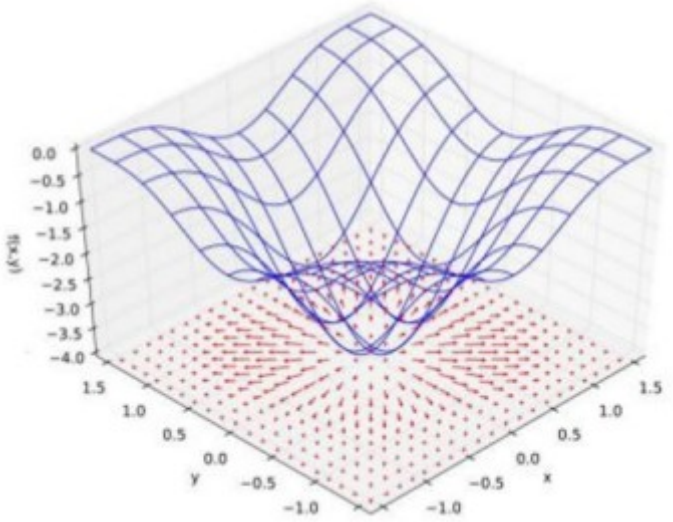
To better understand why there is a minus sign in the formula, let's delve a bit deeper into gradients:

- The gradient is the vector of partial derivatives with respect to all the components of the weights:

  $$\nabla_{\mathbf{w}} L = \left[\frac{\partial L}{\partial w_1} \; \frac{\partial L}{\partial w_2} \cdots \frac{\partial L}{\partial w_N} \right]$$

- Each partial derivative measures **how quickly the loss changes in one direction**.

- When the gradient is zero, i.e., all the partial derivatives are zero, the loss is not changing in any direction.



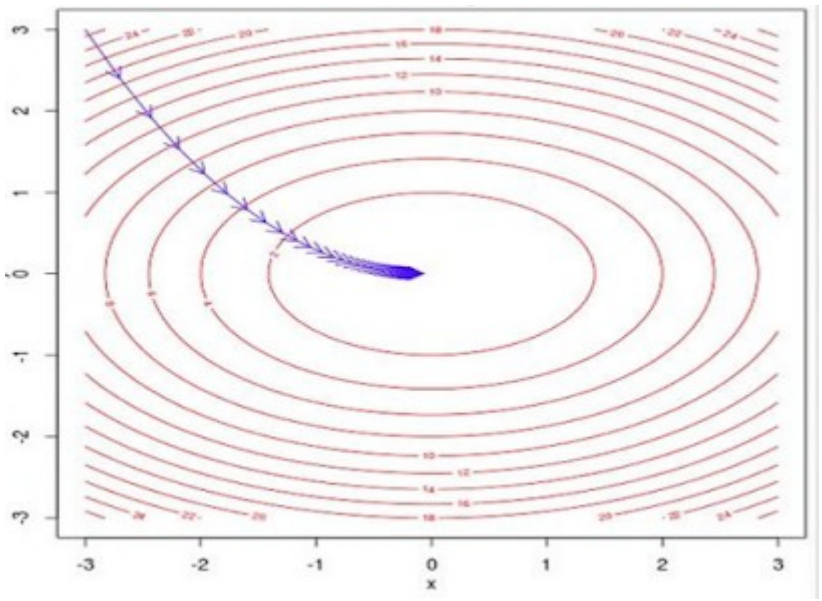
The arrows (gradients) extend outward from a minimum toward a maximum, and that's why we include the minus sign, as we aim to move from a maximum to a minimum.

The formal procedure for gradient descent is as follows:

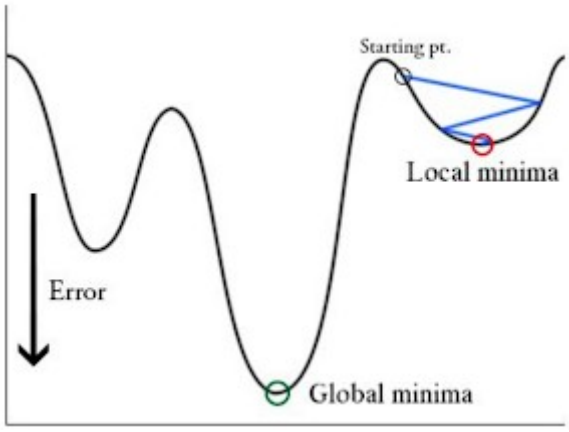
Algorithm 21 GRADIENTDESCENT($\mathcal{F}$, K, η_1, ...)

1. $z^{(0)} \leftarrow \langle o, o, \ldots, o \rangle$ // initialize variable we are optimizing
2. **for** $k = 1 \ldots K$ **do**
3. $\quad g^{(k)} \leftarrow \nabla_z \mathcal{F}|_{z^{(k-1)}}$ // compute gradient at current location
4. $\quad z^{(k)} \leftarrow z^{(k-1)} - \eta^{(k)} g^{(k)}$ // take a step down the gradient
5. **end for**
6. **return** $z^{(K)}$

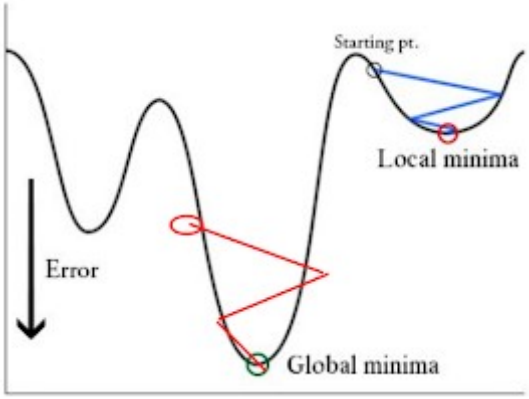
Here's a two-dimensional illustration featuring a convex function:

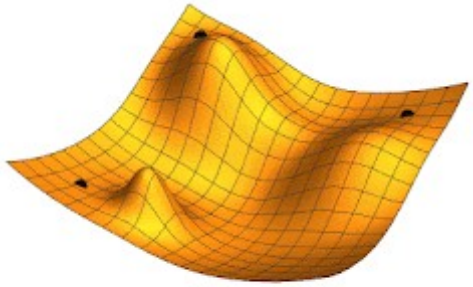


Let's discuss a final point about gradient descent.
In problems where the optimization problem is non-convex, gradient descent may likely converge to **local minima**. An example is neural networks. In fact, for a long time, people avoided using neural networks because they preferred methods that guaranteed finding the optimal solution.

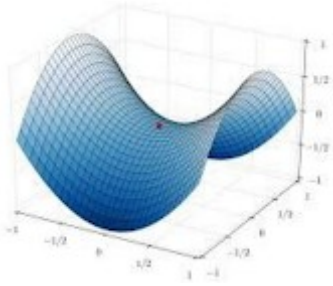


In such cases, the starting point significantly influences the solution's quality, leading to techniques to determine the best starting point.





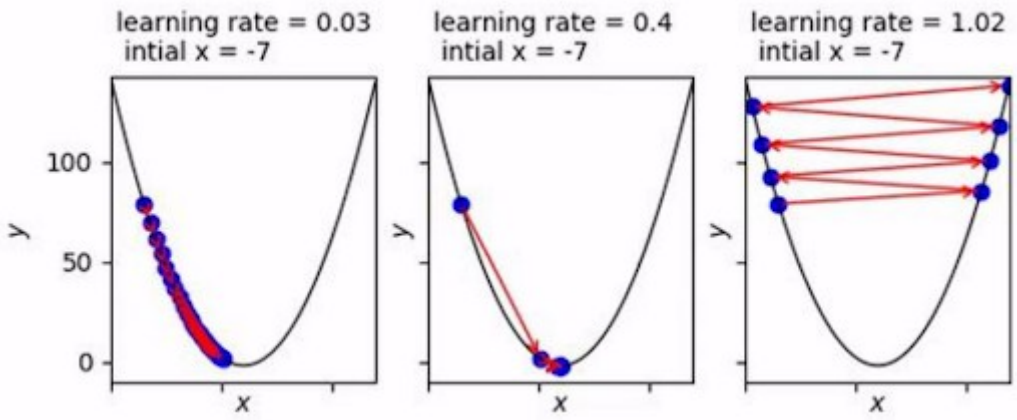
When discussing gradient descent more broadly, beyond convex functions, an important concept is the **saddle point**.
Saddle points, prevalent in **high-dimensional spaces**, pose challenges in optimization landscapes. They occur where certain directions exhibit upward curvature while others display downward curvature. At these points, the gradient vanishes (is 0), although it's not at a minimum, potentially causing stagnation in optimization processes. However, even when seemingly trapped at a saddle point, a slight deviation from its exact location can facilitate movement towards more favorable regions, highlighting the dynamic nature of optimization in navigating such complex terrains.



In linear problems and the examples we've explored so far, the number of parameters is the same as the number of features. However, in neural networks, the number of parameters is significantly higher, and we encounter a highly non-convex objective function, increasing the likelihood of getting stuck in a saddle point.

As mentioned earlier, the learning rate is a crucial hyperparameter. Choosing a "decent" value allows approaching the minimum with fewer iterations:

- A too small value of the learning rate prolongs reaching the minimum.
- A too high value of the learning rate may prevent reaching the minimum altogether.



In summary:

- Model-based machine learning involves:
    - Defining a model
    - Objective function (i.e., loss function)
    - Minimization algorithm
- Gradient descent minimization algorithm:
    - So far, we've considered cases where the loss function is convex; otherwise, we could encounter issues.
    - It involves making small updates towards lower losses.
- Perceptron learning algorithm and gradient descent/exponential loss function (adjusted by a learning rate)
- Gradient descent in general

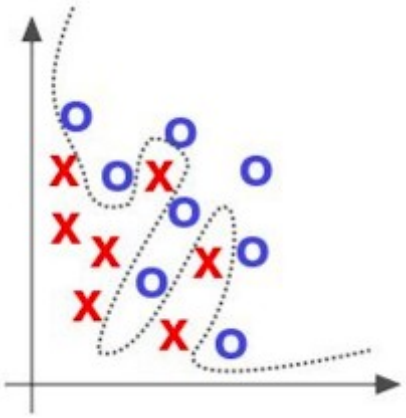
In this chapter, we have grasped firsthand the relationship between machine learning and optimization. Indeed, many machine learning algorithms rely on principles studied in optimization theory.

Regularization

We've already introduced the concept of **regularization**; now let's delve deeper into it.

Avoid Overfitting

One of the most significant risks in machine learning is **overfitting**, where the model learns too much from the training dataset and fails to generalize to unseen data.



Consider the objective function we discussed in the previous chapter:

$$\text{argmin}_{w,b} \sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b))$$

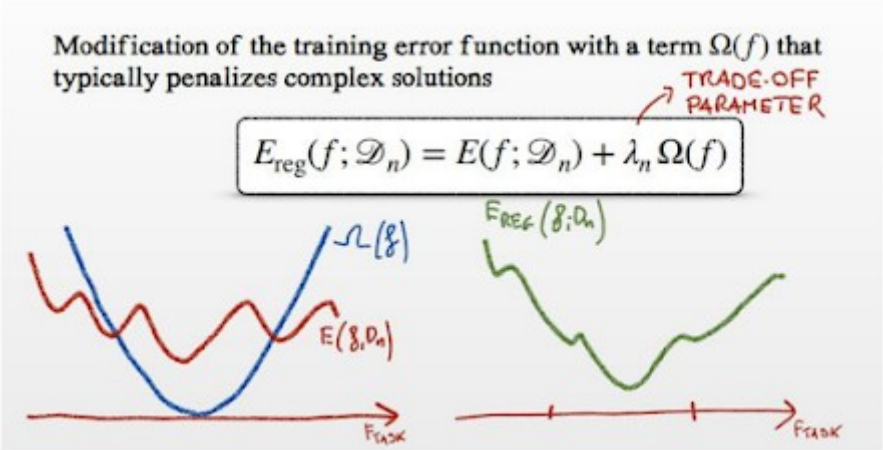
The choice of weights and bias depends on the **training dataset** because we're optimizing them to minimize the error on that specific dataset. Since we only have access to the training dataset during training, we must ensure that our model generalizes well to unseen data. Therefore, we need to be cautious of **overfitting**: minimizing the loss on the training set does not necessarily guarantee minimal loss on the test set.

This is where **regularization** comes in.
A **regularizer** is an additional criterion added to the loss function to prevent overfitting (although it doesn't completely eliminate it; rather, it mitigates the issue). It's termed a regularizer because it aims to maintain the parameters more normal or regular, acting as a bias on the model that encourages certain types of weights over others.

$$\text{argmin}_{w,b} \sum_{i=1}^{n} loss(yy') + \lambda \ regularizer(w,b)$$

The regularizer is a function solely of the parameters. The hyperparameter λ regulates the tradeoff between minimizing the training set loss and minimizing the regularizer.

This is the term we've already seen which penalizes complex solutions:



Regularizers

The concept behind regularization is to limit the range of possible weights to simplify the final model.
Typically, we aim to prevent excessively large weights within our model. This is because when weights are magnified, even a slight change in a feature could lead to a significant shift in prediction, potentially introducing instability or overfitting. Additionally, there's often a preference for assigning weights of 0 to features deemed irrelevant or non-contributory to the prediction task. This not only simplifies the model but also enhances its interpretability by effectively disregarding extraneous variables that offer little or no predictive value.

Let’s explore how we can practically encourage small weights while penalizing large weights.

$$\text{argmin}_{w,b} \sum_{i=1}^{n} loss(yy') + \lambda\ regularizer(w,b)$$

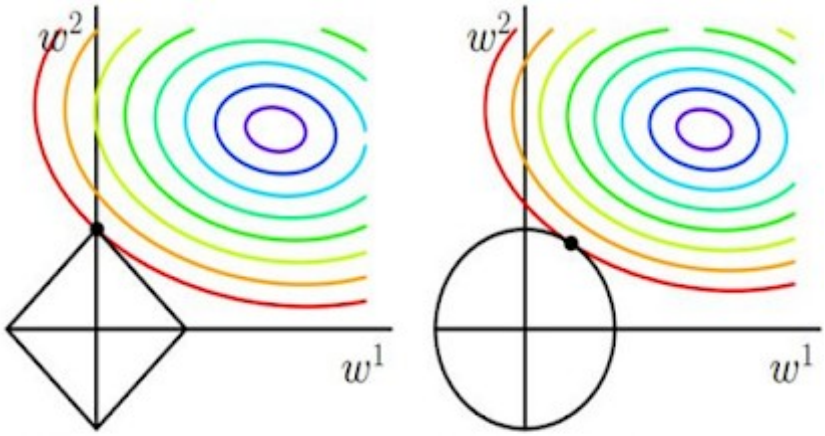
The first method is called **L1-regularization** (the sum of the absolute values of the weights), and the second is called **L2-regularization** (the square root of the sum of the squares of the weights). These are the two most common regularizers. The difference between them lies in:

- L1 regularization penalizes small values more.
- L2 regularization penalizes large values more.

sum of the weights (1-norm) $r(w,b) = \sum |w_j|$

sum of the squared weights (2-norm) $r(w,b) = \sqrt{\sum |w_j|^2}$

The illustration of them is as follows:



(a) ℓ_1-ball meets quadratic function. ℓ_1-ball has corners. It's very likely that the meet-point is at one of the corners.

(b) ℓ_2-ball meets quadratic function. ℓ_2-ball has no corner. It is very unlikely that the meet-point is on any of axes.

The colored plots/circles correspond to the objective function (a convex function).
Without the regularizer, the algorithm will end inside the purple circle in both cases. However, with the regularizer, the algorithm will end at a tradeoff represented by the red circle.
We can observe that in the L1 regularization, one of the two coordinates is exactly 0, and in the L2 regularization, the weights are much smaller than the purple circle.

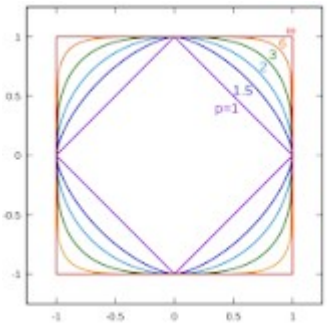
Considering the p-norm:

p-norm $$r(w,b) = \sqrt[p]{\sum |w_j|^p} = \|w\|^p$$

We can note that:

- Smaller values of p ($p<2$) encourage sparser vectors (vectors with many 0 values).
- Larger values of p discourage large weights more and tend to favor similar weights.

The illustration of p-norm is depicted here:



all p-norms penalize larger weights

p < 2 tends to create sparse (i.e. lots of 0 weights)

p > 2 tends to like similar weights

So our procedure will have this slight adjustment:

1. pick a model

$$0 = b + \sum_{j=1}^{n} w_j f_j$$

2. pick a criteria to optimize (aka objective function)

$$\sum_{i=1}^{n} loss(yy') + \lambda regularizer(w)$$

3. develop a learning algorithm

$$\text{argmin}_{w,b} \sum_{i=1}^{n} loss(yy') + \lambda regularizer(w)$$

Find w and b that minimize

λ is our choice:

- If $\lambda = 0$, we revert to the previous situation.
- If λ is higher, it indicates that the regularizer is more aggressive, and we place more trust in the regularizer.

Like all other hyperparameters, there's no fixed rule for choosing the 'correct' λ; it often involves trial and error, evaluating the performance on the validation set.

Now, let's discuss the minimization of this new objective function.
We already know how to solve convex minimization problems using gradient descent:

$$\text{argmin}_{w,b} \sum_{i=1}^{n} loss(yy')$$

If we can ensure that the loss function + regularizer is convex, then we can still use gradient descent:

$$\text{argmin}_{w,b} \sum_{i=1}^{n} loss(yy') + \lambda regularizer(w)$$

make convex

Remembering that a convex function is one where the line segment between any two points on the function lies above or on the function. A more formal definition of a convex function is:

f is convex if for all x_1, x_2:

$$f\left(t\,x_1 + (1-t)\,x_2\right) \le t f\left(x_1\right) + (1-t) f\left(x_2\right) \qquad \forall\, 0 < t < 1$$

This means that the value of f at any point on the line segment between x_1 and x_2 is less than or equal to the linear interpolation between $f\left(x_1\right)$ and $f\left(x_2\right)$.

If both the regularizer and loss are convex, then their sum is also convex.

$$\text{argmin}_{w,b} \sum_{i=1}^{n} loss(yy') + \lambda regularizer(w)$$

convex as long as both loss and regularizer are convex

An important point to note is that p-norm regularizers are convex for $p \geq 1$.

$$r(w,b) = \sqrt[p]{\sum |w_j|^p} = \|w\|^p$$

So we can select the L2-norm regularizer and continue utilizing gradient descent to minimize the objective function with the regularizer.

1. Pick a model

$$0 = b + \sum_{j=1}^{n} w_j f_j$$

2. Pick a criteria to optimize (aka objective function)

$$\sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b)) + \frac{\lambda}{2}\|w\|^2$$

3. Develop a learning algorithm

$$\text{argmin}_{w,b} \sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b)) + \frac{\lambda}{2}\|w\|^2$$

Find w and b that minimize

The $\frac{1}{2}$ in the regularizer is added for convenience, but it doesn't affect the minimum, as it's just a constant.

In summary:

- The loss function penalizes examples where the prediction differs from the label.
- The regularizer penalizes large weights.

The objective function above is convex, allowing us to utilize gradient descent. Thus, we can apply the same algorithm we've seen in the previous chapter:

- pick a starting point (w)
- repeat until loss doesn't decrease in any dimension:
  - pick a dimension
  - move a small amount in that dimension towards decreasing loss (using the derivative)

$$w_j = w_j - \eta \frac{d}{dw_j}(loss(w) + regularizer(w,b))$$

$$\text{argmin}_{w,b} \sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b)) + \frac{\lambda}{2}\|w\|^2$$

In the case of L2-norm regularizer, we obtain this:

$$\frac{d}{dw_j} objective = \frac{d}{dw_j} \sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b)) + \frac{\lambda}{2}\|w\|^2$$

$\vdots$ (some math happens)

$$= -\sum_{i=1}^{n} y_i x_{ij} \exp(-y_i(w \cdot x_i + b)) + \lambda w_j$$

So, in addition to the derivative we've seen before, we introduce another term:

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i(w \cdot x_i + b)) - \eta \lambda w_j$$

learning rate — direction to update — constant: how far from wrong — regularization

The regularizer has the following effect:

- If w_j is positive, it reduces w_j.
- If w_j is negative, it increases w_j.

In practice, this means that the regularizer moves w_j towards 0.

In the case of L1-norm regularizer, we obtain this:

$$\text{argmin}_{w,b} \sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b)) + \|w\|$$

$$\frac{d}{dw_j} \sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b)) + \lambda \|w\|$$

$$= -\sum_{i=1}^{n} y_i x_{ij} \exp(-y_i(w \cdot x_i + b)) + \lambda sign(w_j)$$

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i(w \cdot x_i + b)) - \eta\lambda sign(w_j)$$

learning rate

direction to update

constant: how far from wrong

regularization

This regularizer has the same effect as before, but the increments/reductions are constant because the regularizer isn't a function of the weight's magnitude, only of its sign. Thus, the regularizer moves w_j towards 0, regardless of its magnitude.

For any "instantiation" of the p-norms, it's possible to derive an update rule:

L1:

$$w_j = w_j + \eta(loss_correction - \lambda sign(w_j))$$

L2: $w_j = w_j + \eta(loss_correction - \lambda w_j)$

Lp: $w_j = w_j + \eta(loss_correction - \lambda c w_j^{p-1})$

It's important to note that gradient descent isn't the only method to find w and b. There are many other approaches to find the minimum, some of which don't even require iteration. The entire field is known as convex optimization.
For example, replacing the exponential loss with the hinge loss (used in SVM) requires a different method, although gradient descent could still be used.

In summary:

- L1 regularization is popular because it tends to result in sparse solutions, meaning that many weights are reduced to zero, which simplifies the model. However, the L1 penalty function is not differentiable at zero weights, meaning it lacks a well-defined derivative at those points. Despite this, it can still be used with gradient descent methods, which are capable of handling functions that are not entirely differentiable by employing techniques that account for this characteristic and finding optimal solutions.
- L2 regularization is also popular because for some loss functions, it can be solved directly without requiring gradient descent, although iterative solvers are often still used.
- Lp regularization is less popular because it does not tend to shrink the weights enough.

Without regularization, the generic update is:

$$w_j = w_j + \eta\, y_i\, x_{ij}\, c$$

where

$c = \exp(-y_i(w \cdot x_i + b))$ exponential

$c = 1[yy' < 1]$ hinge loss

In fact, many other loss functions have this update rule. The squared error, despite being similar, is a little different:

$w_j = w_j + \eta(y_i - (w \cdot x_i + b)x_{ij})$ squared error

There exist many methods that work with specific loss functions and specific regularizers:

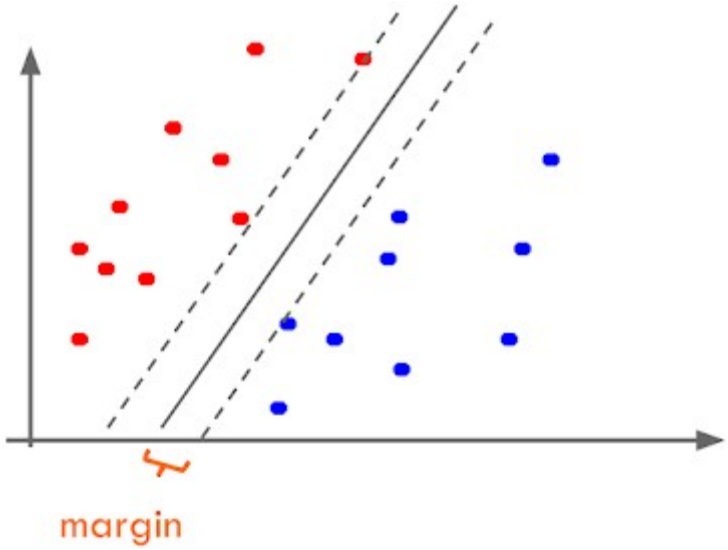
MANY METHODS

- (Ordinary) Least squares: squared loss
- Ridge regression: squared loss with L2 regularization
- Lasso regression: squared loss with L1 regularization
- Elastic regression: squared loss with L1 AND L2 regularization
- Logistic regression: logistic loss

Support Vector Machines

With SVMs, we're still interested in learning a linear model. The reason we continue to use linear models is because they offer a convenient representation, involving learning a number of parameters equal to the number of features. Thus, if you don't have many features, you'll end up with a very compact model, which is crucial in terms of time during the inference phase.

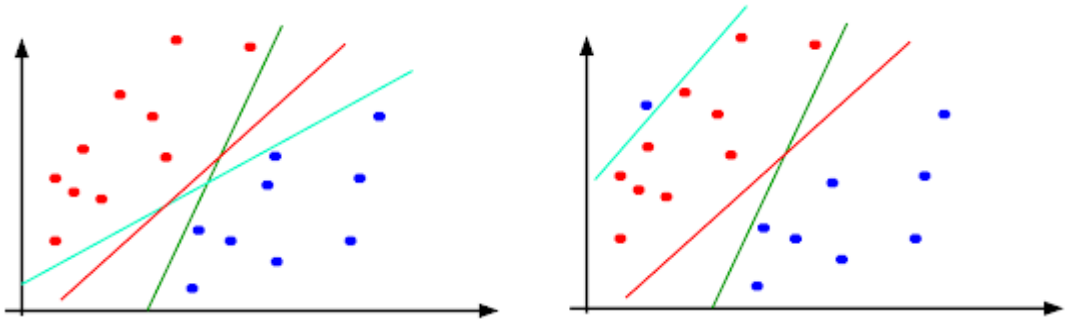
The idea of SVM is illustrated in the following picture:



In contrast to what we've seen before, with SVM, we're interested in learning **a specific linear model**: one that **maximizes the margin**.

There are two main aspects in linear classifiers:

- Which hyperplane they choose when the data is linearly separable.
- How they handle data that is not linearly separable.



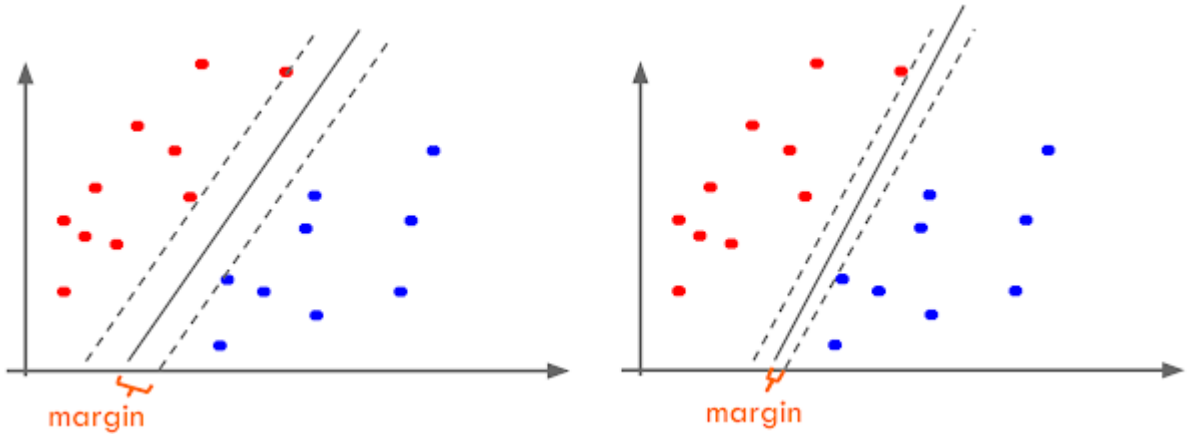
In the perceptron, we've observed that:

- We don't know a priori which linear model the perceptron will end up with; we don't require any specific characteristics from the model, we only seek one linear model. The perceptron finds **some** hyperplane that separates the data.
- We can handle non-linearly separable data by imposing a constraint on the maximum number of iterations.

The perceptron will continue to adjust as it iterates through the examples, and the final hyperplane will depend on which examples it saw recently.

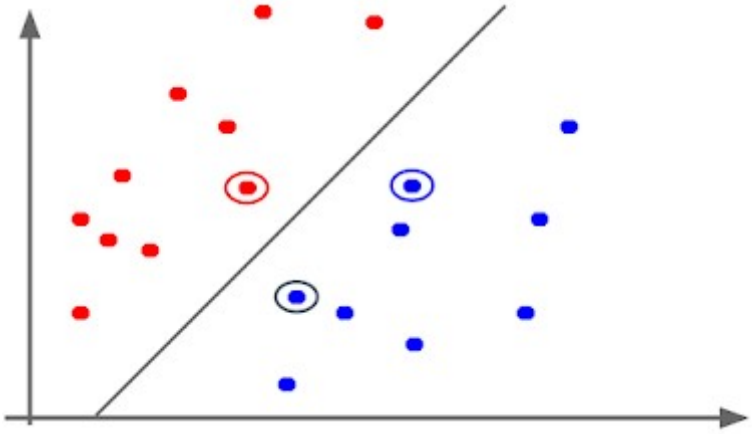
With SVM, we can specify some constraints to be more specific about which linear model we want to obtain.

Let's introduce **large margin classifiers**.
The **margin** of a classifier is the distance to the closest points of either class. **Large margin classifiers** attempt to maximize this.



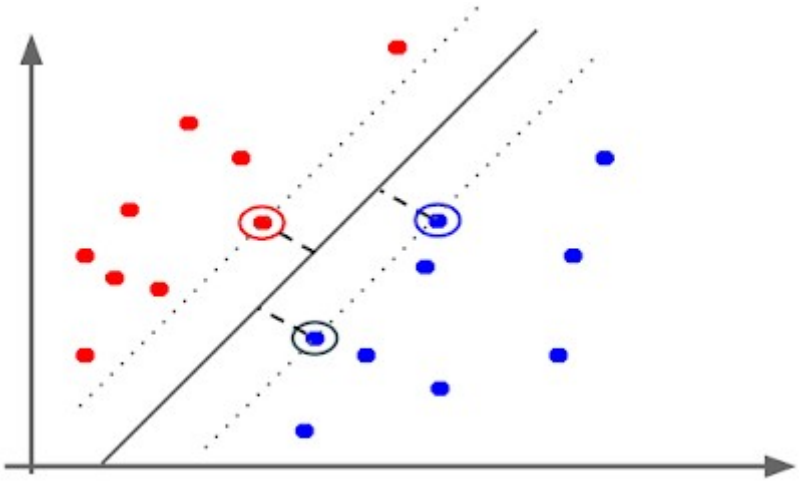
The idea of learning a linear model that maximizes the margin is that the hyperplane built in this manner possesses greater generalization capabilities. In fact, a larger margin implies a greater separation between classes, leading to a more robust and generalizable model.

For any separating hyperplane, there exists some set of "closest points"; these are called the **support vectors**. These are "special" samples to which we direct our attention because they have practical relevance.
For n dimensions, there will be at least $n+1$ support vectors.

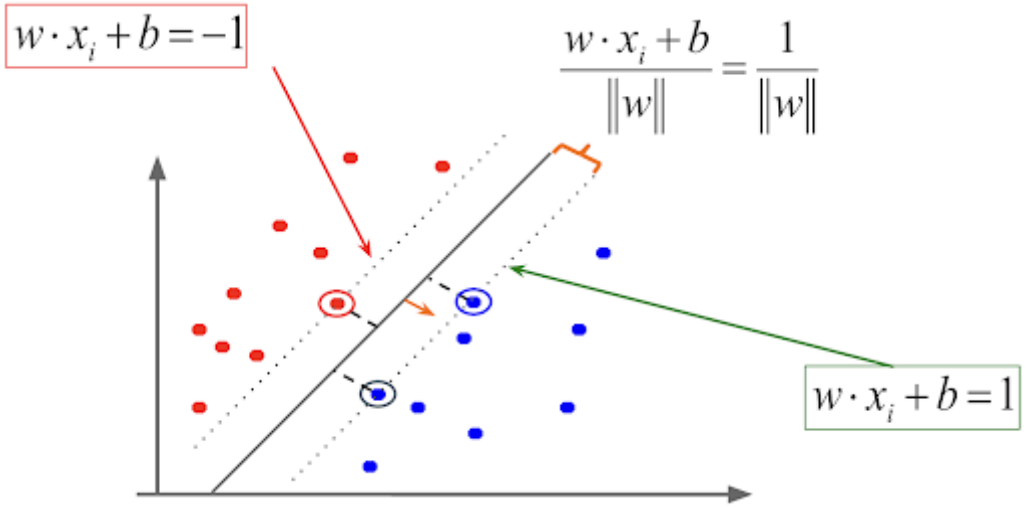


Maximizing the margin is good because it implies that **only support vectors matter**; other training examples are ignorable (and with millions of data points, this is an enormous advantage).
This is an anticipation of why the support vectors are important.

So the margin is the distance to the support vectors, i.e., the “closest points”, on either side of the hyperplane.



Let’s see how we can calculate the value of the margin (motivation for the formulas: Link1 Link2).



If our hyperplane decision equation is $w \cdot x_i + b = 0$, then, up to some normalization which depends on the L2-norm of the weights, the equations for the other hyperplanes are $w \cdot x_i + b = -1$ and $w \cdot x_i + b = 1$. In this case, the margin can be written as:

$$m = \frac{|w \cdot x_i + b|}{\|w\|} = \frac{1}{\|w\|}$$

So, I’m interested in finding $max_{w,b} margin(w, b)$, which is the objective function.

Obviously, I’m not interested only in maximizing the margin, but also in correctly classifying all the points, so I add this constraint:

$$\max_{w,b} \quad \text{margin}(w,b)$$
subject to:
$$y_i(w \cdot x_i + b) \geq 1 \quad \forall i$$

This constraint ensures that the data points are linearly separable by our hyperplane. It also ensures that all the data points are outside the margin. Thus, our problem becomes a **constrained optimization problem**.

Substituting with the value of the margin, we obtain:

$$\max_{w,b} \quad \frac{1}{\|w\|}$$
subject to:
$$y_i(w \cdot x_i + b) \geq 1 \quad \forall i$$

But this isn't convenient to compute directly, so it is typically preferred to compute this:

$$\min_{w,b} \quad \|w\|$$
subject to:
$$y_i(w \cdot x_i + b) \geq 1 \quad \forall i$$

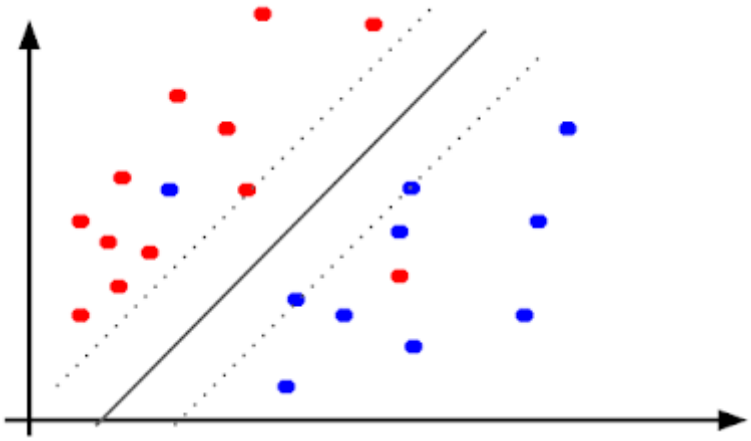
Because maximizing the margin is equivalent to minimizing the norm of the weights (subject to separating constraints). This has another advantage since minimizing $\|w\|$ is very beneficial because it also regularizes the solution.

To make it even easier to calculate, we can consider the square of the function, since it's simpler to find the minimum for quadratic functions:

$$\min_{w,b} \quad \|w\|^2$$
subject to:
$$y_i(w \cdot x_i + b) \geq 1 \quad \forall i$$

This is called the primal optimization problem for SVM, and this is a version of a **quadratic optimization problem**: maximize/minimize a quadratic function subject to a set of linear constraints (because it is a linear function of the weights). Being a known and studied problem, there exist many techniques to find the values in an optimal way.

Unfortunately, this is only the beginning, because it covers only the case in which the data are effectively linearly separable. I need changes for the cases in which the data are non-linearly separable, like in this situation:



We would like to learn something like this, but our constraints do not allow it.

Soft Margin Classification

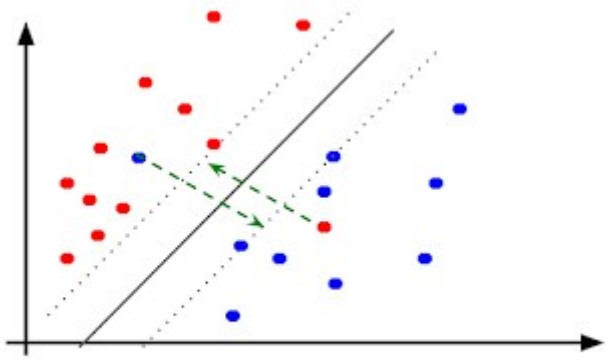
To handle these situations, we rewrite the optimization problem to tolerate this error, a concept known as **soft margin classification**. We introduce another set of variables called **slack variables**:

$$\min_{w,b} \ \|w\|^2$$
subject to:
$$y_i(w \cdot x_i + b) \geq 1 \quad \forall i$$

$$\min_{w,b} \ \|w\|^2 + C\sum_i \varsigma_i$$
subject to:
$$y_i(w \cdot x_i + b) \geq 1 - \varsigma_i \quad \forall i$$
$$\varsigma_i \geq 0$$

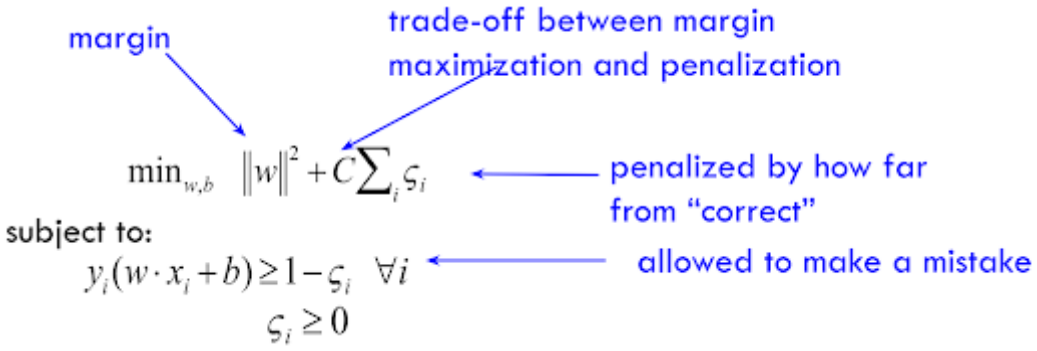
These slack variables are not included in the variables to learn because our ultimate goal is to learn the hyperplane. Instead, they serve as additional information to compute the most accurate model possible (minimizing their values). However, adding these slack variables does increase computation time.

There is one slack variable ς_i for each example, and they are all greater than or equal to zero. Practically, the slack variable indicates how far the data point is from its side of the hyperplane.



slack penalties

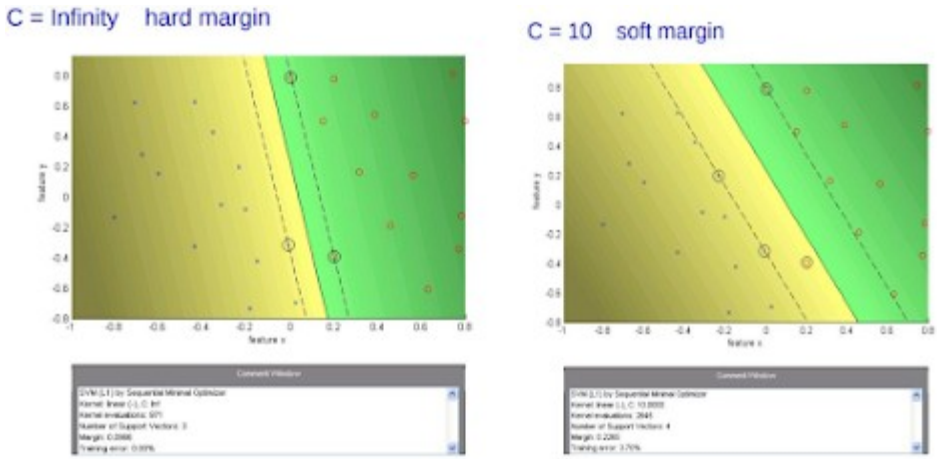
The more of these slack variables are zero, the more the dataset tends to be linearly separable. If they are all zero, then the dataset is effectively linearly separable.



Parameter C, the main hyperparameter of an SVM, defines the trade-off between margin maximization and penalization (how much we can violate the constraint).
It can be viewed as a way to **control overfitting**: it "trades off" the relative importance of maximizing the margin and fitting the training data. C is a regularization parameter:

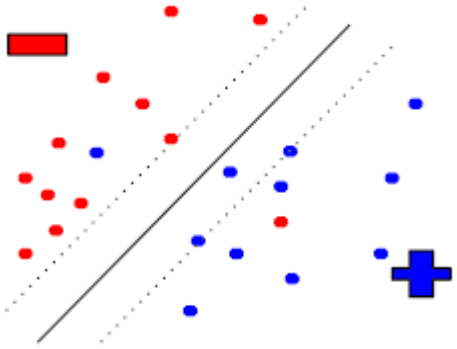
- Small C allows constraints to be easily ignored, leading to a large margin.
- Large C makes constraints hard to ignore, resulting in a narrow margin.
- $C = \infty$ enforces all constraints, resulting in a hard margin.

Here's a practical example representing the different results based on the choice of C:



It is important to note that this is still a **quadratic optimization problem** because the objective function remains quadratic and the constraints remain linear. Therefore, we can use the same solver that we use in large margin SVM.

Given the optimal solution (w, b), we can determine the slack penalties for each point.



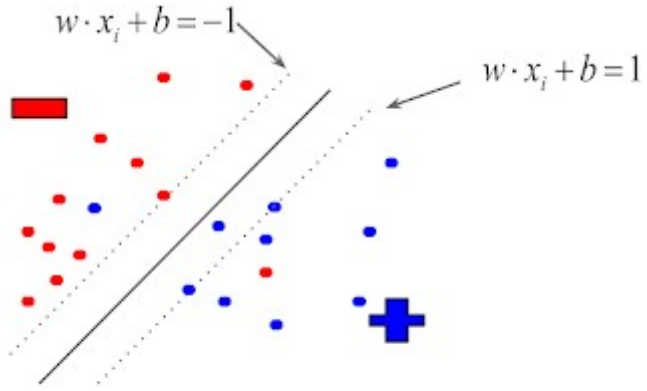
$$\min_{w,b} \quad \|w\|^2 + C\sum_i \varsigma_i$$

subject to:

$$y_i(w \cdot x_i + b) \geq 1 - \varsigma_i \quad \forall i$$

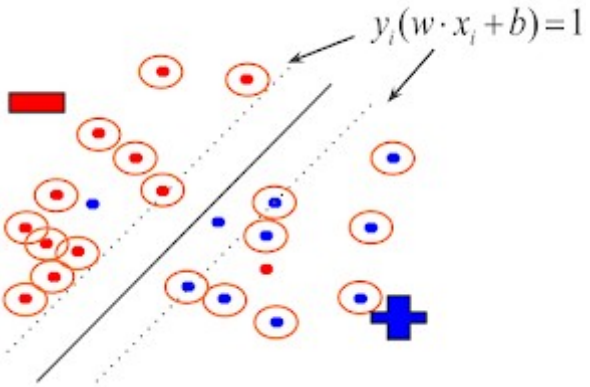
$$\varsigma_i \geq 0$$

As mentioned earlier, the two equations of the margin hyperplanes can be combined into a single equation:



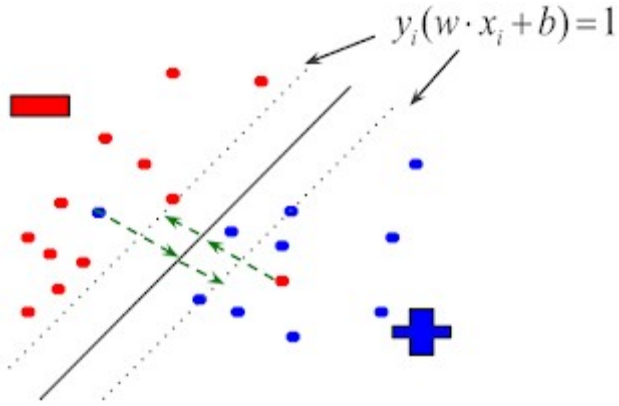
or: $\boxed{y_i(w \cdot x_i + b) = 1}$

The slack values for points outside (or on) the margin and correctly classified are 0. This is because the slack variables must be greater than or equal to zero, and if they are on or beyond the margin, then $y_i\left(x \cdot x_i + b\right) \geq 1$ already.



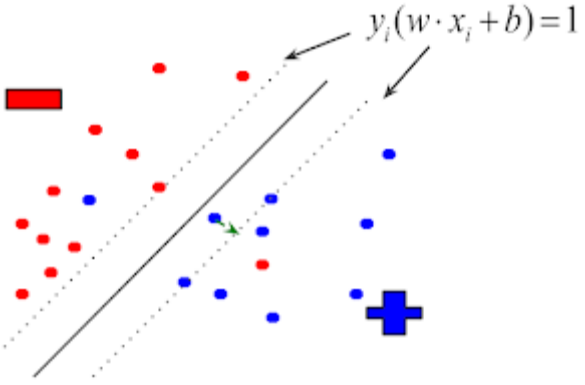
For points that are incorrectly classified, the slack value is the “distance” from the point to the hyperplane (- $y_i\left(w \cdot x_i + b\right)$) plus the “distance” from the hyperplane to the margin (which is 1).

$$\varsigma_i = 1 - y_i(w \cdot x_i + b)$$



The slack values for points inside the margin and classified correctly remain the same as before:

$$\varsigma_i = 1 - y_i(w \cdot x_i + b)$$



Combining these observations, we can rewrite the constraint as follows:

$$\varsigma_i = \begin{cases} 0 & if\ y_i(w \cdot x_i + b) \geq 1 \\ 1 - y_i(w \cdot x_i + b) & otherwise \end{cases}$$

which can be further simplified to:

$$\varsigma_i = \max(0, 1 - y_i(w \cdot x_i + b))$$
$$= \max(0, 1 - yy')$$

We can substitute this definition of ς_i into our objective function:

$$\min_{w,b} \ \|w\|^2 + C\sum_i \varsigma_i$$
subject to:
$$y_i(w \cdot x_i + b) \geq 1 - \varsigma_i \quad \forall i$$
$$\varsigma_i \geq 0$$

$$\varsigma_i = \max(0, 1 - y_i(w \cdot x_i + b))$$



$$\min_{w,b} \ \|w\|^2 + C\sum_i \max(0, 1 - y_i(w \cdot x_i + b))$$

This yields an **unconstrained problem** to solve soft margin SVMs. This means that if desired, we can solve this problem with gradient descent.

This formulation also aligns with the model-based approach we discussed earlier, incorporating an L2-norm regularizer and the sum of the loss (in this case, hinge loss):

$$\min_{w,b} \ \|w\|^2 + C\sum_i loss_{hinge}(y_i, y_i')$$

Does this look like something we have seen before?

$$\text{argmin}_{w,b} \sum_{i=1}^{n} loss(yy') + \lambda \ regularizer(w, b)$$

Where $\lambda = \frac{1}{C}$.

Additionally, we can observe that ς_i is the **hinge loss**:

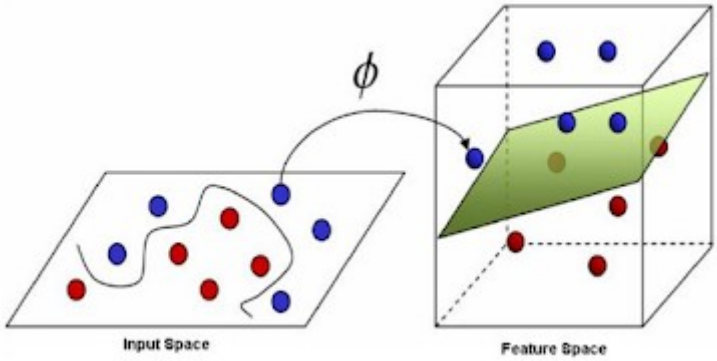
Hinge: $l(y,y') = \max(0, 1 - yy')$

Squared loss: $l(y,y') = (y - y')^2$

0/1 loss: $l(y,y') = 1[yy' \le 0]$

Non Linearly Separable Data

The advantage of SVMs lies in their ability to handle not only problems where non-linear separability is “weak” but also those where non-linear boundaries are necessary.



Therefore, SVMs gained fame for their capability to handle non-linearly separable data.

Let’s summarize the SVM **primal problem**:

$$\min_{w,b} \ \|w\|^2$$

subject to:

$$y_i(w \cdot x_i + b) \ge 1 \quad \forall i$$

This is a version of a **quadratic optimization problem**

Maximize/minimize a quadratic function subject to a set of linear constraints

This is typically referred as **primal problem**

In optimization theory, we deal with various classes of **optimization problems**:

Linear programming (LP): linear problem, linear constraints

$$\min_{\mathbf{x}} \quad \mathbf{c}^T\mathbf{x}$$
$$\text{s.t.} \quad \mathbf{A}\mathbf{x} = \mathbf{b}, \quad \mathbf{x} \geq 0$$

Quadratic programming (QP): quadratic objective and linear constraints, it is convex if Q is positive semidefinite

$$\min_{\mathbf{x}} \quad \mathbf{c}^T\mathbf{x} + \frac{1}{2}\mathbf{x}^T\mathbf{Q}\mathbf{x}$$
$$\text{s.t.} \quad \mathbf{A}\mathbf{x} = \mathbf{b}, \quad \mathbf{C}\mathbf{x} \geq \mathbf{d}$$

Nonlinear programming problem (NLP): in general non-convex

$$\min_{\mathbf{x}} \quad f(\mathbf{x})$$
$$\text{s.t.} \quad g(\mathbf{x}) = 0, \quad h(\mathbf{x}) \geq 0$$

SVMs belong to **Quadratic Programming**, enabling us to utilize existing solvers for solving the primal problem.
Moreover, the same form applies to another problem called the **dual problem**, which has significant implications on how solutions are derived.

Quadratic optimization problems are a well-known class of mathematical programming problems, for which several (non-trivial) algorithms exist.
One approach involves constructing a dual problem from a primal problem, where a Lagrange multiplier α_i is associated with every inequality constraint in the primal (original) problem:

$$\max_{\boldsymbol{\alpha}} \quad \sum_i \alpha_i - \frac{1}{2}\sum_i \sum_j \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$
$$\text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad \alpha_i \geq 0, \forall i$$

Here's a brief explanation:

- Each datapoint has a Lagrange multiplier α_i.
- The objective function is quadratic (due to the presence of α_i and α_j).
- The loss function is linear.
- The objective function incorporates the dot product between pairs of datapoints and the product among the associated labels.
- α_i is what the solver infers.
- $y_i y_j x_i^T x_j$ is a number computed beforehand

Solving this problem (the dual problem) or solving the primal problem is equivalent:

- Given a solution to $\alpha_i, \ldots, \alpha_n$ for the dual problem, the solution to the primal problem is:

$$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$$

$$b = y_k - \sum_i \alpha_i y_i \mathbf{x}_i^T \mathbf{x}_k$$

- Each non-zero α_i indicates that the corresponding x_i is a support vector. Then, the classifying function is (note that we don't need w explicitly):

$$f(\mathbf{x}) = \sum_i \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b$$

  This point is crucial because if you have 1 million datapoints in the training set, but only 100 are support vectors, you will compute the equation only for those 100 α_i.

Since we use only α_i both in training and inference, we don't need to calculate w explicitly anymore, which is fundamental because it enables us to handle the non-linearly separable case.

Two important observations are:

- The solution relies on an inner product between the test point x and the support vectors x_i, so the inference depends only on the support vectors.
- Solving the optimization problem involves computing the inner products between all training points, so if you have a large training set, the computation becomes substantial.

The dual problem is similar in the soft margin case, but note the constraints:

$$\max_{\boldsymbol{\alpha}} \quad \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$\text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, \forall i$$

Again, x_i with non-zero α_i will be support vectors.

In summary:

- The classifier is a separating hyperplane.
- Most "important" training points are support vectors; they define the hyperplane.
- Quadratic optimization algorithms can identify which training points are support vectors with non-zero Lagrange multipliers α_i.

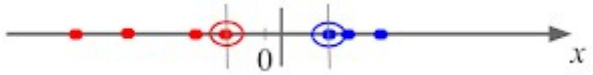
- Both in the dual formulation of the problem and in the solution, training points appear only inside dot products:

$$\max_{\alpha} \quad \sum_i \alpha_i - \frac{1}{2}\sum_i \sum_j \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \qquad f(\mathbf{x}) = \sum_i \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b$$

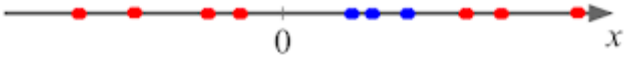
$$\text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \le \alpha_i \le C, \forall i$$

The reason we embraced this new formulation is because it is convenient for handling non-linearly separable data.

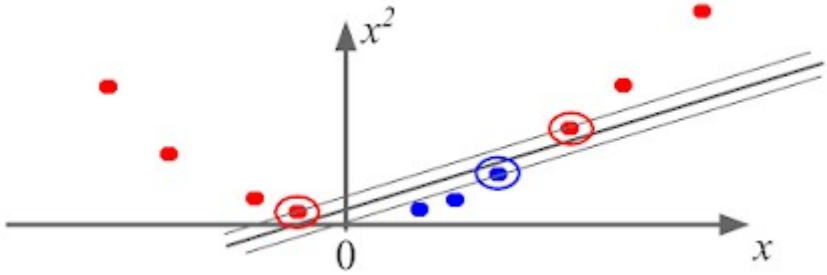
Datasets that are linearly separable with some noise work out great:



But it can happen that the dataset is too challenging:

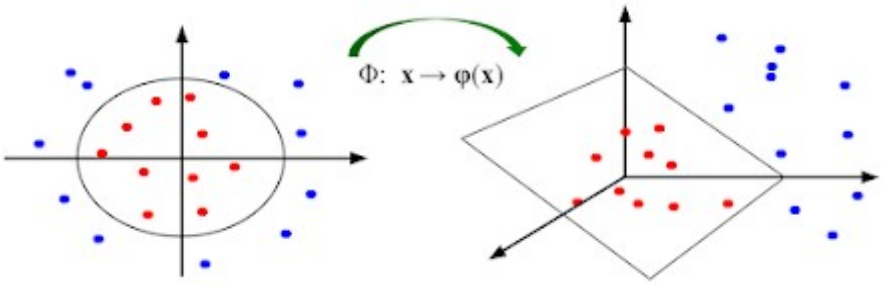


In these cases, it is possible to map the data to a higher-dimensional space:



And then find a way to linearly separate the datapoints that aren't separable in a lower-dimensional space.

The general idea of non-linear SVMs is that the original feature space can always be mapped, with a mapping function φ, to some higher-dimensional feature space where the training set is separable:



Now, let's discuss the **kernel trick**.

The linear classifier relies on the inner product between vectors: $K(x_i, x_j) = x_i^T x_j$.

If every datapoint is mapped into the high-dimensional space via some transformation $\Phi: x \rightarrow \varphi(x)$, the inner product becomes:

$$K(x_i, x_j) = \varphi(x_i)^T \varphi(x_j)$$

A kernel function is a function that is equivalent to an inner/dot product in some feature space.

The transformation function φ remains implicit. The only explicit component is the kernel because it is used in the objective function and is chosen by us.

Let's consider an example:

Example:

2-dimensional vectors $\mathbf{x}=[x_1 \ x_2]$; let $K(\mathbf{x}_i,\mathbf{x}_j)=(1+\mathbf{x}_i^T\mathbf{x}_j)^2$

Need to show that $K(\mathbf{x}_i,\mathbf{x}_j)=\varphi(\mathbf{x}_i)^T\varphi(\mathbf{x}_j)$:

$$K(\mathbf{x}_i,\mathbf{x}_j)=(1+\mathbf{x}_i^T\mathbf{x}_j)^2 = 1+x_{i1}^2x_{j1}^2+2x_{i1}x_{j1}x_{i2}x_{j2}+x_{i2}^2x_{j2}^2+2x_{i1}x_{j1}+2x_{i2}x_{j2}=$$
$$=[1 \ x_{i1}^2 \ \sqrt{2}x_{i1}x_{i2} \ x_{i2}^2 \ \sqrt{2}x_{i1} \ \sqrt{2}x_{i2}]^T[1 \ x_{j1}^2 \ \sqrt{2}x_{j1}x_{j2} \ x_{j2}^2 \ \sqrt{2}x_{j1} \ \sqrt{2}x_{j2}] =$$
$$=\varphi(\mathbf{x}_i)^T\varphi(\mathbf{x}_j), \quad \text{where } \varphi(\mathbf{x}) = [1 \ x_1^2 \ \sqrt{2}x_1x_2 \ x_2^2 \ \sqrt{2}x_1 \ \sqrt{2}x_2]$$

A kernel function **implicitly** maps data to a high-dimensional space (without the need to compute each $\varphi(x)$ explicitly).

For some functions $K(x_i,x_j)$, checking that $K(x_i,x_j)=\varphi(x_i)^T\varphi(x_j)$ can be cumbersome.
There exists a theorem that states how to check if a function is a valid kernel function:

Mercer's theorem:

- Every positive semidefinite symmetric function is a kernel
- A positive semidefinite symmetric functions correspond to a positive semidefinite symmetric Gram matrix:

$$K=\begin{bmatrix} K(x_1,x_1) & K(x_1,x_2) & K(x_1,x_3) & \dots & K(x_1,x_n) \\ K(x_2,x_1) & K(x_2,x_2) & K(x_2,x_3) & & K(x_2,x_n) \\ & & & & \\ \dots & \dots & \dots & \dots & \dots \\ K(x_n,x_1) & K(x_n,x_2) & K(x_n,x_3) & \dots & K(x_n,x_n) \end{bmatrix}$$

A symmetric matrix is positive semidefinite if and only if all eigenvalues are non-negative.

There has been a lot of research on inventing possible kernel functions to suit a massive variety of different data types.

The three possible kernel functions that work with vectors are:

- Linear: $K(\mathbf{x}_i,\mathbf{x}_j)= \mathbf{x}_i^T\mathbf{x}_j$
- Polynomial of power p: $K(\mathbf{x}_i,\mathbf{x}_j)= (1+ \mathbf{x}_i^T\mathbf{x}_j)^p$
- Gaussian (radial-basis function): $K(\mathbf{x}_i,\mathbf{x}_j) = e^{\frac{\|\mathbf{x}_i-\mathbf{x}_j\|^2}{2\sigma^2}}$
  - Mapping Φ: $\mathbf{x} \rightarrow \varphi(\mathbf{x})$, where $\varphi(\mathbf{x})$ is *infinite-dimensional*

When you opt for a more complex kernel (like the Gaussian one), you will introduce some hyperparameters (like σ). So you're not only asked to set C with your validation set but also to determine the optimal values for the hyperparameters.

In the case of the Gaussian kernel, the mapping can't be recovered because it maps the data points into an infinite-dimensional space.

So, in summary, the equation that describes SVMs in the non-linear case is:

Dual problem formulation:

$$\max_{\alpha} \quad \sum_i \alpha_i - \frac{1}{2}\sum_i\sum_j \alpha_i\alpha_j y_i y_j K(\mathbf{x}_i,\mathbf{x}_j)$$
$$\text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad \alpha_i \geq 0, \forall i$$

The solution is:

$$f(\mathbf{x}) = \sum_i \alpha_i y_i K(\mathbf{x}_i,\mathbf{x}) + b$$

Optimization techniques for finding α_i's remain the same!

In the inference phase, you compute the kernel of the support vectors and the example.

SVM Remarks:

- SVMs were originally proposed by Boser, Guyon, and Vapnik in 1992 and gained increasing popularity in the late 1990s.
- They have been successfully applied to a variety of classification tasks, ranging from text to genomic data, and they have strong theoretical generalization guarantees.
- SVMs can be applied to complex data types beyond feature vectors (e.g., graphs, sequences, relational data) by designing kernel functions for such data.
- SVM techniques have been extended to various tasks such as regression [Vapnik et al. '97], principal component analysis [Schölkopf et al. '99], etc.
- The most popular optimization algorithms for SVMs use decomposition to hill-climb over a subset of αi's at a time, e.g., SMO [Platt '99] and [Joachims '99].

- However, tuning SVMs remains a challenging task, as selecting a specific kernel and parameters is typically done through a trial-and-error approach (grid search).

SVMs are not inherently designed to prioritize accuracy, so they may not be suitable for fields where accuracy is crucial, such as cancer detection.

Let's now explore some practical applications of SVMs.
SVMs were the method of choice for many applications two decades ago.
For instance, SVMs were utilized for **pedestrian detection in computer vision**, which involves detecting (localizing) standing humans in an image.



In this context, the object detection problem is reduced to a binary classification problem: does an image contain a person or not?

Practically, all possible windows, across various locations and sizes, are examined, and each window is assigned a label (+1 if it contains a pedestrian, -1 otherwise).

To train a model, a dataset containing both positive and negative samples with diverse conditions such as different positions, lighting conditions, locations, and sizes is used:

- Positive data – 1208 positive window examples



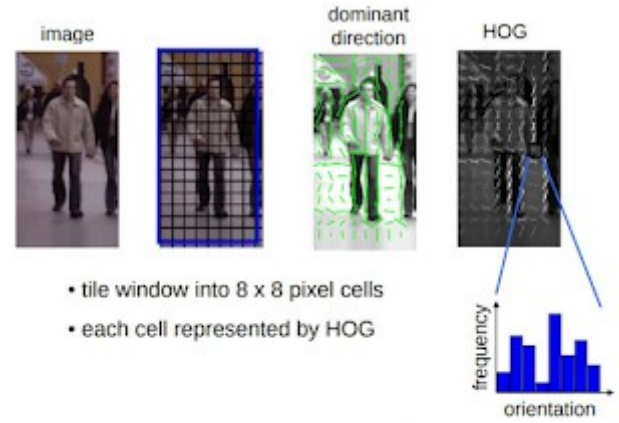
- Negative data – 1218 negative window examples (initially)



The critical aspect of this project was determining the relevant features.

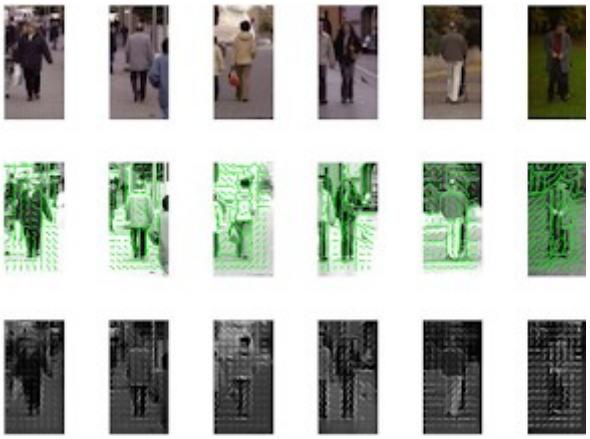
One significant innovation, developed 20 years ago, is a technique called **Histogram of Oriented Gradients** (HOG):

- Each image is divided into small, fixed-size segments called cells, usually 8x8 pixels.
- Each cell is represented by a feature descriptor known as the Histogram of Oriented Gradients (HOG). The HOG descriptor captures **edge orientation** and **gradient** information within the cell, which are distinctive features for detecting pedestrians.
- The feature vector dimension is calculated based on the tiling of cells and the number of orientations. For instance, if we divide the window into 16x8 cells and consider 8 different orientations, the feature vector dimension would be $(16 \cdot 8 \cdot 8)$, resulting in a 1024-element vector for each image window.



Feature vector dimension = 16 x 8 (for tiling) x 8 (orientations) = 1024

HOG is an effective feature for pedestrian detection, especially in low-resolution images, as it captures the consistent orientation of body parts, such as the verticality of legs and the horizontality of shoulders.
The feature vectors of images containing a pedestrian exhibit similarities in this aspect:

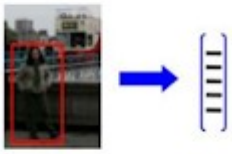


Though this approach was quite empirical, it was highly effective at the time.

The process of creating a pedestrian detection system follows this scheme:

Training (Learning)

- Represent each example window by a HOG feature vector



$\mathbf{x}_i \in \mathbb{R}^d, \text{ with } d = 1024$

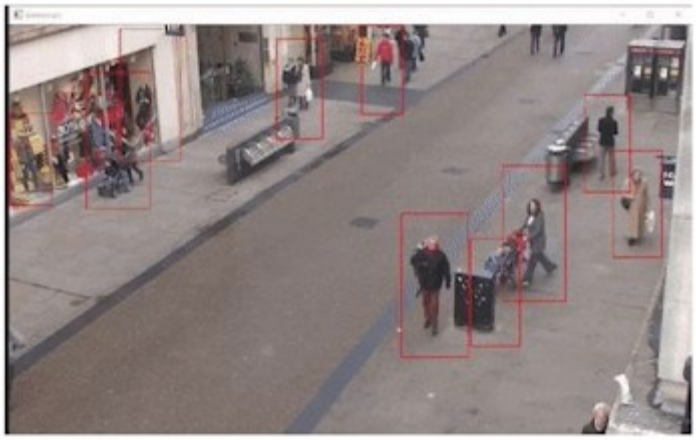
- Train a SVM classifier

Testing (Detection)

- Sliding window classifier

$$f(x) = \mathbf{w}^\top \mathbf{x} + b$$

The testing process is computationally expensive as it requires analyzing every potential window within the image. Hence, it is crucial to keep the feature vector as compact as possible without sacrificing the ability to address the problem effectively.



Decision Trees and Random Forests

Let's first review what supervised learning entails:
In supervised learning, we are given an input space X, which represents the set of all possible instances, and an output space Y, indicating the set of all possible labels. Our goal is to find the mapping $h \in Y^X$.
We are provided with a training set:

$$\left\{<x_i, y_i>\right\}_{i=1}^{n} = \left\{<x_1, y_1>, \ldots, <x_n, y_n>\right\}$$

To introduce decision trees, let's consider this example:

Example: Tennis match

- The green columns denote the different features
- The rows denote the different labeled instances
- The class labels indicate whether a tennis match was played

$\langle x_i, y_i \rangle$

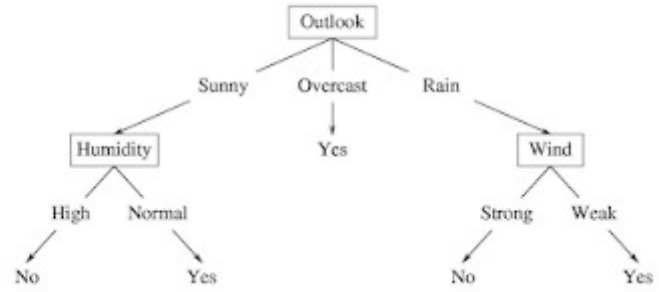
Predictors				Response
Outlook	Temperature	Humidity	Wind	Class
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Binary classification problem

Suppose we want to predict whether a tennis match will be held based on certain features, such as weather conditions.
In this example, the features have discrete and fixed possible values.

This example illustrates a classification problem that can be modeled using a decision tree:

- A possibile tree...



Characteristics of a decision tree include:

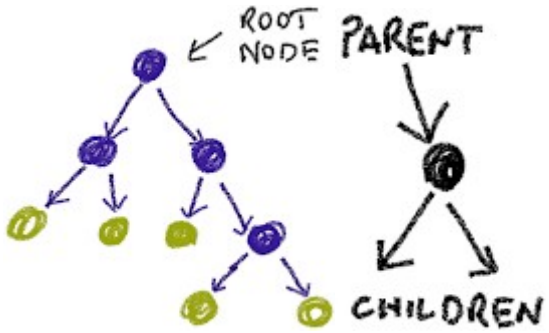
- Each internal node tests one feature/attribute.
- Each branch from a node selects one value for that attribute.
- Each leaf node performs a prediction.

One significant advantage of decision trees, contributing to their popularity, is the **explainability** of the model. You can inspect the model and understand the reasoning behind its choices.

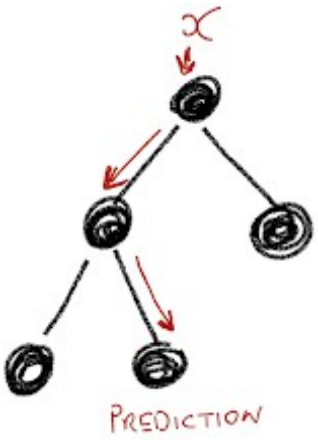
Now, let's describe decision trees more abstractly, assuming **binary trees**:
A decision tree is a tree-structured prediction model. It consists of terminal (or leaf) nodes and non-terminal nodes:

- **Non-terminal nodes** have two or more children (although we'll consider the case of two children for simplicity) and implement a routing function.
- **Leaf nodes** have no children (i.e., terminal) and implement a prediction function.

There are no cycles, and all nodes have at most one parent (except for the root node, which has no parent).

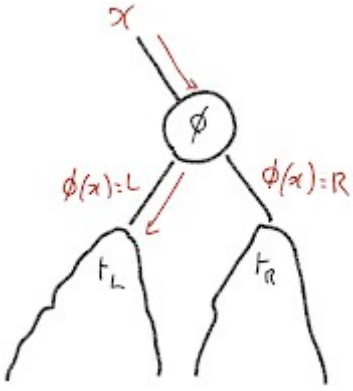


Let's explore how decision trees function.
A decision tree takes an input $x \in X$ and routes it through its nodes until it reaches a leaf node, as illustrated in the example:



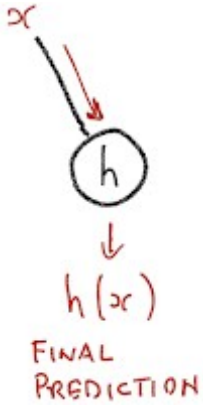
At the leaf node, a prediction is made.

Each non-terminal node $Node(\phi, t_L, t_R)$ holds a **routing function** $\phi \in \{L, R\}^X$, a **left child** t_L, and a **right child** t_R.
When x reaches the node, it proceeds to either the left child t_L or the right child t_R based on the value of $\phi(x) \in \{L, R\}$.

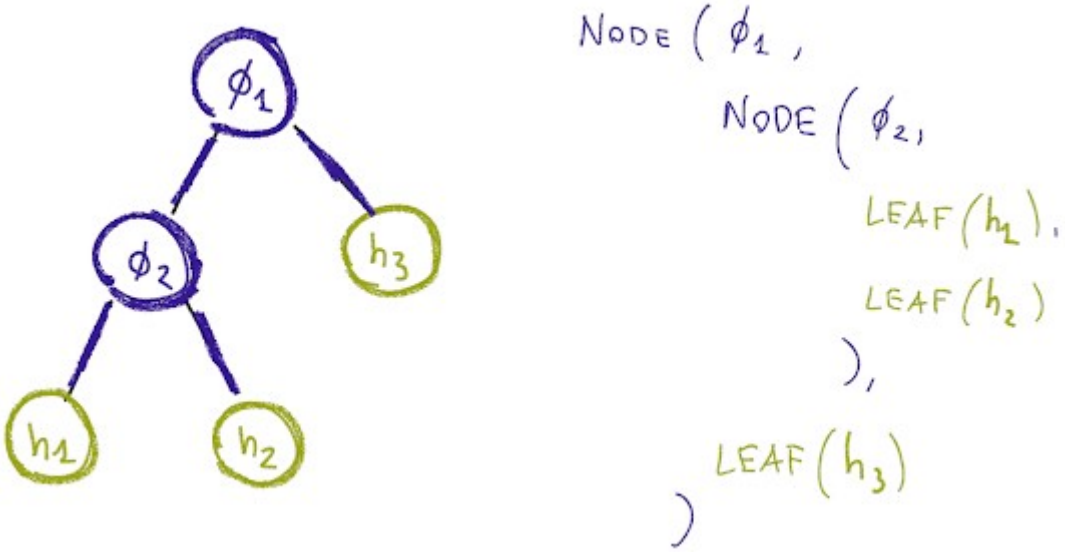


Each leaf node $Leaf(h)$ holds a prediction function $h \in F_{task}$ (typically a constant, as seen in our examples).

The prediction function depends on the task we want to solve, such as $h \in Y^X$ for classification or regression, $h \in \triangle(X)$ for density estimation, or others.
Once the leaf is reached, the final prediction is given by $h(x)$.



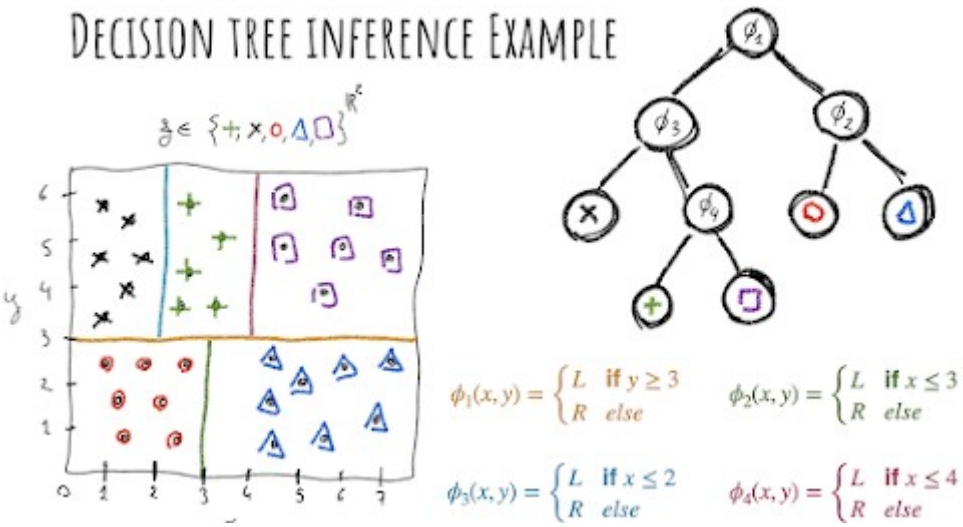
Trees can be represented graphically or textually:



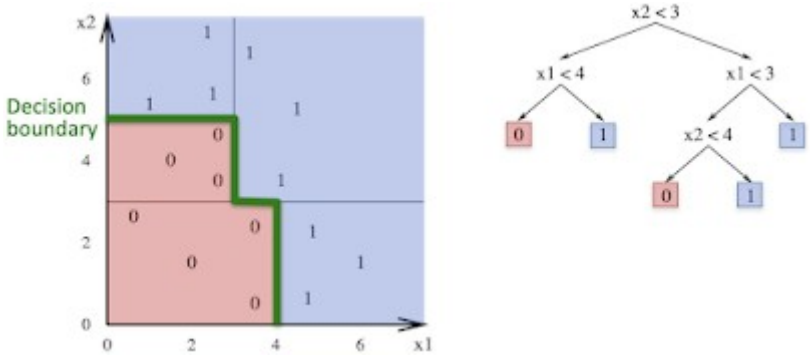
During the inference or testing phase of decision trees, f_t is the function returning the prediction for input $x \in X$ according to the decision tree t.
It is **recursively defined** as:

$$f_t(x) = \begin{cases} h(x) & \textbf{if } t = \texttt{Leaf}(h) \\ f_{t_{\phi(x)}}(x) & \textbf{if } t = \texttt{Node}(\phi, t_L, t_R) \end{cases}$$

Let's dive into a practical example of decision tree inference:



Now, let's discuss how we determine the routing functions when building the decision tree. But before that, it's essential to understand decision boundaries in decision trees.
Decision trees partition the feature space into axis-parallel (hyper-)rectangles. Each rectangular region is assigned a single label.
For instance, in a binary classification problem:



Now, let's delve into the learning algorithm of decision trees, which is how we construct the tree.
Given a training set $D_n = \{z_1, \ldots, z_n\}$, we aim to find f_{t^*}, where

$$t^* \in argmin_{t \in T} \; E(f_t; D_n)$$

with T representing the set of all possible decision trees.

Unlike in other methods where we use iterative optimization, gradient descent, or quadratic optimization, the optimization problem in decision trees is straightforward (with many solutions) only if we don't impose constraints, such as the most compact tree. Otherwise, it could be NP-hard.
In practice, a solution can be found using a simple greedy strategy, considering a relatively reduced set of decision trees.

Hereafter, we'll assume:

$$E(f_t; D) = \frac{1}{|D|} \sum_{z \in D}^{\square} l(f; z)$$

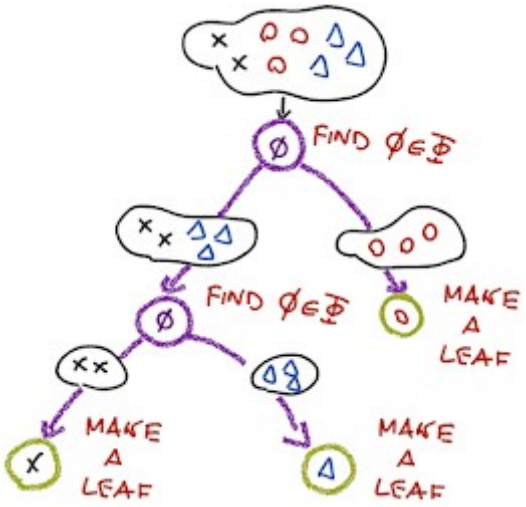
where $l(f; z)$ is a sort of elementary loss, similar to what we've encountered before.

Training is conducted in **batch mode**, requiring the entire training set to minimize that function and find the best tree.

So, we:

- Fix a set of leaf predictions $H_{leaf} \subset F_{task}$ (e.g., constant functions, for simplicity).
- Fix a set of possible routing (or split) functions $\Phi \subset \{L, R\}^X$ (again, for simplicity, we'll consider a binary tree and only threshold functions).

The method for building a tree involves a tree-growing strategy that recursively partitions the training set and **decides whether to grow leaves or non-terminal nodes**.



In simpler terms, we begin with the original training set and find a way to partition it, defining the routing function that will split it. We repeat this procedure until the subset is somewhat pure, meaning it contains only samples of a certain class:



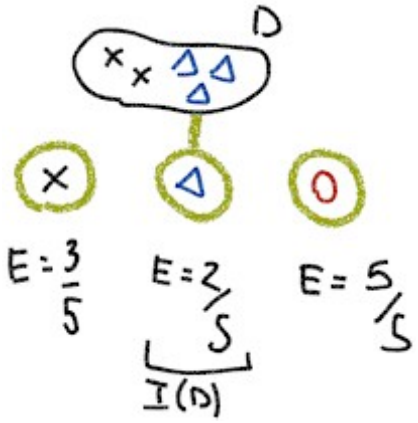
Now, let's discuss when we will grow a leaf node.
Given $D = \{z_1, \ldots, z_n\}$, the training set reaching the node, the optimal leaf predictor can be computed as:

$$h_D^* \in argmin_{h \in H_{leaf}} E(h; D)$$

The optimal error value is also called **impurity measure**

$$I(D) = E\left(h_D^{\star}; D\right)$$

The impurity measure consists of a number representing the proportion of samples of other classes in respect to the most common class in the set.
To better understand what the impurity measure entails, let's look at an example:



If certain criteria are met, we grow a leaf $Leaf\left(h_D^{\star}\right)$, such as:

- Purity: $I(D) < \epsilon$
- Minimum cardinality: $|D| < k$
- Others

The reason for including additional termination criteria beyond purity is to prevent overfitting.

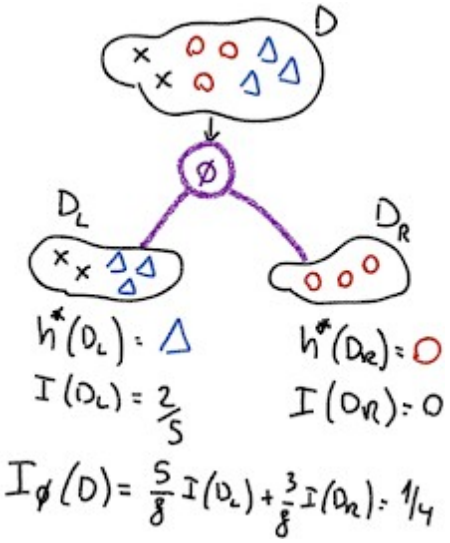
If no stopping criterion is met, we find an optimal split function:

$$\phi_D^{\star} \in argmin_{\phi \in \Phi}\ I_{\phi}(D)$$

The **impurity** $I_{\phi}(D)$ **of a split function** ϕ given a training set D is computed in terms of the **impurity of the split data**:

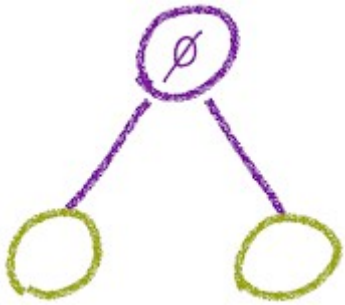
$$I_{\phi}(D) = \sum_{d \in \{L,R\}}^{\square} \frac{|D_d^{\phi}|}{|D|} I\left(D_d^{\phi}\right)$$

with $D_d^{\phi} = \{(x, y) \in D;\ \phi(x) = d\}$



In practice, we test all possible split functions (partitions) to minimize all the impurity measures.

The impurity of a split function represents the lowest training error that can be attained by a tree consisting of a root and two leaves.



Ultimately, we grow a node $Node\left(\phi^{\star}, t_L, t_R\right)$, where $\phi^{\star}$ is the optimal split. t_L and t_R are obtained by recursively applying the learning algorithm to the associated training set splits.

In summary, we have this recursive equation to build a tree:

$$\texttt{Grow}(\mathscr{D}) = \begin{cases} \texttt{Leaf}(h^{\star}_{\mathscr{D}}) & \textbf{if stopping criterion met} \\ \texttt{Node}(\phi^{\star}_{\mathscr{D}}, \texttt{Grow}(\mathscr{D}^{\star}_L), \texttt{Grow}(\mathscr{D}^{\star}_R)) & \textbf{otherwise} \end{cases}$$

$$\mathscr{D}^{\star}_d = \{(x,y) \in \mathscr{D} \,;\, \phi^{\star}_{\mathscr{D}}(x) = d\}$$

Sometimes, the selection of the best split function is not based on minimizing the impurity of the split but on the equivalent **maximization of information gain**, given by:

$$\Delta_{\phi}(D) = I(D) - I_{\phi}(D)$$

Information gain is nonnegative $\Delta_{\phi}(D) \geq 0$ for any $\phi \in \{L, R\}^X$ and any training set $D \subset X \times Y$. With this criterion, the impurity will never increase for any randomly chosen split.

Let's delve further into leaf predictions.
The leaf prediction offers a solution to a simplified problem involving only data reaching the leaf. This solution can be any arbitrary function $h \in F_{task}$, but in practice, we constrain it to a subset H_{leaf} of simple ones.

The simplest predictor one can construct is a function returning a constant, such as a prefixed class label. The set of all possible constant functions can be represented as:

$$H_{leaf} = \{\, y \in Y \,\}^X$$

Now, let's examine the impurity measures for classification.
For classification with $Y = \{c_1, \ldots, c_k\}$ and $D \subset X \times Y$, let $D^y = \{(x, y') \in D \;:\; y' = y\}$ denote the subset of training samples in D with class label y.
Consider the following error function:

$$E(f;D)=\frac{1}{|D|}\sum_{z\in D}^{\square} l(f;z)$$

Now, let's explore the different impurity measures we can have for various losses and leaf nodes:

If $\ell(f;(x,y))=1_{f(x)\neq y}$ and $\mathscr{H}_{\text{leaf}}=\cup_{y\in\mathscr{Y}}\{y\}^{\mathscr{X}}$ then the impurity measure is the classification error:

$$I(\mathscr{D})=1-\max_{y\in\mathscr{Y}}\frac{|\mathscr{D}^y|}{|\mathscr{D}|}$$

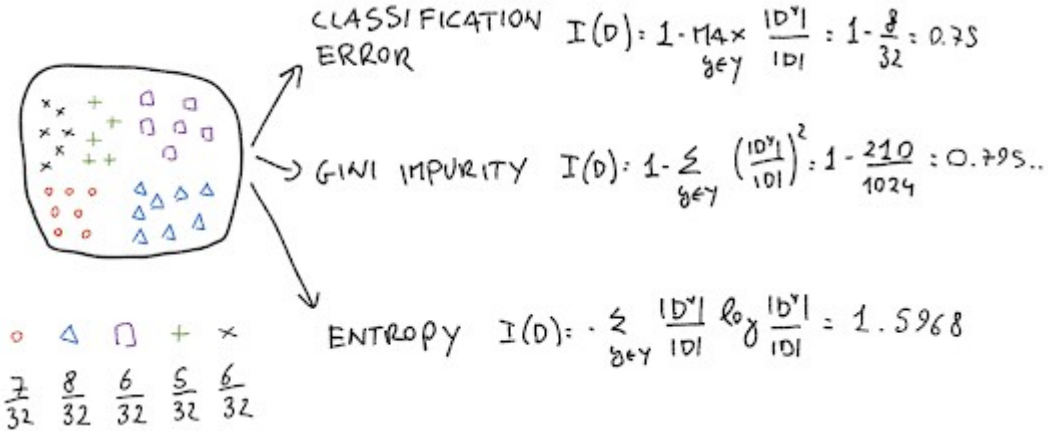
If $\ell(f;(x,y))=-\log f_y(x)$ and $\mathscr{H}_{\text{leaf}}=\cup_{\pi\in\Delta(\mathscr{Y})}\{\pi\}^{\mathscr{X}}$ (constant label distribution as leaf prediction) then the impurity measure is the **entropy**:

$$I(\mathscr{D})=-\sum_{y\in\mathscr{Y}}\frac{|\mathscr{D}^y|}{|\mathscr{D}|}\log\frac{|\mathscr{D}^y|}{|\mathscr{D}|}$$

If $\mathscr{H}_{\text{leaf}}=\cup_{\pi\in\Delta(\mathscr{Y})}\{\pi\}^{\mathscr{X}}$ and $\ell(f;(x,y))=\sum_{c\in\mathscr{Y}}[f_c(x)-1_{c=y}]^2$ then the impurity measure is the Gini impurity:

$$I(\mathscr{D})=1-\sum_{y\in\mathscr{Y}}\left(\frac{|\mathscr{D}^y|}{|\mathscr{D}|}\right)^2$$

Let's illustrate this with an example for better understanding:



This choice has practical implications for the type of tree we're going to obtain.

Moving on to impurity measures for regression.
For regression with $Y\subset R^d$ and $D\subset X\times Y$:

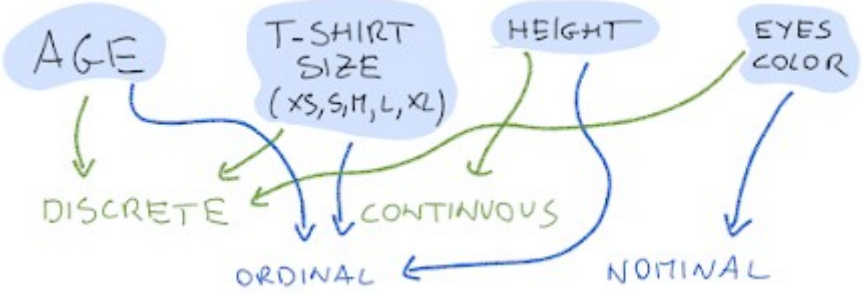
- For regression $\mathcal{Y} \subset \mathbb{R}^d$ and $\mathcal{D} \subset \mathcal{X} \times \mathcal{Y}$
- If $\ell(f;(x,y)) = \|f(x) - y\|_2$ and $\mathcal{H}_{\text{leaf}} = \cup_{y \in Y} \{y\}^{\mathcal{X}}$ then the impurity measure is the variance:

$$I(\mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{(x,y)\in\mathcal{D}} \|x - \mu_{\mathcal{D}}\|^2$$

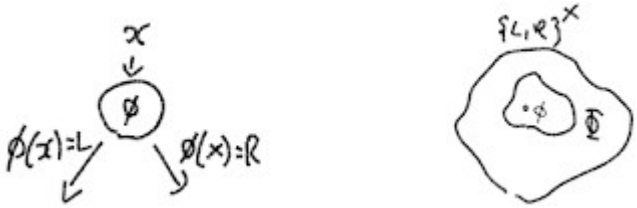
$$\mu_{\mathcal{D}} = \frac{1}{|\mathcal{D}|} \sum_{(x,y)\in\mathcal{D}} x$$

One reason decision trees are highly useful is their flexibility, as they can be applied to different problems simply by altering the notion of impurity.

Another reason is their ability to handle a wide range and types of features/attributes.
A data point $x \in X$ could be d-dimensional, with each dimension taking heterogeneous types of values (discrete, continuous) and having a total ordering or not (ordinal or nominal).



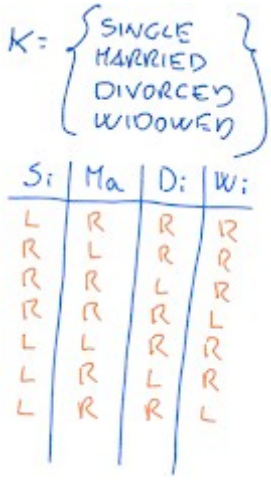
The routing or split $\phi \in \{L, R\}^X$ determines whether a data point $x \in X$ should move left (i.e., $\phi(x) = L$) or right (i.e., $\phi(x) = R$).
The possible split functions are restricted to some predefined set $\Phi \subset \{L, R\}^X$ depending on the nature of the feature space.
The prototypical split function for a d-dimensional input first selects one dimension and then applies a 1-dimensional splitting criterion.



The split function can be used to handle different types of features.
Assuming discrete nominal features taking values in K (e.g. colors, marital status, etc.), the split function can be implemented given a partition of K into K_R and K_L

$$\phi(x) = \begin{cases} L & \text{if } x \in \mathcal{X}_L \\ R & \text{if } x \in \mathcal{X}_R \end{cases}$$

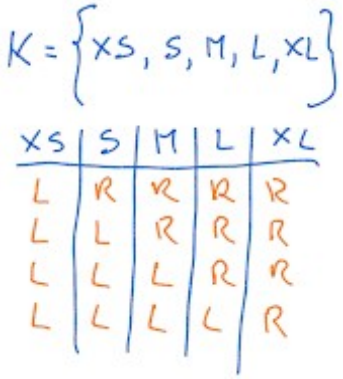
Finding the optimal split requires testing $2^{|K|-1}-1$ bi-partitions



Assuming ordinal features, i.e., those that can be sorted (numbers, t-shirt sizes, etc.), taking values in K, the split function can be implemented given a threshold $r \in K$

$$\phi(x) = \begin{cases} L & \text{if } x \leq r \\ R & \text{if } x > r \end{cases}$$

- If $|K| \leq |D|$, finding the optimal split requires testing $|K|-1$ thresholds.
- Otherwise, it requires sorting the input values in D, where D is the training set reaching the node, and testing $|D|-1$ thresholds

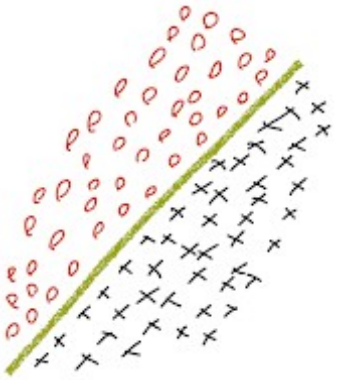


Sometimes it is advantageous to split using multiple features at once. Such split functions work with continuous features and are called oblique because they can generate oblique decision boundaries.

If $x \in R^d$, then the split function can be implemented given $w \in R^d$ and $r \in R$.

$$\phi(x) = \begin{cases} L & \text{if } w^\top x \leq r \\ R & \text{otherwise} \end{cases}$$

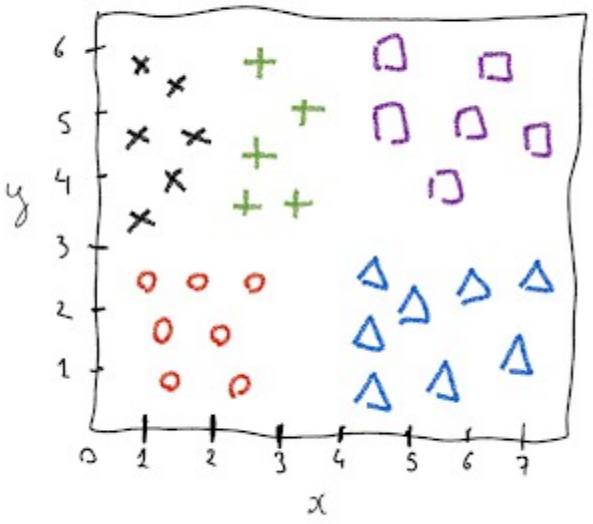
A disadvantage is that it is more difficult to optimize.



Let's now see an example, starting with the choice we made for building it:

DECISION TREE LEARNING EXAMPLE

- Classification problem
- We optimize classification error
- Constant leaf predictions
- 1-dimensional split functions
- Stop with zero impurity

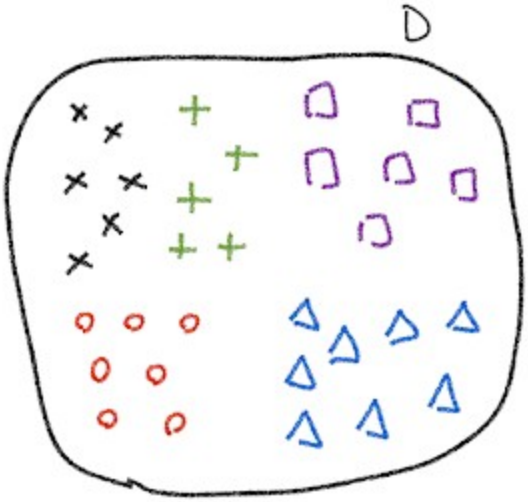


$Y = \{\circ, +, \times, \triangle, \square\}$

$X = \mathbb{R}^2$

DECISION TREE LEARNING EXAMPLE - ROOT

$$I(\mathcal{D}) = 1 - \max_{y \in \mathcal{Y}} \frac{|\mathcal{D}^y|}{|\mathcal{D}|}$$



MAKE A LEAF OR A NODE ?

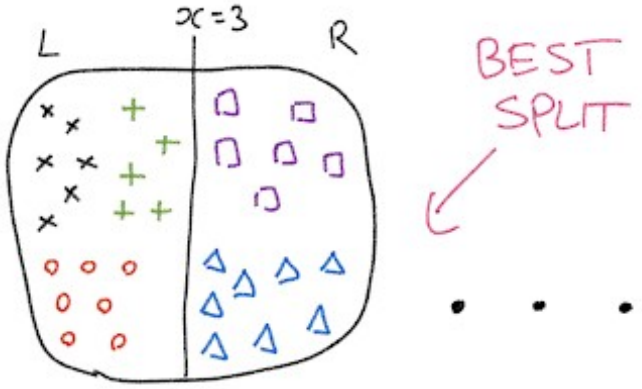
$$I(D) = \frac{24}{32} > 0$$

⇓

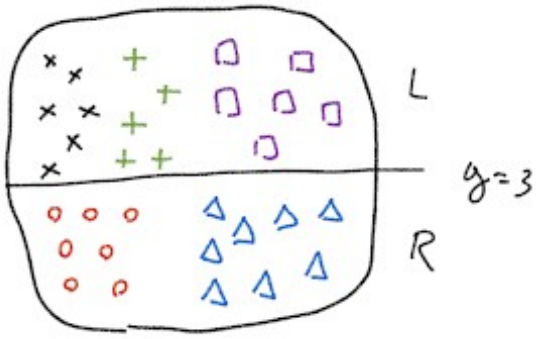
MAKE A NODE

DECISION TREE LEARNING EXAMPLE - BEST SPLIT

$$I_\phi(\mathcal{D}) = \sum_{d \in \{L,R\}} \frac{|\mathcal{D}_d^\phi|}{|\mathcal{D}|} I(\mathcal{D}_d^\phi)$$



BEST SPLIT

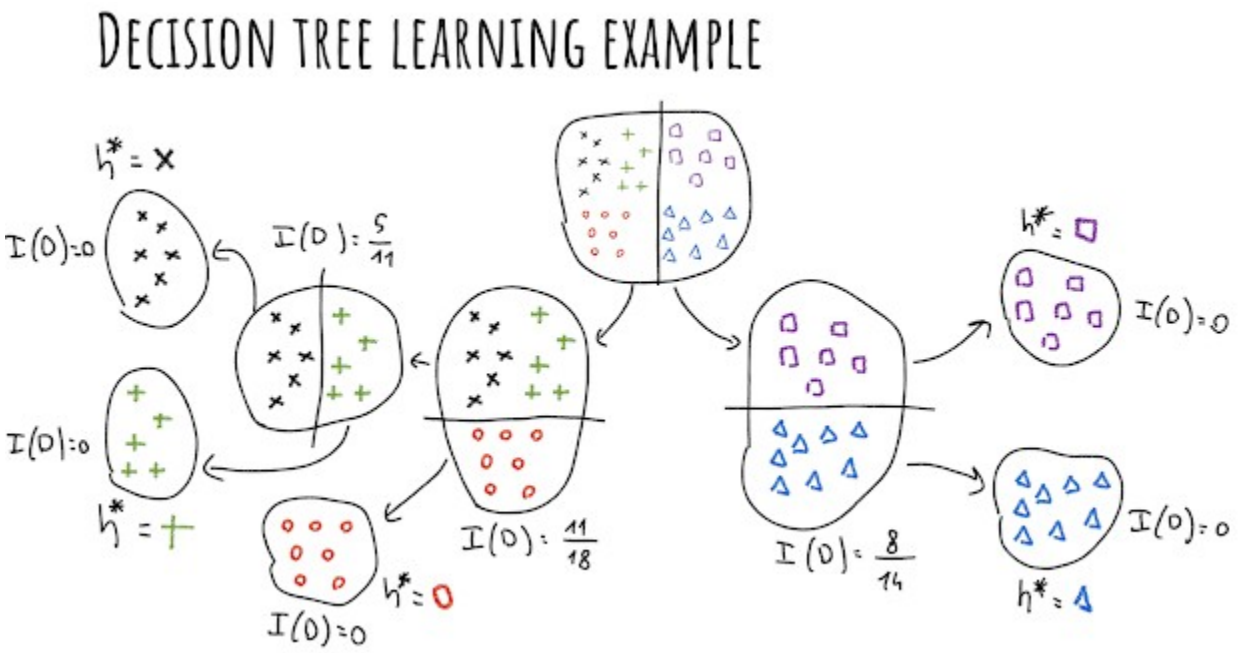


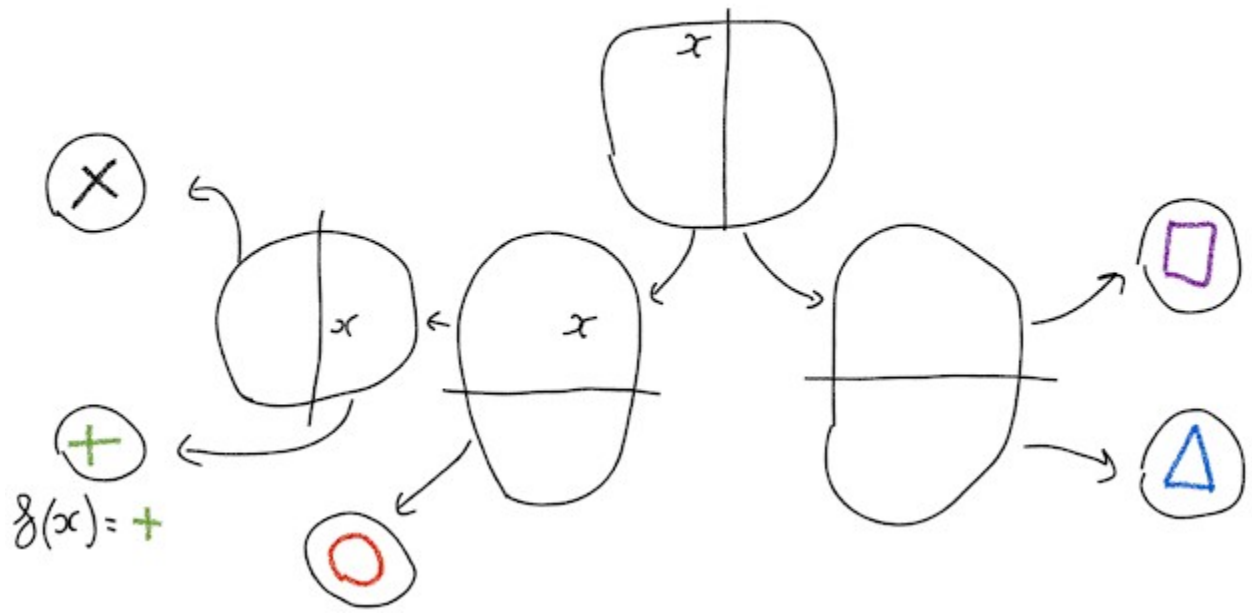
$I_\phi(D) = \frac{18}{32} \cdot I(D_L) + \frac{14}{32} I(D_R) = \frac{17}{32}$

$I(D_L) \downarrow \frac{11}{18}$ $I(D_R) \downarrow \frac{6}{14}$

$I_\phi(D) = \frac{17}{32} \cdot I(D_L) + \frac{15}{32} I(D_R) = \frac{18}{32}$

$I(D_L) \downarrow \frac{11}{17}$ $I(D_R) \downarrow \frac{7}{15}$





The pseudocode of a top-down learning algorithm for decision trees is as follows:

```
node = root of DT
Main loop:
1. Find the "best" decision attribute A (e.g. information
gain) for the next node
2. Assign A as decision attribute for node.
3. For each value of A, create a new descendant of node.
4. Sort  training examples to leaf  nodes.
5. If training examples are perfectly classified, stop.
     Else, recurse over new leaf nodes.
```

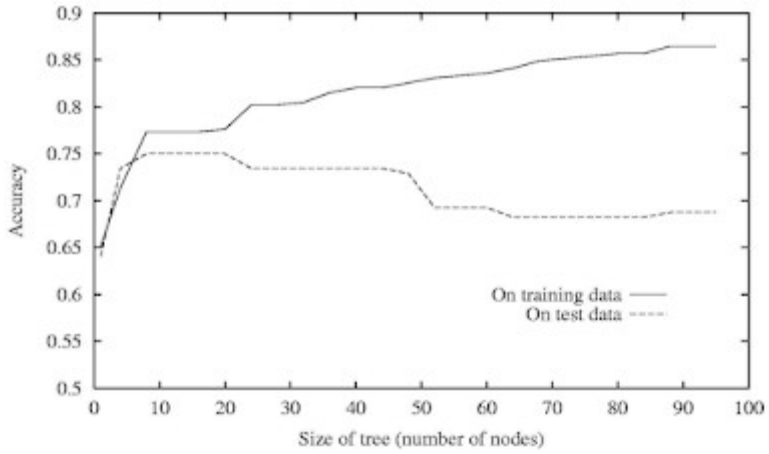
Let's discuss overfitting.

Decision trees possess a structure that adapts to the data they are trained on. Consequently, they exhibit flexibility and can readily conform to the intricacies of the training set, albeit with a heightened risk of **overfitting**.
Common strategies employed to bolster generalization also extend to decision trees: early stopping, regularization, data augmentation, complexity reduction, and ensembling. One such technique aimed at post hoc complexity reduction is known as **pruning**. Pruning involves replacing an entire subtree with a leaf node.

Various forms of "noise" may infiltrate the training data:

- Some attribute values may be erroneous due to flaws in the data acquisition process.
- Instances might be mislabeled.
- Certain attributes may prove inconsequential to the decision-making process.

Irrelevant attributes can exacerbate overfitting by misleadingly fitting the training data. With an abundance of attributes, spurious patterns may emerge, further complicating the issue.
One approach to mitigate overfitting involves the **removal of irrelevant attributes**, albeit this is often a manual endeavor and not always feasible.



In summary:

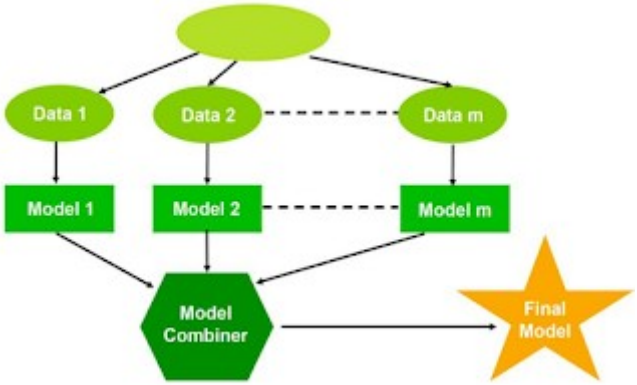
- Decision trees are widely used in practice.
- Strengths:
    - Fast and simple to implement (greedy strategy).
    - Can be converted to rules (explainability).
    - They can adapt well to many real-world applications due to the power of the split functions in making decisions.
- Weaknesses:
    - Univariate feature split (to maintain computational feasibility).
    - Requires fixed-length feature vectors.
    - Non-incremental (it is a batch method).

Now, let’s differentiate between Decision Trees and K-Nearest Neighbors:

- **K-Nearest Neighbors**:
    - Decision boundaries: piece-wise linear (Voronoi partition).
    - Test complexity: depends on all training examples.
- **Decision Trees**:
    - Decision boundaries: axis-aligned.
    - Test complexity: determined by the number of attributes and the splits in the tree.

To introduce the concept of random forests, which address the tendency of overfitting in decision trees, let's first discuss **ensemble learning**.
Ensemble learning is a powerful technique in machine learning where multiple models are combined to improve the overall system's performance. Instead of relying on a single model, ensemble methods leverage the collective wisdom of multiple models to make predictions or classifications.

The fundamental idea behind ensemble learning is that by aggregating or averaging the predictions of multiple models, the weaknesses of individual models can be mitigated, and their strengths can be amplified. This leads to more robust (in terms of overfitting) and accurate predictions than what any single model could achieve alone.



The issue with this approach is that although we have only one training set, we need to create multiple distinct models.
The solution is **bagging**, which stands for **b**ootstrap **agg**regat**ing**, a technique introduced by Leo Breiman in 1994.

This technique utilizes **bootstrap sampling**, which involves repeatedly sampling from the original training set D. Each bootstrap sample is a variation of the original dataset, containing randomly selected examples with replacement.
Bootstrap sampling involves, given a set D containing N examples, creating a new dataset D' by randomly selecting N examples from the original dataset D with replacement. This means that some examples may be selected multiple times, while others may not be selected at all.

The **bagging** process is the following:

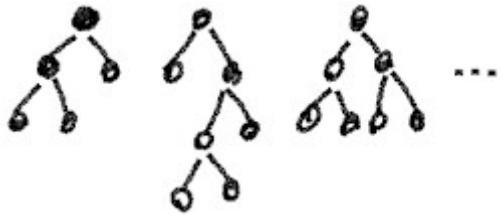
- **Creating Bootstrap Datasets**:
  Create k bootstrap datasets, denoted as D_i, by repeating the bootstrap sampling

process k times. Each bootstrap dataset D_i is a variation of the original training set D, containing randomly selected examples with replacement.

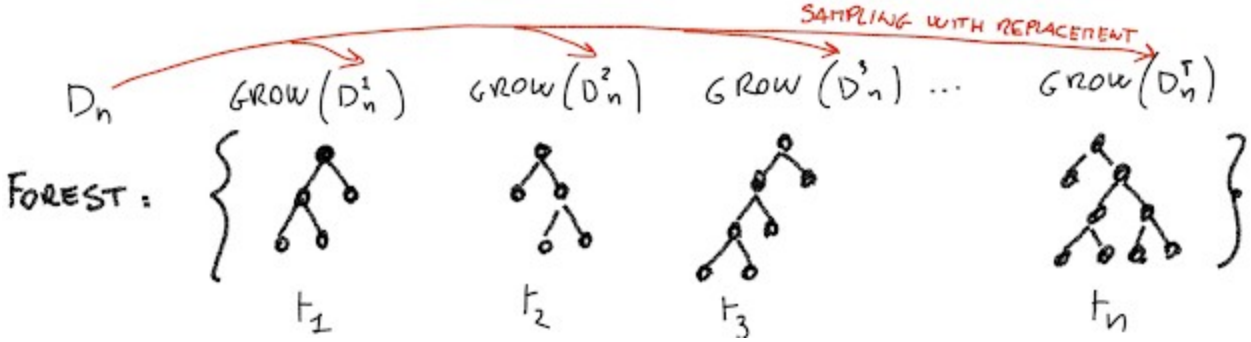
- **Training Distinct Classifiers**:
  Train a distinct classifier on each bootstrap dataset D_i. These classifiers could be of the same type or different types, depending on the application.
- **Classifying New Instances**:
  When classifying a new instance, each classifier makes a prediction. The final prediction for the new instance is determined through a majority vote (for classification tasks) or an average (for regression tasks) of the predictions made by all classifiers. This ensures robustness and reduces the variance of the predictions.

Bagging leverages the diversity of multiple classifiers trained on bootstrap datasets to improve the overall performance and stability of the model. By combining the predictions from multiple models, bagging reduces overfitting and provides more reliable predictions for both classification and regression tasks.

Random forests are ensembles of decision trees.



Each tree is typically trained on a bootstrapped version of the training set (sampled with replacement, i.e. independent samples).



Split functions are optimized on **randomly sampled features**, or are sampled completely at random (extremely randomized trees). This helps obtain decorrelated decision trees and improved performances.

The final prediction of the forest is obtained by averaging the prediction of each tree in the ensemble $Q=\{t_1,\ldots,t_n\}$:

$$f_Q(x)=\frac{1}{n}\sum_{i=1}^{n} f_{t_i}(x)$$

This is the pseudocode of the random forest algorithm:

1. For $b = 1$ to B:
   (a) Draw a bootstrap sample $\mathbf{Z}^*$ of size N from the training data.
   (b) Grow a random-forest tree T_b to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size n_{min} is reached.
      i. Select m variables at random from the p variables.
      ii. Pick the best variable/split-point among the m.
      iii. Split the node into two daughter nodes.
2. Output the ensemble of trees $\{T_b\}_1^B$.

To make a prediction at a new point x:

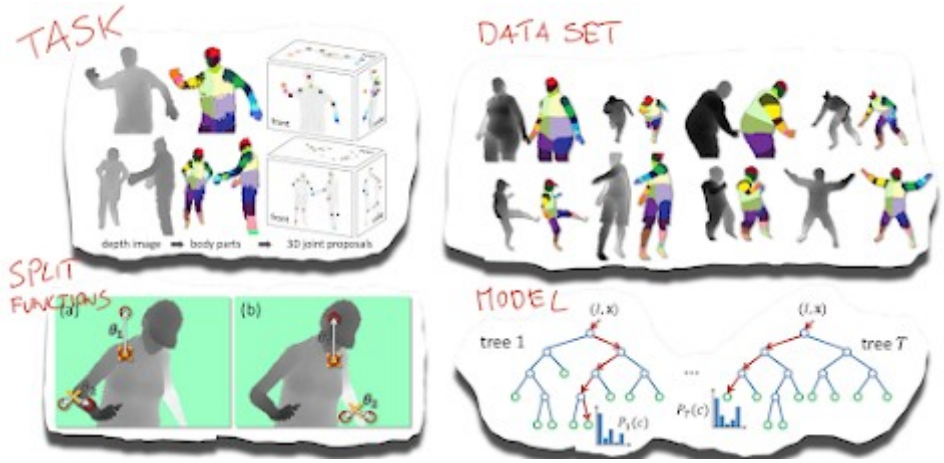
Regression: $\hat{f}_{\text{rf}}^B(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$.

Classification: Let $\hat{C}_b(x)$ be the class prediction of the bth random-forest tree. Then $\hat{C}_{\text{rf}}^B(x) = \textit{majority vote}\ \{\hat{C}_b(x)\}_1^B$.

A real application of random forest is the Xbox Kinect; in fact, random forest was the algorithm used to predict the position of the joints given, as input, the depth map produced by the Kinect sensors (infrared).



The main difficulty in doing this was deciding how to implement the split functions, and it was decided to use distances from one point to another, which worked successfully.



There was a significant effort in constructing the dataset because it required individuals to annotate the depth image parts to train the random forest.

They chose the random forest because it is extremely fast during inference time, and since it was a live activity, this aspect was crucial.
Another reason was the flexibility; it provided a simple way of having many simple functions.

Unsupervised Learning

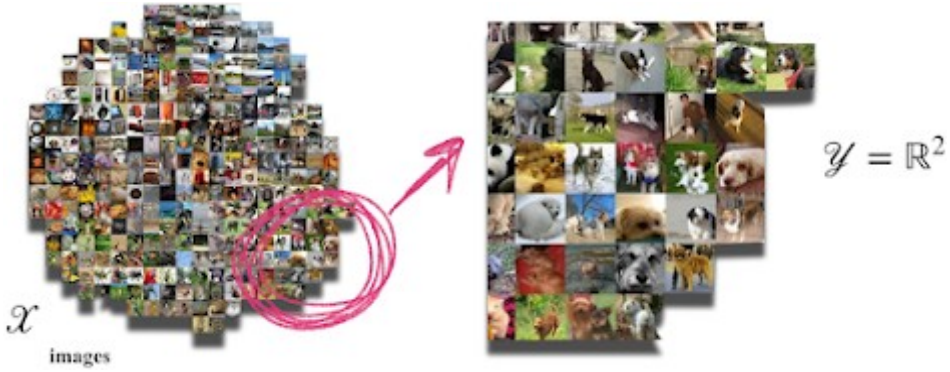
In the subfield of unsupervised learning, we will discuss algorithms addressing three distinct main needs:

- Dimensionality reduction
- Clustering
- Density estimation

As we've already mentioned, in unsupervised learning we handle and observe data from an (unknown) distribution $p_{data} \in \Delta(X)$, for which we don't have any associated labels. In fact, we lack observations of a target variable (e.g., class labels).
Implicit supervision is provided through the design of the objective function, which is tailored to the specific task we aim to accomplish.

Let's revise what these tasks consist of:

- **Dimensionality Reduction**
  Find a function $f \in Y^X$ mapping each (high-dimensional) input $x \in X$ to a lower-dimensional embedding $f(x) \in Y$, where $dim(Y) << dim(X)$.



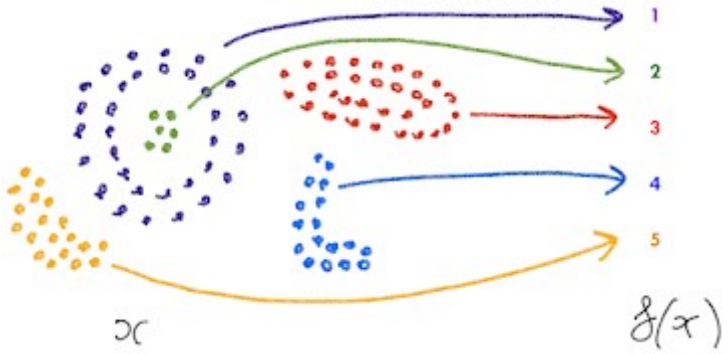
Dimensionality reduction consists of compressing the input data by **reducing the feature dimensionality** while preserving as much information as possible. The way information loss is measured yields different algorithms.

This is done to:

- **Reduce the time** of subsequent data processing and/or **storage**
- Enable **better visualization** of data
- Reduce the **curse of dimensionality**
- **Remove** features that correspond to **noise** rather than real information

- **Clustering** (Partitional)

  Find a function $f \in N^X$ that assigns each input $x \in X$ a cluster index $f(x) \in N$. All points mapped to the same index form a cluster.

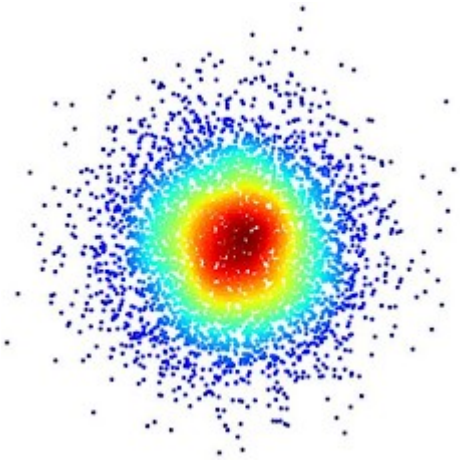


  Clustering allows us to **analyze data by grouping together** data points that exhibit some regular pattern or similarity under some predefined criterion. It can be used to **compress data** by reducing the number of data points as opposed to reducing the feature dimensionality.

  Application examples: cluster users by preference (e.g., based on movie ratings), group gene families from gene sequences, find communities in social networks, etc.

- **Density Estimation**

  Find a probability distribution $f \in \Delta(X)$ that fits the data $x \in X$.



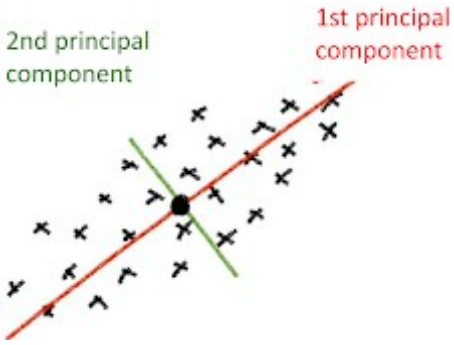
Density estimation allows us to get an **explicit estimate** of the unknown probability distribution that generated the training data. It enables the **generation of new data** by sampling from the estimated distribution and enables **detecting anomalies/novelties/outliers** in terms of data points that exhibit low probabilities according to the estimated distribution.
In the past, this task was simpler because we could assume that the distribution was a Gaussian one, but today the distributions are more complex because we are dealing with texts and images. We will discuss only deep models and not shallow techniques.

Principal Component Analysis

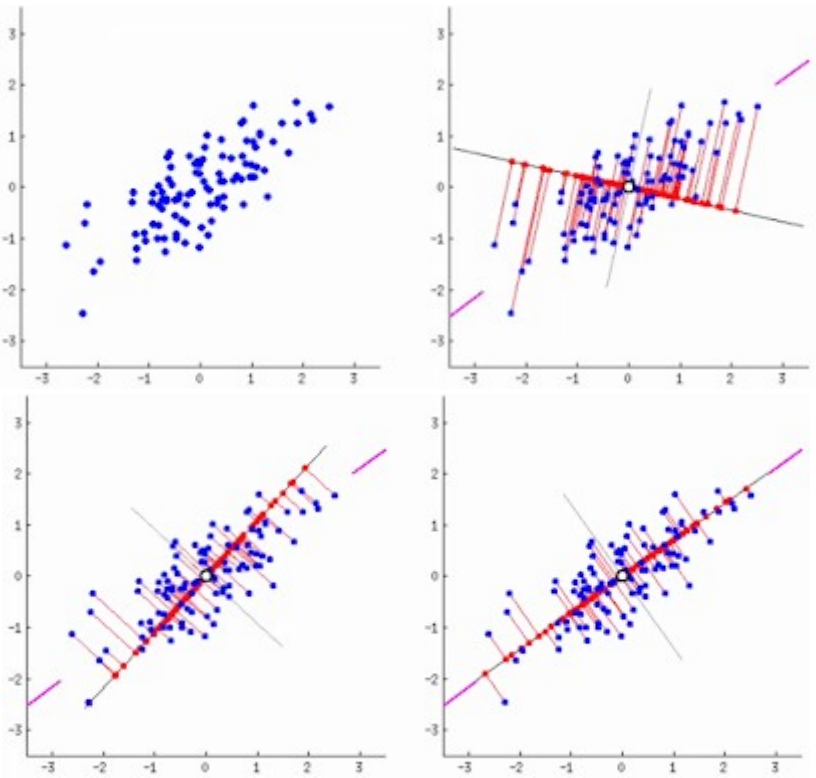
The first technique of unsupervised learning we're going to discuss is **Principal Component Analysis** (PCA), which is the simplest method to perform dimensionality reduction. The intuition behind PCA is to extract a new set of coordinates from the data points:

- The first principal component represents the direction of maximum spread of the data points.
- The second principal component represents the second direction of maximum spread of the data points and it is orthogonal to the first.

This concept can be extended to an arbitrary number of directions.



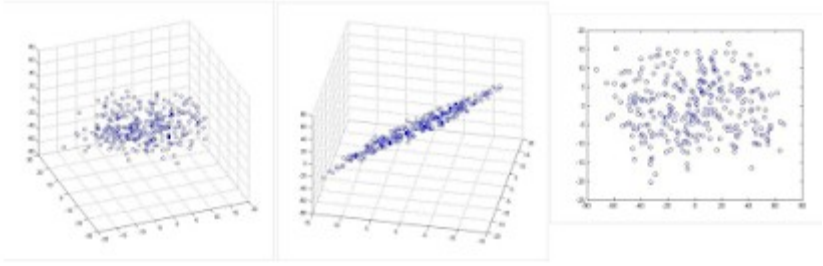
To be more formal, PCA involves identifying orthogonal directions of maximum variance in order to change the coordinate system and subsequently dropping dimensions with the least variance. We make the important assumption that considering the variance to select which dimension to remove is a good choice.



In this case, where we have only two dimensions, our aim is to drop one of the two. Thus, we project our 2-dimensional points (the blue ones) onto the first principal component, which represents the dimension with the maximum variance/spread of the data points, resulting in the red points.

This is a linear method to perform dimensionality reduction because the transformation applied to the original data to map them into a reduced space is a linear one. We can understand it better later with eigenvectors and eigenvalues.

We've discussed Principal Component Analysis in a 2-dimensional scenario; however, it can be applied to datasets of arbitrary dimensions.



Let’s start with some math. We need to compute the variance along the unit direction w.

We want to project the points in the direction of w. We consider w such that

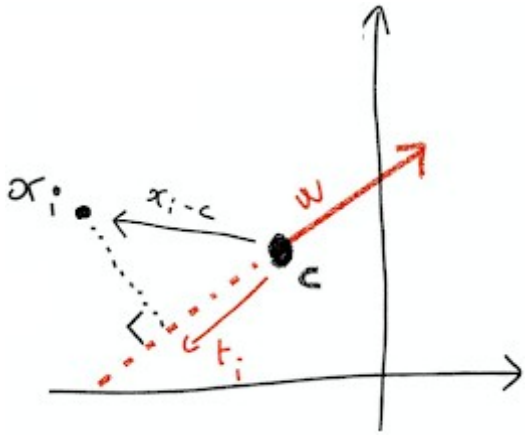
$$w^T w = 1$$

ensuring w is a unit vector; otherwise, the problem won’t have a unique solution.

To obtain t_i, which is the projection of x_i onto w, we compute

$$t_i = \left(x_i - c\right)^T w$$

We do this for every point in the dataset.



c is an arbitrary point useful only for the computation. In fact, it simplifies the computation of the mean and the variance.
The value of the mean can be computed in this way:

$$\mathbf{E}[t] = \frac{1}{n}\sum_{i=1}^{n} t_i = \frac{1}{n}\sum_{i=1}^{n} (\boldsymbol{x}_i - \boldsymbol{c})^\top \boldsymbol{w}$$
$$= \bar{\boldsymbol{x}}^\top \boldsymbol{w} - \boldsymbol{c}^\top \boldsymbol{w}$$

$\frac{1}{n}\sum_i x_i$

And the mean is used to compute the variance:

$$\mathbf{Var}[t] = \frac{1}{n}\sum_{i=1}^{n} (t_i - \mathbf{E}[t])^2 = \frac{1}{n}\sum_{i=1}^{n} \left[(\boldsymbol{x}_i - \bar{\boldsymbol{x}})^\top \boldsymbol{w}\right]^2$$
$$= \frac{1}{n}\sum_{i=1}^{n} \left(\bar{\boldsymbol{x}}_i^\top \boldsymbol{w}\right)^2 = \boldsymbol{w}^\top \left[\frac{1}{n}\bar{X}\bar{X}^\top\right] \boldsymbol{w} = \boldsymbol{w}^\top C \boldsymbol{w}$$

$t_i = (\boldsymbol{x}_i - \boldsymbol{c})^\top \boldsymbol{w}$

$\bar{X} = [\bar{\boldsymbol{x}}_1, \ldots, \bar{\boldsymbol{x}}_n]$

covariance matrix

$C = \left[\frac{1}{n}\underline{X}\ \ \underline{X}^T\right]$ is the covariance matrix for the training set.

For the main problem, we want to find the direction w such that the variance, which can be expressed as $w^T Cw$, is maximized. We aim to maximize it because we want to retain as much information as possible.

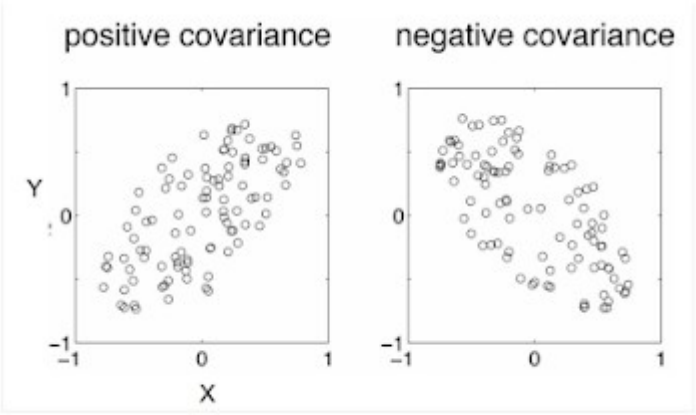
Let's make a recap of what variance and covariance mean.
They are used to measure the "**spread**" of a set of points around their center of mass (mean).

- **Variance**: Measure of the deviation from the mean for points in one dimension.

- **Covariance**: Measure of how much each of the dimensions vary from the mean **with respect to each other**. It is measured between two dimensions and it sees if there is a relation between the two dimensions.

If the **covariance** between two variables is **positive**, it means that they tend to increase or decrease together. In other words, when one variable has a high value, the other variable also tends to have a high value, and when one variable has a low value, the other variable also tends to have a low value. This positive covariance indicates a positive linear relationship between the variables.

Conversely, if the **covariance** between two variables is **negative**, it means that they tend to change in opposite directions. When one variable has a high value, the other variable tends to have a low value, and vice versa. This negative covariance indicates a negative linear relationship between the variables.



Positive: Both dimensions increase or decrease together Negative: While one increase the other decrease

Eigenvalues and eigenvectors are fundamental concepts in linear algebra that play a crucial role in various fields such as physics, engineering, and computer science. They provide valuable insights into the behavior of linear transformations represented by square matrices.

Let's break down the components:

- **Eigenvalues (λ)**: Eigenvalues are scalar values that represent how a linear transformation, represented by a square matrix, stretches or compresses vectors during the transformation process. In simpler terms, they are the factors by which an eigenvector is scaled when the transformation is applied. Eigenvalues can be real or complex numbers.
- **Eigenvectors (x)**: Eigenvectors are non-zero vectors that remain in the same direction after the linear transformation represented by the matrix is applied. They may change in length but not in direction. Eigenvectors corresponding to different eigenvalues can be linearly independent.
- **Matrix Representation (A)**: In the context of eigenvalues and eigenvectors, A represents a square matrix, meaning the number of rows is equal to the number of

columns. The size of the matrix determines the number of eigenvectors and eigenvalues it possesses.

The relationship between a square matrix A, its eigenvalues λ, and eigenvectors x is expressed through the equation:

$$Ax = \lambda x$$

This equation essentially states that when the matrix A operates on the eigenvector x, the result is the same as multiplying x by the scalar eigenvalue λ. In other words, the eigenvector x only changes in scale, and its direction remains unchanged when transformed by the matrix A.

Understanding eigenvalues and eigenvectors allows us to decompose matrices into simpler forms, which simplifies various mathematical operations and provides insights into the behavior of systems described by linear transformations. They are extensively used in various fields, including quantum mechanics, computer graphics, and data analysis, to name a few.

Here's an example:

Example

$$\text{Show } x = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \text{ is an eigenvector for } A = \begin{bmatrix} 2 & -4 \\ 3 & -6 \end{bmatrix}$$

$$\text{Solution}: Ax = \begin{bmatrix} 2 & -4 \\ 3 & -6 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\text{But for } \lambda = 0, \ \lambda x = 0 \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\text{Thus, } x \text{ is an eigenvector of } A, \text{and } \lambda = 0 \text{ is an eigenvalue.}$$

We can find eigenvalues by solving

$$det(A - \lambda I) = 0$$

Example

Find eigenvalues of **A**:

$$A = \begin{bmatrix} 2 & -12 \\ 1 & -5 \end{bmatrix}$$

$$|\lambda I - A| = \begin{vmatrix} \lambda - 2 & 12 \\ -1 & \lambda + 5 \end{vmatrix} = (\lambda - 2)(\lambda + 5) + 12$$

$$= \lambda^2 + 3\lambda + 2 = (\lambda + 1)(\lambda + 2)$$

Two eigenvalues: -1, - 2

Eigenvalue decomposition is a powerful method in linear algebra used to decompose a symmetric matrix into a set of eigenvalues and corresponding eigenvectors. Let's break down the key components of the eigenvalue decomposition:

- **Given a Symmetric Matrix (A)**: Eigenvalue decomposition is typically applied to symmetric matrices, denoted as $A \in R^{m \times m}$, where m is the number of rows (which is also equal to the number of columns since A is square).
- **Existence of Eigenvectors and Eigenvalues**: For a symmetric matrix A, there exists a matrix $U = [u_1, \ldots, u_m] \in R^{m \times m}$, composed of eigenvectors, and a diagonal matrix

$\Lambda = (\lambda_1, \ldots, \lambda_m)^T \in R^m$, composed of corresponding eigenvalues, such that:

$$A = U\Lambda U^\top = \sum_{j=1}^{m} \lambda_j u_j u_j^\top \quad \text{and} \quad U^\top U = UU^\top = I$$

This equation is the essence of eigenvalue decomposition. It shows that the original matrix A can be expressed as a product of the matrix of eigenvectors U, a diagonal matrix Λ containing the eigenvalues, and the transpose of the matrix of eigenvectors U^T.

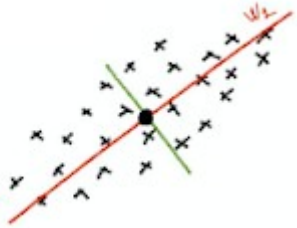
- **Properties of** U:
  - The columns of matrix U are the eigenvectors of A (u_j is an eigenvector and λ_j is the corresponding eigenvalue)
  - U is orthogonal (as we see above, where I is the identity matrix).
- **Descending Order of Eigenvalues**: The eigenvalues $\lambda_1, \ldots, \lambda_m$ are ordered in descending order, meaning $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_m$.
- **Usage of Eigenvalue Decomposition**: Eigenvalue decomposition allows us to understand the behavior of the original matrix in terms of its eigenvectors and eigenvalues. It's often used in various applications, such as principal component analysis (PCA), solving systems of linear differential equations, and spectral analysis in signal processing.

In summary, eigenvalue decomposition provides a useful way to represent a symmetric matrix in terms of its eigenvalues and eigenvectors, facilitating analysis and manipulation of the matrix in various contexts.

Returning to PCA, the **first principal component** is computed as follows:

$$w_1 \in \arg\max \{ w^\top C w : w^\top w = 1 \}$$

Variance along w



It can be shown that the largest eigenvalue of C is the variance along the first principal component, so the first principal component w_1 is the corresponding eigenvector.

The **second principal component** is computed as follows:

$$w_2 \in \arg\max \{ w^\top C w : w^\top w = 1, w \perp w_1 \}$$

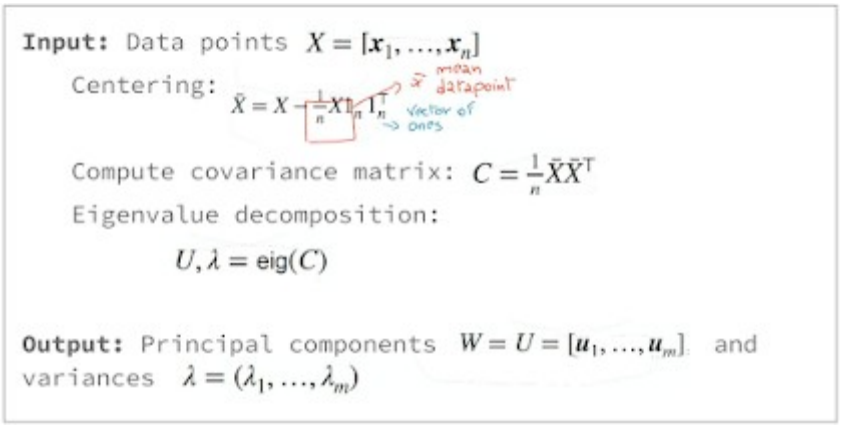
Similarly, it can be shown that the second largest eigenvalue of C is the variance along the second principal component, so the second principal component w_2 is the corresponding eigenvector.

The i-**th principal component**, computed as:

$$\boldsymbol{w}_i \in \arg\max \left\{ \boldsymbol{w}^\top C \boldsymbol{w} \ : \ \boldsymbol{w}^\top \boldsymbol{w} = 1 \, , \boldsymbol{w} \perp \boldsymbol{w}_j \textbf{ for } 1 \leq j < i \right\}$$

is the corresponding eigenvector of the i-th largest eigenvalue of C, which represents the variance along the i-th principal component.

The algorithm to perform PCA using eigenvalue decomposition is the following:



First, we center the data points by subtracting the mean value of the training set from each point. This step ensures that the data is centered around the origin, which simplifies the computation of the covariance matrix.
Next, we compute the covariance matrix from the centered training set. The covariance matrix captures the pairwise relationships between the different dimensions of the data, reflecting how they vary together.
After obtaining the covariance matrix, we proceed to compute its eigenvalues and eigenvectors. The eigenvalues represent the amount of variance explained by each principal component, while the eigenvectors represent the directions of these principal components in the original feature space.
The principal components are then determined based on the eigenvalues. The eigenvector corresponding to the highest eigenvalue is identified as the first principal component, as it accounts for the maximum variance in the data. Similarly, the eigenvector corresponding to the second highest eigenvalue becomes the second principal component, and this process continues for subsequent components.

One method to assess the suitability of dimensionality reduction is by examining the spread of eigenvalues to gauge whether they deviate significantly from equality.

Another way to compute the principal components is **Singular Value Decomposition (SVD)**, which is based on a different notion of matrix decomposition.

SVD: Let $A \in \mathbb{R}^{m \times n}$. There exist $U \in \mathbb{R}^{m \times k}, s \in \mathbb{R}^{k}$, with $s_1 \geq \ldots \geq s_k > 0$ and $V \in \mathbb{R}^{n \times k}$ such that:

$$A = USV^\top \quad \textbf{and} \quad U^\top U = V^\top V = I$$

$S = \begin{bmatrix} s_1 & & \\ & \ddots & \\ & & s_N \end{bmatrix}$

Note that A isn't necessarily a square matrix.

Input: Data points $X = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n]$

Centering:

$$\bar{X} = X - \frac{1}{n} X \mathbf{1}_n \mathbf{1}_n^\top$$

($\bar{x}$ mean datapoint; $\mathbf{1}_n$ vector of ones)

SVD decomposition:

$$U, \boldsymbol{s}, V = \text{SVD}(\bar{X})$$

Output: Principal components $U = [\boldsymbol{u}_1, \ldots, \boldsymbol{u}_k]$ and variances

$$\left(\frac{s_1^2}{n}, \ldots, \frac{s_k^2}{n} \right)$$

Because $\bar{X} = USV^\top$ and $C = \frac{1}{n} \bar{X} \bar{X}^\top = \frac{1}{n} USV^\top VSU^\top = U \frac{S^2}{n} U^\top$

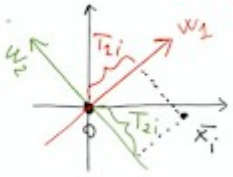
After calculating the principal components (the eigenvectors), we can proceed with dimensionality reduction.

Let $\widehat{W} = [w_1, \ldots, w_k]$ hold the first k principal components derived from the data points $\underline{X} = [\underline{x}_1, \ldots, \underline{x}_n]$.

We then transition to a reduced coordinate system with the k principal components as axes.

Principal component scores

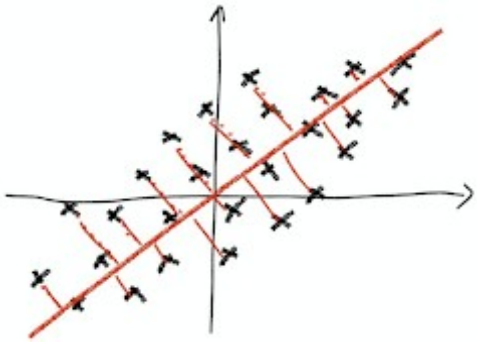
$$T = \hat{W}^\top \bar{X} \in \mathbb{R}^{k \times n}$$



We project the centered data points using the projection matrix $\widehat{W}$, which contains the first k principal components.

We can use eigenvalue decomposition or SVD algorithms, which allow us to avoid computing the full decompositions

The first principal component can be interpreted as the line in space with **minimal squared distance** from the data points. A similar interpretation can be given to the other principal components.



As with other algorithms in machine learning, the scale of the features matters in PCA. It's recommended to standardize features to a unit standard deviation when they are expressed in different units before applying PCA.

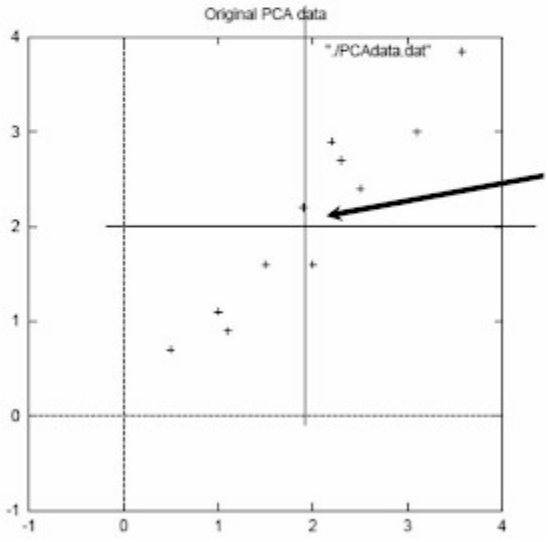
Let's now see a numerical example to better understand how PCA works.

EXAMPLE

Step 1: Data Centering

DATA:

x	y
2.5	2.4
0.5	0.7
2.2	2.9
1.9	2.2
3.1	3.0
2.3	2.7
2	1.6
1	1.1
1.5	1.6
1.1	0.9



Mean: this this becomes the new origin

Step 2: Compute the covariance matrix $\boldsymbol{C}$

$$\boldsymbol{C} = \begin{bmatrix} 0.616 & 0.615 \\ 0.615 & 0.716 \end{bmatrix}$$

Since the non-diagonal elements in $\boldsymbol{C}$ are positive, both the x and y variable increase together.

Step 3: Calculate the eigenvectors and eigenvalues of the covariance matrix

$$\boldsymbol{U} = \begin{bmatrix} -0.735 & -0.677 \\ 0.677 & -0.735 \end{bmatrix}$$

$$[\lambda_1 \quad \lambda_2] = [0.04 \; 1.28]$$

Eigenvectors are plotted as diagonal dotted lines (they are perpendicular to each other)

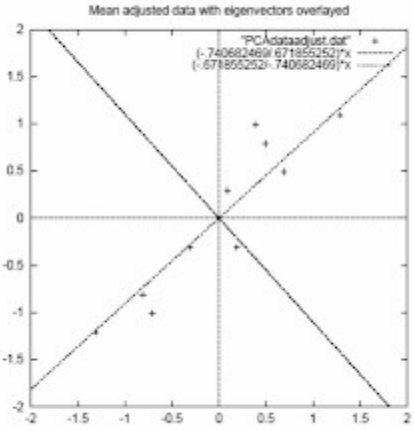


Figure 3.2: A plot of the normalised data (mean subtracted) with the eigenvectors of the covariance matrix overlayed on top.

Step 4:

We can either form a feature **vector** with both of the eigenvectors or or, we can choose to leave out the smaller, less significant component and only have a single column in $\boldsymbol{U}$

We can derive new data coordinates multiplying the matrix with the eigenvectors in the columns transposed and the matrix of the centered data

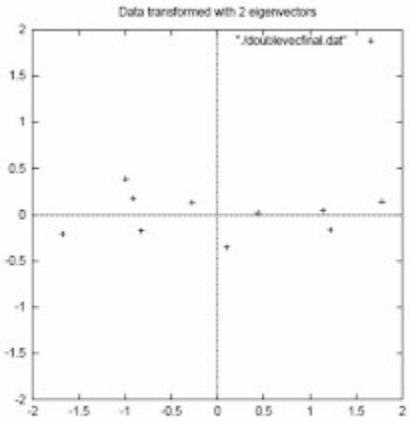
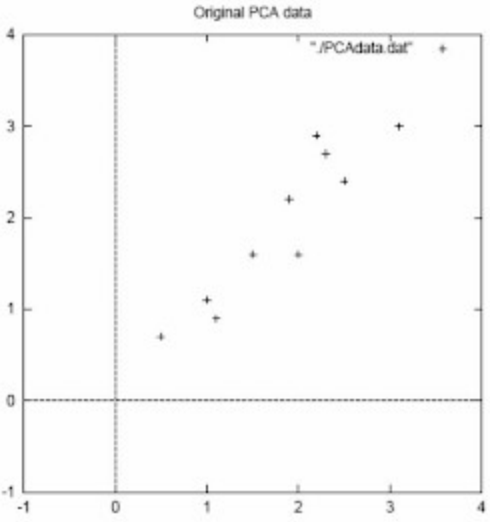
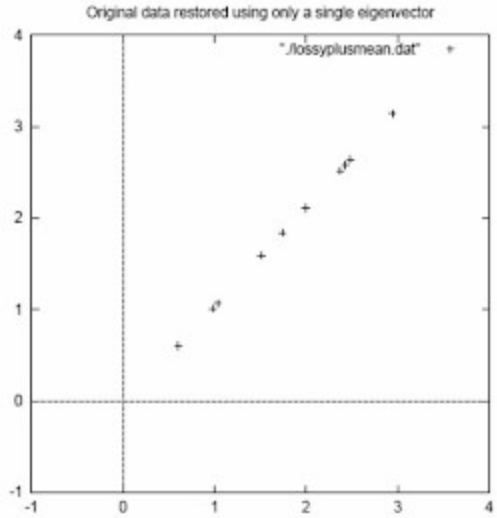


Figure 3.3: The table of data by applying the PCA analysis using both eigenvectors, and a plot of the new data points.

Reducing the dimensions...





The last important decision is determining how many principal components to retain. The **number of components** for dimensionality reduction depends on the goal and application. There are no definitive means of validating it unless we use it in the context of a supervised method (e.g., reducing input dimensionality for a supervised algorithm), in which it acts like an additional hyperparameter.

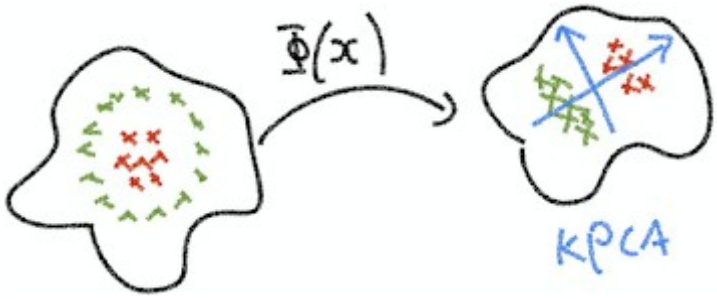
However, we can compute the **cumulative proportion of explained variance**, which is the cumulative sum of the proportions of variance explained by each principal component. For the first k principal components, it is given by:

EIGENVALUE DECOMPOSITION $\frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{m} C_{jj}}$ SVD $\frac{\sum_{j=1}^{k} s_j^2}{\sum_{ij} \bar{X}_{ji}^2}$

- If the cumulative proportion of explained variance is high, it indicates that the retained principal components capture a large portion of the total variance present in the original dataset. This suggests that the selected components effectively represent the data and that the dimensionality reduction process retains most of the important information.
- If the cumulative proportion of explained variance is low, it implies that the retained principal components explain only a small portion of the total variance. In this case, the dimensionality reduction may result in a significant loss of information, and the reduced dataset may not adequately represent the original data.

This allows us to estimate the amount of information loss.

The main limitation of PCA is that it is a linear dimensionality reduction technique. However, by using the **kernel trick**, one can apply PCA in a higher-dimensional space, yielding a non-linear transformation in the original space. This variation is called **Kernel PCA** (KPCA).



Some Dimensionality Reduction Techniques are:

- **PCA (Principal Component Analysis)**: The most famous technique, which finds projections that maximize the variance.
- **Multidimensional Scaling**: Finds projections that best preserve inter-point distances.
- **LDA (Linear Discriminant Analysis)**: A supervised technique that maximizes the component axes for class separation.

There are numerous advanced techniques for performing dimensionality reduction, and we still utilize these techniques for neural networks to examine the features that the network can learn across its different layers.

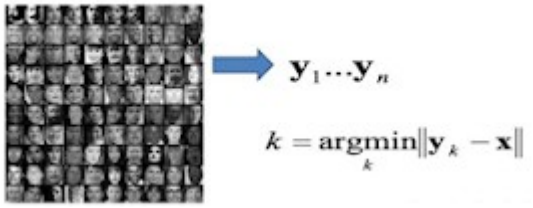
PCA has an important practical application in **face recognition**. Nowadays, we use neural networks for this task, but in the past, PCA was the best technique available.

We start with grayscale face images of relatively low resolution (for simplicity), and initially, face recognition was performed using the nearest neighbor method.

1. Treat face image as a vector of intensities



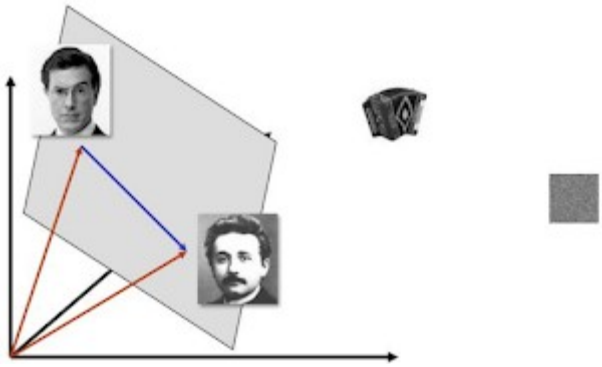
2. Recognize face by nearest neighbor in database



However, directly working in the pixel space was not effective, so researchers sought better approaches.

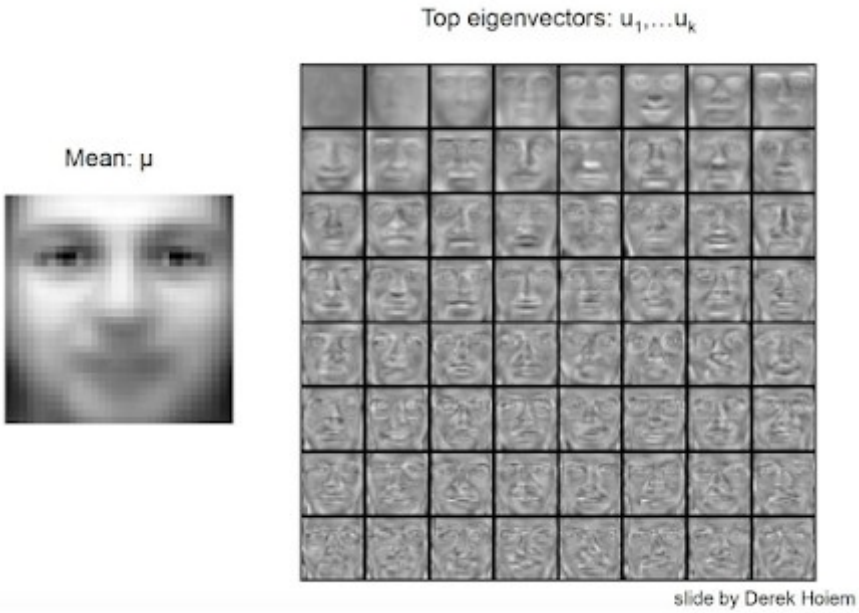
Face images, when represented as vectors of pixel values, become incredibly high-dimensional. For instance, a seemingly modest 100x100 pixel image results in a vector of 10,000 dimensions.
Not all possible combinations of these 10,000 dimensions yield valid face images. The objective is to efficiently capture and represent the subset of these dimensions that correspond to valid face images, essentially modeling the "space" of face images.

In fact, the set of faces forms a "subspace" of the set of all images.



We can find the best subspace using PCA.

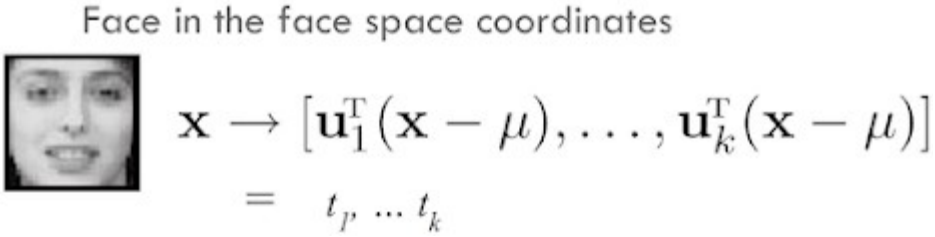
To do this, we take a dataset of faces and compute the mean, eigenvalues, and eigenvectors of our training set.



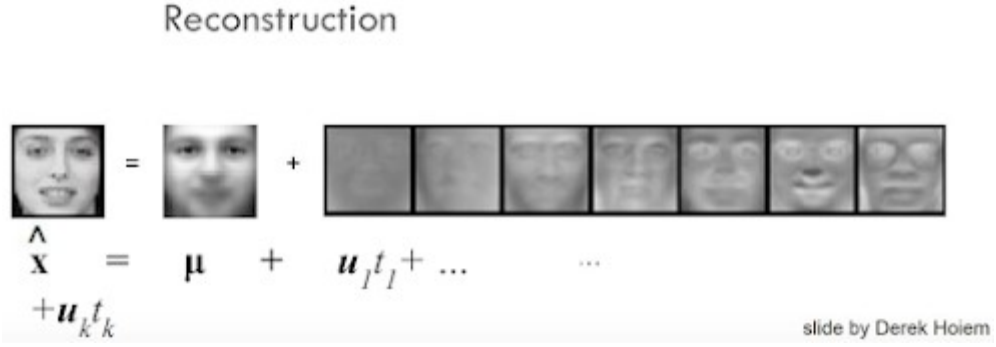
The eigenvectors will represent different factors, like gender, facial expression, etc.

This procedure requires aligning all the faces in the images because it assumes the positions of the facial features are fixed.

Once we have a new face, we can represent it in terms of eigenvalues and eigenvectors. We center the face and then project it into the new space.



By summing up the eigenvectors and the mean, we can reconstruct the face.



Obviously, considering different numbers of eigenvalues/eigenvectors will result in varying qualities of the reconstructed image:



The algorithm for this process is as follows:

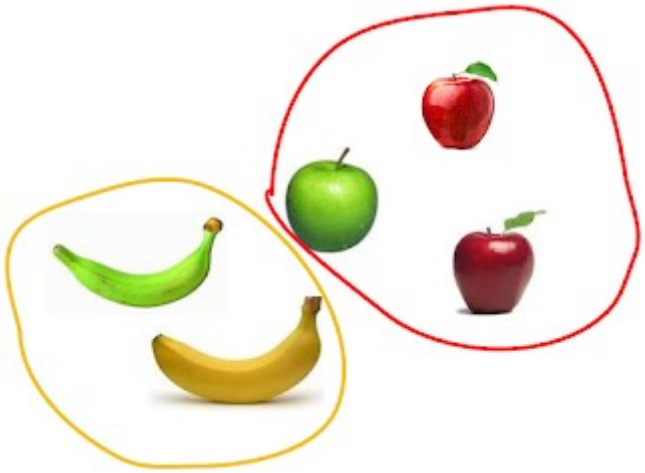
ALGORITHM: DETECTION & RECOGNITION WITH EIGENFACES

1. Process the image database (set of images with labels)
    - Run PCA: compute eigenfaces
    - Calculate K coefficients for each image
2. Given a new image (to be recognized) $\mathbf{x}$, calculate K coefficients
3. Detect if $\mathbf{x}$ is a face (distance between $\mathbf{x}$ and reconstructed image < threshold)
4. If it is a face, who is it?
    - Find closest labeled face in database (KNN in K-D space)

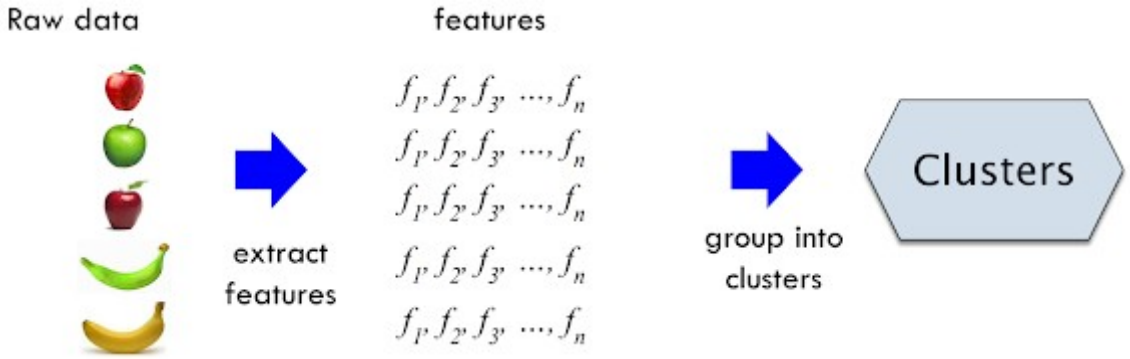
k-Means Clustering

k-Means Clustering is a widely used algorithm for clustering tasks, despite its simplicity.

In clustering, there are no labels provided for the data; it is an unsupervised learning setting where we are given examples but no labels.

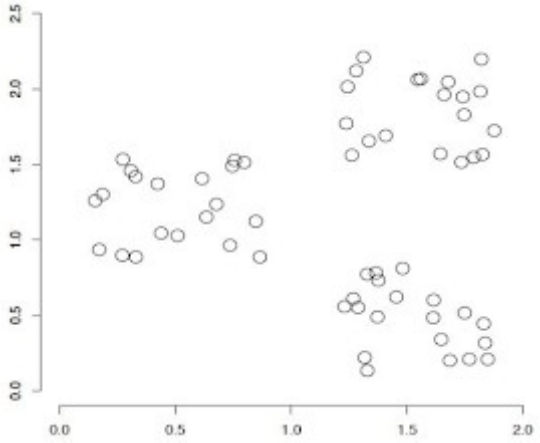


The process is as follows:



In clustering, there is no supervision; we are solely provided with data and aim to identify inherent groupings within it.

Essentially, clustering involves grouping a set of objects into classes of similar objects.



In k-Means clustering, we assume that we know the number of clusters (k) in advance. However, in practice, we often do not know the correct number of clusters, and there is no definitive method to determine it. Another challenge is defining what "similar" means.

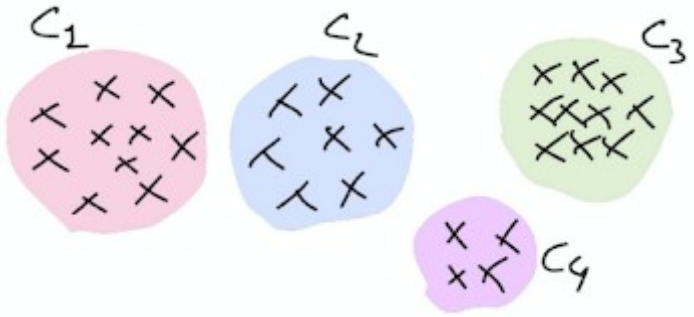
There are many algorithms that perform clustering, each with its own advantages and disadvantages. The most well-known and popular clustering algorithm is **k-Means**:

- Given data points $X = [x_1, \ldots, x_n] \in R^{d \times n}$
- Fix a number of clusters k
- Find a partition of data points into k sets minimizing the variation within each set:

  $min_{C_1,\ldots,C_k} \sum_{j=1}^{k} V(C_j)$

- The variation $V(C_j)$ is typically given by: $V(C_j) = \sum_{i \in C_j}^{\square} \|x_i - \mu_j\|^2$

  where the centroid μ_j is computed as: $\mu_j = \frac{1}{|C_j|} \sum_{i \in C_j}^{\square} x_i$



The algorithm is as follows:

Initialization: Use some initialization strategy to get some initial cluster centroids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$

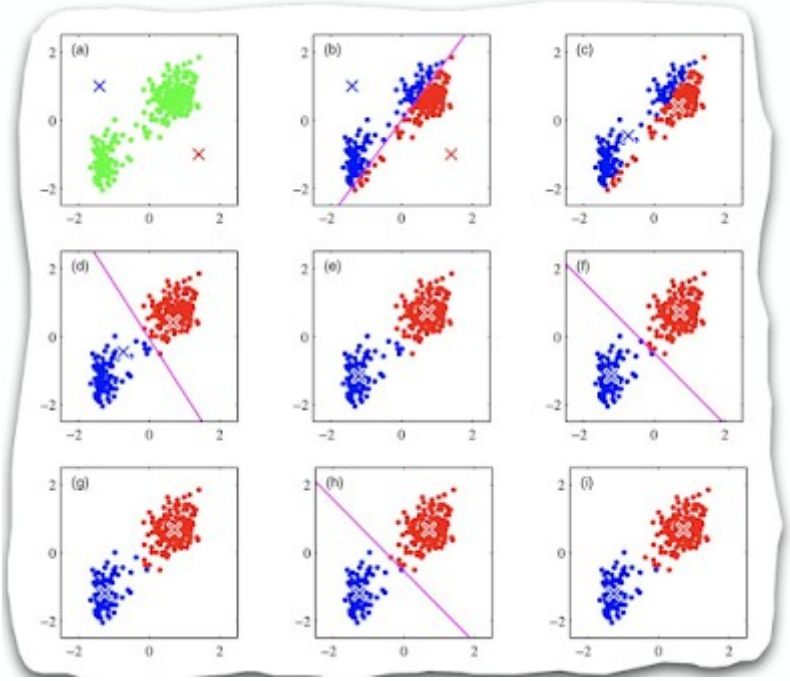
while clusters change **do**

Assign each datapoint to the closest centroid forming new clusters, i.e.

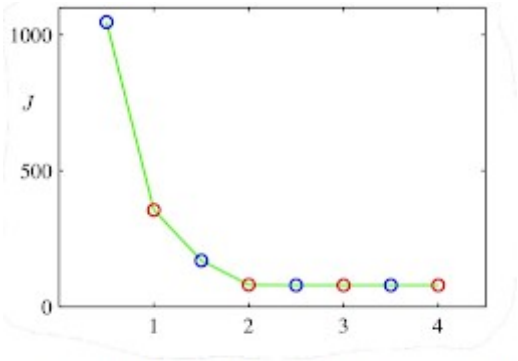
$$\mathscr{C}_j = \{i \in \{1,\ldots,n\} \ : \ j = \arg\min_{\ell} \|\boldsymbol{x}_i - \boldsymbol{\mu}_\ell\|\}$$

Compute cluster centroids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$

Let’s explore a visual example of the k-Means algorithm:



Due to the while loop, the value of the loss will always decrease until it converges to a local minimum:



Value of objective function for previous example

Some important properties of this algorithm include:

- **Sensitivity to Feature Scale:** The algorithm is sensitive to the scale of features, necessitating feature normalization when dealing with features of varying scales. This property applies to many machine learning algorithms.
- **Guaranteed Convergence:** The algorithm is guaranteed to converge because it consistently improves the objective function whenever there is a change in the cluster assignment, and the set of possible partitions is finite.
  Each step of k-means moves towards reducing the loss function (or at least not increasing it). Specifically:
  - **Assignment Step:** Each data point is assigned to the cluster that minimizes the distance to its centroid, ensuring that any alternative assignment would result in a larger loss.
  - **Centroid Computation Step:** The centroid of each cluster is recalculated to minimize the squared error, representing how well the centroid represents the cluster's data points.

However, it is not guaranteed to find the global minimum but rather a **local minimum**, which is influenced by the choice of initial centroids. The k-Means loss function is generally not convex, and for most problems, it has many minima. We are only guaranteed to find one of these minima.

As mentioned, the initial centroid selection significantly impacts the outcome. Results can vary drastically depending on the random seed selection. Some seeds may lead to a poor convergence rate or suboptimal clustering.
Common heuristics for selecting initial centroids include:

- Selecting random points in the space (not examples) as initial centroids.
- Randomly picking examples from the dataset.
- Choosing points that are least similar to any existing center (known as the furthest centers heuristic).
- Experimenting with multiple starting points to find the best result.
- Initializing with the results of another clustering method.

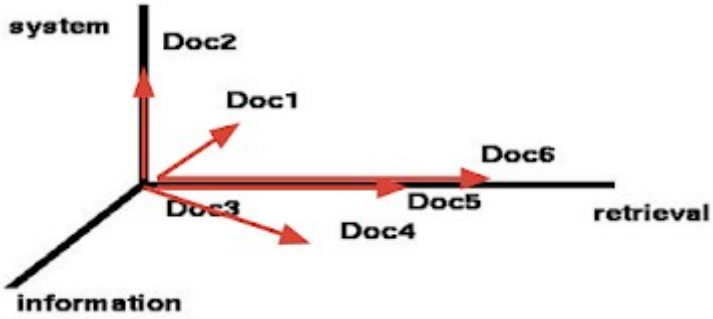
Regarding the **computational running time** of the algorithm, with k centroids and n points:

- **Assignment Step**: $O(kn)$ time
- **Centroid Computation Step**: $O(n)$ time

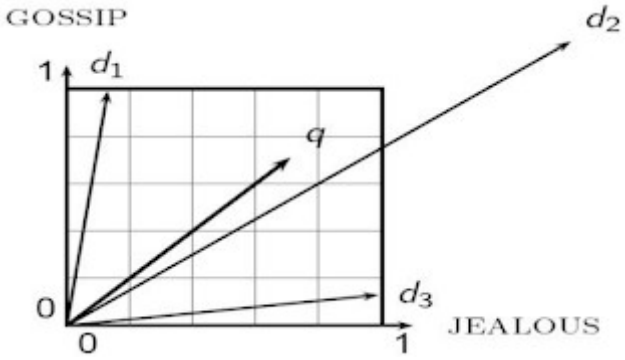
By default, k-Means clustering uses **Euclidean distance** to calculate the centroid and make the assignments:

$$d(x,y)=\sqrt{\sum_{i=1}^{n}(x_i-y_i)^2}$$

However, Euclidean distance is not always the best choice. For example, in clustering documents, where each document is represented as a vector in a high-dimensional space, each feature corresponds to a word, and its value represents the frequency of occurrence:



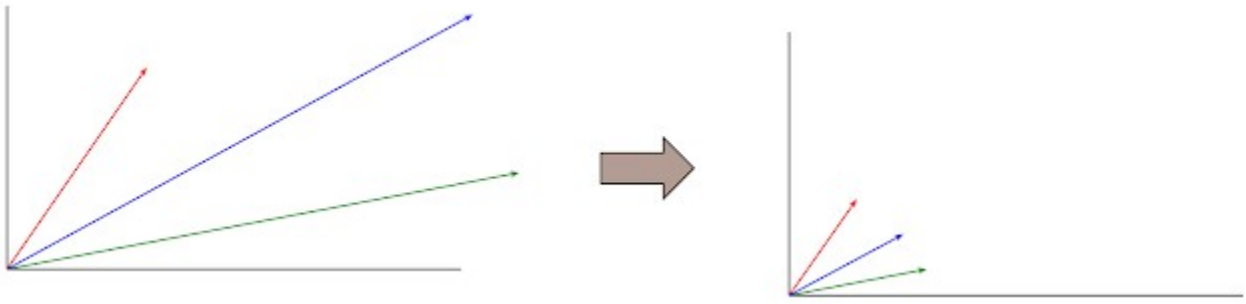
In a 2-dimensional space:



We can observe that the Euclidean distance between q and d_2 is large despite the similar distributions of terms in q and d_2.

A solution to this problem is using cosine similarity, which correlates with the angle between two vectors:

$$sim(x,y)=\frac{x\bullet y}{|x||y|}=\frac{x}{|x|}\bullet\frac{y}{|y|}=\frac{\sum_{i=1}^{n}x_iy_i}{\sqrt{\sum_{i=1}^{n}x_i^2}\sqrt{\sum_{i=1}^{n}y_i^2}}$$



Cosine Similarity is a metric that ranges between 0 and 1, making it particularly suitable for analyzing text data and many other real-world datasets. Unlike Euclidean distance, it focuses on the angle between vectors rather than their magnitude, which can be advantageous in scenarios like document clustering. One of the key advantages of cosine similarity is its computational efficiency, as it only considers features with non-zero values for both examples. This allows it to effectively capture similarities between vectors while disregarding irrelevant or sparse features.

Instead of using Euclidean distance, we use cosine distance, which quantifies the degree of dissimilarity between vectors:

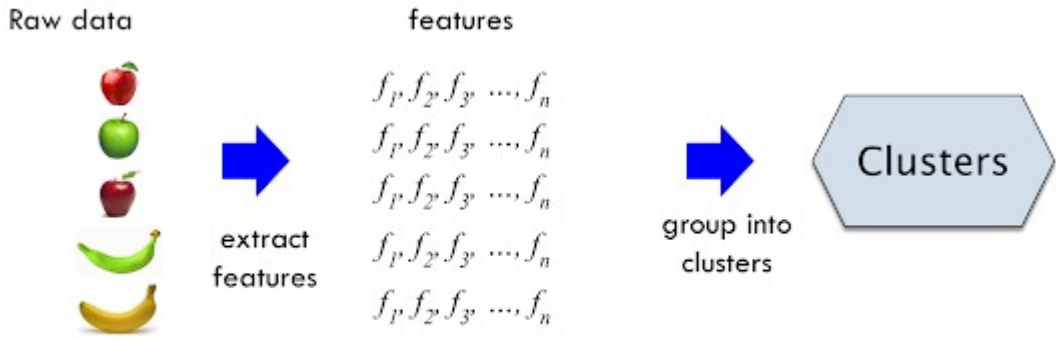
$$d(x,y) = 1 - sim(x,y)$$

k-Means clustering is relatively efficient and can be used for reasonably large training sets. It is popular due to its simplicity.
At a certain point, researchers started to develop deep clustering methods, with the first one incorporating k-Means clustering.

Clustering

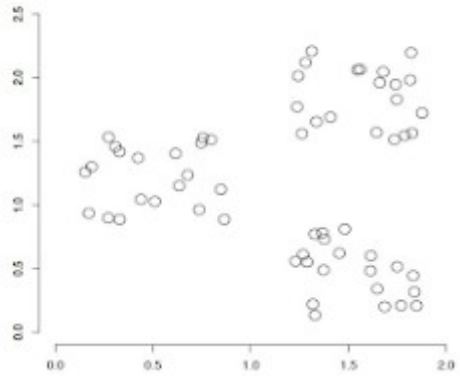
Let’s now have a little discussion about alternative possible methods to k-Means Clustering.





No supervision: we are only given data and want to find groupings

Clustering: the process of grouping a set of objects into classes of similar objects

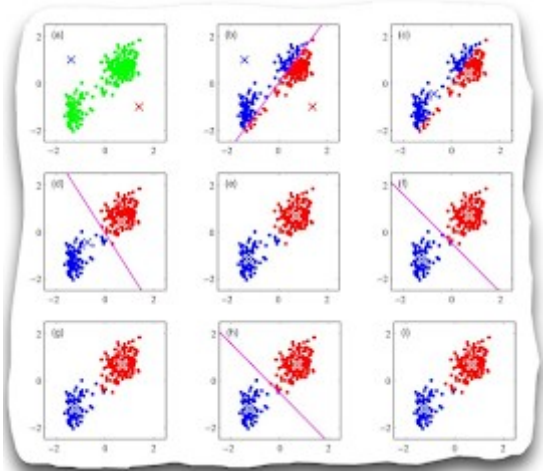


Which clustering algorithms can we use?

What are some of the issues for clustering?

As we've already said, the most well-known and popular clustering algorithm is **k-Means**, which:

- Start with some initial cluster centroids
- Iterate:
    - Assign each example to closest centroid
    - Recalculate centers as the mean of the points in a cluster



Let's now see what the general **issues** are for clustering algorithms.

- **Representation of Examples**
  Determining how to represent examples is pivotal. While using a vector of features is common, it may result in information loss, particularly in scenarios involving complex structures like graphs.
- **Defining a Suitable Similarity Metric**
  Selecting an appropriate metric to measure the similarity or distance between examples is crucial for accurate clustering outcomes.
- **Choosing Between Flat or Hierarchical Clustering**
  Deciding between flat clustering, which assigns each data point to a label, or hierarchical clustering, where similar data points are grouped as siblings in a hierarchical tree, depends on the nature of the data and the desired clustering objectives.
- **Determining the Number of Clusters**
  One of the key challenges in clustering is determining the optimal number of clusters. This can be approached in two ways:
    - **Fixed a Priori**: Predetermining the number of clusters based on prior knowledge or specific requirements.
    - **Data-Driven**: Letting the data guide the determination of the number of clusters, often through techniques like the elbow method or silhouette analysis.

The clustering algorithms can be divided in:

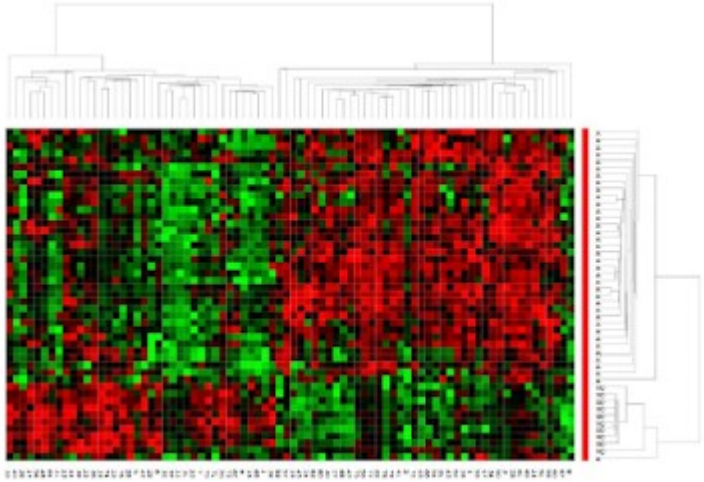
- **Flat algorithms**
    - K-means clustering (usually start with a random partitioning and refine it iteratively)
    - Spectral clustering



- **Hierarchical algorithms**
    - Bottom-up, agglomerative
    - Top-down, divisive



This is an example of hierarchical clustering:

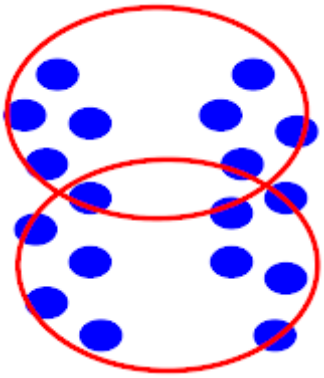


This is a very convenient representation because, with the same dendrogram (the hierarchical tree), we can extract any number of clusters we desire simply by cutting the dendrogram without the need for recomputation. The lower the cut, the more clusters will result.

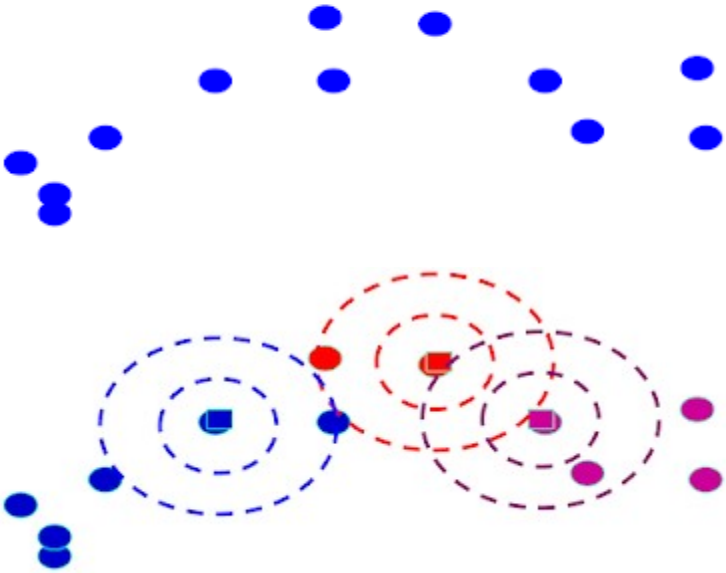
Another type of classification among clustering methods is the following:

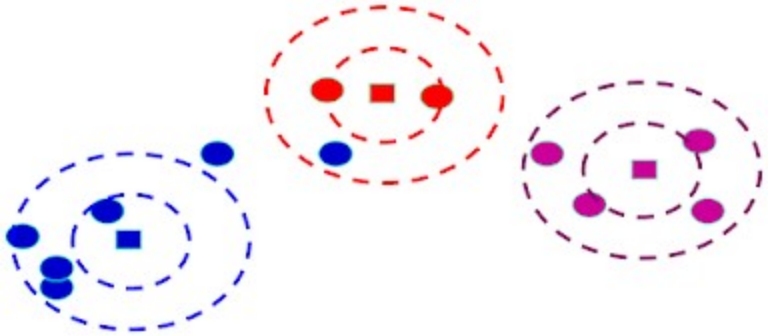
- **Hard Clustering** (e.g., k-Means)
  In hard clustering each example is exclusively assigned to one cluster.
- **Soft Clustering** (probabilistic methods)
  Contrastingly, soft clustering allows examples to belong to more than one cluster, typically represented as probabilities.
  This approach is particularly suitable for applications where instances may exhibit overlap across multiple categories. For instance, in creating browsable hierarchies, it might be desirable to categorize a pair of sneakers into both "sports apparel" and "shoes" clusters simultaneously

One issue with k-Means clustering is its tendency to identify spherical clusters. This stems from the model's reliance on Euclidean distance, leading to an inherent assumption of spherical cluster shapes.



In fact it iteratively learning a collection of spherical clusters:

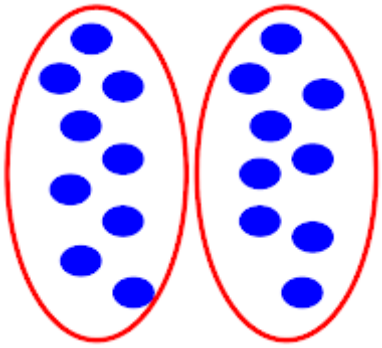


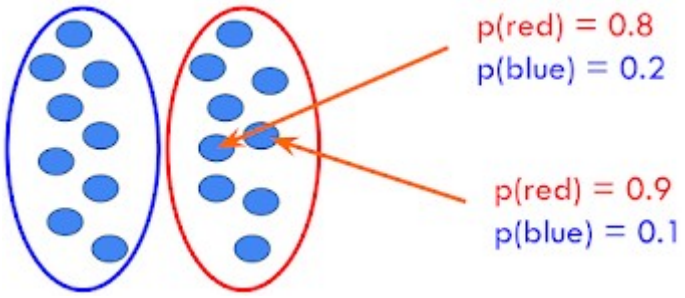


In light of the limitations posed by k-Means clustering in identifying solely spherical clusters, let's explore a more sophisticated alternative: **Expectation Maximization** (EM) Clustering.
Expectation Maximization (EM) operates under the assumption that data stems from a **mixture of Gaussians**, accommodating non-spherical or elliptical data distributions. EM employs a probabilistic approach, allowing for **soft clustering** by assigning data points to clusters with certain probabilities.
This flexibility makes EM Clustering particularly valuable in scenarios where data exhibits complex shapes and distributions that deviate from spherical clusters.

EM Clustering shares a high-level resemblance with K-means, operating through iterative steps of assigning points to clusters and updating cluster centers. However, it distinguishes itself through two key differences:

- **Elliptical Cluster Assumption**: Unlike K-means, which assumes spherical clusters, EM Clustering assumes clusters to have elliptical shapes. This accommodates more diverse data distributions, enabling the algorithm to capture complex patterns effectively.
- **Soft Clustering**: Another significant departure from K-means is EM's soft clustering approach. Rather than strictly assigning data points to clusters, EM assigns probabilities, allowing for more nuanced interpretations of cluster membership. This flexibility is particularly advantageous in scenarios where data points exhibit overlapping or ambiguous cluster assignments.

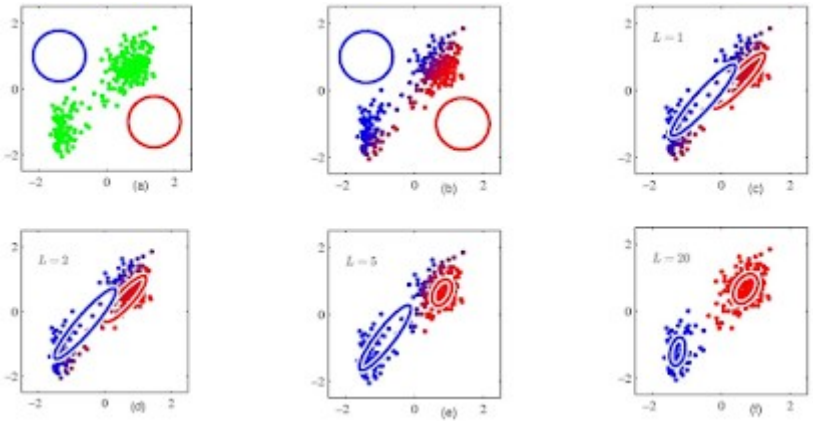




Let's see at high level how the EM clustering algorithm works:

- Start with some initial cluster centers.
- Iterate through the following steps:
    - Softly assign points to each cluster based on their probabilities.
      Calculate $p(\theta_c | x)$, the probability of each point belonging to each cluster.
    - Recalculate the cluster centers based on the current soft assignments.
      Calculate new cluster parameters θ_c representing the maximum likelihood cluster centers given the current soft clustering.

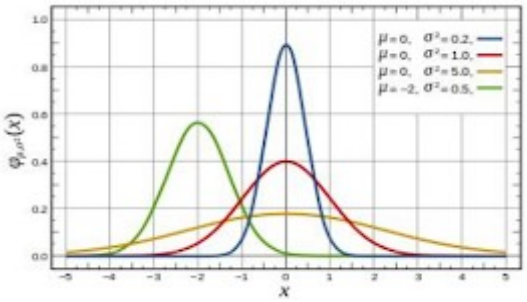
So, we begin by establishing initial cluster centers. Next, we determine the membership probabilities of each point to each cluster, allowing for soft, probabilistic assignment. Then, we recalculate the cluster centers, which represent elliptical clusters and are influenced by the weighted contributions from points (maximum likelihood fitting).



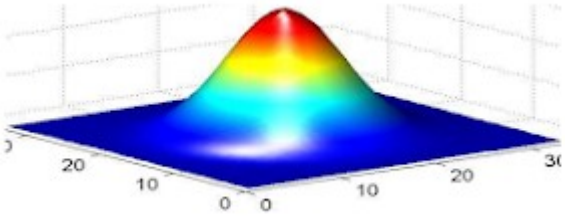
Let's now see how to define a Gaussian (i.e. ellipse).
In one dimension it is defined in the following way:

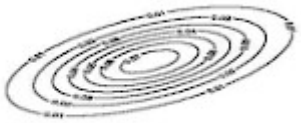
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{\frac{(x-\mu)^2}{2\sigma^2}}$$

And it is parameterized by the mean and the standard deviation, which are what we need to find and optimize.



In two dimensions the representation is the following:





Covariance determines the shape of these contours

And in general (since clustering is applied to *n* dimensions), in an *n* dimensional space the formula of the gaussian is:

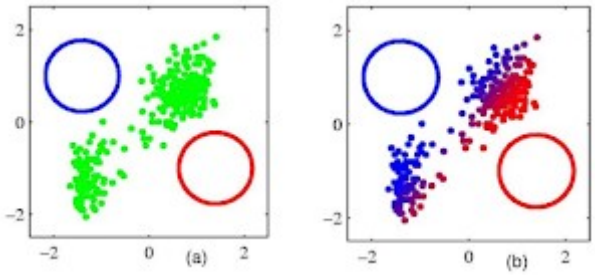
$$N[x;\mu,\Sigma] = \frac{1}{(2\pi)^{d/2}\sqrt{\det(\Sigma)}} \exp[-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)]$$

The data points will be fitted with a mixture of gaussians.
We learn the **means of each cluster** (i.e. the center) and the **covariance matrix** (i.e. how spread out it is in any given direction) which best fit our current data.

So, in step 1, the soft assignment of points to each cluster involves simply computing the probabilities once we have the Gaussians.

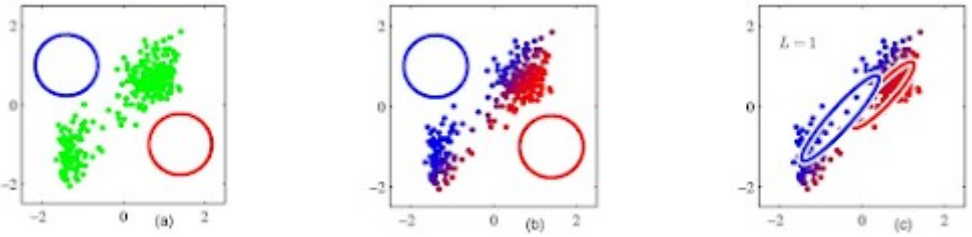
Calculate: $p(\theta_c|x)$

(the probability of each point belonging to each cluster)



These probabilities are obtained by plugging the data points into the Gaussian equation for each cluster (normalized to ensure they form a probability distribution).

In step 2, the recalibration of the cluster centers involves calculating new cluster parameters θ_c, which entails finding the maximum likelihood cluster centers based on the current soft clustering.



Let's now see what it means to fit the gaussian.
If we take this data in a 1-dimensional space

$$10, 10, 10, 9, 9, 8, 11, 7, 6, \ldots$$

And recalling the 1-dimensional gaussian equation:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

We can observe that the Maximum Likelihood Estimation (MLE) is just the mean and variance of the data!

The Phases of EM (Expectation Maximization) are:

- **Expectation Phase**:
  Given the current model, determine the expected probabilities of the data points belonging to each cluster $p(\theta_c|x)$.
- **Maximization Phase**:
  With the probabilistic assignment of all points, estimate a new model θ_c. This process employs maximum likelihood estimation.

And they are similar to k-Means:

- Assign/cluster each point to closest center

  $p(\theta_c|x)$ Soft assignment

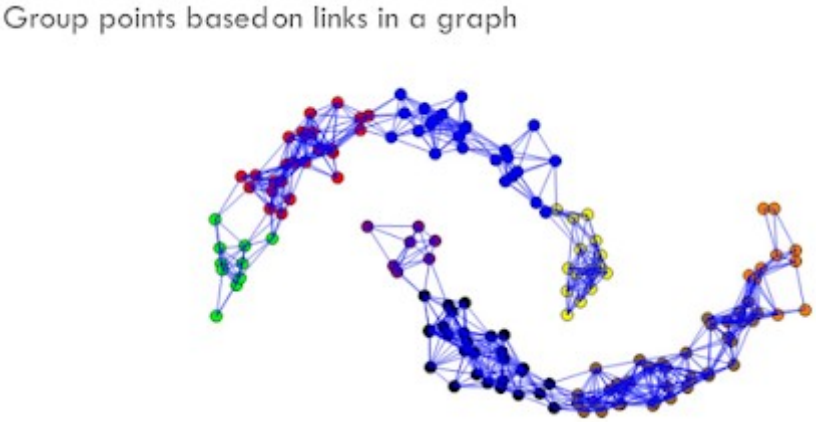
- Recalculate centers as the mean of the points in a cluster

  Estimate a new model θ_c

Expectation Maximization (EM) serves as a versatile algorithm with broader applications beyond clustering, distinguishing it from the more specialized K-Means approach. While sharing similarities with K-Means, such as iterative refinement and the objective of maximizing data likelihood, EM offers a more generalized framework. Each iteration of EM incrementally enhances the likelihood of the data, ensuring convergence, albeit to a local optimum (like k-Means).
EM and its alternation between its two steps are a general procedure which is also used for other purposes in Machine Learning (like training graphical models).

k-Means Clustering and EM Clustering are widely recognized as two of the most popular methods for clustering. However, they may not be suitable for all clustering tasks.
For instance, these methods struggle with datasets that exhibit non-Gaussian distributions. While k-Means seeks a solution for clustering based on a global perspective, optimizing a global objective function, there are scenarios where a local approach might be more appropriate.

Just as in classification, where linear models offer global decisions and k-Nearest Neighbors (k-NN) provide local decisions, there exists a family of clustering algorithms known as **spectral clustering**. These methods adopt a local decision-making approach for clustering tasks and were very useful in fields like computer vision.



The concept is to calculate the distance between each pair of points. As depicted in the figure, some distances are "dropped" (with similarity set to 0) because they are excessively high and deemed irrelevant.

Let's formalize this concept.
We have two main options for constructing the graph:

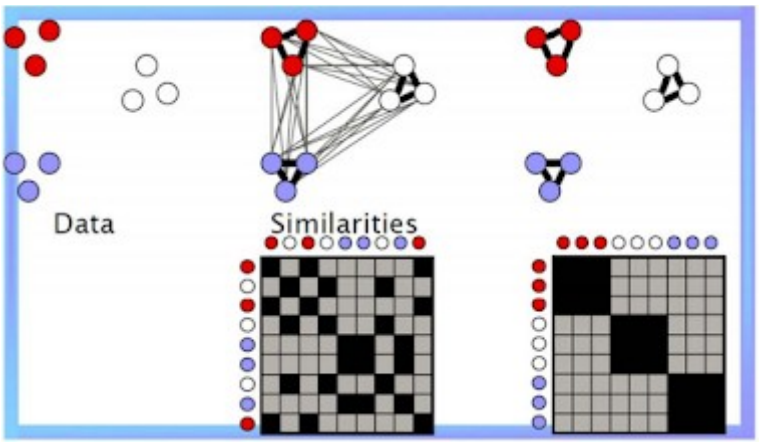
- **Fully Connected Graph**: Each node is directly connected to every other node in the dataset.
- **K-Nearest Neighbor Graph**: Each node is connected only to its K closest neighbors, promoting local connectivity.

We establish connections between nodes based on some measure of similarity. A common approach is to utilize a Gaussian Kernel to determine the similarity between nodes:

$$W(i,j) = \exp\frac{-\left|x_i - x_j\right|^2}{\sigma^2}$$

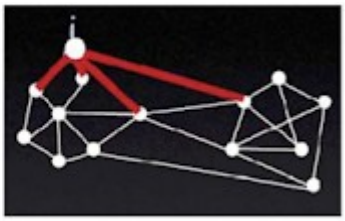
Returning to what was previously mentioned, if $W(i,j)$ is too small, it can be simply dropped.

You create a similarity matrix based on these similarities, and then you prune the graph (graph partitioning) by removing edges with similarities below a certain threshold.



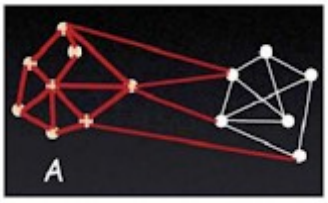
Two important concepts are the **degree of a node**:

$$d_i = \sum_j w_{i,j}$$

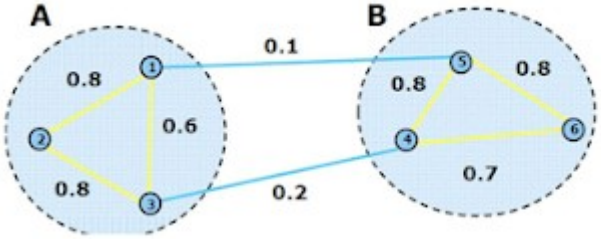


and the **volume of a set**:

$$vol(A) = \sum_{i \in A} d_i, A \subseteq V$$



The concept of graph cut $Cut(A, B)$ involves considering a partition of the graph into two parts, A and B, and it consists in the sum of the weights of the edges connecting these two groups. The intuitive objective is to **find the partition that minimizes the cut**.



$Cut(A, B) = 0.3$

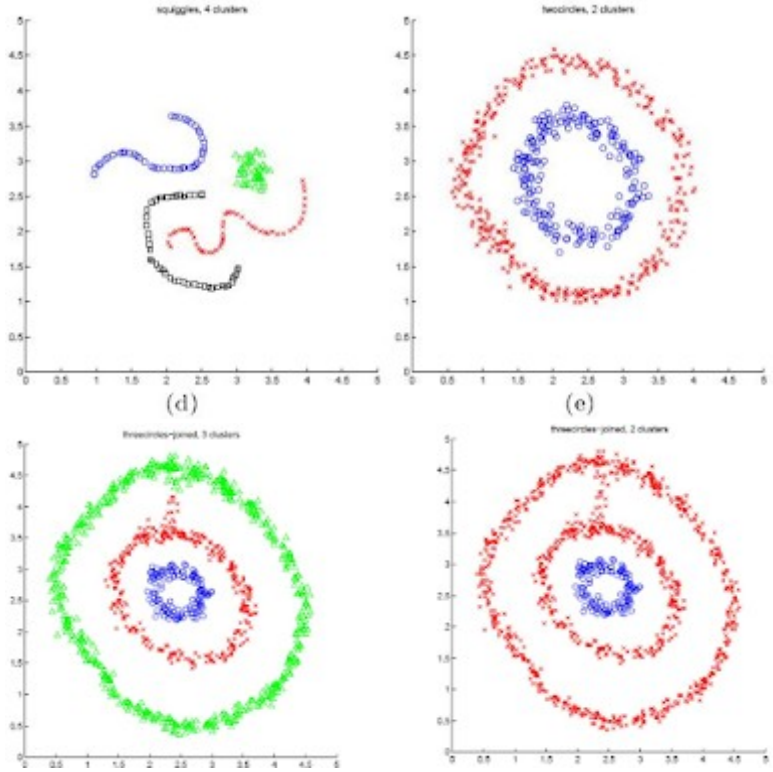
In practice we aim to minimize the **normalized cut**:

$$NCut(A, B) = \frac{Cut(A, B)}{Vol(A)} + \frac{Cut(A, B)}{Vol(B)}$$

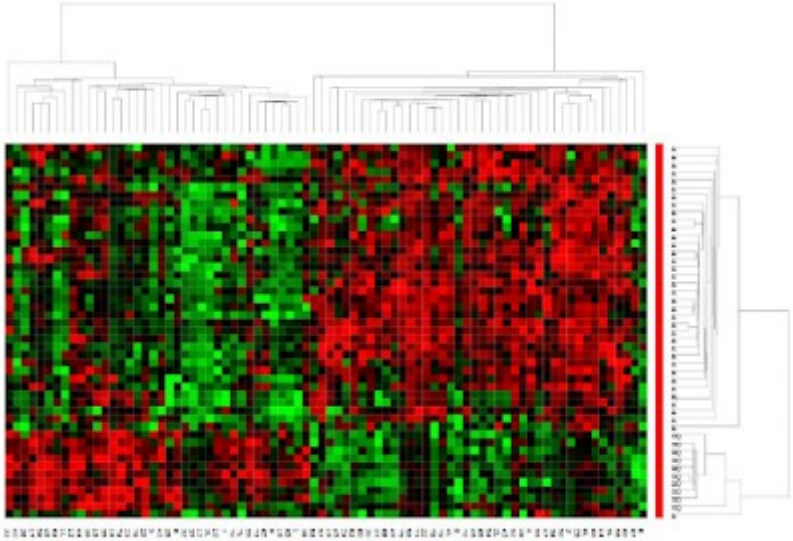
This problem can be formalized as a discrete optimization problem, which means allocating certain variables with binary values (0 or 1). However, solving problems with only integer

variables can be challenging. To address this challenge, the problem can be relaxed or mapped into the continuous domain, eventually transforming it into a generalized eigenvalue problem.

Let's see some examples where k-Means fails, while the spectral clustering algorithm performs exceptionally well:



Let’s see how hierarchical clustering works.

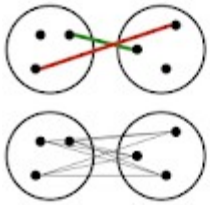


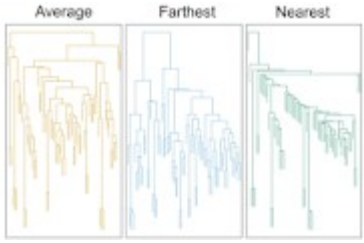
Hierarchical clustering is a method that produces a hierarchical structure of nested clusters, which are organized in the form of a tree known as a **dendrogram**. This dendrogram visually

represents the relationships and similarities between data points, with each branch and node indicating the merging of clusters at various levels of granularity.

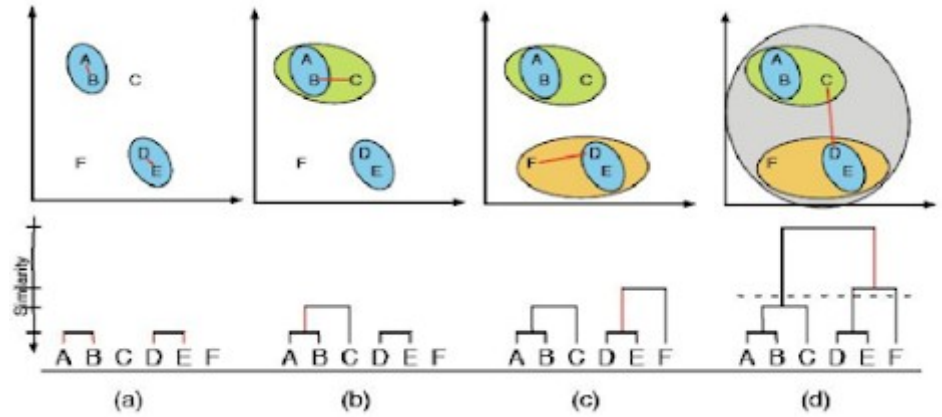
The idea to build a dendrogram is to first merge very similar instances and then incrementally build larger clusters out of smaller clusters.
Here's a step-by-step explanation of how to build a dendrogram in hierarchical clustering:

1. **Calculate the Distance Matrix**: First, you need to calculate the distance between each pair of data points.
2. **Determine Linkage Criteria**: Hierarchical clustering uses different linkage criteria to decide how to merge clusters at each step of the algorithm, since the concept of distance between objects of multiple elements isn't defined. The most common linkage criteria include:
    - Single Linkage (closest pair): The distance between two clusters is defined as the shortest distance between any two points in the two clusters.
    - Complete Linkage (farest pair): The distance between two clusters is defined as the maximum distance between any two points in the two clusters.
    - Average Linkage: The distance between two clusters is defined as the average distance between all pairs of points in the two clusters.
    - Ward's Method: This criterion minimizes the variance when merging clusters. It tends to merge clusters that minimize the increase in total within-cluster variance.





3. **Initialize Clusters**: Start by considering each data point as a separate cluster.
4. **Merge Clusters**: Iteratively merge clusters based on the chosen linkage criterion and the distance matrix. At each step, merge the two closest clusters according to the chosen criterion. Update the distance matrix to reflect the distances between the newly formed clusters and the remaining clusters.
5. **Construct the Dendrogram**: As clusters are merged, a dendrogram is constructed to illustrate the hierarchy of the clustering process. The x-axis of the dendrogram represents the data points or clusters, and the y-axis represents the distance or dissimilarity between them. The height of each fusion in the dendrogram represents the distance at which the clusters were merged.
6. **Stop when there is only one cluster left**: The hierarchical clustering process continues until all data points are merged into a single cluster. At this point, the dendrogram is complete, and the hierarchical structure of the data is fully represented.



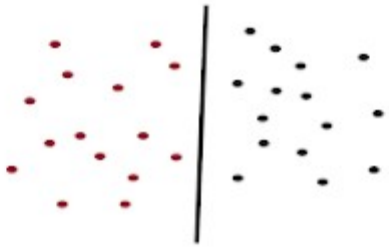
This algorithm produces not just one clustering, but a **family of clusterings** represented by a dendrogram.

Introduction to Neural Networks

Let's first review the concept of linear models.

LINEAR MODELS

- A **linear model** is a model that assumes the data is linearly separable
- We studied the **perceptron**



We've seen the perceptron learning algorithm:

repeat until convergence (or for some # of iterations):

for each training example $(x_{i1}, x_{i2}, \ldots, x_{in}, y)$: label is -1/1

$$prediction = \sum_j w_j x_{ij} + b$$

if $prediction \cdot y_i \leq 0$: // they don't agree

for each w_j:

$w_j = w_j + x_{ij} y_i$

$b = b + y_i$

Algorithm 5 PERCEPTRONTRAIN(D, *MaxIter*)

```
w_d ← 0, for all d = 1...D                          // initialize weights
b ← 0                                               // initialize bias
for iter = 1 ... MaxIter do
  for all (x,y) ∈ D do
    a ← Σ_{d=1}^{D} w_d x_d + b                     // compute activation for this example
    if ya ≤ 0 then
      w_d ← w_d + yx_d, for all d = 1...D           // update weights
      b ← b + y                                     // update bias
    end if
  end for
end for
return w_0, w_1, ..., w_D, b
```

Algorithm 6 PERCEPTRONTEST($w_0, w_1, \ldots, w_D, b, \hat{x}$)

```
a ← Σ_{d=1}^{D} w_d x̂_d + b                         // compute activation for the test example
return SIGN(a)
```

And we've also seen that this procedure is very similar to the gradient descent one.

PERCEPTRON LEARNING & GRADIENT DESCENT

repeat until convergence (or for some # of iterations):

for each training example $(x_{i1}, x_{i2}, \ldots, x_{in}, y)$:

$$prediction = \sum_j w_j x_{ij} + b$$

if $prediction \cdot y_i \leq 0$: // they don't agree

Note: for gradient descent, we always update

for each w_j:

Perceptron update: $w_j = w_j + x_{ij} y_i$

$b = b + y_i$

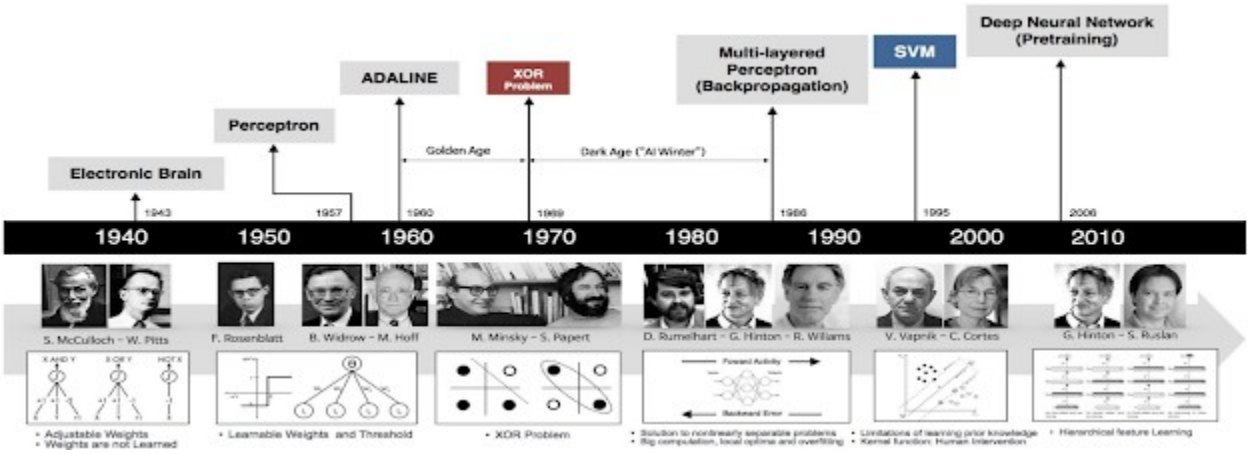
Gradient descent update: $w_j = w_j + \eta y_i x_{ij} \exp(-y_i(w \cdot x_i + b))$

In practice $w_j = w_j + x_{ij} y_i c$ where $c = \eta \exp(-y_i(w \cdot x_i + b))$

From Perceptron to Deep Networks

The reason we reviewed the perceptron is that it serves as the foundational building block for constructing a Feedforward Neural Network.

The history of Neural Networks is as follows:

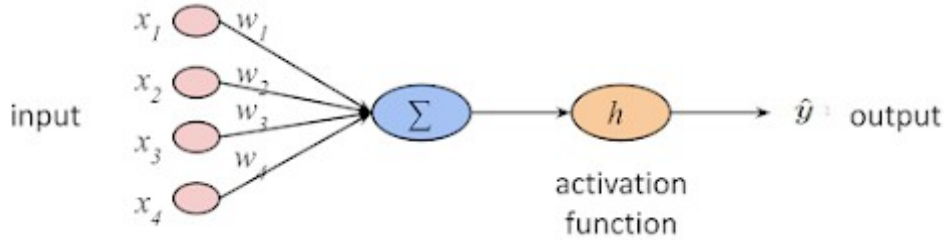


Understanding this history is crucial for grasping the basic concepts that form the foundation of neural networks. Here are the key milestones:

- **Electronic Brain (1943): McCulloch & Pitts**
  Introduced the concept of a simplified brain cell, the McCulloch-Pitts neuron. The weights in this model are not learned but are set manually.
- **Perceptron (1957): Rosenblatt and ADALINE (1960): Widrow & Hoff**
  The Perceptron, invented by Rosenblatt, was the first neural network with the ability to learn weights from data. Widrow and Hoff's ADALINE was another important milestone, introducing the concept of error-correction learning.
- **XOR Problem (1969): Minsky & Papert**
  Minsky and Papert highlighted one of the key weaknesses of early neural networks: their inability to solve the XOR problem, a simple problem that linear models can't handle.
- **Multi-layered Perceptron (Backpropagation) (1986): Rumelhart, Hinton & Williams**
  The introduction of backpropagation, a powerful learning algorithm, made it possible for multi-layered Perceptrons to solve problems that earlier models couldn't. However, they also introduced new challenges, such as the risk of overfitting and getting stuck in local optima.
- **SVM (1995): Vapnik & Cortes**
  Support Vector Machines (SVMs) represent a different approach to supervised learning. They introduced the concept of maximizing the margin to improve generalization. However, they require the use of kernel functions, which often need to be carefully chosen and can involve human intervention.
- **Deep Neural Network (Pretraining) (2006): Hinton & Ruslan**
  Deep Neural Networks, with many layers of non-linear processing units, came into the

spotlight with the advent of efficient training methods. These networks learn hierarchical representations of data, which can lead to superior performance on many tasks.

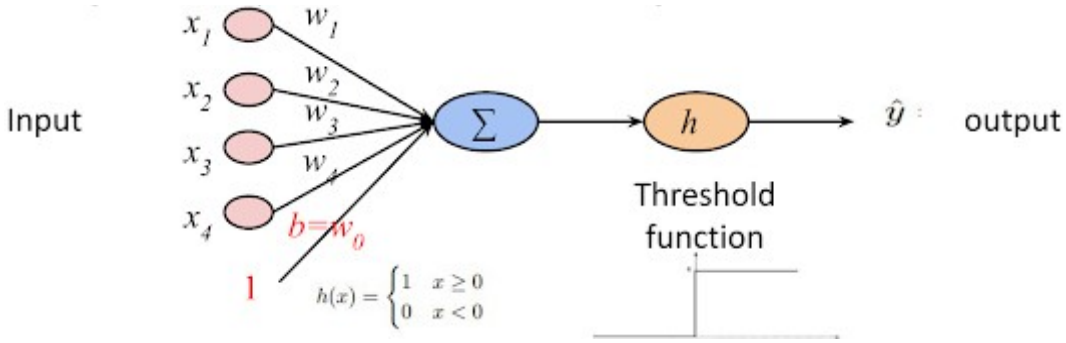
We've already seen the perceptron as a technique to learn a linear model. Let's delve into a more complex definition of what a perceptron is:
An **artificial neuron**, or **perceptron**, is a non-linear parameterized function with a restricted output range.



In this image the bias is missing because it isn't the core of the thing.

An **activation function** is a crucial element that determines the output of the perceptron based on its input. It helps introduce non-linearity to the model, allowing it to learn and represent complex patterns in data. The activation function then decides whether the perceptron should "fire" (output 1) or "not fire" (output 0) based on the weighted sum.
In the original perceptron model, a step function (thresholding) is used as the activation function. However, this function has limitations in terms of learning complex patterns in data.
Modern neural network architectures use various activation functions to introduce non-linearity and enable the network to learn more complex relationships in data. Popular activation functions include the sigmoid function, hyperbolic tangent (tanh) function, and Rectified Linear Unit (ReLU) function, among others.
These activation functions allow neural networks to model and approximate a wide range of functions, making them powerful tools for tasks such as classification, regression, and pattern recognition.

Rosenblatt's work in 1958 introduced the perceptron, conceptualizing it as a machine primarily designed for linear classification tasks. The main idea behind the perceptron was to learn the appropriate weights for each input and account for bias, thereby enabling it to make decisions based on input data.
In its simplest form, the perceptron assigns one weight to each input feature. These weights are then multiplied by their respective inputs, and the resulting products are summed along with a bias term. This aggregated value is compared to a predefined threshold. If the sum exceeds the threshold, the perceptron outputs a 1, indicating one class; otherwise, it outputs a 0, representing another class.
Essentially, the perceptron's algorithm follows a straightforward process reminiscent of the sign function we've encountered previously. However, its significance lies in its ability to autonomously adjust weights and bias during the learning process, thereby enabling it to iteratively improve its classification performance over time. This foundational concept laid the groundwork for more complex neural network architectures and learning algorithms that followed.

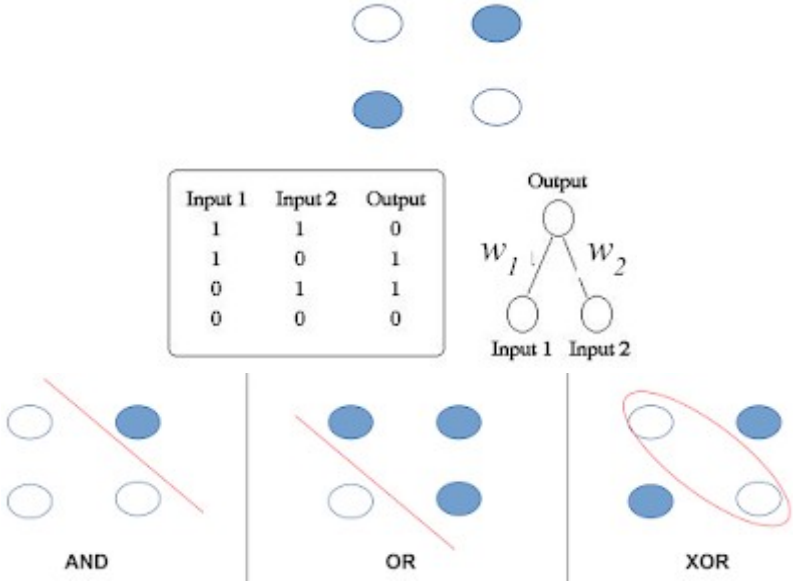


Minsky and Papert's seminal work in 1969 shed light on a crucial limitation of perceptrons: their inability to solve non-linearly separable problems, exemplified by the Exclusive OR (XOR) function.

The XOR function is a classic example of a problem that cannot be solved by a single-layer perceptron. Unlike linearly separable problems where a single straight line can divide the data into distinct classes, the XOR function requires a non-linear decision boundary to properly classify its inputs. This means that no matter how the weights and biases of a perceptron are adjusted, it cannot effectively capture the XOR function's underlying pattern.

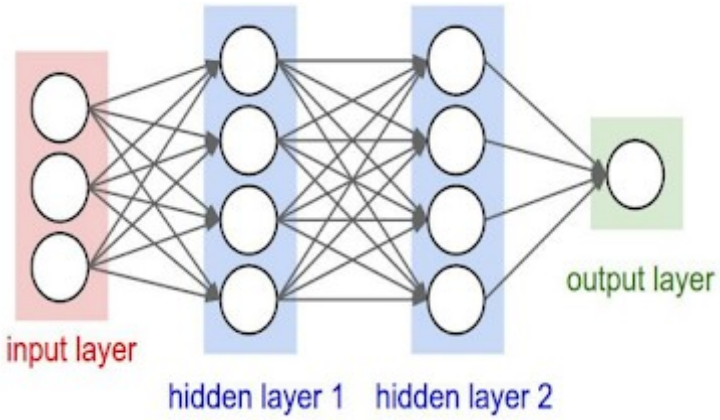
This discovery was significant as it highlighted the perceptron's limitations in handling more complex data structures. It prompted researchers to explore more sophisticated neural network architectures, such as multi-layer perceptrons (MLPs), which could overcome the XOR problem by introducing hidden layers capable of capturing non-linear relationships in the data.

Minsky and Papert's work underscored the importance of understanding the capabilities and limitations of early neural network models, paving the way for the development of more powerful and versatile architectures in the field of artificial intelligence.



The advent of the **Multi-layer Perceptron** (MLP) marked a significant breakthrough in neural network research, particularly in overcoming the limitations posed by single-layer perceptrons, as highlighted by Minsky and Papert's work.

With an MLP, solving complex problems like XOR became possible. The key innovation lay in its architecture, which allowed for densely connected layers of artificial neurons. By stacking multiple layers and interconnecting neurons within each layer, MLPs could effectively capture and represent compositions of non-linear functions.
This architectural design enables MLPs to learn and model intricate patterns in data, making them versatile tools for a wide range of supervised learning tasks. Whether it's classification, regression, or other tasks, MLPs demonstrate remarkable effectiveness in handling arbitrary supervised learning problems.
By leveraging the flexibility and expressive power of MLPs, researchers and practitioners have been able to tackle increasingly complex real-world challenges in fields such as computer vision, natural language processing, and beyond. The development of MLPs marked a crucial milestone in the evolution of neural network models, paving the way for the modern deep learning revolution.



In a Multi-Layer Perceptron (MLP), we typically encounter three types of layers:

- **Input Layer**
  The input layer is where the data is introduced into the neural network. Each neuron in this layer represents a feature of the input data. The number of neurons in the input layer corresponds to the number of input features in the dataset.
- **Hidden Layers**
  Hidden layers are the intermediate layers between the input and output layers. These layers perform complex transformations on the input data through weighted connections and activation functions. The term "hidden" indicates that the output of these layers is not directly observed in the training data; they serve as intermediary representations that enable the network to learn and capture intricate patterns in the data. MLPs can have one or more hidden layers, each containing numerous neurons.
- **Output Layer**
  The output layer is where the final predictions or outputs of the neural network are generated. The number of neurons in the output layer depends on the nature of the task. For instance, in a classification task with multiple classes, each neuron in the output layer typically corresponds to one class, and the network's output represents the predicted probabilities or class labels. In regression tasks, the output layer may consist of a single neuron representing the predicted continuous value.

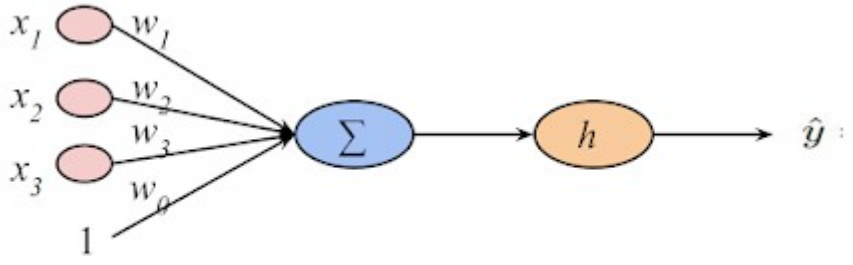
These layers work together in a feedforward manner, where the output of each layer serves as the input to the next layer. Through the process of forward propagation, data is transformed as it passes through the network, ultimately producing the desired output. The architecture and configuration of these layers play a crucial role in determining the MLP's ability to learn and generalize from the input data.

The concept of aggregating multiple perceptrons is encapsulated in what is commonly referred to as a Multilayer Perceptron (MLP) or **Feed-Forward Neural Network** (FFNN). These networks possess several key characteristics:

- **Directed Acyclic Graph (DAG)**:
  Information flows unidirectionally from the inputs through the hidden layers to the outputs without any feedback loops (and this is why they are called also Feed-Forward Neural Network), forming a Directed Acyclic Graph (DAG). This absence of cycles ensures that each computation is solely based on the inputs and the model parameters, thereby avoiding the complexities introduced by feedback connections.
- **One or more non-linear functions**:
  Feed-forward neural networks compute one or more non-linear functions. This computation is achieved through the composition of numerous algebraic functions implemented by the connections, weights, and biases of the hidden and output layers. These connections, often represented as weighted edges in the network, are adjusted during the training process to minimize the difference between the model's predictions and the actual outputs, utilizing a technique known as backpropagation.
- **Hidden layers compute intermediate representations**:
  The hidden layers within these networks play a crucial role in the learning process. They act as intermediate computational layers, extracting and transforming the input data into higher-level representations that facilitate the network's ability to generalize patterns and make accurate predictions. Through the iterative process of forward and backward passes during training, these hidden layers gradually refine their representations, enabling the network to learn complex relationships within the data.

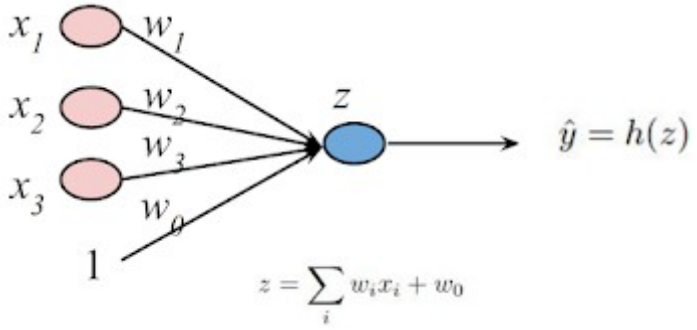
Let's now observe, through images and formulas, how we can transition from a classical perceptron to a Feed-Forward Neural Network.

In a **single perceptron** the output is the activation function applied to the dot product between the feature vector and the weights plus the bias:

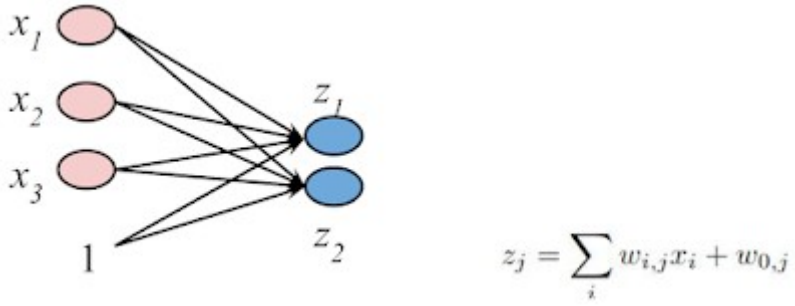


$$\hat{y} = h(\sum_i w_i x_i + w_0) = h(\mathbf{w}^T \mathbf{x} + w_0)$$

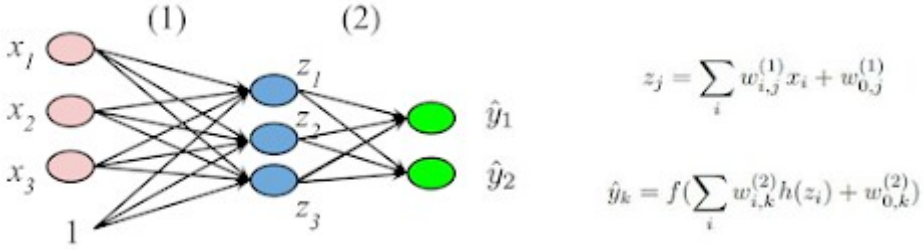
We can redraw the diagram into a simplified model by removing the node representing the activation function within the network.



Now we can create multiple outputs by creating multiple z's, meaning that we will use different weights and biases for each z (each edge is associated with a different weight):



From this, we can obtain a **single-layer neural network** by adding another layer of edges to other nodes:



The first layer is calculated in the same manner as previously described. The second layer takes as input the outputs of the previous layer and computes the result in a similar fashion. Each layer applies the activation function to the dot product between the vector obtained from the previous layer and the weights of that layer. Typically, the activation function remains consistent within the same neural network, but the function applied to the output layer can be different.
We use the apexes to distinguish between the weights of the various layers.

It is possible to add as many layers as we want and in this way we can perform any kind of arbitrary complex function.

The onset of the **First AI Winter** coincided with the challenge of training a Multilayer Perceptron (MLP). The issue stemmed from having knowledge of only the desired target for the **output layers**, while lacking information for the **hidden layers**. The traditional perceptron learning rule,

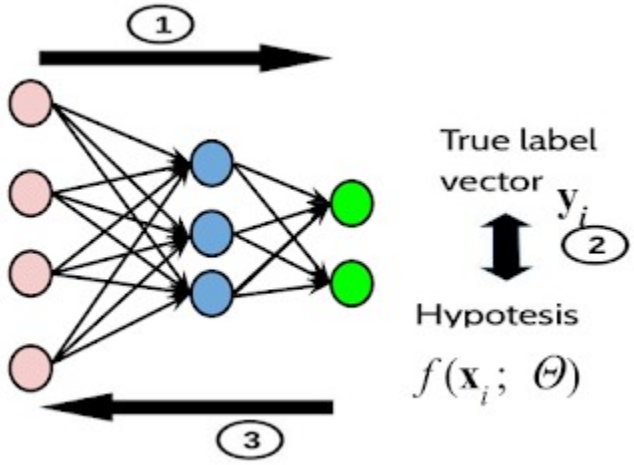
which is the primary method for setting weights in a perceptron and relies on knowing the desired target, proved inapplicable in this scenario.

The AI Winter persisted from 1969 to 1986, until the invention of the **backpropagation algorithm** revitalized the field. With backpropagation, learning Multilayer Perceptrons (MLPs) for complex functions became solvable. Backpropagation is an efficient algorithm capable of processing large training sets. Today, this algorithm remains at the core of neural network training.

Here's a concise overview of how backpropagation works:

1. **Forward Propagation**:
   During forward propagation, inputs are summed, activations are produced, and the signal is fed forward through the network.
   Inputs are multiplied by weights, summed, and then passed through an activation function (such as sigmoid or ReLU) to produce the output of each neuron.
   This process is repeated layer by layer until the final output is generated.
2. **Error Estimation**:
   Once the forward pass is complete and the output is obtained, the next step is to calculate the error or the difference between the predicted output and the actual output.
   This error estimation is crucial for understanding how well the network is performing and for guiding the adjustment of weights during the training process.
3. **Backpropagation of Error**:
   After error estimation, the error signal is backpropagated through the network.
   This involves propagating the error backward from the output layer to the hidden layers.
   The error at each layer is used to adjust the weights in a way that minimizes the overall error of the network.
   The gradient of the error with respect to each weight is calculated using techniques like the chain rule of calculus, and weights are updated accordingly using optimization algorithms like stochastic gradient descent (SGD).

By iteratively performing forward propagation, error estimation, and backpropagation, neural networks are trained to minimize their error and improve their ability to make accurate predictions. This process lies at the heart of training deep learning models and has enabled significant advancements in various fields such as image recognition, natural language processing, and robotics.



A neural network built in this way is extremely flexible because it can accommodate a wide range of loss functions.

Similarly to other algorithms, such as Support Vector Machines (SVM), learning in neural networks revolves around solving an optimization problem. Here's the main idea:

- Given a set of training samples $T = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$, the goal is to adjust all the weights of the network Θ such that a cost function is minimized:

  $min_{\Theta} \sum_{i}^{\square} L(y_i, f(x_i, \Theta))$

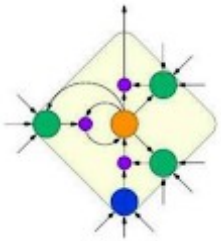
  The cost function measures the discrepancy between the predicted outputs of the neural network and the true labels of the training samples.
- Choose your **loss function**: Common choices include square loss, cross-entropy loss, or any other appropriate loss function depending on the nature of the problem.
- The weights of each layer in the neural network are updated iteratively using optimization algorithms like **gradient descent**.
  Gradient descent aims to minimize the cost function by adjusting the weights in the direction of the steepest descent of the cost surface.
- **Backpropagation** of the error signal is used to **efficiently compute the gradient** of the cost function with respect to each weight in the network. This allows for efficient calculation of the gradient and facilitates weight updates during training.

By iteratively minimizing the cost function using gradient-based optimization methods and backpropagating the error signal through the network, neural networks are trained to accurately predict outputs for unseen data, making them powerful tools for a wide range of tasks in machine learning and artificial intelligence.

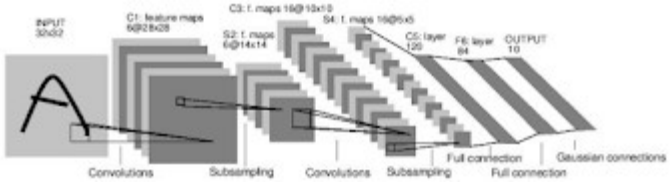
As of 2024, we have neural networks capable of performing unsupervised learning tasks. However, for now, we will focus only on supervised learning.

Following the invention of backpropagation, researchers began exploring the capabilities of feedforward neural networks (FFNNs) for solving complex problems. It became evident that FFNNs were highly adaptable and flexible. Instead of solely designing these networks, creativity was encouraged in designing various types of neurons and layers. Two significant advancements emerged:

- **Recurrent Long-Short Term Memory Networks** LSTM (Schmidhuber, 1997):
  A specialized model designed for handling sequences, such as words in a document. Unlike traditional feedforward networks, recurrent neural networks (RNNs) possess loops within their architecture, allowing them to retain memory and process sequential data efficiently.



- **Convolutional Neural Networks** CNN (LeNet in 1998):
  CNNs revolutionized tasks like Optical Character Recognition (OCR).
  CNNs are tailored for processing structured data, particularly matrices like images. Their architecture incorporates convolutional layers, pooling layers, and fully connected layers, enabling effective feature extraction and pattern recognition in image data.



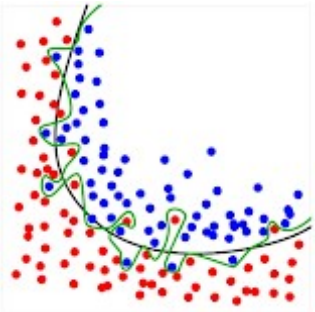
These advancements were pivotal to:

- **Handle Variable-Length Sequences**:
  Utilizing FFNNs for representing sequences like documents poses challenges due to variable lengths.
  RNNs, particularly LSTM networks, excel in processing sequences of varying lengths, making them ideal for tasks involving text data.
- **Preserving Spatial Information**:
  Although images can be flattened into vectors, this representation loses crucial spatial information.
  CNNs preserve spatial relationships through convolutional operations, enabling them to effectively capture features within images.

On Asimov Institute's Neural Network Zoo, it is possible to find an overview of the different types of neural networks invented over the years. This illustrates the vastness of the field.

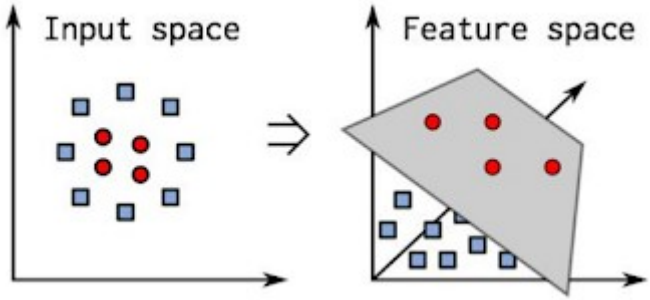
The **second AI winter** followed a period of excitement that lasted around ten years. Three main causes contributed to this downturn are:

- **Complex problems that neural networks can't handle**: The increasing complexity of problems necessitated deep networks with many layers and parameters. However, neural networks faced challenges such as overfitting and the vanishing gradient problem, hindering effective training.
  The vanishing gradient problem occurs during neural network training when gradients become increasingly small as they propagate backward through many layers. This can hinder the ability of earlier layers to update their weights effectively, leading to slower learning or stagnation in training progress.
- **Lack of Processing Power**: Limited processing power, particularly the absence of Graphics Processing Units (GPUs), constrained the ability to train deep neural networks efficiently.
- **Insufficient Data**: The lack of large annotated datasets hindered the training of neural networks, limiting their ability to generalize effectively to real-world scenarios.

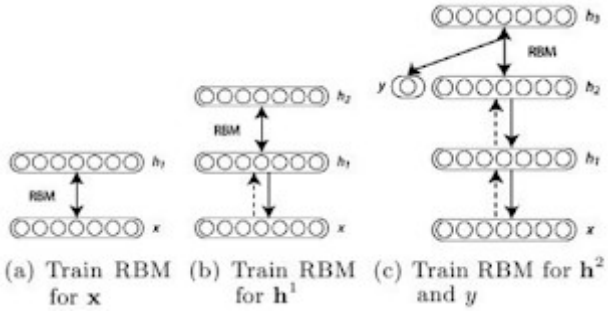


In the 1990s, the choice between Neural Networks (NN) and Support Vector Machines (SVMs) arose. Kernel Machines, particularly SVMs, gained sudden popularity, becoming a substitute for NNs for many years. The reasons behind this shift were:

- **Comparable Accuracies**: SVMs demonstrated similar accuracies to neural networks in various tasks. They performed well, particularly when coupled with effective feature extractors, as observed in applications like pedestrian detection.
- **Simplicity in Training**: SVMs offered advantages in terms of simplicity, with fewer heuristics and parameters compared to neural networks. Training SVMs involved solving a convex optimization problem, ensuring the existence of a unique minimum, unlike the complexities associated with gradient descent in neural networks.
- **Generalization Guarantees**: SVMs provided reassuring proofs on generalization capabilities. Unlike neural networks, SVMs offered guarantees on generalization, enhancing confidence in their performance across diverse datasets and applications.
- **Efficiency with Non-linear Problems**: Handling non-linear problems with neural networks often required scaling up complexity by appending many layers. SVMs with the kernel trick proved to be more efficient in addressing non-linearities, offering a more streamlined approach compared to the increased complexity of neural networks.

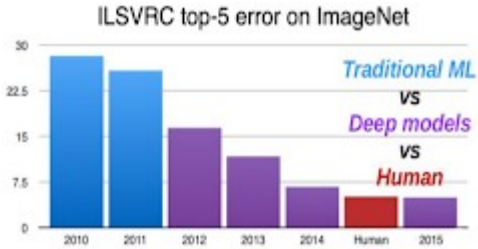


SVMs dominated the scene until 2006 when Geoffrey Hinton, often referred to as the "godfather" of deep learning for his pivotal contributions such as the invention of backpropagation, along with his students, introduced **Learning Deep Belief Nets**. This innovative approach involves initializing network weights by training each layer one by one with unsupervised training, utilizing techniques like contrastive divergence. This method yields significantly better weight initializations than random values.
Subsequently, the network's weights were fine-tuned through a round of supervised learning. This approach yielded state-of-the-art performance, especially when trained on benchmark datasets like MNIST, surpassing the capabilities of SVMs.



Another significant acceleration occurred when Hinton created the first neural network for a complex task: image classification. Hinton's group developed **AlexNet**, a convolutional neural network (CNN) similar to LeNet [LeCun, 1998], but trained on the ImageNet dataset using two GPUs. With additional technical enhancements such as ReLU activation functions, dropout regularization, and data augmentation, AlexNet demonstrated groundbreaking performance in image classification tasks.



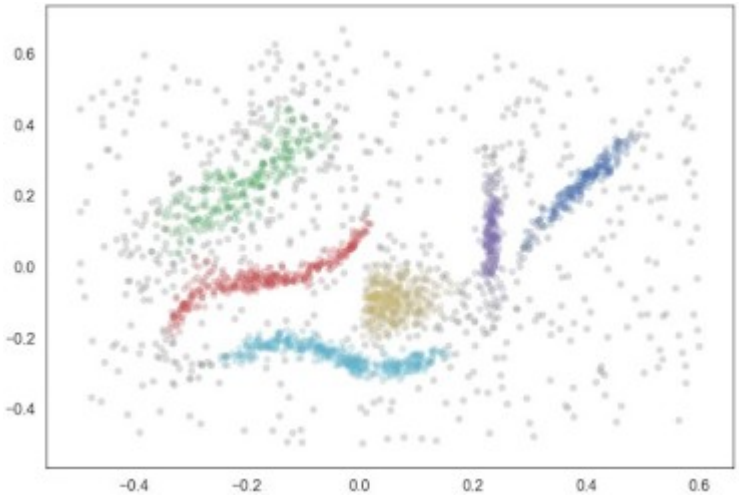


The example of AlexNet was mentioned because computer vision was at the forefront of AI, but the Learning Deep Belief Nets changed the entire field, impacting other areas as well.

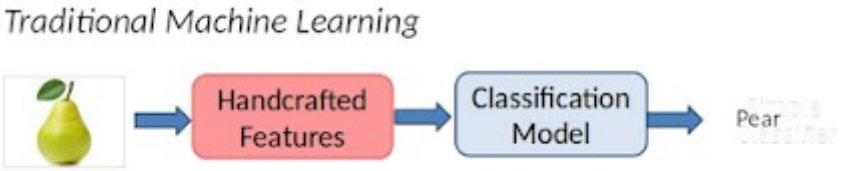
For most of the problems encountered in machine learning, we rely on a function to transform our examples into features, which inevitably leads to a loss of information. Our machine learning algorithm doesn't directly perceive the original data



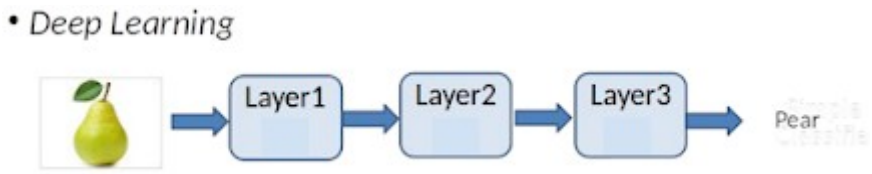
but rather operates on this transformed representations



One of the primary distinctions between traditional machine learning and deep learning lies in the separation of feature extraction and classification phases. Traditional methods typically involve human-performed feature extraction, known as handcrafted features, followed by learning. This human effort heavily influences the final result.



In contrast, deep learning adopts an end-to-end learning approach, where a learnable representation is directly obtained from the data. The process is opaque regarding where feature extraction and classification occur. Consequently, the algorithm autonomously extracts relevant features during training, and modifying aspects like the loss function can alter the representation.

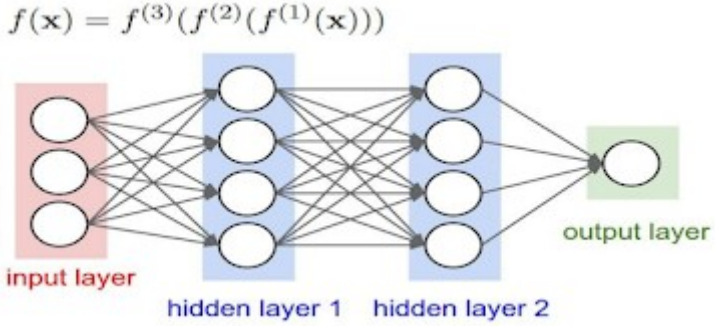


This shift in approach transfers the burden from engineers designing feature extractors to designing deep learning architectures. It enables more automated and flexible feature extraction as neural networks autonomously learn to extract relevant features directly from the data during training.

A neural network is composed of **modules** (building blocks), forming a series of hierarchically connected functions, with each having its own parameters.

Feedforward Networks: Basic Elements

Let's delve into the technical details of Feedforward Networks.



A Feedforward Neural Network consists of a series of nested functions. Each function includes a dot product with weights corresponding to multiple edges, along with an activation function that introduces non-linearity.

The purpose of a Feed-Forward Neural Network is to approximate some unknown ideal function $f^* : X \rightarrow Y$. To achieve this, it defines a parametric mapping $f(x_i;\Theta)$, where Θ represents the parameters, and then learns these parameters to obtain a good approximation of f^* from available samples.
The information flow in function evaluation starts at the input, proceeds through intermediate computations, and culminates in producing the output. As previously mentioned, the function f is a composition of multiple functions. This function composition can be described by a Directed Acyclic Graph, where $f^{(1)}$ is the first layer, $f^{(2)}$ is the second layer, and so on.
The depth is the maximum i in the function composition chain, and the final layer is called the output layer.

Now, let's discuss the **training** process of a Feed-Forward Neural Network.
The objective is to optimize the parameters Θ to drive $f(x_i;\Theta)$ towards $f^*(x)$. The training data comprises evaluations of $f^*(x)$ at various x instances (i.e., expected outputs).

However, it only specifies the output of the output layer. The outputs of intermediate layers are not specified, hence the nomenclature "hidden layers".

Designing and training a neural network is not significantly different from training any other machine learning model using gradient descent. However, it introduces challenges such as non-convex optimization problems and lack of convergence guarantees.

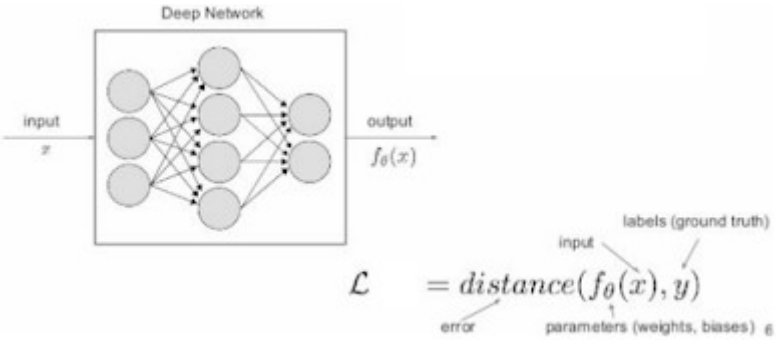
To apply gradient descent, it is necessary to specify a model, a cost function, and the output representation. These components are crucial for defining the neural network architecture and guiding the optimization process during training.

When designing a neural network, AI engineers must make several crucial modeling choices:

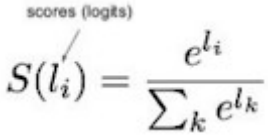
- **Cost Function**:
  The choice of cost function determines how the network's performance is evaluated during training. Common choices include mean squared error for regression tasks and cross-entropy loss for classification tasks. The selection of an appropriate cost function depends on the specific task and the nature of the data.
- **Form of Output**:
  The form of output specifies how the network's predictions are represented. This could be a single scalar value for regression tasks, a probability distribution over classes for classification tasks, or a sequence of values for sequence prediction tasks. Selecting the appropriate output representation is essential for ensuring that the network outputs are meaningful and interpretable.
- **Activation Functions**:
  Activation functions introduce non-linearity into the network, allowing it to learn complex relationships in the data. Common activation functions include sigmoid, tanh, and ReLU (Rectified Linear Unit). The choice of activation function affects the network's ability to capture non-linear patterns and can influence the training dynamics.
- **Architecture**:
  The architecture of the neural network, including the number of layers, the size of each layer, and the connections between layers, plays a crucial role in determining its capacity and expressiveness. Deep networks with multiple layers can capture intricate patterns in the data but may require more computational resources and longer training times.
- **Optimizer**:
  The optimizer determines how the network's parameters are updated during training to minimize the chosen cost function. Popular optimizers include stochastic gradient descent (SGD), Adam, and RMSprop. The choice of optimizer can impact the convergence speed and final performance of the network.

By carefully considering these modeling choices, AI engineers can design neural networks that effectively learn from data and achieve superior performance on their intended tasks.

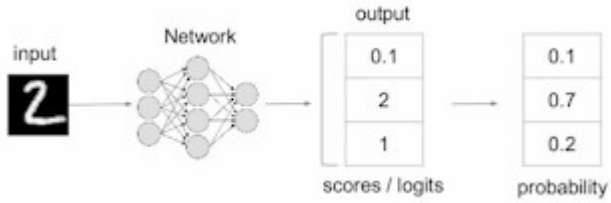
Let's discuss the **cost function**. It evaluates how well the network performs on training data. When the network takes input and produces output, we need a function that enables us to compare the ground truth with the predictions made by our network, which are functions of weights and biases.



Of course, we can apply all the losses we have seen so far (e.g., square loss for regression). For classification in neural networks the most famous loss is called cross-entropy loss.
It is common, for classification, to convert/normalize outputs (which are scores/logits) into probabilities using **softmax**.

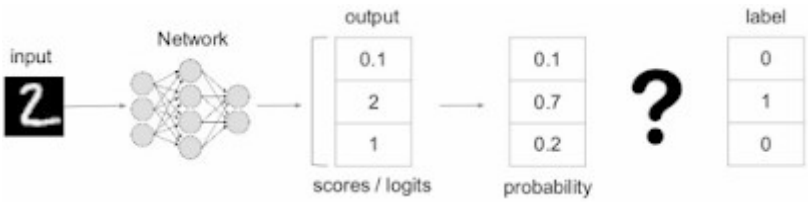


This is important because, for example, if I'm addressing the problem of classifying an image into the set of all possible digits, I can have ten outputs, each indicating the probability for that image to be a particular number, enabling multiclass classification.



This approach is more efficient than multiclass classification in SVM, where obtaining probabilities is much more challenging.

After obtaining the Neural Network, which outputs probabilities, I require a loss function that quantifies the disparity between the model's prediction and the ground truth label. To achieve this, I can convert the label into the one-hot encoding form, signifying that the label '1' will be transformed into a vector of ten zeros with a '1' in the corresponding position.



In this case, a suitable loss function to replace the question mark is the cross entropy loss, which compares the ground truth with the normalized logits:

$$\mathcal{L}_i = -\sum_k y_k \log(S(l_k)) = -\log(S(l))$$

While there's no restriction against using other loss functions, softmax + cross entropy is typically the preferred choice for classification tasks in neural networks.

The selection of the loss function is intricately tied to the choice of the output unit, as the latter should encapsulate a function that aligns with the specific problem domain and complements the chosen loss function effectively.

Let's delve into the type of **Output Units**, which are important not only for the loss but also for the quality of the backpropagation.
Given features h, a layer of **linear output units** computes $\hat{y} = W^T h + b$. Linear units do not have saturation, which simplifies gradient-based optimization algorithms since the gradients do not vanish as the input grows. However, linear units are not suitable for problems where the output needs to be interpreted as probabilities, and they do not handle cases where the gradient of the model's output approaches zero well, potentially leading to problems in learning.

To address these issues, **softmax output units** are often employed. Softmax normalization is desirable in scenarios where it is necessary to produce normalized probabilities at the output layer. Unlike a linear layer, which produces unbounded values, the softmax function transforms the output into a probability distribution, where each component represents a normalized probability, summing up to 1.

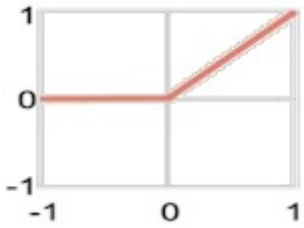
Inside a hidden unit, we always undergo the following steps:

1. Accept input x.
2. Compute the affine transformation $z = W^T x + b$, considering the weights W and the bias b of the layer.
3. Apply an element-wise non-linear function $h(z)$.
4. Obtain the output $h(z)$.

The AI engineer is confronted with numerous choices for h. The design of hidden units and the selection of h represent active areas of research within the field.

For many years, the go-to activation function in neural networks has been **Rectified Linear Units** (ReLU). It gained prominence through its usage in Hinton's AlexNet and remains widely adopted. The ReLU function outputs the input directly if it's positive; otherwise, it outputs zero

$$h(x) = max(0, x)$$



This non-linearity is crucial for neural networks to discern complex patterns in data.

The advantage is that the gradient of ReLU is either 0 or 1, akin to linear units, making it relatively easy to optimize and to use in backpropagation. In fact

- For $x > 0$, the function is linear with a slope of 1. So, the gradient in this region is 1. This means that if the input is positive, any small change in the input will result in an equal change in the output, hence the gradient is 1.
- For $x \leq 0$, the function outputs 0. Since the function is constant in this region, the gradient is 0. This implies that any change in the input in the negative region does not affect the output of the ReLU function.

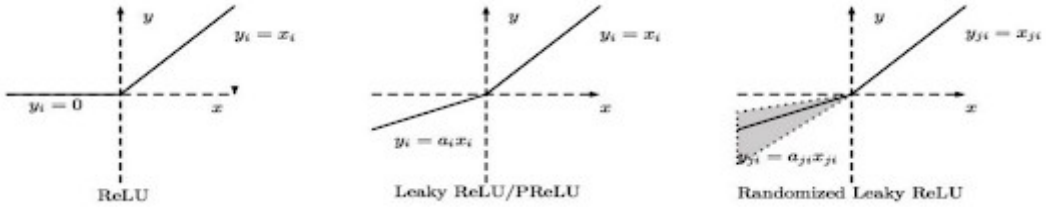
It yields large and consistent gradients when active, which is advantageous for backpropagation. Despite not being everywhere differentiable (in $x = 0$ it isn't), in practice, this isn't problematic; one-sided derivatives at $x = 0$ suffice. Furthermore, gradient-based optimization is inherently subject to numerical error, mitigating concerns about differentiability.

While ReLU activation function is widely used, it has a drawback known as the "dying ReLU" problem, where some units may become inactive during training if their output consistently remains negative, resulting in zero gradients.
To address this issue, several variations of ReLU have been proposed:

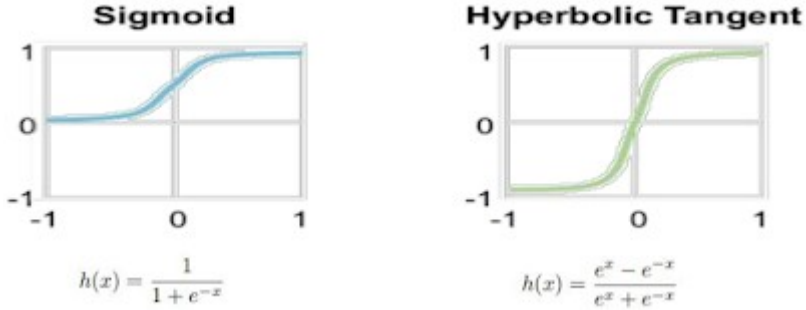
- **Leaky ReLU / PReLU**: Leaky ReLU (or Parametric Rectified Linear Unit) introduces a small slope α for negative inputs, instead of zero. Mathematically, it is defined as:

  $$f(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{otherwise} \end{cases}$$

  where α is a small positive constant. This prevents the gradient from being exactly zero for negative inputs, thus alleviating the dying ReLU problem.
- **Randomized Leaky ReLU**: Randomized Leaky ReLU randomly selects the slope α for negative inputs from a predefined range during training. This randomness adds robustness to the model and prevents units from becoming inactive due to consistently negative outputs.



ReLU was introduced to address limitations encountered with earlier activation functions like Sigmoid and Tanh. Perceptrons, akin to neurons, possess a threshold activation, motivating the emulation of this behavior in neural networks.
Before ReLU, Sigmoid and Tanh were explored due to their differentiable nature. Sigmoid squashes the input into the range [0,1], while Tanh squashes it into [-1,1]. Both aim to introduce a non-linear behavior, essential for learning complex patterns.



However, Sigmoid and Tanh were eventually supplanted by ReLU due to inherent drawbacks. They saturate across most of their domains, exhibiting strong sensitivity only when inputs are close to zero. This saturation poses challenges for gradient-based learning, hindering convergence during training.

ReLU's simple yet effective approach of maintaining linearity for positive inputs while setting negative inputs to zero circumvents the saturation issue. This allows for more robust gradient propagation and faster convergence during optimization, making it a preferred choice in modern neural networks.

Let's discuss some final points about the architecture of a neural network.
Determining the appropriate **depth and width of a neural network** remains an open challenge. While there are some theoretical insights, much of the decision-making relies on empirical observations.
A seminal **theoretical result** by Cybenko (1989) states that a 2-layer neural network with a linear output and squashing non-linearities in the hidden units can approximate any continuous function over a compact domain to arbitrary accuracy, provided there are enough hidden units.

This theorem extends to other non-linear functions, such as ReLU.
It implies that, regardless of the function we aim to learn, a sufficiently large multi-layer perceptron (MLP) can represent it.
However, practical challenges may arise. Despite the theoretical expressiveness of large MLPs, there's no guarantee that our training algorithm will effectively learn the function. Issues like optimization problems (we've seen that it isn't always possible to reach the theoretical minima) and overfitting can hinder the training process, necessitating careful consideration during model development and training.

In neural networks, especially when addressing complex problems, the design of the network plays a pivotal role. Both the architectural choices and the selection of the optimizer can profoundly impact the model's performance and behavior.

Neural networks are inherently flexible models capable of approximating complex functions, but their effectiveness heavily relies on how they are designed and trained. While we have theoretical guarantees that neural networks can converge to good solutions, empirical evidence suggests that certain design principles and training strategies tend to yield better outcomes in practice.
In essence, while theoretical guarantees assure us of the potential of neural networks, it is through empirical validation and careful design choices that we can harness their full power and effectiveness in addressing real-world challenges.

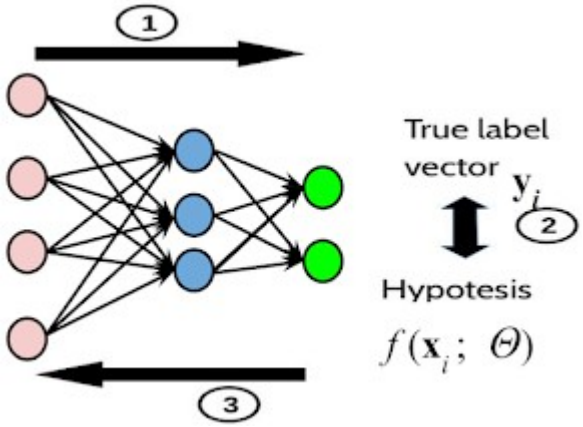
In practice, neural networks are often designed with a large number of parameters to handle complex tasks. However, this can lead to overfitting, where the model learns to memorize the training data rather than generalize to unseen data.
Then, to mitigate overfitting, we employ various strategies such as Regularization Techniques, Data Augmentation, Early Stopping and Model Complexity Control.

In essence, rather than aiming to design the "best" model from scratch, we often start with a large, complex model and then apply various regularization techniques to control its complexity and prevent overfitting. This iterative process helps strike a balance between model complexity and generalization performance.

Backpropagation

As we've already mentioned, backpropagation provides a method to learn weights and efficiently compute gradients. Backpropagation consists of three steps:

1. **Feedforward Propagation**:
    - Accept input x
    - Pass through intermediate stages
    - Obtain the output
2. **Compute the Cost**:
    - Use the computed output to **calculate a scalar cost** based on the loss function
    - Compare outputs with the ground truth to get the **error signal**
3. **Backpropagation of Errors**:
    - Allow information to flow backwards from the cost to compute the gradient
    - **Backpropagate** the error signal to get derivatives for each weight for learning

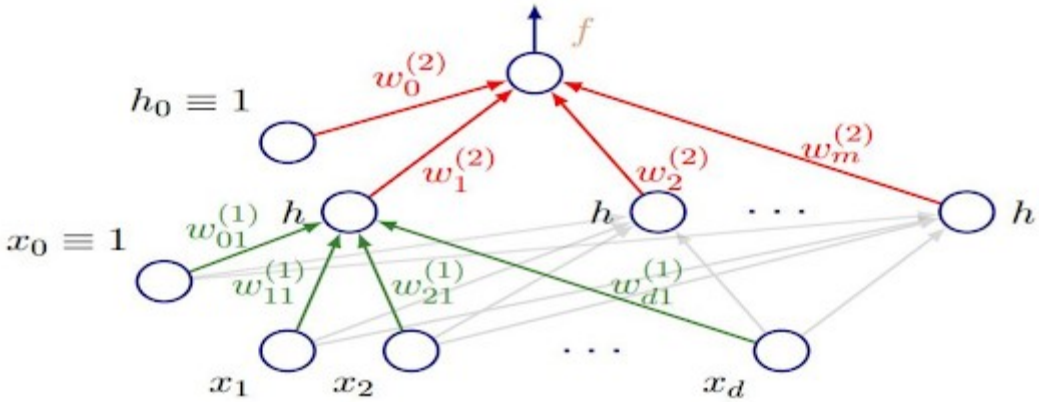


Let's see the formulas which allow us to implement this procedure.
To implement backpropagation we use **gradient descent**, so we need the error derivatives with respect to all the weights in the network.
For example, for the weight of the first layer, the update rule is:

$$w_{11}^{(1)} := w_{11}^{(1)} - \eta \frac{\partial L}{\partial w_{11}^{(1)}}$$

We need the derivative of the loss with respect to each associated weight, incorporating the learning rate.
This computation is challenging because we don't have direct access to the ground truth associated with the hidden layers. Since **we don't know the exact values for the hidden units** in the training data, we can instead compute how quickly the error changes as we adjust a hidden unit's activity. Thus, we use **error derivatives with respect to hidden activities** rather than with respect to the weights.
Each hidden unit can affect many output units, having **separate effects on the error**. Thus we need a mechanism combining these effects to compute the overall error derivatives.
By **efficiently computing error derivatives for hidden units**, we can then easily derive the error derivatives for the weights connecting into those units in a recursive way.

Let's take into consideration the following Feed-Forward Neural Network:



This network has one hidden layer and one output layer.

$x_1, \ldots, x_d$ is the input and x_0 is the bias of the first layer. The green weights are the weights of the first layer.
The red weights are the weights of the second layer and h_0 is the bias for that layer.
As we've already mentioned, the function applied to the output f is typically different from the function applied to the intermediate results h.

So the **step 1**, which is the feedforward propagation, consists in computing the output with the following operation:

$$\hat{y}(\mathbf{x}; \mathbf{w}) = f\left(\sum_{j=1}^{m} w_j^{(2)} h\left(\sum_{i=1}^{d} w_{ij}^{(1)} x_i + w_{0j}^{(1)}\right) + w_0^{(2)}\right)$$

This process is consistent regardless of the number of layers in the network.

Step 2 involves computing the loss. For simplicity, let's assume we use a squared loss.
Thus, to calculate the error of the network on a training set, we use the following formula:

$$L(X; \mathbf{w}) = \sum_{i=1}^{N} \frac{1}{2} (y_i - \hat{y}(\mathbf{x}_i; \mathbf{w}))^2$$

If $\hat{y}$ were a linear function, the minimization of the loss would have a closed-form solution. However, since $\hat{y}$ is a nonlinear function, there is no closed-form solution. The good news is that we can still apply gradient descent, even though the problem isn't convex.

We need to evaluate the derivative of L with respect to the weights for each individual example. For the output, it is possible to consider a simple linear model by setting f as the identity function.

$$\hat{y} = \sum_j w_j x_{ij}.$$

Under these conditions, the derivative of L with respect to the weights is simply computed as the difference between the labels and the predicted outputs, multiplied by the current input:

$$\frac{\partial L(\mathbf{x}_i)}{\partial w_j} = \underbrace{(\hat{y}_i - y_i)}_{\text{error}} x_{ij}.$$

Step 3 involves backpropagation.
The general unit activation in a multilayer network is computed as follows:

$$z_t = h\left(\sum_j w_{jt} z_j\right)$$



Here, z_t is the value of the activation of a node in the network. For simplicity, we denote the summation as a_t.

$$a_t = \sum_j w_{jt} z_j$$

Next, we need to compute the derivative of the loss with respect to any arbitrary weight that connects to node t in the neural network.

$$\frac{\partial L}{\partial w_{jt}} = \frac{\partial L}{\partial a_t}\frac{\partial a_t}{\partial w_{jt}} = \frac{\partial L}{\partial a_t} z_j$$

Using the chain rule, we obtain two derivatives that can be easily computed.
We denote the derivative of L with respect to a_t as δ_t.

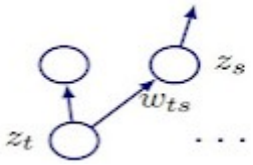
$$\frac{\partial L}{\partial w_{jt}} = \underbrace{\frac{\partial L}{\partial a_t}}_{\delta_t} z_j$$

In the output layer, with a linear activation function, this derivative corresponds to the difference between the labels and the predicted outputs.

$$\delta_t = \hat{y} - y$$

In a hidden layer, we need to aggregate all the errors from the neurons S to which the hidden unit t sends signals.

$$\delta_t = \sum_{s \in S} \frac{\partial L}{\partial a_s}\frac{\partial a_s}{\partial a_t} = h'(a_t)\sum_{s \in S} w_{ts}\delta_s$$



$$a_s = \sum_{j:j \to s} w_{js} h(a_j)$$

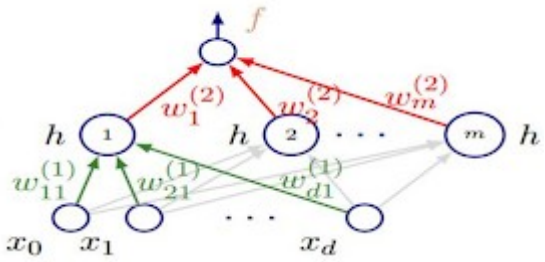
For a hidden layer, δ_t includes a term involving the derivative of the activation function plus an additional term. The presence of the activation function's derivative explains its critical role in backpropagation.

Let's look at an **example** to better understand this process.
First, we set the functions f and h:

- Output: $f(a) = a$
- Hidden:

$$h(a) = \tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}},$$

$$h'(a) = 1 - h(a)^2.$$



Next, we perform feedforward propagation with the inputs to obtain the output:

Input to hidden $\quad a_j = \sum_{i=0}^{d} w_{ij}^{(1)} x_i,$

Hidden output $\quad z_j = \tanh(a_j),$

Net output $\quad : \hat{y} = a = \sum_{j=0}^{m} w_j^{(2)} z_j.$

Then, we choose the loss function and compute the derivative for the output unit:

Error on example $\mathbf{x}$: $L = \frac{1}{2}(y - \hat{y})^2.$

Output unit: $\delta = \frac{\partial L}{\partial a} = y - \hat{y}.$

The derivative for the hidden units is a bit more complex. We use the formula we've seen before to compute it:

Next, compute δ for the hidden units:

$$\delta_j = (1 - z_j)^2 w_j^{(2)} \delta$$

Derivative with respect to weights:

$$\frac{\partial L}{\partial w_{ij}^{(1)}} = \delta_j x_i, \qquad \frac{\partial L}{\partial w_j^{(2)}} = \delta z_j.$$

We have small differences between the derivatives of the weights in the two layers.

Finally, we update the weights:

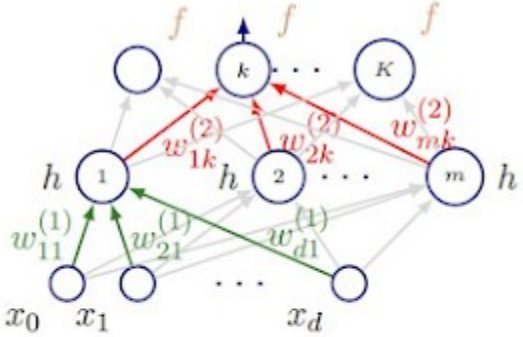
Update weights: $w_j \leftarrow w_j - \eta \delta z_j \qquad w_{ij}^{(1)} \leftarrow w_{ij}^{(1)} - \eta \delta_j x_i.$

But typically, a neural network isn't composed of a single hidden layer, but many of them. Therefore, the formula for the hidden units needs to be applied to multiple layers.

And the version where we have multiple nodes in the output layer is the following:

Loss on example

$$\frac{1}{2} \sum_{k=1}^{K} (y_k - \hat{y}_k)^2$$



Now, for each output unit $\delta_k = y_k - \hat{y}_k$;

For hidden unit j:

$$\delta_j = (1 - z_j)^2 \sum_{k=1}^{K} w_{jk}^{(2)} \delta_k.$$

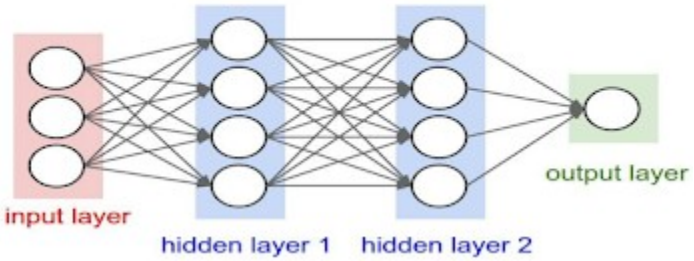
We've covered only regular feed-forward neural networks, but nowadays there are much more complex networks.

Neural Networks II

Training a Neural Network

We've already seen that in a Feed-Forward Neural Network, the function f is a composition of multiple functions:

$$f(\mathbf{x}) = f^{(3)}(f^{(2)}(f^{(1)}(\mathbf{x})))$$



We've also seen that the goal is to approximate some unknown ideal function $f^* : X \to Y$. To achieve this, we define a parametric mapping $f(x_i; \Theta)$, and we learn the parameters to obtain a good approximation of f^* from the available samples.

We have seen all the modeling choices we need to make to build the neural network; the only one left to explore in depth is the **optimizer**. We've seen that selecting the optimizer is crucial for training. The learning of a task involves optimizing a certain cost function.
The main idea is to adjust all the weights of the network, given the training samples $T = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$, such that the cost function is minimized:

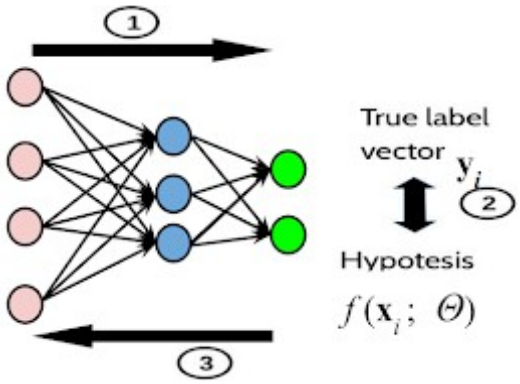
$$\min_{\Theta} \sum_i L(\mathbf{y}_i, f(\mathbf{x}_i; \Theta))$$

To achieve this, we need to:

1. Choose the **loss/cost function** (e.g., square loss, cross-entropy loss).
2. Update the weights of each layer using **gradient descent** (since the cost function is very complex, this is the only way to optimize it).
3. Use **backpropagation to compute the gradient efficiently**.

We've seen that backpropagation consists of three steps:

1. **Forward Propagation**: Sum inputs, produce activations, and feed-forward.
2. **Error Estimation**: Calculate the error between the predicted output and the actual output.
3. **Backpropagation**: Propagate the error signal backward through the network and use it to update the weights.



In principle, a Feed-Forward Neural Network can be trained with Vanilla Gradient Descent (or Regular Gradient Descent), which is the simplest way of implementing gradient descent:

```
# Vanilla Gradient Descent

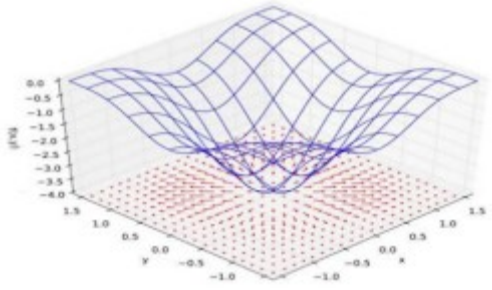
while True:
  weights_grad = evaluate_gradient(loss_fun, data, weights)
  weights += - step_size * weights_grad # perform parameter update
```

In vanilla gradient descent, we compute the gradient (using backpropagation) and then apply the gradient descent update rule, where the new weight values are the previous weight values minus the learning rate multiplied by the gradients.

Just as a reminder, let's see what a gradient is.
The gradient is the vector of partial derivatives with respect to all the coordinates of the weights:

$$\nabla_{\mathbf{w}} L = \left[\frac{\partial L}{\partial w_1} \quad \frac{\partial L}{\partial w_2} \cdots \frac{\partial L}{\partial w_M} \right]$$

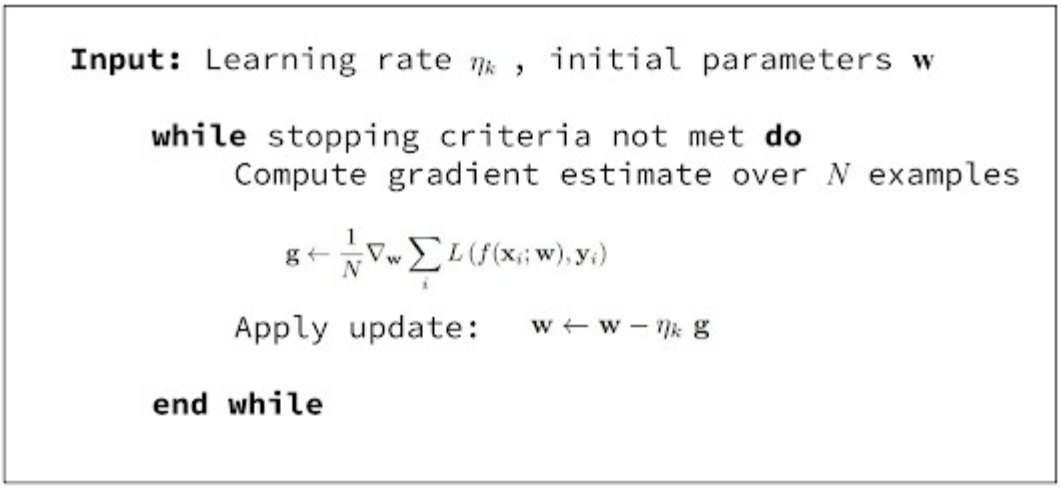
Each partial derivative measures how fast the loss changes in one direction. When the gradient is zero, i.e., all the partial derivatives are zero, the loss is not changing in any direction.
The issues are local minima (a point where the function value is lower than at any nearby points, but it might not be the lowest overall value) and saddle points (a point where the gradient is zero, but unlike a local minimum or maximum, it is not a point of extreme value: in some directions, the function increases, while in others, it decreases).



Gradient Descent is an iterative procedure that aims to find the set of parameters that minimizes the loss function. The change in parameters (the update) depends on the gradients of the loss with respect to the network weights.

We don’t need to focus on how the gradient is computed anymore because we know that backpropagation is the method for computing gradients. Now, we need to determine whether regular gradient descent is sufficient or not.

In traditional Machine Learning (like the methods we've seen before Neural Networks), we typically apply the following:



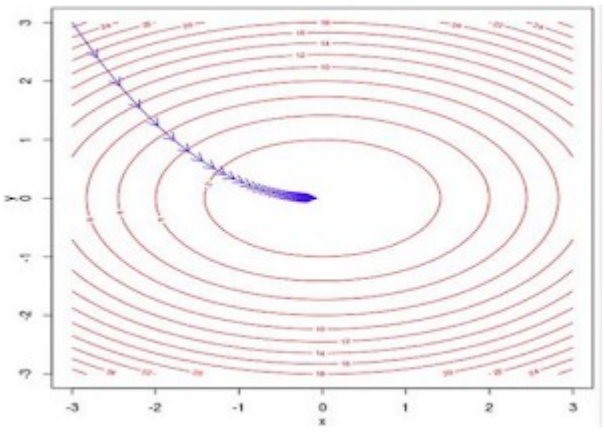
So, what we're actually applying is technically called **Batch Gradient Descent** (BGD):

- Set the learning rate and the initial values of the parameters.
- Decide on a stopping criterion (e.g., number of iterations).
- Iterate by computing the gradient over all examples in the training set and applying the update using the update rule.

The learning rate may change at each step; typically, it decays linearly to balance the trade-off between rapid convergence and fine-grained optimization.

Using Batch Gradient Descent in Neural Networks is not ideal. Nowadays, we deal with huge networks and vast amounts of data. Computing the summation for all data points in the training set at each step becomes incredibly slow.
As a solution, we give up on the idea of computing the gradient by considering all samples in the training set. Instead, we opt for a more efficient approach: taking a small random subset of samples and computing the gradient only on this subset, known as mini-batch gradient descent.

Let’s explore the scenario where we have the fortunate situation of dealing with a convex function and computing the gradient by considering all examples for every step:



Pros:

- **Stable Gradient Estimates**: The gradient smoothly converges towards the minimum, ensuring stability.

Cons:

- **Need for Computing Gradients over Entire Training Set for Each Update**: This ideal condition of regular batch gradient descent is impractical in real-world scenarios due to the extensive computational time it requires.

Stochastic Gradient Descent (SGD) follows a similar procedure to Batch Gradient Descent, but instead of using the entire dataset each iteration, it randomly selects only one datapoint from the training set and computes the gradient solely for that datapoint.

Input: Learning rate η_k , initial parameters $\mathbf{w}$

while stopping criteria not met **do**
Sample one datapoint from training set $(\mathbf{x}_i, \mathbf{y}_i)$
Compute gradient estimate

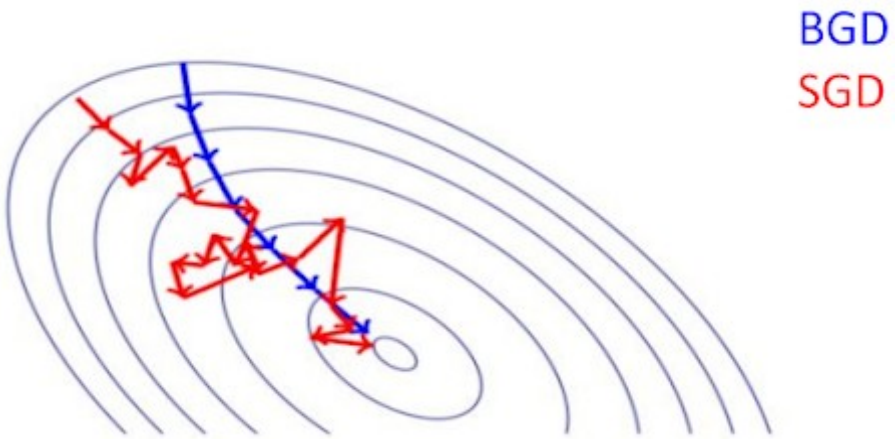
$$\mathbf{g} \leftarrow \nabla_{\mathbf{w}} L\left(f(\mathbf{x}_i; \mathbf{w}), \mathbf{y}_i\right)$$

Apply update: $\mathbf{w} \leftarrow \mathbf{w} - \eta_k \mathbf{g}$

end while

It is crucial that the datapoint is chosen stochastically.

The difference in the paths taken by these two algorithms is as follows:



In Stochastic Gradient Descent, because you sample stochastically, you won't experience smooth convergence but rather something more unstable and "chaotic."

This randomness can be seen as a positive attribute because it can aid in escaping local minima and saddle points.

The difference in the algorithm lies in the fact that Batch Gradient Descent uses all the samples, while Stochastic Gradient Descent uses only one.

BGD

$$\mathbf{g} \leftarrow \frac{1}{N} \nabla_{\mathbf{w}} \sum_i L\left(f(\mathbf{x}_i; \mathbf{w}), \mathbf{y}_i\right)$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta_k \, \mathbf{g}$$

SGD

$$\mathbf{g} \leftarrow \nabla_{\mathbf{w}} L\left(f(\mathbf{x}_i; \mathbf{w}), \mathbf{y}_i\right)$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta_k \, \mathbf{g}$$

Batch Gradient Descent is not always impractical; in fact, in cases where the training set is small and a simple loss function is used, it can be employed without any issues and may yield better results.

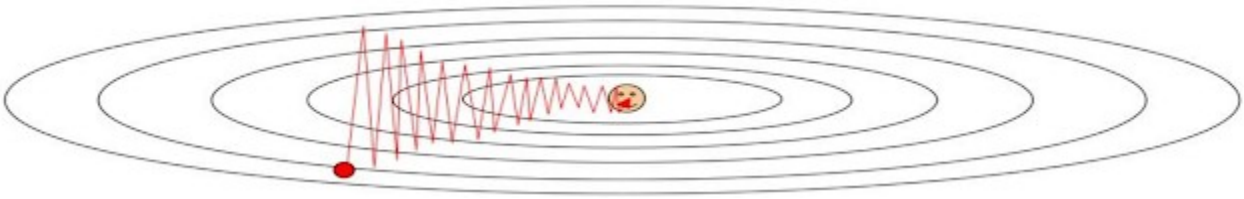
Since Stochastic Gradient Descent produces noisy gradient estimates due to considering only one point at a time, one obvious solution is to utilize a small set of samples, referred to as **mini-batches**, instead of just one datapoint.
The advantages of mini-batches are:

- The computation time per update does not depend on the number of training examples, N.
- It facilitates computations on extremely large datasets.
- It often allows for parallel implementation.

- When using GPUs, power-of-2 batch sizes commonly offer improved runtime, as some types of hardware achieve better performance with specific array sizes.

Now we have everything that allows us to train a Neural Network (how to create the architecture, backpropagation, stochastic gradient descent). However we rarely used Stochastic Gradient Descent in its naive way, but we are using more sophisticated variations of it.

A problem with SGD is that it can result in very slow progress along flat directions and jittery movements along steep ones. In certain regions, the error surface may become skewed, leading to oscillating updates that require more time to reach the minimum. To address this issue, strategies exist that consider the history of updates and ensure a more linear path toward the minimum.



One of these strategies involves what is called Stochastic Gradient Descent with **momentum**. A new variable is introduced: the velocity v. The velocity is an exponentially decaying moving average of the negative gradient.

Input: Learning rate η_k , initial parameters $\mathbf{w}$, initial velocity $\mathbf{v}$, momentum parameter α

while stopping criteria not met **do**
Sample one datapoint from training set $(\mathbf{x}_i, \mathbf{y}_i)$
Compute gradient estimate

$$\mathbf{g} \leftarrow \nabla_{\mathbf{w}} L\left(f(\mathbf{x}_i; \mathbf{w}), \mathbf{y}_i\right)$$

Compute velocity update

$$\mathbf{v} \leftarrow \alpha\, \mathbf{v} - \eta_k\, \mathbf{g}$$

Apply update: $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}$

end while

So, in addition to the hyperparameter for the learning rate, we introduce another one which is the momentum parameter, and we also need an initial value for the velocity vector.
Once the gradient is obtained, instead of directly using it to compute the update, I compute the velocity with it and then use the velocity to make the update.
The velocity vector takes into account the history of the gradients.
The higher the momentum parameter, the more importance we give to the gradients computed in past iterations.

The momentum enables the algorithm to converge faster towards the minima.

However, what we've seen so far is not sufficient. Nowadays, we utilize methods known as **Adaptive Learning Rate Methods** to train neural networks.
Assigning the same learning rate to all features (weights) may not be efficient if the features vary in importance and frequency.
A 2-dimensional visualization of the problem is as follows:



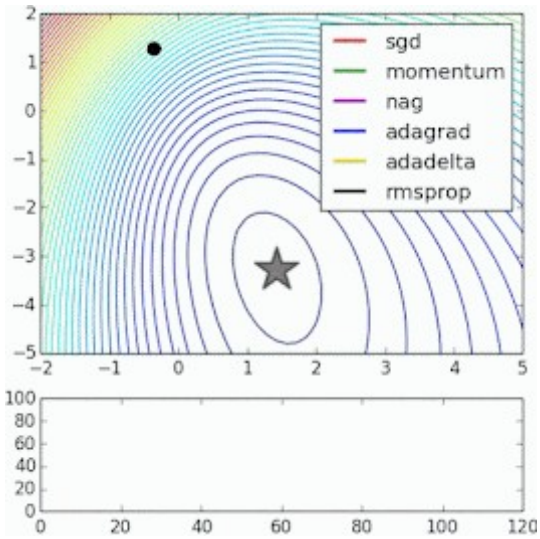
Easier: all the features important

Harder

The learning rate is one of the most challenging hyperparameters to set in neural networks. Adaptive Learning Rate Methods automatically determine the best learning rates for certain types of parameters (weights) and update the gradient accordingly.

The community is actively working on inventing new optimizers to address this challenge.

Let’s visualize how different methods perform:



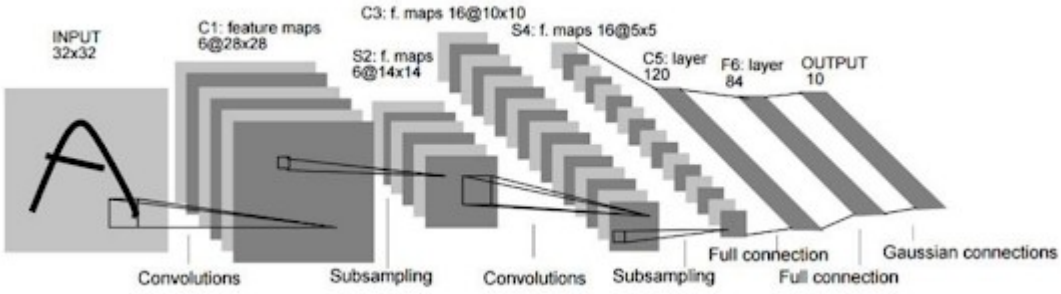
In Stochastic Gradient Descent, the accuracy approaches the upper bound very slowly, and the weights take a considerable amount of time to converge to the optimal values.

With the introduction of momentum strategies (such as Momentum and Nesterov Accelerated Gradient), the convergence speed increases, albeit with a tendency to overshoot.

However, the family of Adaptive Learning Rate Methods (such as Adagrad, Adadelta, and RMSprop) offers significantly faster convergence rates compared to traditional Stochastic Gradient Descent, making them preferred choices in many scenarios.

Convolutional Neural Networks

A Convolutional Neural Network (CNN) is a specialized variant of the Feed-Forward Neural Network designed to process data organized in spatial dimensions, such as images.

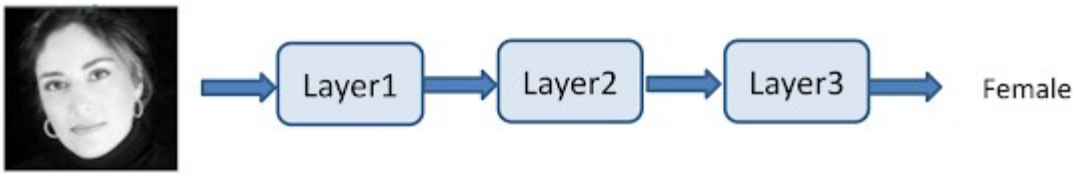


The progress of Feed-Forward Neural Networks applied solely to vectors has not been particularly remarkable in recent years. However, significant advancements have been witnessed in fields requiring structured data handling, such as computer vision, natural language processing, and audio processing.
These advancements stem not only from the development of new activation functions, regularizers, or optimizers, but also from the innovation of methods tailored for structured data handling.
Many applications inherently involve locally structured input spaces, whether **spatial** or temporal (e.g., **images**, language). In such cases, neural networks exhibit exceptional power and efficacy.

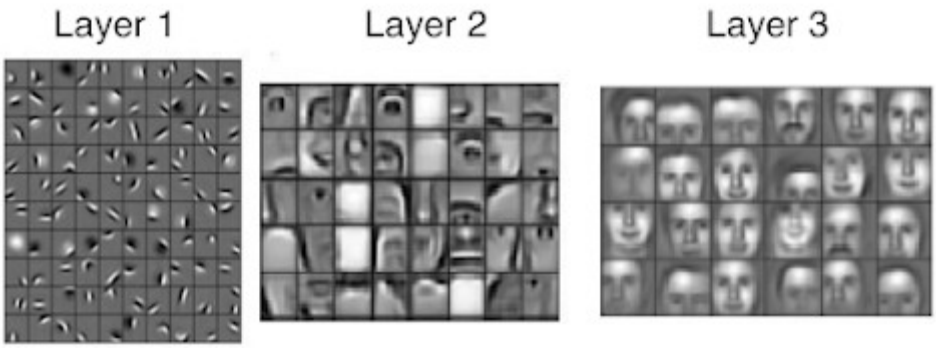


Tomorrow, and tomorrow, and tomorrow; creeps in this petty pace from day to day, until the last syll-able of recorded time. And all our yesterdays have lighted fools the way to dusty

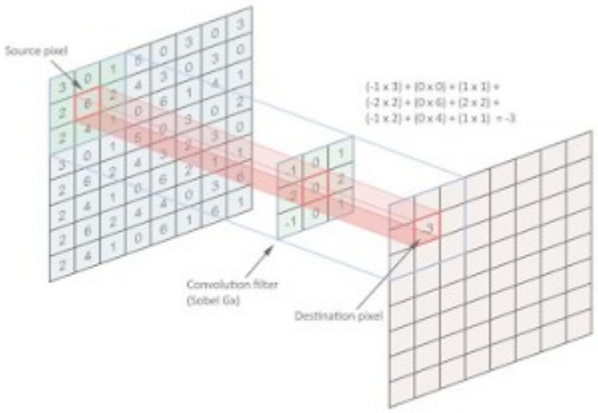
Neural networks, thanks to their end-to-end processing capabilities, directly learn hierarchical representations of features, enabling effective representation of structured data. Each layer in the hierarchy extracts features from the output of the previous layer. All layers are trained jointly.



In Convolutional Neural Networks, we can envision that within the various layers, the network learns structured patterns. In the initial layers, it grasps locally structured patterns, gradually progressing towards the output where it learns more global patterns.



Convolutional networks are neural networks that utilize **convolution** instead of general matrix multiplication **in at least one of their layers**.
Convolution is a fundamental operation for processing images, serving as a general-purpose filter.

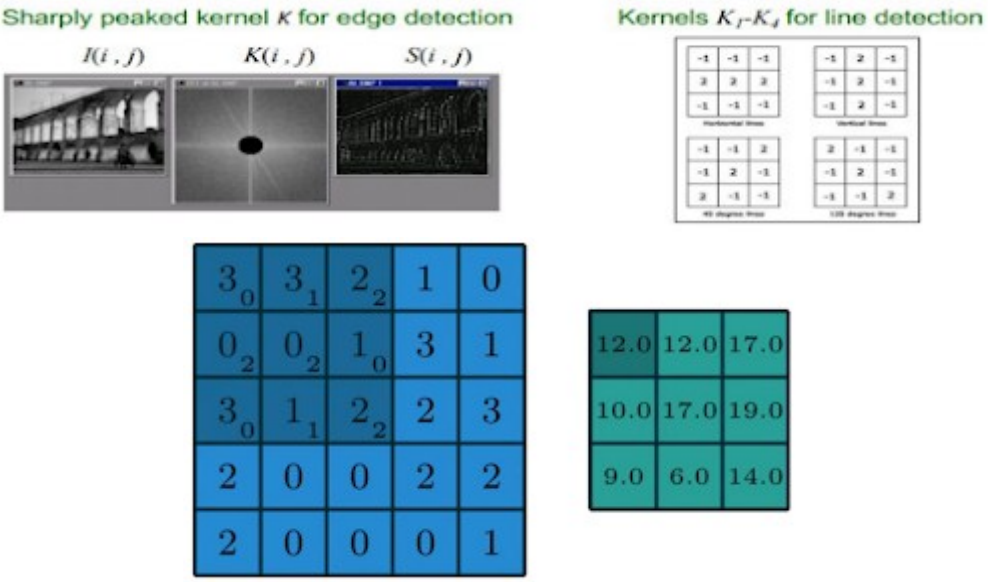


During convolution, a kernel matrix is applied to an image, determining the value of a central pixel by aggregating the weighted values of its neighboring pixels (with the same size as the kernel matrix).
The output of the process is a modified, filtered image.

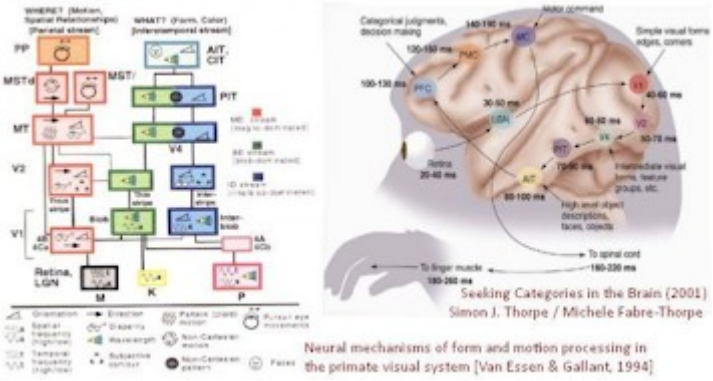
$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(m,n)K(i-m, j-n).$$

Typically, the output matrix has smaller dimensions than the input matrix, but adding padding (borders with zeros) allows for obtaining a matrix of the same dimensions. This technique can be applied for various enhancements, including smoothing, sharpening, or feature extraction. Crucially, convolution is a commutative operation.

People employ convolution beyond and before Neural Networks to manipulate signals. Historically, we manually crafted kernels to filter images, enhancing certain patterns. For instance, a kernel with zeros on the side columns and a constant value in the central column enhances vertical edges in the output image.



There's a rich history behind neural networks, often regarded as biologically inspired. In fact the concept of extracting localized patterns in Convolutional Neural Networks mirrors the mammalian process of visual information processing.



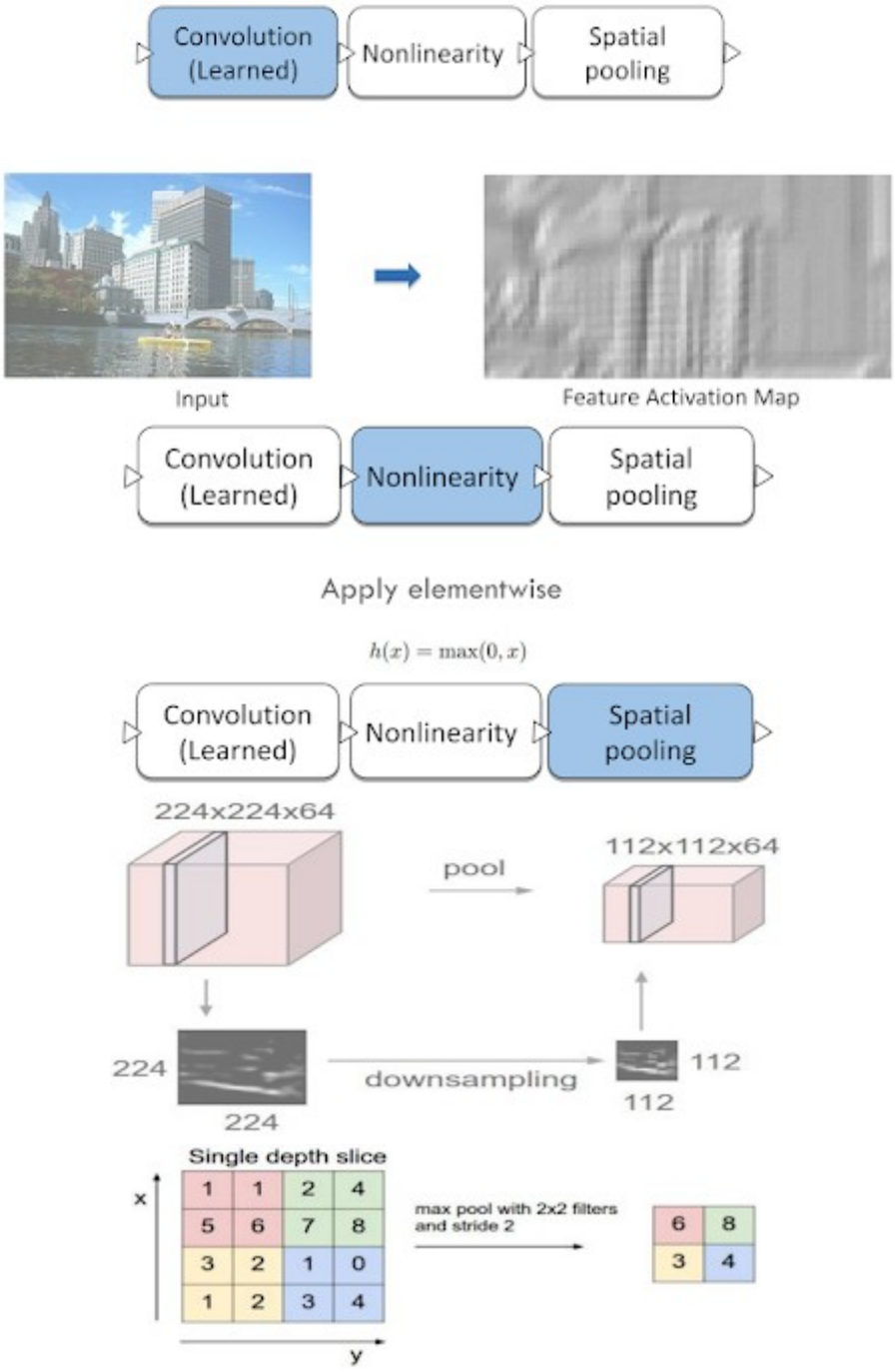
The visual cortex comprises a complex network of cells sensitive to small sub-regions of the visual field, known as **receptive fields**. These cells function as local filters across the input space, capitalizing on the strong spatial correlations inherent in natural images.
There are two basic cell types:

- **Simple cells**, which respond maximally to specific edge-like patterns within their receptive field.
- **Complex cells**, possessing larger receptive fields and exhibiting local invariance to the precise position of the pattern.

To bridge the concepts of convolution and Feed-Forward Neural Networks, we introduce **Convolutional Neural Networks** (CNNs). A CNN is essentially a Feedforward neural network with a specialized connectivity structure.
In CNNs, layers perform transformations on the input matrix to generate class predictions. Unlike traditional feedforward networks, where multiplication is employed, CNNs utilize

convolution. Notably, the kernel weights are not manually specified but instead are learned through backpropagation.
Several distinct operations are integral to CNN layers:

1. **Convolution**:
    - Function: The convolutional layer is the core component of CNNs. It comprises a set of **learned filters** or kernels. Each filter covers a small spatial portion of the input data, termed the receptive field.
    - Process: These filters are convolved across the dimensions of the input data (**receptive field**), generating a **multi-dimensional feature map** (using multiple filters in the same layer).
    - Intuition: The network learns the best filters in order to activate when specific features are detected at certain spatial positions within the input. This enables the network to extract meaningful features from the data.
2. **Non-linearity**:
    - Function: Non-linearity is introduced through the application of an activation function. This enhances the nonlinearity of the entire CNN architecture while preserving the receptive fields established by the convolutional layers.
    - Process: Each neuron's output from the convolutional layer is passed through the activation function (like ReLU), introducing nonlinearity to the network.
    - Intuition: Activation functions introduce complex mappings between input and output, enabling the network to learn and represent intricate relationships within the data.
3. **Pooling**:
    - Function: Pooling operations are a form of downsampling which are employed to provide **invariance to translations** (ability to recognize an object regardless of its position within the input image), **reduce spatial dimensions, parameters, computation, and control overfitting**.
    - Process: Pooling layers progressively reduce the spatial size of the feature maps generated by the convolutional layers. Common pooling methods include max pooling, which retains the maximum value within a fixed-size window, and average pooling.
    - Intuition: By reducing the spatial dimensions of the feature maps, pooling operations enable the network to focus on the most salient features while discarding irrelevant details. This results in a more compact representation of the data, facilitating efficient processing and reducing computational complexity. This is done to reduce the number of parameters in the input of the next layer.



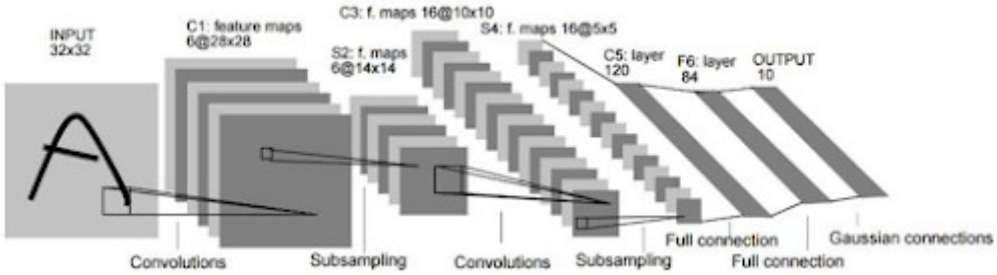
The stride determines how many input cells move to produce the next output cell.

The distinction between convolution and pooling lies in the fact that the kernel in convolution is learned, whereas pooling is deterministic and solely used to reduce the input size.

Convolutional Neural Networks Architectures

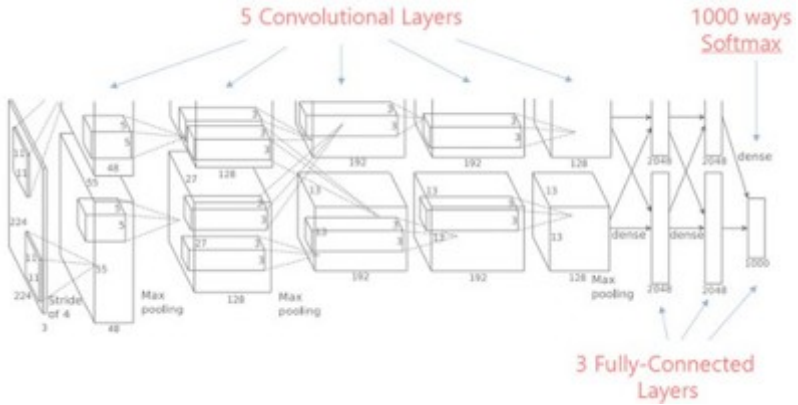
We've observed that following the development of Feed-Forward Neural Networks and the advent of backpropagation, researchers began to focus on specialized architectures.

One notable specialization is the transition from Feed-Forward Neural Networks to Convolutional Neural Networks (CNNs), exemplified by **LeNet** (1998).
Originally designed for optical character recognition, LeNet features a structured architecture comprising alternating convolutional and pooling layers. These layers progressively reduce the dimensions of the filtered images until they are sufficiently small to be flattened into a vector. The final layers consist of fully connected layers, akin to those in traditional feed-forward neural networks.



This architecture had a manageable number of parameters because the computational power for training was limited, and the data used were images at a reasonably low resolution. Therefore, it was relatively manageable.

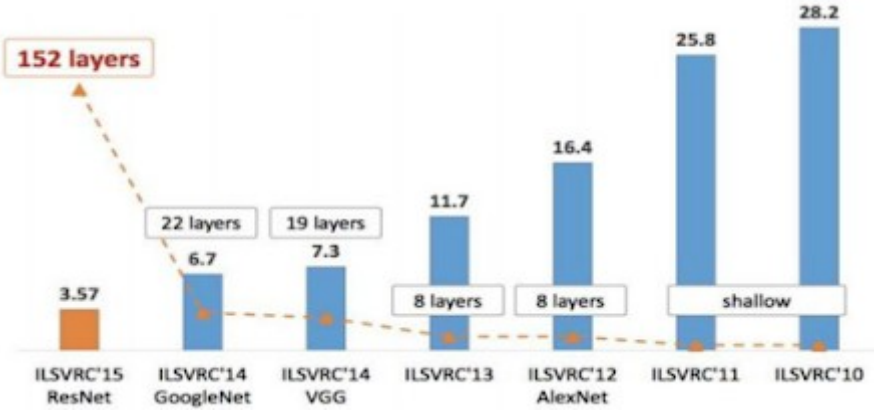
AlexNet, introduced in 2012, follows a similar framework to LeNet but boasts a larger model. It comprises 7 hidden layers, 650K units, and a staggering 60 million parameters. Additionally, it benefits from a more extensive dataset, with 10^6 classes compared to LeNet's 10^3. Notably, AlexNet leverages GPU implementation, yielding a significant speedup of 50 times over CPU processing. It was trained on two GPUs for a week to achieve its remarkable performance. This growth in the architecture dimension was required due to the increase in the resolution of the images.



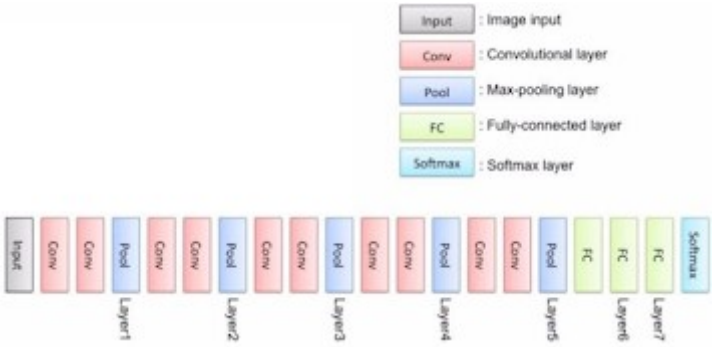
Let's delve deeper into the classification task by examining the top algorithms that have won the ImageNet Challenge over the years:

- 2010 and 2011: The winning models were shallow, with features automatically extracted but handcrafted, and used sophisticated SVMs with multiple kernel learning.
- 2012: AlexNet was introduced, significantly improving performance with its deep architecture.

- 2013: An architecture very similar to AlexNet was used, but with efforts to understand the inner workings of the Neural Network.
- 2014: VGG, another architecture similar to AlexNet but deeper, with more layers.
- 2014 and 2015: It became evident that simply adding more layers to the same convolution-pooling architecture wasn't effective. This led to the development of radically different architectures like GoogleNet and ResNet. ResNet, in particular, is much deeper but surprisingly has fewer parameters due to its innovative design.



The 2014 winner, VGG, was proposed by Oxford and features the same alternation of convolution and pooling layers.



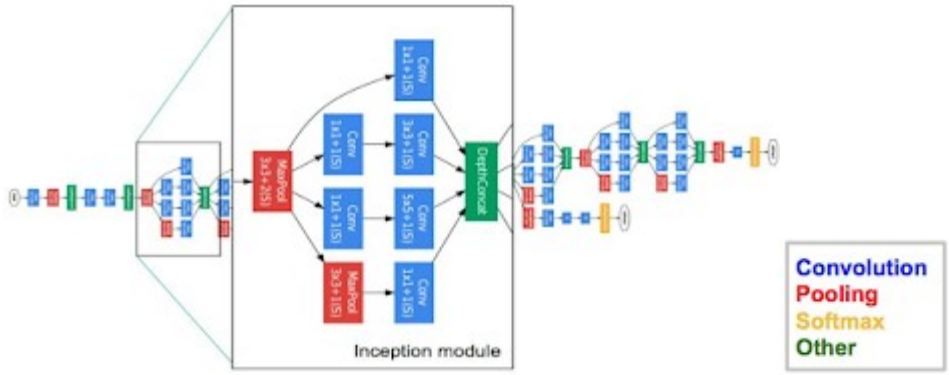
When designing a Convolutional Neural Network, it is important to choose a kernel size that is neither too large nor too small because:

- If it is too big, it may capture too much noise and unnecessary details, leading to overfitting. It also increases the computational complexity and the number of parameters, which can slow down the training process.
- If it is too small, it may not capture enough context or broader features, leading to a lack of precision. Smaller kernels might miss important spatial hierarchies in the data.

In the early days of Convolutional Neural Networks (CNNs), the design typically involved a straightforward alternation of convolutional and pooling layers. This structure was effective and laid the foundation for many groundbreaking applications in image recognition and processing. However, as the field of deep learning has advanced, the architectures of CNNs have evolved significantly. Modern CNNs now often employ more sophisticated designs that do not adhere strictly to the pattern of alternating convolution and pooling layers. Instead, they incorporate a

variety of layers and techniques such as skip connections, residual blocks, and attention mechanisms, which enhance their performance and capability to capture complex patterns in data. The limitation of the early alternation architecture is its inability to scale efficiently due to the large number of parameters it requires, prompting the development of more efficient and effective CNN architectures like GoogleNet and ResNet.

GoogleNet is deeper but has 12 times fewer parameters than AlexNet. This efficiency is achieved through the use of the Inception Module:



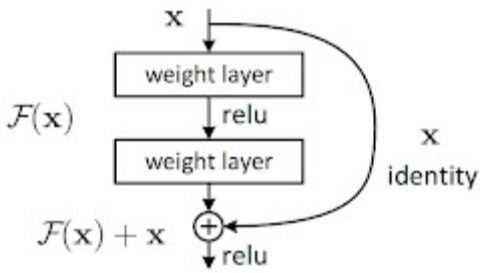
The Inception module introduces different parallel paths from the input to the output, incorporating convolutions at various sizes. This design choice facilitates the creation of a multiscale representation within the architecture.

The design of GoogleNet is notable for two key features:

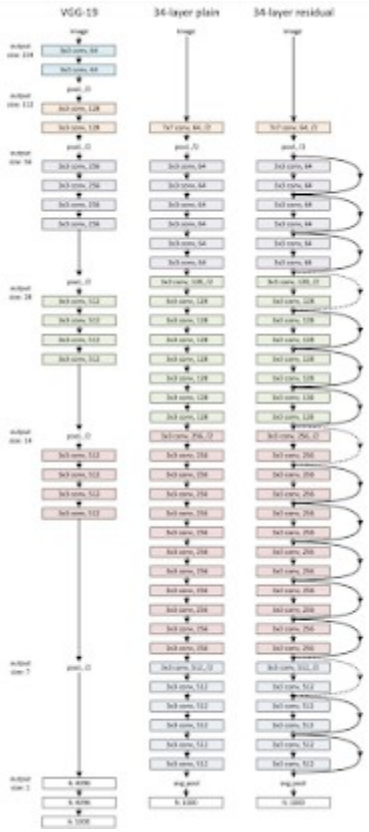
- **Elimination of Fully Connected Layers**: Unlike traditional CNN architectures, GoogleNet removes fully connected layers at the end, reducing the number of parameters and computational complexity.
- **Multiple Losses in the Network**: GoogleNet incorporates multiple loss functions throughout the network, not just at the output layer. This ensures that features learned at intermediate layers are also optimized for the task, addressing the issue of vanishing gradients.

The decision regarding the number of losses to incorporate into the network is often based on empirical observations and experimentation.

ResNet is a neural network constructed by assembling **Residual Blocks**, leading to improved performance of very deep networks.
To address the **degradation problem**, ResNet enables deeper layers to propagate information directly from shallower layers via identity mapping. This mechanism allows the neural network to skip unnecessary representations if not needed for a particular task.

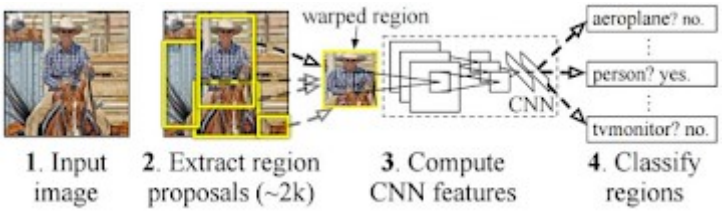


ResNet also introduces batch normalization layers to enhance training. After each convolutional block, the output is renormalized before passing it to the next layer, akin to the normalization/standardization techniques seen in shallower models. This helps improve backpropagation by ensuring that activation functions operate within more optimal ranges.



The output layers can be designed with a linear function and square loss for regression tasks, or softmax with cross-entropy for classification tasks.

One advantage of these architectures is their versatility; they can be repurposed for tasks beyond classification, such as **detection** (learning the bounding box that contains the classified object). The initial approach was R-CNN (Regions with CNN Features), which involved training on ImageNet classification and fine-tuning the CNN on PASCAL-VOC.



R-CNN uses an external algorithm to select areas of interest (where the object could be) and then reshapes/warps the bounding boxes into fixed dimensions before passing them to the CNN. The CNN is then used to determine whether the bounding box contains the object of interest or not.

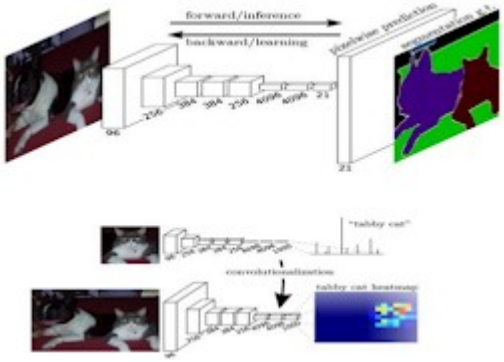
However, its longevity was limited due to several reasons:

- An external algorithm is required to select regions.
- Poor warping quality leads to distortion of object shapes.
- Indirect handling of the detection task using an architecture not specifically designed for it.

As a result, more sophisticated methods have emerged to address these shortcomings.

Two other tasks that can be accomplished with Convolutional Neural Networks are:

- **Semantic Segmentation**
  In semantic segmentation, cross-entropy and softmax can be used. However, a challenge arises when the feature map needs to have the same dimensions as the input. This can be addressed by employing the transpose operation of convolution, also known as transposed convolution or deconvolution.



- **Structured Regression**
  In structured regression, the objective is to learn the position of each joint in the image. Continuous outputs are required for the network. Ideally, the neural network learns

dependencies among different outputs while learning features.



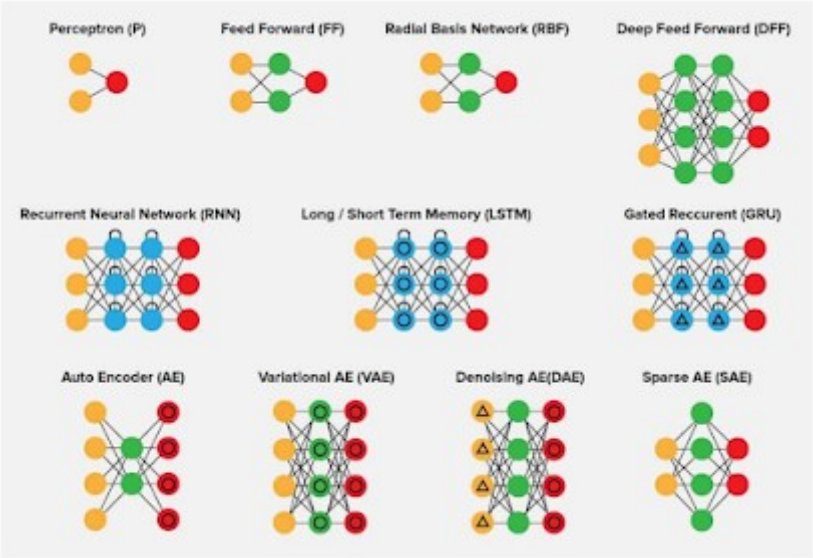
In summary:

- In a feedforward neural network, units are arranged into layers, with each layer receiving input only from units in the layer below.
- CNNs are specialized feedforward networks with a **spatial arrangement** of units.
- In CNNs, units at each layer are organized into 2D grids called **feature maps**.
- Each feature map is the result of a convolution operation, where the **same convolutional filter** is applied at each location. However, the weights are different across feature maps.
- A unit at a particular location on the 2D grid receives input only from units at a similar location in the layer below.
- CNNs require labeled data (images with annotations) as they are supervised learning models.
- They are flexible and applicable to various tasks and domains.

Other Neural Networks

The world of neural networks is vast (Neural Network Zoo). This means that, at some point, people start to specialize in only a limited number of models, each with many variations.

There exist many models for different needs:

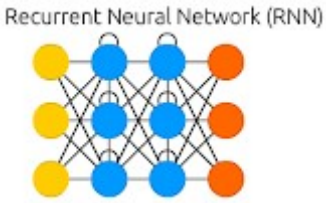


We are going to focus on a very small subset of models.
We've already discussed the Perceptron, the Feedforward, and the Deep Feedforward networks.
The Radial Basis Network is a type of Feedforward network with a special activation function called the Radial Basis Function. It was commonly used in the past to solve regression problems.

Recurrent Neural Networks (RNNs) are a family of models designed to deal with sequences. They are neural networks with cycles. Improved versions include Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRUs).

Autoencoders are used for unsupervised learning, such as dimensionality reduction. Improved versions include Denoising Autoencoders (DAE) and Sparse Autoencoders (SAE). Variational Autoencoders (VAE) are another type of Autoencoder used for density estimation and sample generation.

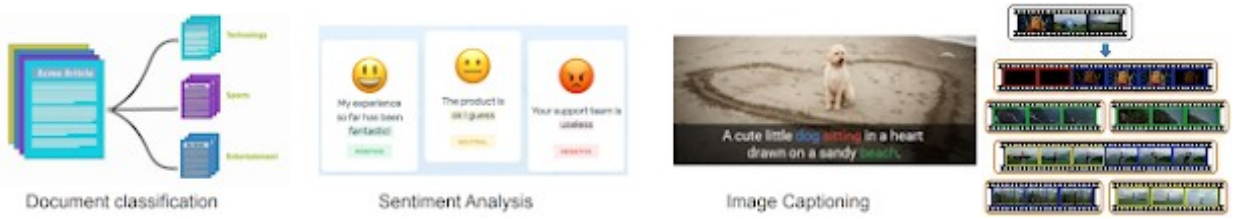
Let’s briefly introduce what a **Recurrent Neural Network** (RNN) is.



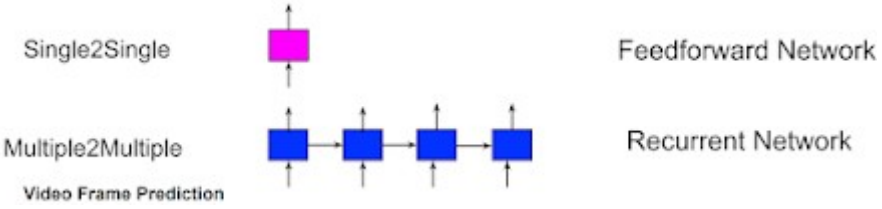
So far, we have mainly focused on prediction problems with **fixed-size inputs and outputs** (vectors, matrices, etc.). We've already discussed the flexibility of CNNs to address a wide range of tasks. However, many applications require solving sequential prediction tasks, where the input and/or output is a **variable-length sequence** (such as text, documents, etc.). To address this need, Recurrent Neural Networks were invented.

Some of the tasks that a Recurrent Neural Network can solve are:

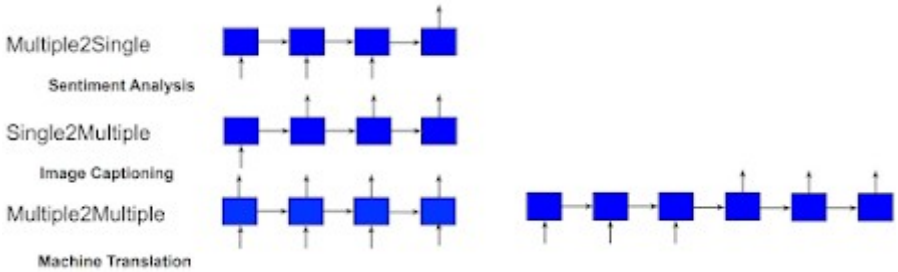
- **Document Classification**
  - Task: Given a document, recognize its class based on its content.
  - Details: The input is a variable-length sequence (the document), and the output is a fixed-size vector representing the class.
- **Sentiment Analysis**
  - Task: Classify the sentiment of a given text (e.g., positive, negative, neutral).
  - Details: The input is a variable-length sequence (the text), and the output is a fixed-size vector representing the sentiment class.
- **Image Captioning**
  - Task: Generate a descriptive caption for a given image.
  - Details: The input is a fixed-size vector (the image features), and the output is a variable-length sequence (the caption).
- **Video Frame Prediction**
  - Task: Predict the class or content associated with each frame in a video.
  - Details: Both the input and output are sequences, where the input is a sequence of video frames and the output is a sequence of predicted classes or contents for each frame.



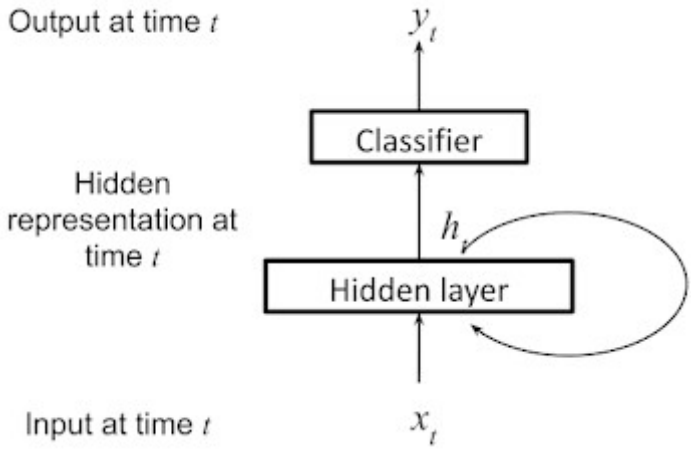
The difference is that in Feedforward Neural Networks and Convolutional Neural Networks, we have a single-to-single mapping (fixed dimensions as input and fixed dimensions as output). In contrast, some Recurrent Neural Networks (like those used for video frame prediction) can achieve a many-to-many mapping.



Recurrent Neural Networks can address a wide range of tasks, utilizing different types of mappings:



All of these scenarios necessitate the utilization of a Neural Network that incorporates past observations. This is achieved by integrating **cycles** to enable **recurrences**.
The structural elements of a Recurrent Neural Network are organized as follows:



1. **Input Encoding**: Initial encoding of the input data (e.g., a vector for each word).
2. **Learning Latent Representations**: The neural network learns the hidden representations h_t, which denote the hidden state at time t.
3. **Utilization for Classification**: Once the neural network has learned the features, they are utilized by the classifier to generate an output (e.g., the label of a review).

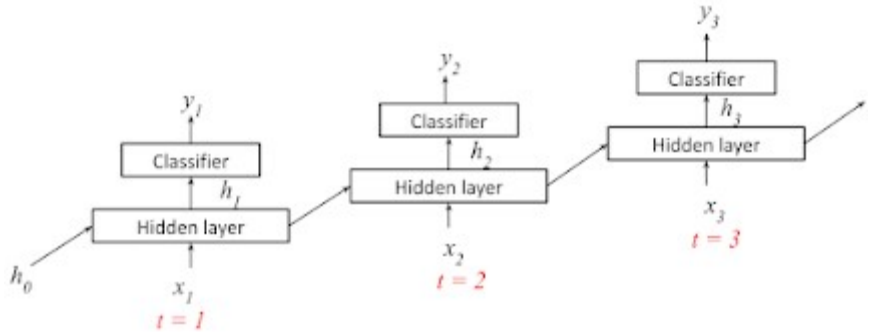
The computation of the hidden representation h_t is as follows:

$$h_t = f_W(x_t, h_{t-1})$$

new state | function of W | input at time t | old state

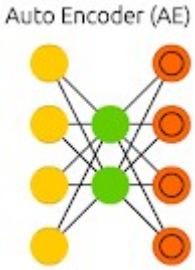
Thus far, we have only encountered parametric functions that take as input the output of the preceding layer, without any consideration of time.
In this scenario, the input also includes the previous hidden representation of the feature.

Another way to represent this logic is the following:



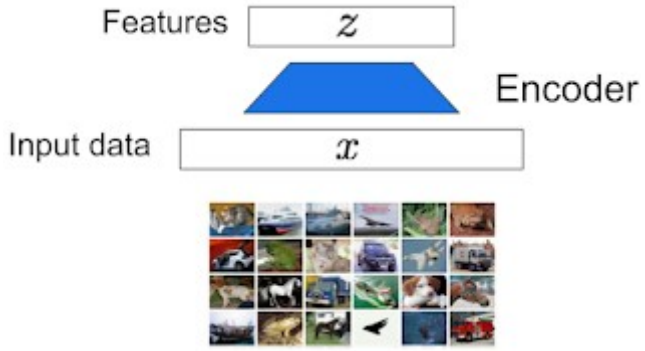
This is actually very similar to a Feedforward Neural Network, which means that it is possible to utilize backpropagation to train this model, despite its slightly different design.

Let’s now talk about **Auto Encoders**.



Autoencoders (AE) share similarities with Feed-Forward Neural Networks (FFNNs), as they represent a different application rather than a distinct architecture. AEs automatically encode information, compressing it without encryption, hence the name. They typically exhibit an hourglass shape, with smaller hidden layers compared to input and output layers. AEs are **symmetric around their middle layer**(s), where information is **most compressed**. The **encoding** portion precedes the middle layer(s), while the **decoding** follows it, with the middle layer(s) serving as the code. Training involves **backpropagation**, where input is fed and the **error is computed as the difference between the input and the output** (reconstruction loss: regression loss). AEs can be symmetric in terms of weights, with encoding and decoding weights being the same.

The purpose of the autoencoder is to construct a nonlinear mechanism for dimensionality reduction. It serves as an unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data. These features aim to capture meaningful factors of variation in the data, with the size of the feature space z typically smaller than the input space x, thereby focusing on the essential aspects of the data. For instance, given a set of unlabeled images, the objective is to map them into a latent space, represented as a vector or matrix.



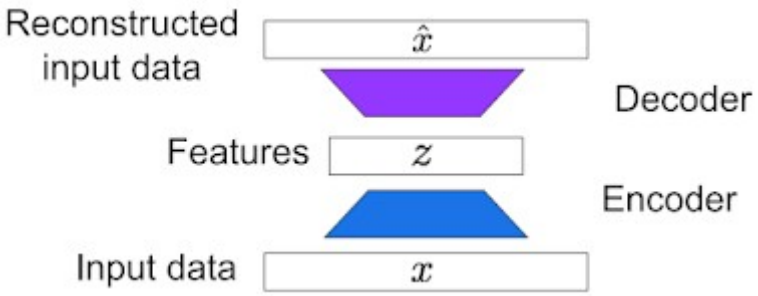
Similar to Principal Component Analysis (PCA), autoencoders operate under the assumption that they retain all crucial information based on factors like loss and design choices. However, their optimality is context-specific, relying on these underlying assumptions.

Autoencoders offer greater versatility compared to Principal Component Analysis (PCA) due to the neural network-based encoder component, which can be implemented in various ways:

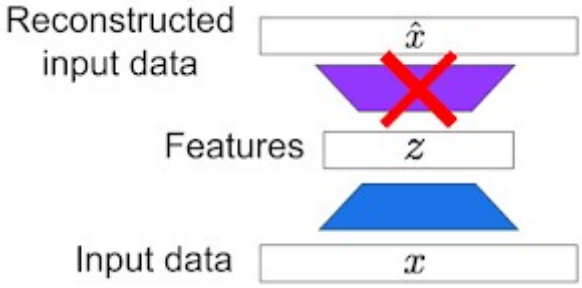
- **Initially**: Single linear layer followed by a nonlinearity (such as sigmoid or hyperbolic tangent).
- **Subsequently**: Introduction of deep structures with multiple layers, fully connected.

- **Recent Advances**: Adoption of Convolutional Neural Networks (CNN) with Rectified Linear Unit (ReLU) activation functions, particularly advantageous for structured inputs like images. Unlike fully connected layers, CNNs preserve spatial information in images, making them a more suitable choice.

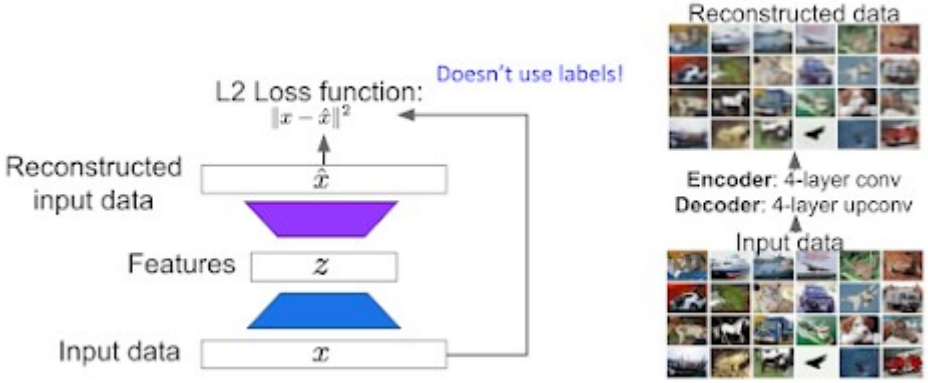
Encoders are tasked with learning a feature representation that enables the reconstruction of the original data (a process known as "**autoencoding**").
To accomplish this during the learning phase, we combine the encoder with a decoder, which is another neural network mirroring the encoder. The decoder's objective is to map the learned features back to the original input space, effectively producing a reconstructed representation of the input.



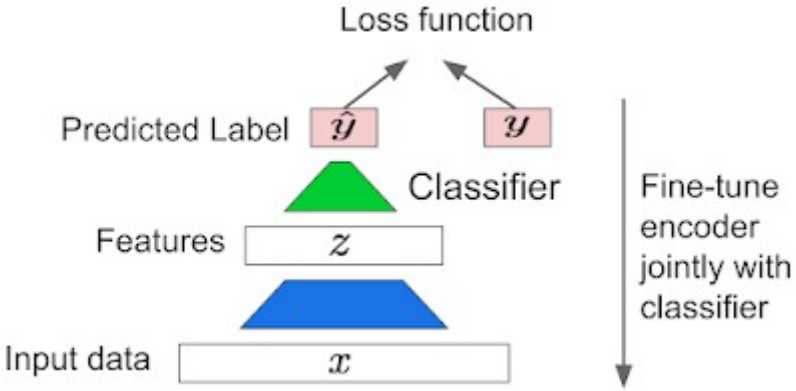
The underlying assumption is that the optimal compressed representation is one that facilitates accurate reconstruction of the input data. Following training, the decoder is discarded, as it is only necessary during the training phase.



Since this model is utilized for unsupervised learning, labels are not available. One approach is to employ the L2 loss (squared loss). Additionally, it's essential for the encoder and decoder to possess an equal number of layers.



The encoder can be used to initialize the input data for a supervised model, such as in a classification task. This approach is analogous to PCA; however, in this case, if a classifier is attached after the encoder, it is possible to fine-tune the entire model. This means that all the weights of the classifier and the dimensionality reduction component can be optimized, starting with the initialization provided by the autoencoder.



Additionally, the autoencoder can learn complex non-linear mappings, whereas PCA (without the kernel) is limited to learning linear mappings.

Deep Generative Models

In the last ten years, progress in generative models has been tremendous. In fact, a few years ago, we weren't able to generate even 28x28 images like those of numbers, but nowadays, it is possible to generate high-resolution images that are very realistic.

The history of generative models is as follows:



There are three families of methods for generating data:

- **Generative Adversarial Networks** (GANs)
- **Variational Autoencoders** (VAEs)
- **Diffusion models**

All of these methods have a substantial mathematical foundation, making the field somewhat complex.
Although we will focus on images, these methods can also be applied to other types of data (such as audio, signals, 3D structures, languages, etc.).

Autoencoders are the first models we encountered that do not perform a supervised learning task using neural networks. Now, we will look at other neural networks used for another unsupervised learning task: density estimation.

Before this, let's define the terminology:

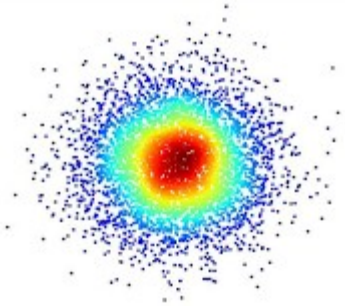
- **Generative models** are statistical models of the data distribution p_X or the joint distribution p_{XY} depending on the availability of target data.
- **Discriminative models** are statistical models of the conditional distribution $p_{Y|X}$ of the target given the input (as we have seen so far, like SVMs).

A discriminative model can be constructed from a generative model via Bayes' rule, but not vice versa.

$$p_{Y|X}(y \mid x) = \frac{p_{XY}(x, y)}{\sum_{y'} p_{XY}(x, y')} \left] P_X(x)\right.$$

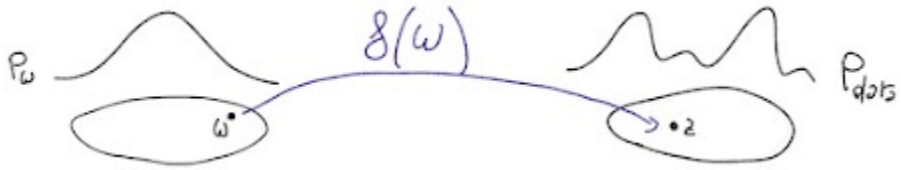
We've mentioned the notion of density estimation several times (learning the underlying probability distribution p_{data} from a set of points), but now we introduce a new concept: the difference between Explicit Density Estimation and Implicit Density Estimation:

- **Explicit Density Estimation** involves finding a probability distribution $f \in \Delta(Z)$ that fits the training data, where $z \in Z$ is sampled from an unknown data distribution $p_{data} \in \Delta(Z)$.
  This involves directly learning a function that describes the data points. This approach is easier but mathematically intensive because it requires designing the likelihood function.



- **Implicit Density Estimation** involves finding a function $f \in Z^{\Omega}$ that generates data $f(\omega) \in Z$ from an input sampled from some predefined distribution $p_{\omega} \in \Delta(\Omega)$ in such a way that the distribution of generated samples fits the (unknown) data distribution $p_{data} \in \Delta(Z)$.
  This method is more complex because it is done indirectly. It uses a simple distribution (e.g., a Gaussian distribution) called the prior or predefined distribution from which samples are drawn. When points from the prior distribution are mapped through the function f, they describe the unknown data distribution. This simplifies the generation

process in practice.



In supervised learning, $Z = X \times Y$; in unsupervised learning, $Z = X$.

Before delving into the details, let's state the problem of Density Estimation more formally:

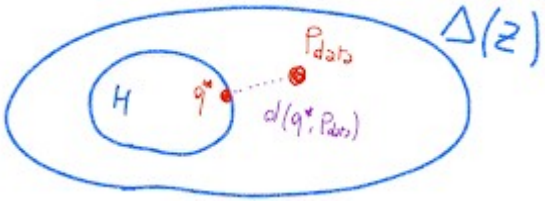
1. Define a **hypothesis space** $H \subset \Delta(Z)$ of models that can represent probability distributions (implicitly or explicitly) and search within it.
2. Define a **divergence measure** $d \in R^{\Delta(Z) \times \Delta(Z)}$ between probability distributions in $\Delta(Z)$ (e.g., Kullback-Leibler divergence) to measure the difference between probability distributions.

   $$D_{\mathrm{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log\left(\frac{P(x)}{Q(x)}\right)$$
3. Find a hypothesis $q^* \in H$ that best fits the data distributed according to p_{data}.

The choice of the divergence measure and the hypothesis space will lead to different models. The optimal q^* will be learned using neural networks, and the network architecture will define the hypothesis space.

The best fit is measured using the divergence d:

$$q^\star \in \arg\min_{q \in \mathcal{H}} d(p_{\text{data}}, q)$$



For the remainder of the chapter, we assume, for simplicity, that $Z = X$

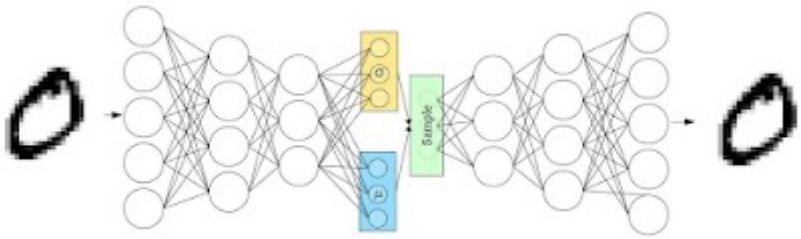
Implicit Density Estimation, despite seeming complex, is a brilliant idea because the mapping and matching can be implemented by the neural network.

For many years, before diffusion models became prominent, we had two main methods for generating samples:

- **Variational Autoencoders (VAEs)**, which use Explicit Density Estimation.
- **Generative Adversarial Networks (GANs)**, which use Implicit Density Estimation.

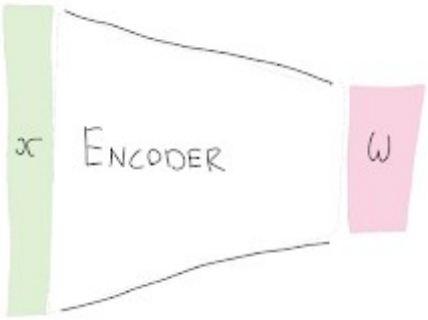
Variational Autoencoders

The first approach we will examine for solving the problem of Density Estimation with Neural Networks is **Variational Autoencoders** (VAEs), introduced in 2014 by Kingma and Welling. The idea is to add a probabilistic aspect to traditional autoencoders:

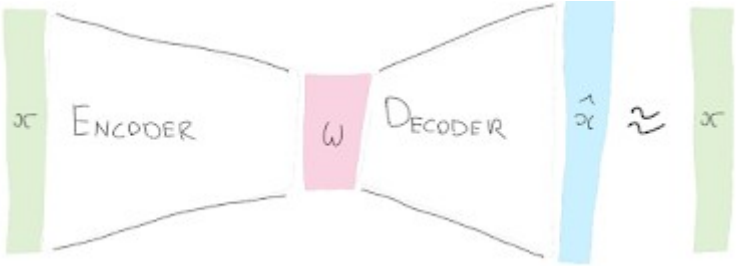


Instead of having deterministic latent representations in the middle, we have representations that can learn a probability distribution. So we are giving a probabilistic flavor to the vanilla autoencoder, but overall, the approach is quite similar.

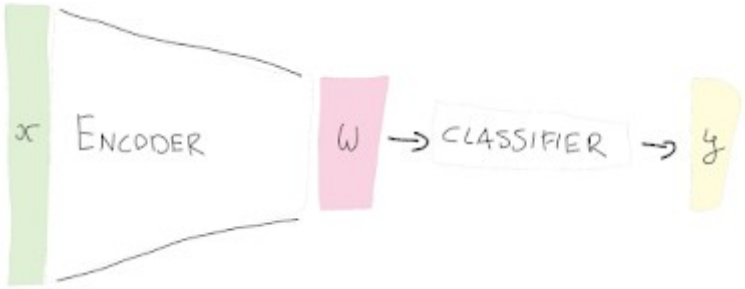
Let's briefly recap autoencoders:
Autoencoders compress high-dimensional data into lower-dimensional representations. An encoder maps input data to a **compressed representation**, preserving meaningful factors of variation in the data. For example, PCA with reduced dimensions can be seen as a linear encoder preserving as much variance in the data as possible.



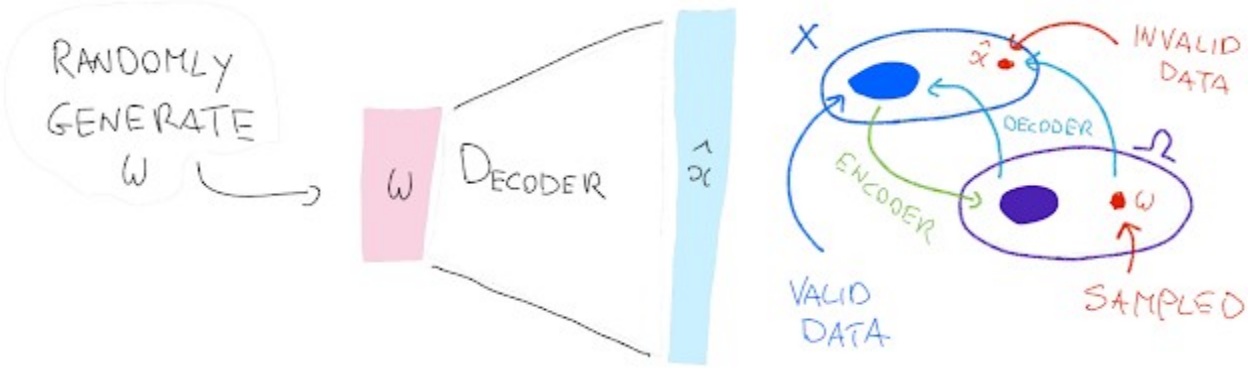
An encoder is trained using a decoder that maps the representation back to the input domain, yielding $\hat{x}$ (reconstruction). Thus, the encoder "autoencodes" its input. The objective is to minimize the divergence between the input and its reconstruction.



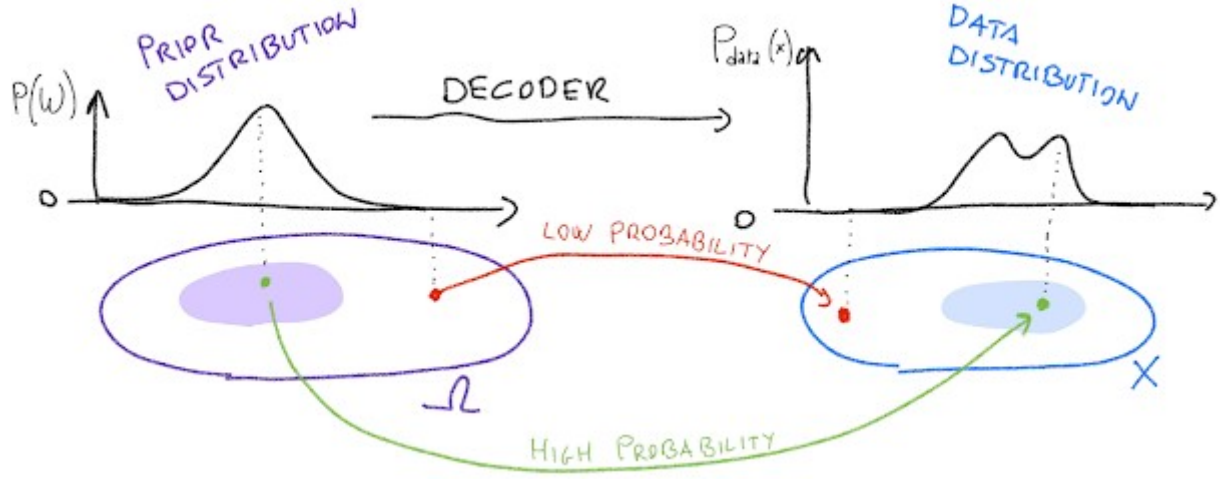
After training, the decoder is no longer needed since it was only functional for estimating the encoder. The encoder can then be used to initialize or precompute features for supervised models.



While the decoder of the autoencoder could be used to generate new data, it will **not generate** data according to the data distribution p_{data}, because we can only feed the decoder with randomly generated inputs.



We would like to have a prior distribution from which to sample data to give to the decoder to obtain data from the original data distribution:



This means that low-probability objects in the prior distribution result in low-probability objects in the data distribution, and the same with high-probability objects.

Let's delve into some mathematics.
The objective is to minimize the discrepancy between two probability distributions: the unknown probability distribution from which we want to sample and the neural network which implements

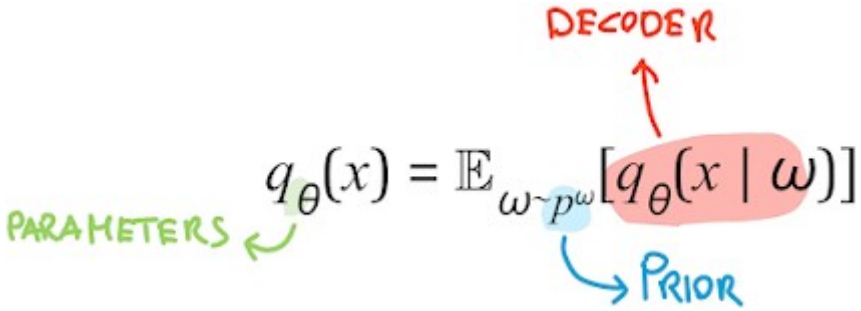
the probability distribution we are trying to learn. This is minimized over the parameters of the neural network.

$$\theta^{*} \in \arg\min_{\theta \in \Theta} d(q_{\theta}, p_{\text{data}})$$

The Kullback-Leibler (K-L) divergence is used as the measure of discrepancy.

$$d_{\text{KL}}(p, q) = \mathbb{E}_{x \sim p}\left[\log \frac{p(x)}{q(x)}\right]$$

The neural network, which implements the probability distribution we are trying to learn q_θ, will be implemented through a decoder that will learn a mapping from the prior space to the data space. The q_θ is computed by sampling in the latent space over a simple distribution that is the prior distribution.



Applying the K-L divergence to the distributions, we obtain the following:

$$\begin{aligned} d_{\text{KL}}(p_{\text{data}}, q_\theta) &= \mathbb{E}_{x \sim p_{\text{data}}}\left[\log \frac{p_{\text{data}}(x)}{q_\theta(x)}\right] \\ &= -\mathbb{E}_{x \sim p_{\text{data}}}\left[\log q_\theta(x)\right] + \text{const} \\ &= -\mathbb{E}_{x \sim p_{\text{data}}}\left[\log \mathbb{E}_{\omega \sim p_\omega}[q_\theta(x \mid \omega)]\right] + \text{const} \end{aligned}$$

INTRACTABLE

$\log\left(p_{data}(x)\right)$ is a constant since it is independent of the parameters, and we rewrite $q_\theta(x)$ in the form we've seen before.

It turns out that minimizing this results in an integral that must be computed over Ω, meaning we have to marginalize over the latent space, which is intractable.

Researchers have devised a way to circumvent this issue: instead of sampling from this prior distribution and attempting to marginalize over Ω and compute that, another neural network is introduced, which is essentially the encoder of the VAE (a network that maps the input data into the latent space).

$$q_\psi(\omega \mid x) \in \Delta(\Omega)$$

with ψ being other parameters. This is called the encoding probability distribution and is implemented with another neural network.

So now, instead of sampling from p_ω, we sample from this $q_\psi(\omega \mid x)$

$$\log \mathbb{E}_{\omega \sim p_\omega}[q_\theta(x \mid \omega)] = \log \mathbb{E}_{\omega \sim q_\psi(\cdot \mid x)}\left[q_\theta(x \mid \omega)\frac{p_\omega(\omega)}{q_\psi(\omega \mid x)}\right]$$

Then, applying the Jensen Inequality (which defines the relation between the logarithm of the expected value and the expected value of the logarithmic function)

$$\overset{\text{by Jensen's inequality}}{\geq} \mathbb{E}_{\omega \sim q_\psi(\cdot \mid x)}\left[\log\left(q_\theta(x \mid \omega)\frac{p_\omega(\omega)}{q_\psi(\omega \mid x)}\right)\right]$$

We approximate the minimization of the discrepancy, achieving an upper bound of the ideal function.

Splitting the logarithmic function, we end up with these two terms:

$$= \mathbb{E}_{\omega \sim q_\psi(\cdot \mid x)}[\log q_\theta(x \mid \omega)] - d_{\mathrm{KL}}(q_\psi(\cdot \mid x), p_\omega)$$

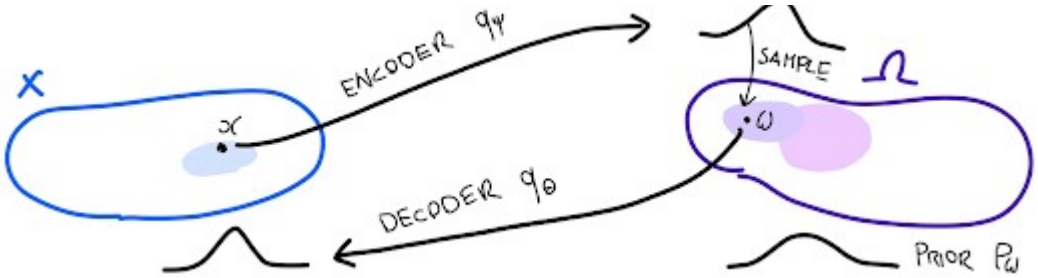
RECONSTRUCTION REGULARIZER

These two terms correspond to two different neural networks.

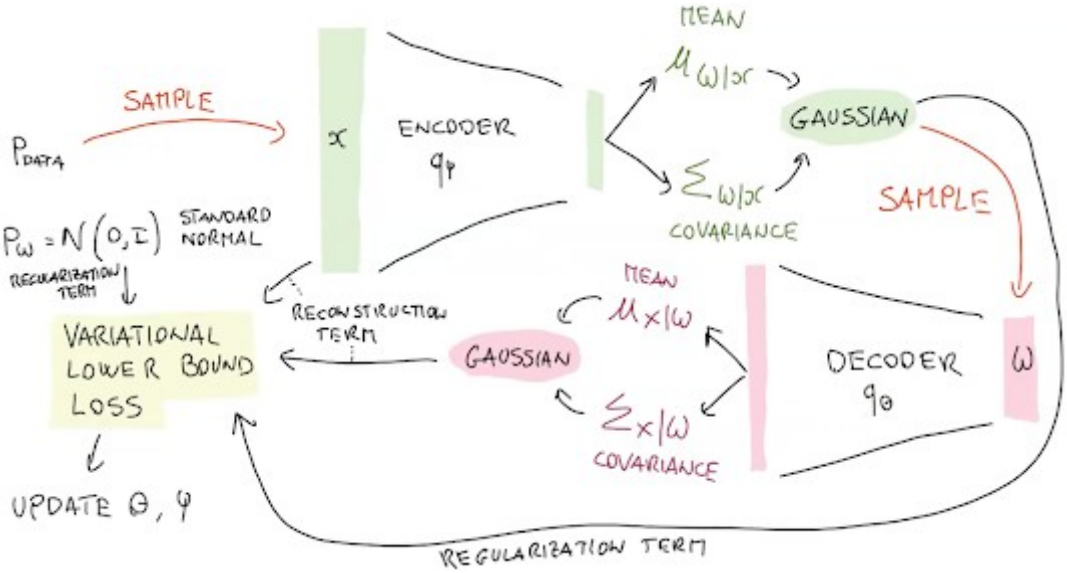
- The **reconstruction term** drives the behavior of the decoder. Although still intractable to compute, we can estimate the gradients of encoder and decoder parameters.
- The **regularizer term** measures the distance between the prior distribution and the distribution that the encoder will learn. This forces our first neural network to learn a Gaussian distribution. Since both are Gaussian, there is a closed-form solution to compute it.

The final result will be something like this:

$$d_{\mathrm{KL}}(p_{\mathrm{data}}, q_\theta) \leq \mathbb{E}_{x \sim p_{\mathrm{data}}}\left[-\mathbb{E}_{\omega \sim q_\psi(\cdot \mid x)}[\log q_\theta(x \mid \omega)] + d_{\mathrm{KL}}(q_\psi(\cdot \mid x), p_\omega)\right] + \text{const}$$



In practice, the training is implemented as follows:



So:

- Sample data points from the training set.
- The encoder doesn't learn a vector but learns the mean and covariance with respect to the probability $\omega \mid x$.
- The mean and covariance learned are used to sample inputs for the decoder.
- The decoder, another neural network, learns the mean and covariance with respect to the probability $x \mid \omega$.

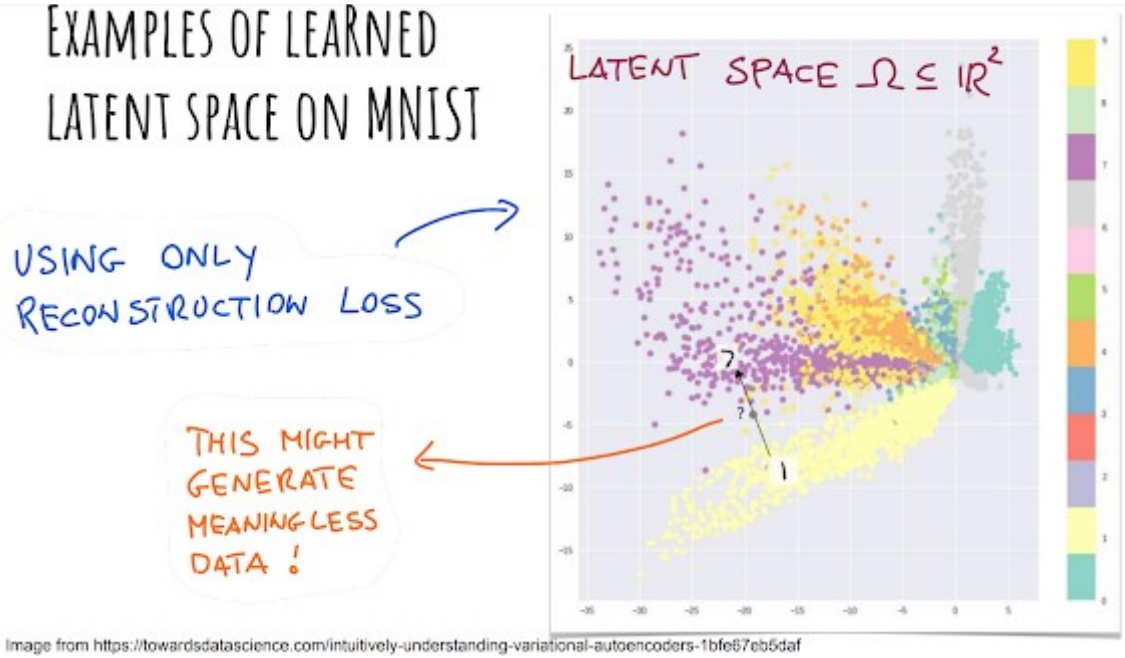
Then, using the variational lower bound loss:

$$\mathbb{E}_{x\sim p_{\text{data}}}\left[-\mathbb{E}_{\omega\sim q_\psi(\cdot|x)}[\log q_\theta(x\,|\,\omega)] + d_{\text{KL}}(q_\psi(\,\cdot\,|x), p_\omega)\right] + \text{const}$$

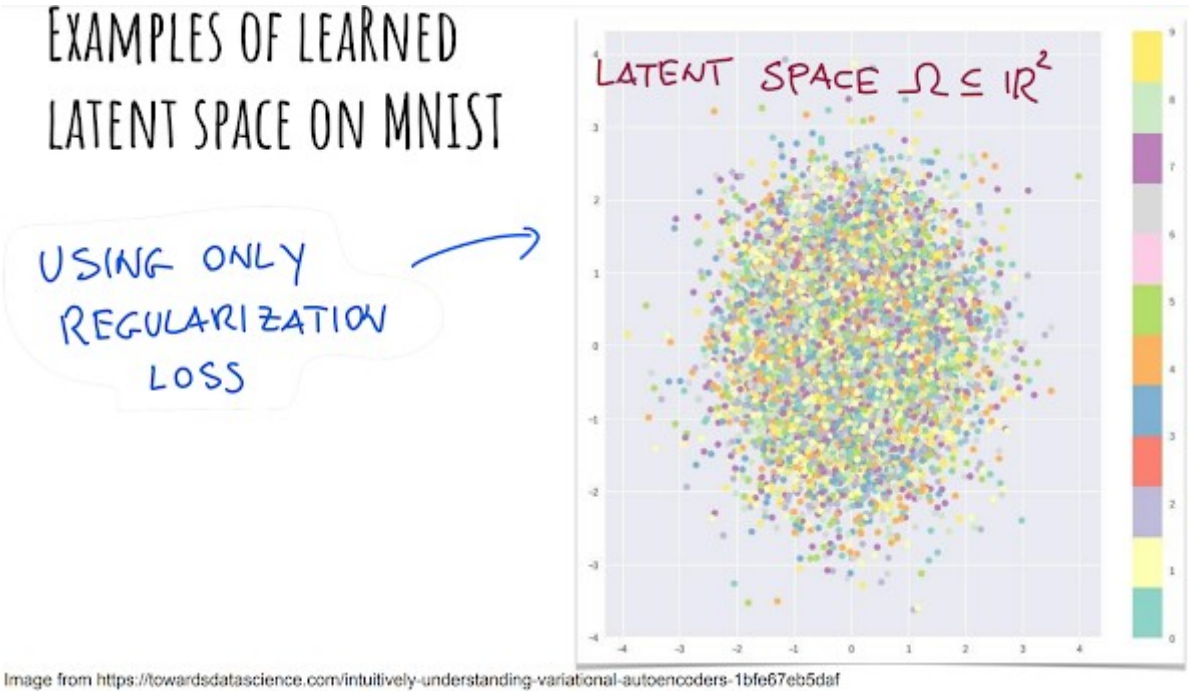
We train both the parameters of the encoder and the parameters of the decoder end-to-end.

Now let's better understand why this works.

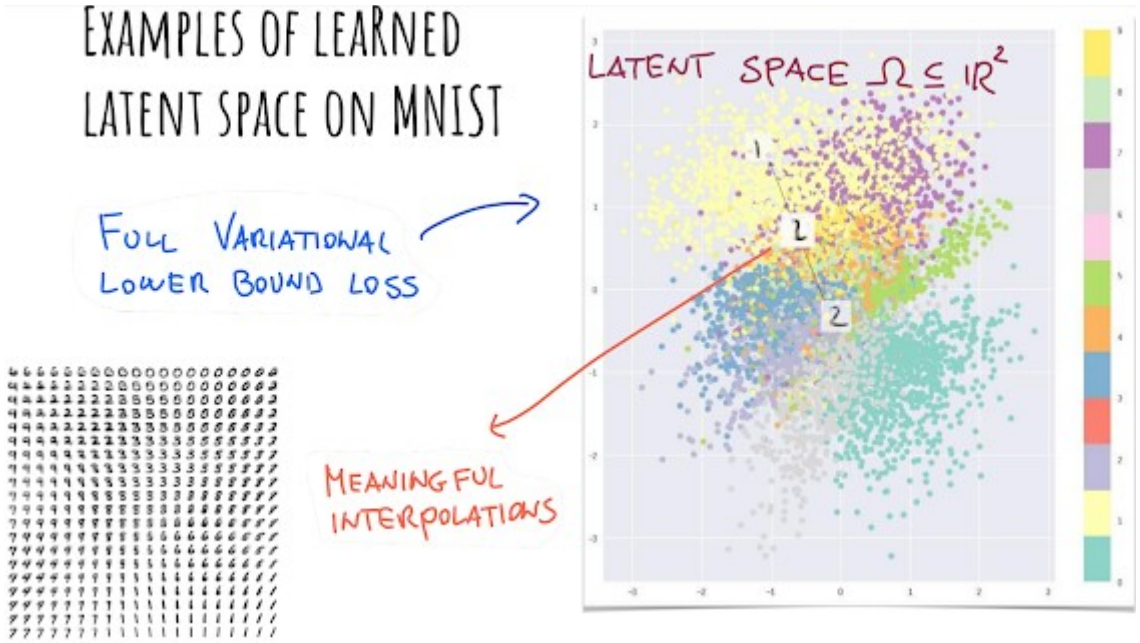
Using only the reconstruction loss, the model is required only to reconstruct the training data as much as possible. Thus, in the latent space, the neural network learns that there are clusters of latent variables corresponding to digits, but taking a point outside these clusters will not generate something meaningful, because that part of the latent space is empty.



Using only the regularization loss, the model learns something resembling a Gaussian distribution, but there is no constraint ensuring that latent representations corresponding to the same digits should be close together.

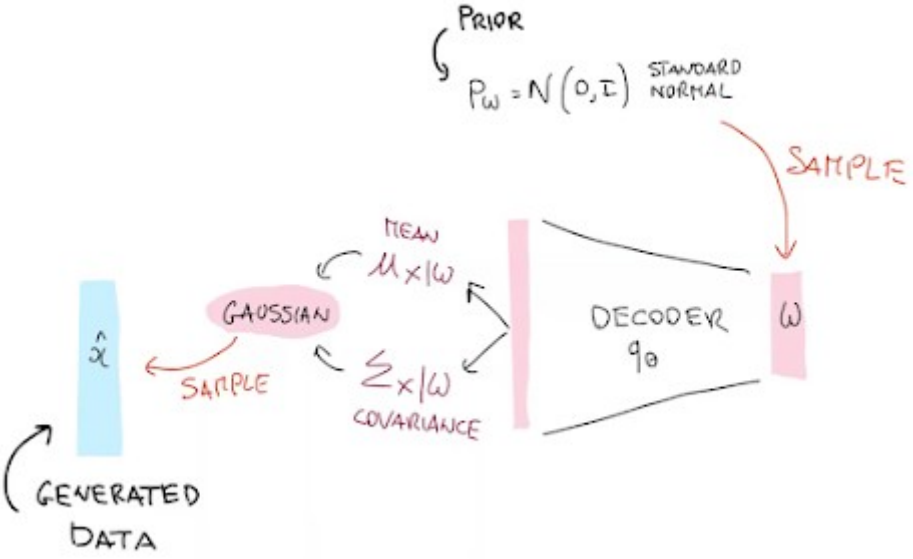


Combining both losses allows us to have a smooth latent space with clustered representations corresponding to similar samples.



In this case, interpolating between two existing latent points will produce results similar to what we expect.

Data generation is implemented as follows:



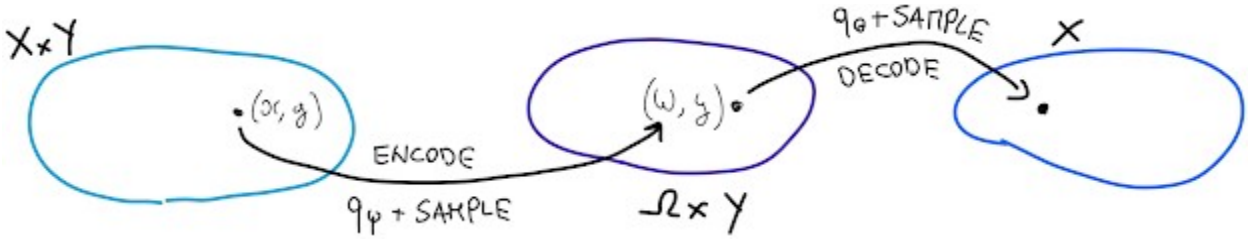
We only use the decoder, sampling from a standard normal distribution. Therefore, VAE has a very simple inference phase: just a feedforward pass.

Here are some examples of what can be achieved with these models:



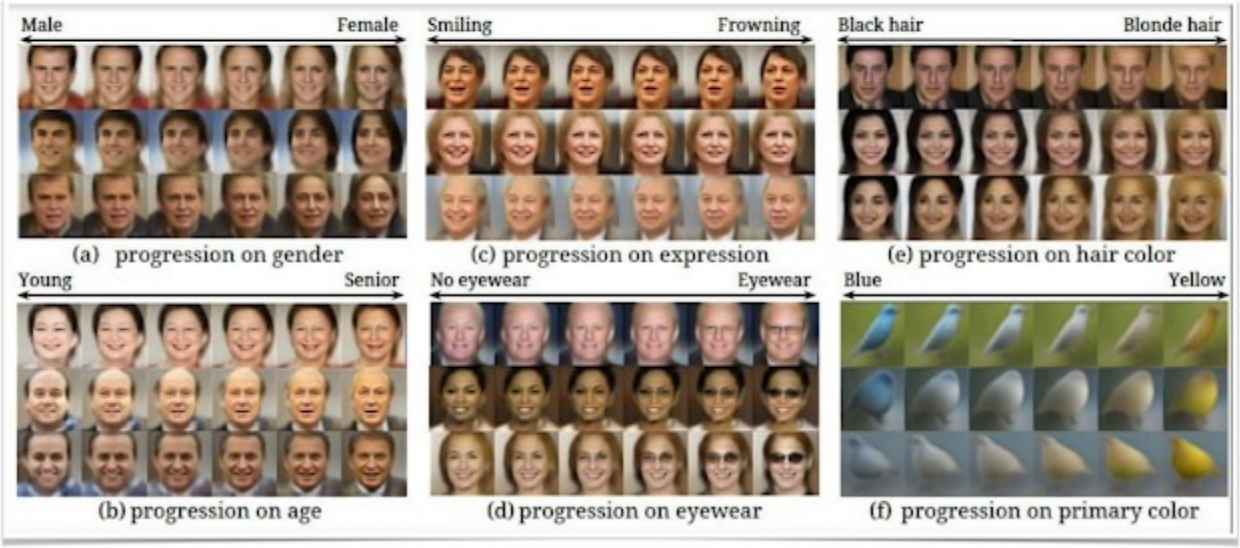
Originally, VAEs were used to generate low-resolution samples. This was a significant achievement because no shallow learning tool existed to perform such tasks (structured output data). Complex neural networks are needed to approximate complex functions, making VAEs a breakthrough in generative modeling.

Let's talk about **Conditional VAEs (CVAE)**.
Assume we have side information $y \in Y$ (e.g., digit labels, attributes, etc.) and we want to generate new data conditioned on this side information (e.g., generate a digit 7, or generate a face with glasses). In this case, we modify the encoder and decoder to take the side information as input, obtaining $q_\psi(\omega \mid x, y)$ and $q_\theta(x \mid \omega, y)$, and define priors conditioned on side information $p_\omega(\omega \mid y)$.



One interesting aspect of forcing the neural network to consider attributes is that you can explore the latent space (which is well defined) to achieve smooth transitions when generating faces with varying feature values.

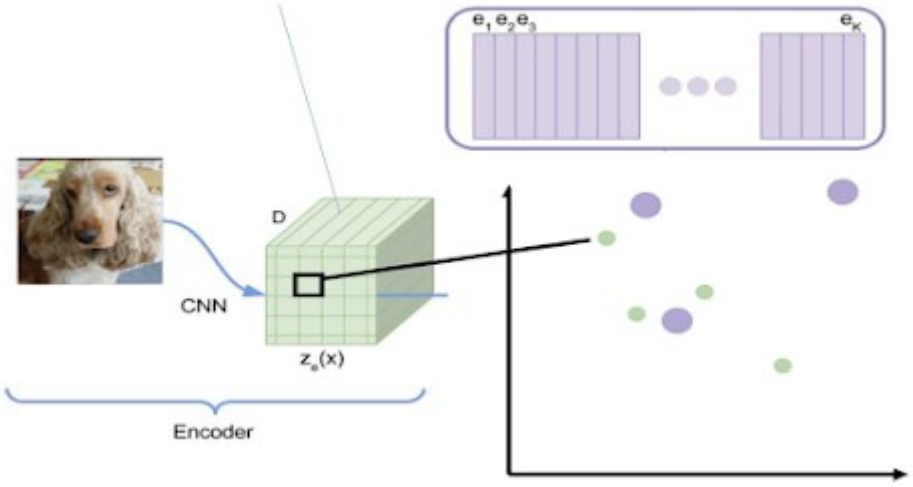
Example



Initially, VAEs had two major issues:

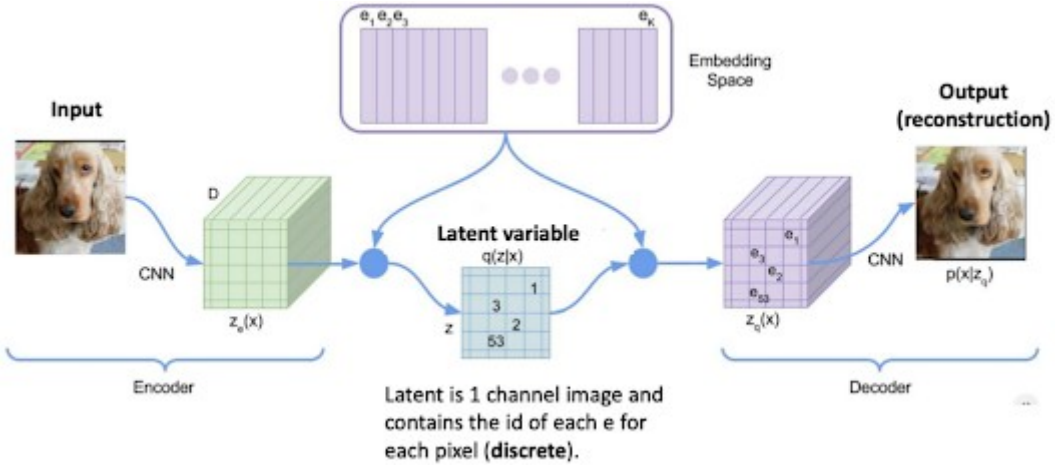
- **Underfitting**: At the initial stages, the regularizer is too strong and tends to diminish the model's capacity. This is because forcing the latent space to be Gaussian limits the model's ability to represent the data accurately. Methods now exist to balance these terms to address this issue.
- **Blurry Samples**: The generator tends to produce blurry data. This problem has been mitigated with more powerful architectures for the encoder and decoder.

The latent space of VAEs is continuous, but researchers have devised a way to quantize the latent representation, transforming it from a continuous to a discrete space, leading to the development of **Vector Quantized-VAE** (VQ-VAE), a popular model for image synthesis. VQ-VAE-2 is an advanced version based on VAEs, featuring a discrete space with a dictionary. It produces high-quality images comparable to those generated by large GAN models.



This involves a few key steps:

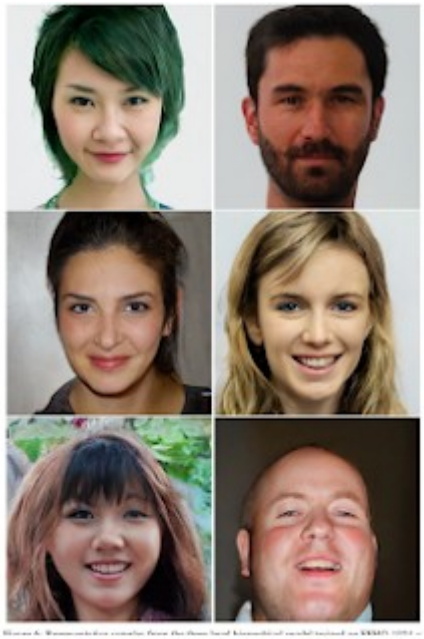
- **Latent Representation**: Similar to a VAE, the data is passed through an encoder, which transforms it into a latent representation (green points). This representation captures the essential features of the data.
- **Dictionary Creation**: VQ-VAE introduces a dictionary or codebook of discrete vectors. Each vector in this dictionary represents a specific cluster or centroid (purple points) in the latent space.
- **Quantization Process**: During the quantization process, each latent vector from the encoder is replaced by the closest centroid from the dictionary. This means each data point is effectively "snapped" to the nearest vector in the dictionary. This process is known as quantization.
    - **Example**: If you have a point in the latent space that is closest to a particular centroid, that point is replaced with the centroid itself.
- **Decoding**: The decoder then reconstructs the original data using these quantized vectors from the dictionary. This reconstruction process often leads to high-quality outputs because the discrete nature of the space helps in capturing and reproducing the essential features of the data



Having a discrete representation of the embedding space allows the use of tools designed for discrete representations, such as autoregressive models, to generate data. This enables the integration of various types of data, including text, into the neural network with VQ-VAE to generate images.

VQ-VAE has enabled the transition from low-quality generation to high-resolution generation.

VQ-VAE



The initial implementation of variational autoencoders produced low-quality samples, but modern VAEs can generate high-quality samples.

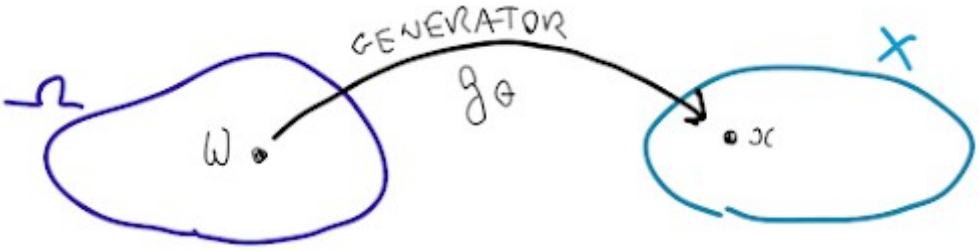
In 2024, the reasons to use VAEs include:

- Direct optimization of an upper bound on the target function.
- Excellent control over the latent space.

Generative Adversarial Networks

The second approach we will examine for solving the problem of density estimation with neural networks is **Generative Adversarial Networks** (GANs).
GANs are relatively easier to understand conceptually, although they involve some heavy mathematics, especially in their various modifications. GANs enable the estimation of implicit densities, making the process a bit more complex.

We assume a prior density $p_\omega \in \Delta(\Omega)$ is given, and a **generator** (or decoder) $g_\theta \in X^\Omega$ that generates data points in X given a random element from Ω.
This approach allows us to learn a data distribution similar to the original one by sampling from a simpler distribution in a different space where we have better control. A neural network then maps the data from this simple distribution to a complex distribution.



If we only have these objects, without any control over what is happening in the complex space, it doesn't work. So, the idea of GANs is to introduce another neural network in the complex space to control what is generated.

Let's now see the mathematical formulation behind GANs.
The density induced by the prior p_ω and the generator g_θ is given by $q_\theta(x)$:

$$q_\theta(x) = \mathbb{E}_{\omega \sim p^\omega} \delta[g_\theta(\omega) - x]$$

where δ is the Dirac delta function, which is a theoretical construct used in mathematics and physics. It is defined as being zero everywhere except at a single point, where it is infinitely high in such a way that its total integral over all space is equal to one. This makes it useful for modeling idealized distributions that are concentrated at a single point, effectively "picking out" the value of a function at that point.
The term $\delta\left[g_\theta(\omega) - x\right]$ effectively "selects" the value of $g_\theta(\omega)$ that matches x. Since $\delta\left[y - x\right]$ is zero for all $y \neq x$ and is infinitely high at $y = x$, this term acts to concentrate the probability mass at the point where the generated sample $g_\theta(\omega)$ equals x.

The original GAN objective is to find $\theta^\star$ such that $q_{\theta^\star}$ best fits the data distribution p_{data} under another divergence measure, the Jensen-Shannon divergence:

$$\theta^\star \in \arg\min_\theta d_{\mathrm{JS}}(p_{\mathrm{data}}, q_\theta)$$

where

$$d_{\mathrm{JS}}(p, q) = \frac{1}{2} d_{\mathrm{KL}}\left(p, \frac{p+q}{2}\right) + \frac{1}{2} d_{\mathrm{KL}}\left(q, \frac{p+q}{2}\right)$$

The Jensen-Shannon divergence is the average between two Kullback-Leibler divergences with reversed terms. Unlike the Kullback-Leibler divergence, which is asymmetric, the symmetric nature of the Jensen-Shannon divergence makes it a better choice for this purpose.

This objective function is intractable to compute directly, and the same holds for the gradient. Using the definitions of Jensen-Shannon divergence and Kullback-Leibler divergence, we obtain the following:

$$
\begin{aligned}
d_{\mathrm{JS}}(p, q) &= \frac{1}{2} d_{\mathrm{KL}}\left(p, \frac{p+q}{2}\right) + \frac{1}{2} d_{\mathrm{KL}}\left(q, \frac{p+q}{2}\right) \\
&= \frac{1}{2}\mathbb{E}_{x\sim p}\left[\log \frac{2p(x)}{p(x)+q(x)}\right] + \frac{1}{2}\mathbb{E}_{x\sim q}\left[\log \frac{2q(x)}{p(x)+q(x)}\right] \\
&= \frac{1}{2}\mathbb{E}_{x\sim p}\left[\log \frac{p(x)}{p(x)+q(x)}\right] + \frac{1}{2}\mathbb{E}_{x\sim q}\left[\log \frac{q(x)}{p(x)+q(x)}\right] + \log(2) \\
&= \log(2) + \frac{1}{2}\max_t \left\{ \mathbb{E}_{x\sim p}\left[\log t(x)\right] + \mathbb{E}_{x\sim q}\left[\log(1-t(x))\right] \right\}
\end{aligned}
$$

In the second-to-last line, we have two logarithms with normalization terms that can be viewed as probabilities, having values between 0 and 1. We can call one term $t(x)$, and the other $1\text{-}t(x)$. Thus, $t(x)$ functions as a binary classifier predicting whether x came from p or q.

The idea is to learn $t(x)$ with a neural network, minimizing the divergence computed this way with respect to the generator's parameters, and maximizing the same objective with respect to the classifier. Let $t_\varphi(x)$ be a classifier (or discriminator) implemented with a neural network. For data points in the training set, we get the following lower bound on our objective:

$$
\begin{aligned}
d_{\mathrm{JS}}(p_{\mathrm{data}}, q_\theta) &= \log(2) + \frac{1}{2}\max_t \left\{ \mathbb{E}_{x\sim p_{data}}\left[\log t(x)\right] + \mathbb{E}_{x\sim q_\theta}\left[\log(1-t(x))\right] \right\} \\
&\geq \log(2) + \frac{1}{2}\max_\varphi \left\{ \mathbb{E}_{x\sim p_{data}}\left[\log t_\varphi(x)\right] + \mathbb{E}_{x\sim q_\theta}\left[\log(1-t_\varphi(x))\right] \right\}
\end{aligned}
$$

CAN BE NEGLECTED
NOT CHANGING
MINIMIZER θ^*

which is minimized to obtain the generator's parameters:

$$
\theta^\star \in \operatorname*{argmin}_\theta \max_\varphi \left\{ \mathbb{E}_{x\sim p_{\mathrm{data}}}\left[\log t_\varphi(x)\right] + \mathbb{E}_{x\sim q_\theta}\left[\log(1-t_\varphi(x))\right] \right\}
$$

Although this still depends on the explicit density, it can be equivalently rewritten (proof skipped):

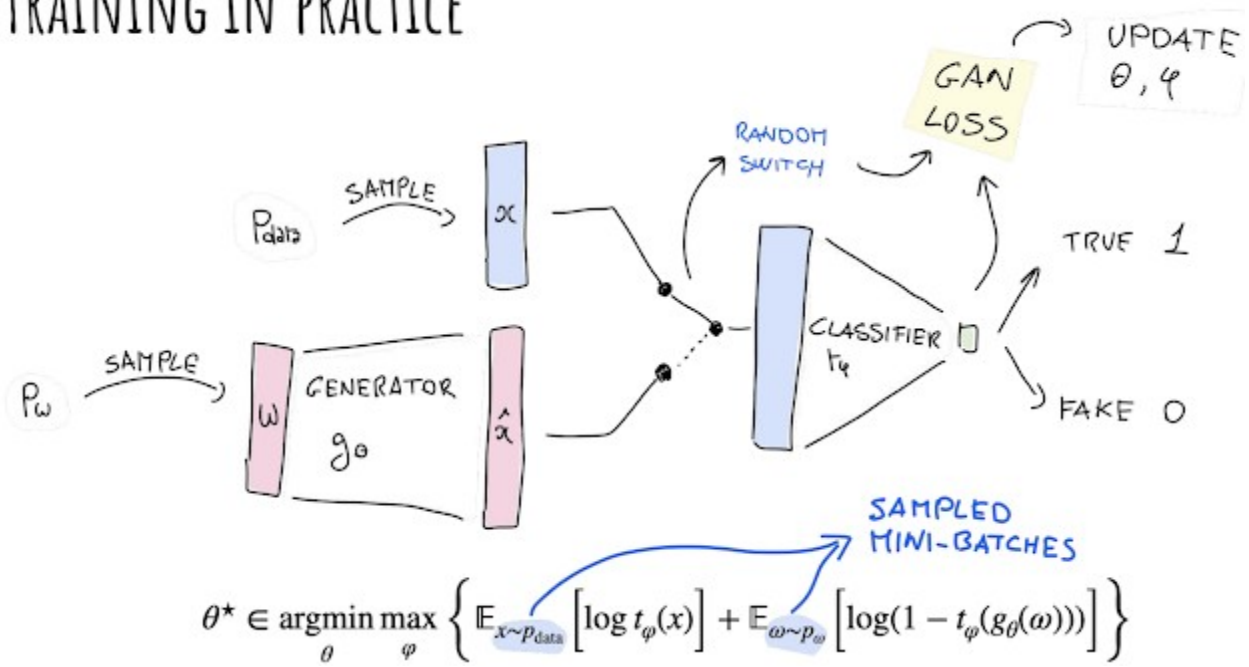
$$
\theta^\star \in \operatorname*{argmin}_\theta \max_\varphi \left\{ \mathbb{E}_{x\sim p_{\mathrm{data}}}\left[\log t_\varphi(x)\right] + \mathbb{E}_{\omega\sim p_\omega}\left[\log(1-t_\varphi(g_\theta(\omega)))\right] \right\}
$$

The exact meaning of this objective is a two-player, zero-sum, non-cooperative game:

- **Player 1 (Generator)**: Tries to generate data that cannot be distinguished from the true data.
- **Player 2 (Discriminator)**: Tries to guess whether the input comes from the true distribution or is "fake." Technically, this is a classifier implemented with cross-entropy loss.

Training is done using gradient descent and specialized approaches for handling the min-max problem, often handled in an alternating fashion.

The training is done in the following way:



To achieve a probability distribution similar to the real data distribution (according to Jensen-Shannon divergence), we use the generator. The generator is a neural network that takes numbers sampled from a simple prior distribution and outputs objects in the final space (e.g., images).

The classifier (or discriminator) then evaluates whether the samples are generated from the real data distribution (true: 1) or from the fake data distribution (false: 0).

The objective is a min-max because:

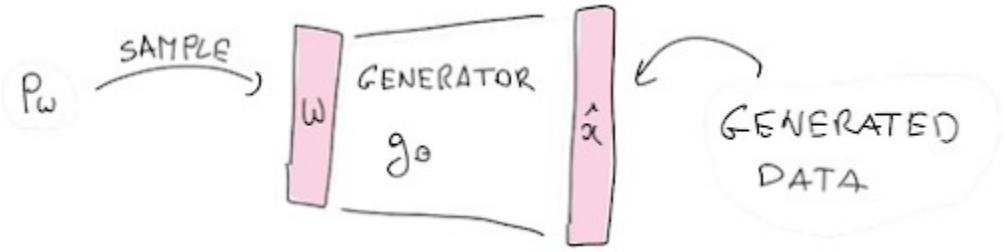
- The generator wants to match the original data distribution as closely as possible.
- The discriminator aims to become as effective as possible at distinguishing between real and fake samples.

Both networks are implemented with neural networks and are in competition, which strengthens their performance.

This is a brilliant idea, but it is much harder to optimize than classical generative models like VAE. The two-player game creates some numerical instability, such as when the discriminator learns too quickly compared to the generator, causing the generator to fail.
A significant difference is made by the architectures used for the generator and the discriminator. It took several years to design good neural networks (originally convolutional neural networks) to implement the generator and the discriminator.

In the inference phase, GANs share a similarity with VAEs in that they are particularly effective at inference time. While training can be challenging, inference is efficient because the discriminator is discarded. Inference involves simply sampling from a Gaussian distribution and

passing these numbers to the generator to get the data. This method was popular because it could generate random faces very quickly. The training is demanding, but inference is fast and practical.



Nowadays, this approach is considered outdated because we now have the ability to condition the generation process with text and other data.

In 2016, one of the first GANs (with a convolutional structure in the generator network) created these low-resolution images of bedrooms with 1 epoch (one complete cycle through the entire training dataset):

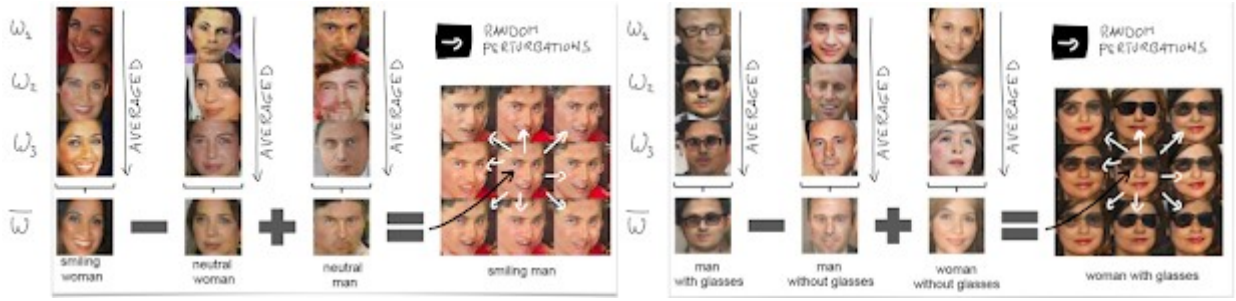


After 5 epochs, the results were images of relatively good quality:



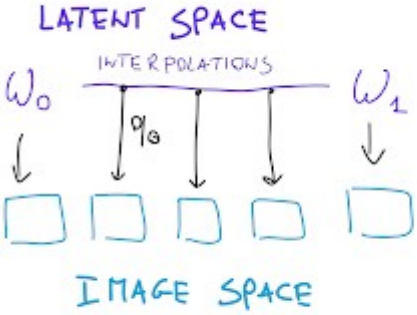
For VAEs, a nice property is the ability to move in the latent space, producing smooth variations in the result.

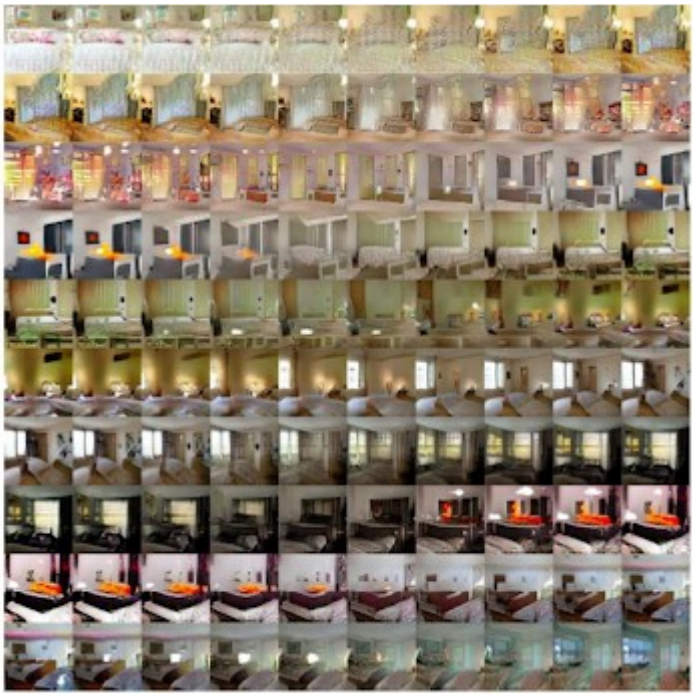
In GANs, even though the latent space isn't as smooth as in VAEs, it is possible to take different samples in the latent space and apply operations on them, such as averaging, addition, subtraction, and slight perturbations.
For example, averaging samples that result in a smiling woman produces a “prototype” of a smiling woman. Then, it is possible to add or subtract other prototypes to add or delete characteristics, such as subtracting a neutral woman and adding a neutral man to obtain a smiling man. Random perturbations can then be applied to the result to obtain different images of a smiling man.



This is done because images with the desired characteristics can be generated using simple operations in the latent space.

Even though the latent space structure isn't "forced" like in VAEs, GANs can still learn meaningful representations in the latent space. Therefore, it is possible to interpolate in the latent space and achieve good results with smooth variations:

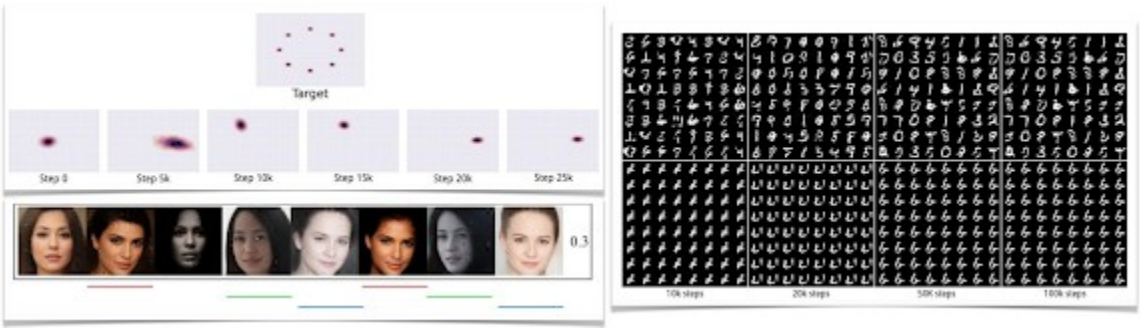




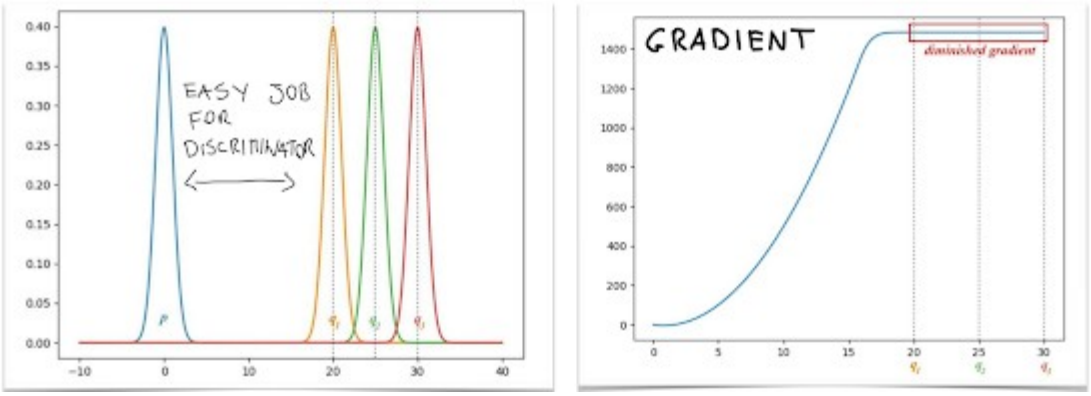
Previously, there were many artifacts, but this problem has been fixed with better convolutional networks.

The issues with GANs include:

- **Training stability:** Alternating updates between the generator and discriminator can cause parameters to oscillate and never converge.
- **Mode collapse:** The generator might learn to generate a few perfect examples from the training set while failing to cover the entire data distribution, resulting in an inability to represent all training data.



- **Metric to evaluate the model:** In the original implementation, the loss was not directly correlated with the quality of the visual images, so minimizing the loss did not necessarily improve image quality.
- **Vanishing gradient:** Using Jensen-Shannon divergence, if the discriminator is very successful, it leaves the generator with little gradient to learn from.



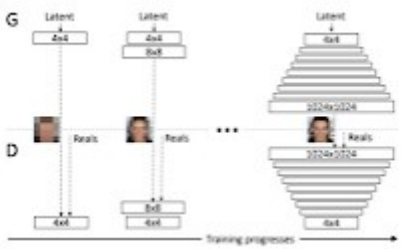
Let's look at several types of GANs:
Different GAN-like models can be constructed by considering various divergences between probabilities:

- **GANs based on Bregman divergences**
- **Wasserstein GANs (WGANs)**: These use the Wasserstein metric, which helps solve the vanishing gradient issue.

GANs and VAEs can also be combined, resulting in the hybrid VAE-GAN.
Additionally, conditional GANs, which incorporate label information, work similarly to conditional VAEs.

One difference between VAEs and GANs is that the optimization problem of a VAE is easier, and VAEs theoretically guarantee that the latent distribution is smooth.
In contrast, GANs lack constraints on the smoothness of the latent distribution and rely on the discriminator to explicitly force the quality of the generated samples, allowing them to generate higher-quality samples.
In recent years, there have been many advances in GAN technology:

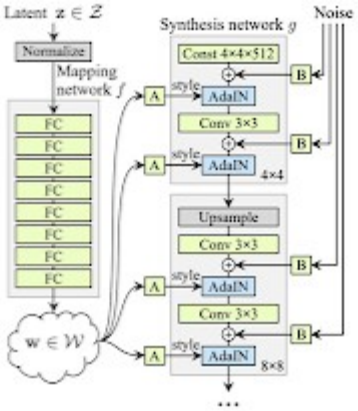
- **Progressive GAN**, a concept pioneered by NVIDIA, involves starting with a GAN composed of two shallow networks for the generator and discriminator, then progressively adding layers and learning the weights of the new layers to generate high-resolution images.





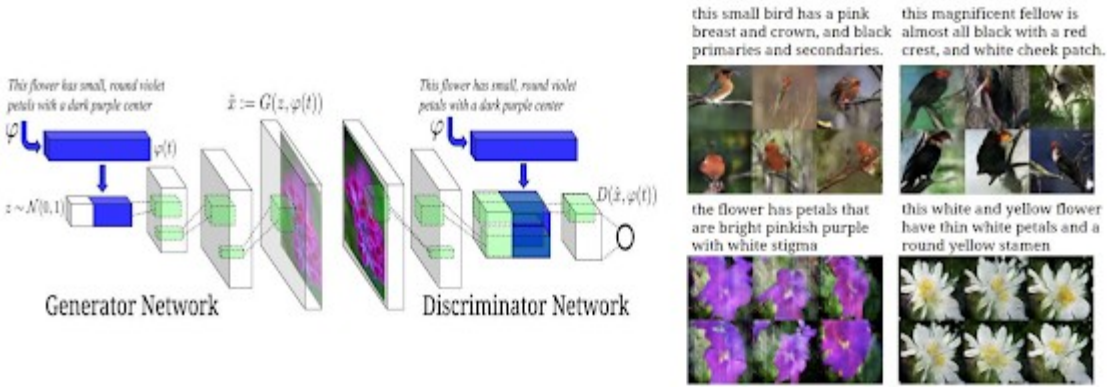
- **StyleGAN** produces incredibly realistic images and can be conditioned with additional information. It consists of several subsequent blocks, with each block controlling different aspects of the image, such as skin color and hair type. Additional vectors inputted into the blocks contain style indications, allowing for precise control over generated faces.



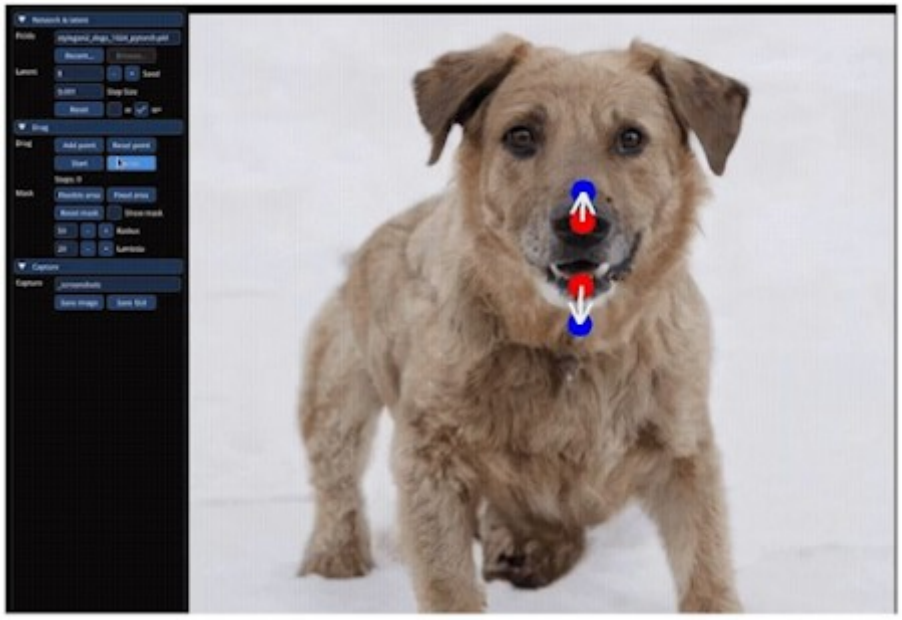


- **Multimodal GANs** take input from various data types, such as text and audio, to generate images. By mapping this information into a vector and feeding it into both the

generator and discriminator, it's possible to achieve this complex task.



- **Drag GAN** attaches the latent representation of the GAN to specific parts of the resulting image. An interplay between the latent representation and keypoint tracking algorithms allows for precise generation, such as generating the mouth of a dog.



Adversarial frameworks like GANs have also been used for deep fake generation methods, including **video generation**. In this task, a neural network must extract keypoint representations describing the movement of a person from a video and apply this representation to static images, assuming the existence of a network capable of disentangling appearance from motion. This task took time to perform effectively.



Diffusion Models

Let’s finish discussing the family of methods used for generating samples of structured data (like images or videos) by talking about diffusion models. We will see only examples about images, but they aren’t restricted to that.

Very popular models like DALL-E, Midjourney, Imagen, and Stable Diffusion are diffusion models.

Let’s revisit the history of generative models:

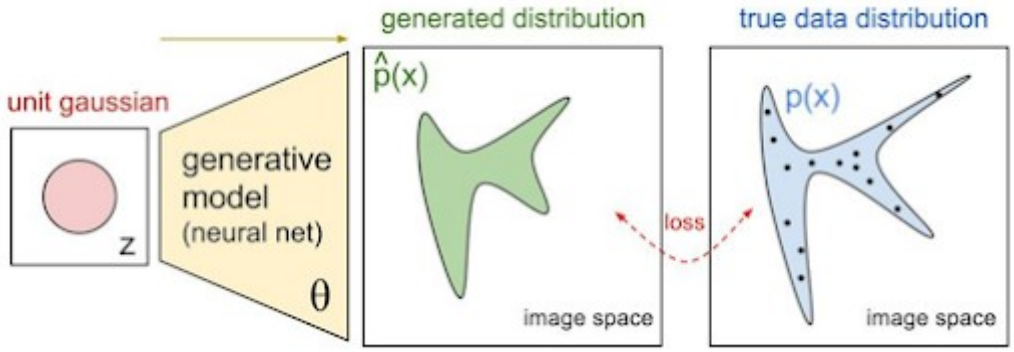


The Variational Autoencoder (VAE) was introduced in 2013, marking a significant development in generative models. A year later, in 2014, Generative Adversarial Networks (GANs) were invented. By 2015, the Deep Convolutional GAN (DCGAN) was developed, which extended GANs to the domain of images and became famous for generating highly realistic bedroom images (those we’ve seen before). These impressive results led many to believe that GANs were the best generative models available, creating a bias in the research community that fostered a continued focus on GANs.

However, in the same year, 2015, the paper titled "Deep Unsupervised Learning using Nonequilibrium Thermodynamics" was published. This work was more oriented towards physics than traditional artificial intelligence and was initially overlooked due to its complex mathematical framework and the challenging nature of its application.

Despite the initial neglect, this paper laid the foundation for diffusion models. These models were invented in 2015, but their practical use took several years to develop. The complexity of the mathematical formulation led to optimization challenges that made the training of these networks difficult. Over time, researchers managed to simplify these challenges, leading to a resurgence in interest and the eventual adoption of diffusion models as prominent tools for data generation.

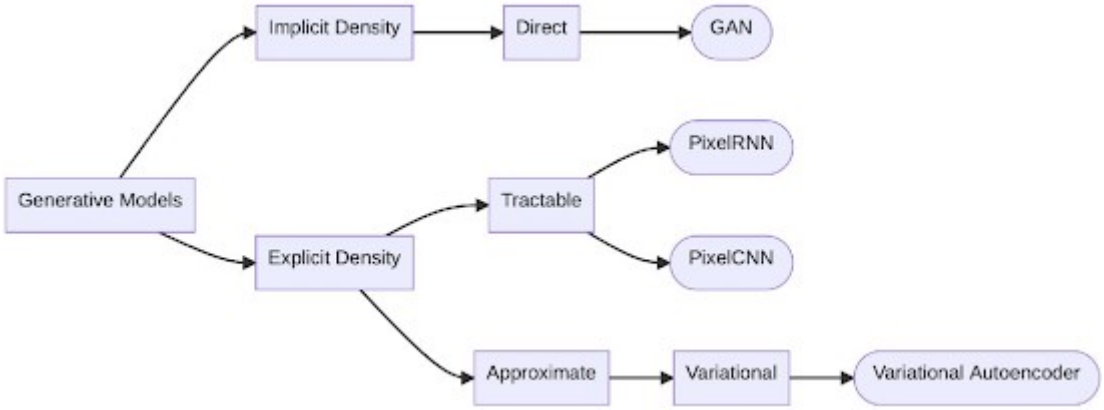
Another way to understand what a generative model is is as follows:



The primary objective is to train a generator that maps from the latent space to the data space, thereby approximating the distribution of the real data. However, the training process can be challenging due to several factors:

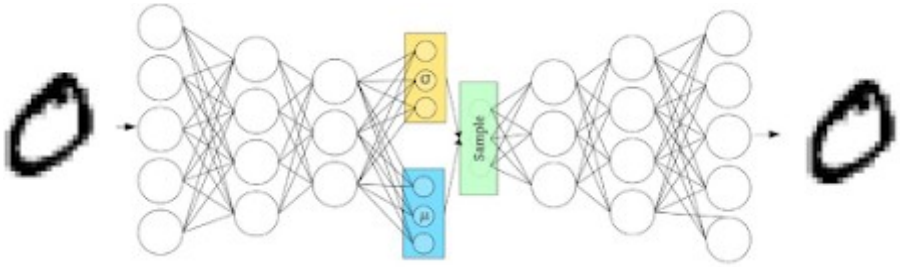
- The selection of hyperparameters
- The quantification of similarity between sets (loss)
- The choice of latent space

A summary of what we've seen is as follows:

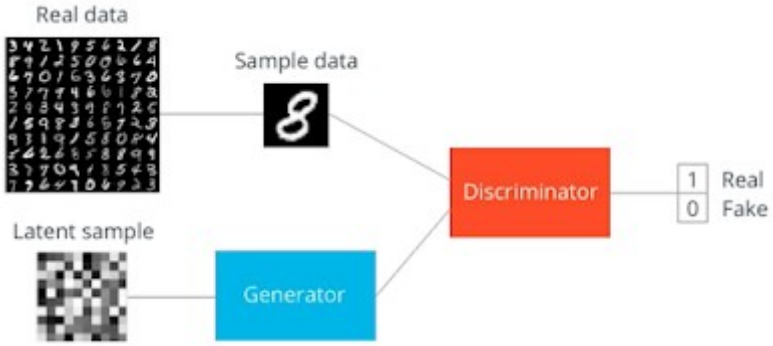


Variational Autoencoders add the ability to generate new samples to autoencoders. There is a probabilistic latent space (assumed to be multivariate Gaussian), and it involves minimizing both

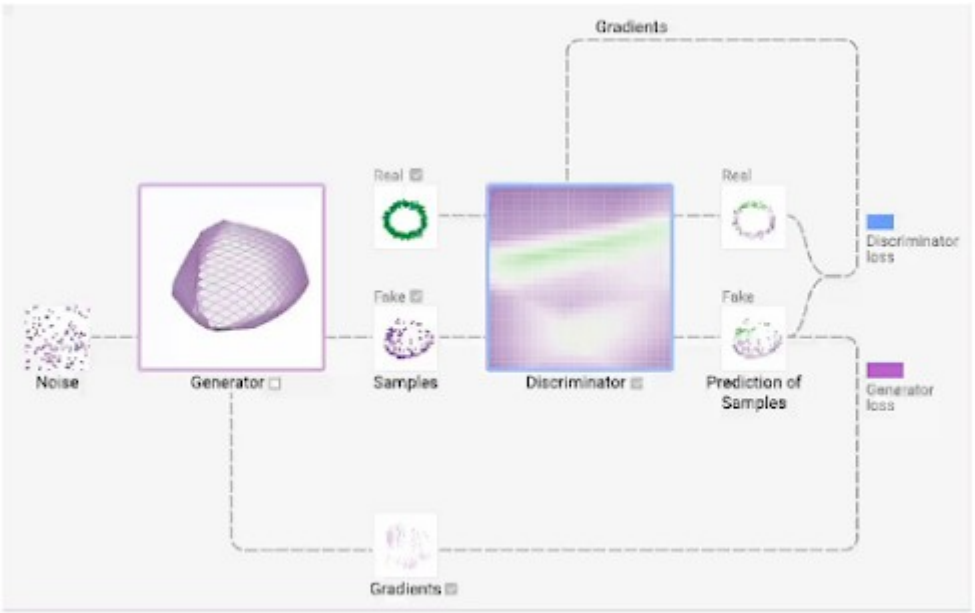
the reconstruction error and the KL divergence between the latent space and $N(0,1)$ (the regularizer). Sampling from the latent distribution allows for generation and interpolation.



Generative Adversarial Networks have a Generator that starts from Gaussian noise and generates a data point to fool the Discriminator, and a Discriminator that tries to distinguish between real and fake data. Training becomes a game between the generator and discriminator, and the solution lies in the Nash equilibrium between the two participants.



A fancy visualization of a GAN is the following:



The results of GANs, as presented in the very first paper on GANs in 2014 by Goodfellow, are as follows:



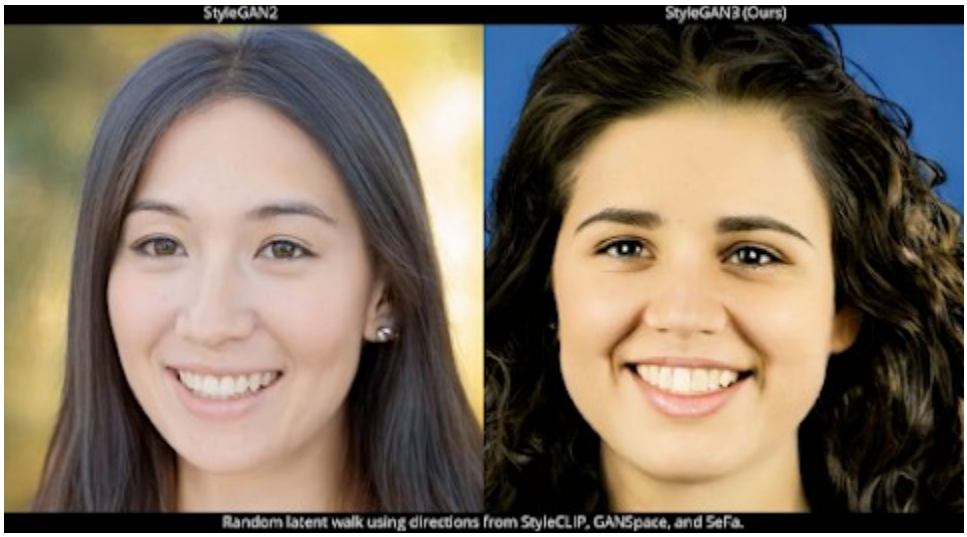
MNIST, Toronto Face Dataset, FC CIFAR-10, Con CIFAR-10 results

The yellow-boxed cells are samples from the real dataset that are most similar to the generated ones. This is satisfactory for the digits, but it's not satisfactory for natural images if they are relatively low resolution because the distribution is much more complex to fit. This was the first model that allowed us to generate structured data, making it a revolutionary development.

Huge improvements were made thanks to several tricks, especially in the StyleGAN family. Smooth walking in the latent space yields meaningful changes in the output. These are the results from 2021:



Over the years, the quality of detail generation with StyleGAN improved significantly.

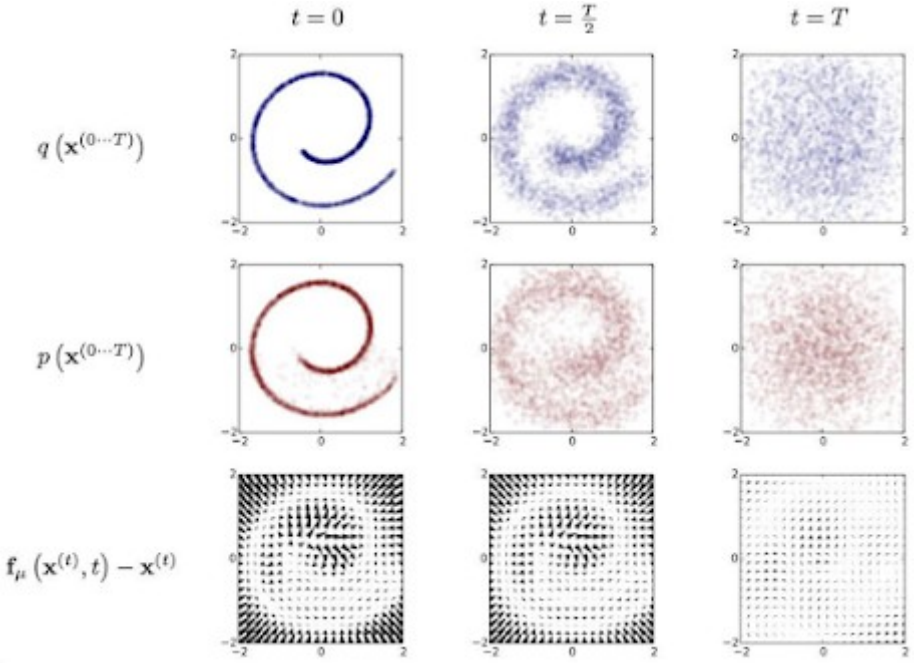






Let’s talk about the paper “Deep Unsupervised Learning using Nonequilibrium Thermodynamics.” The reason why this paper didn’t gain traction initially was this picture, which

has now become extremely popular. At the time, people didn't understand that this mathematical object was actually very powerful.



The meaning of this picture is that one starts from some data distribution and then progressively applies noise to the data, ending up with Gaussian noise. The diffusion model should be able to reverse this process, learning to go from this Gaussian noise back to the initial data distribution using a neural network.

If we compare the images in the first diffusion model paper with the images in the first GAN paper (just one year earlier), we can observe that the images generated by the diffusion model are actually better than those by the GAN.



However, despite this, diffusion models did not initially receive much attention.

After people “unlocked” these models, they became really strong for generating high-quality images, leading to amazing results. Here’s an example of a Denoising Diffusion Probabilistic Model results.

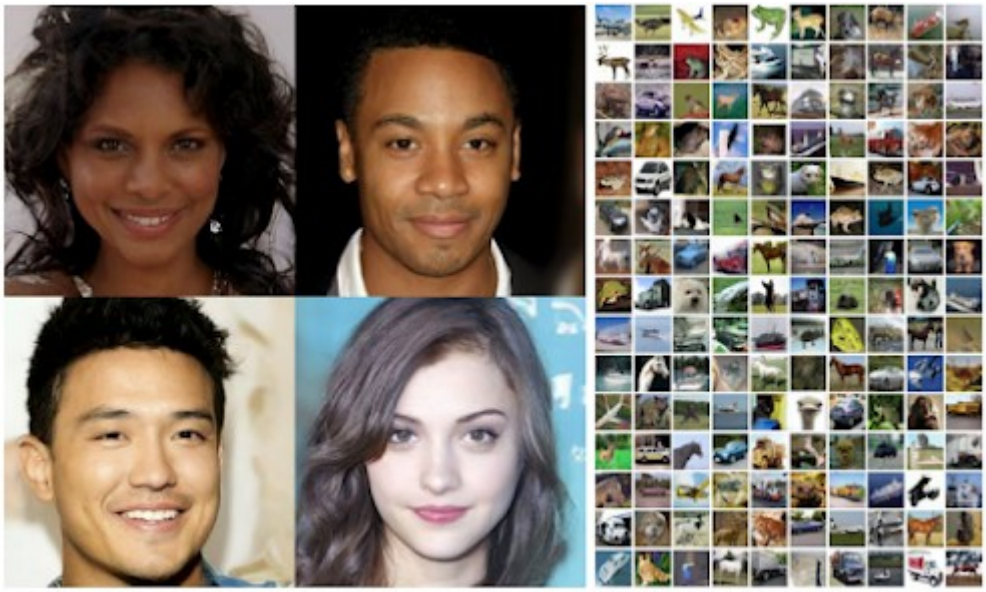
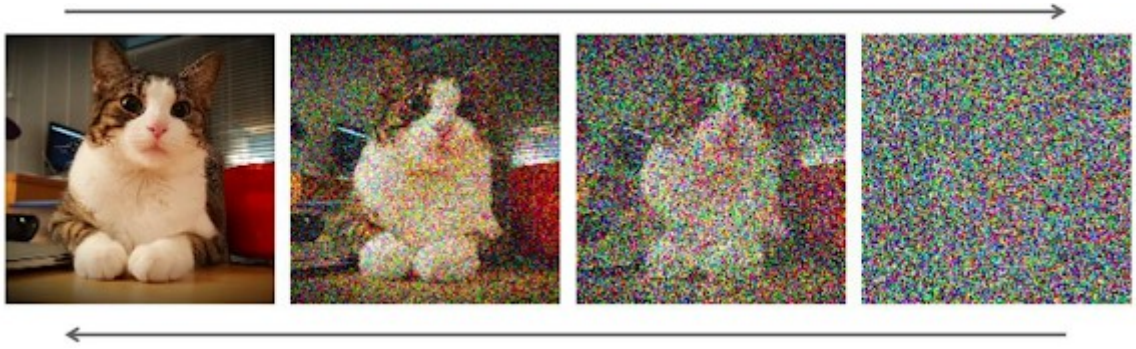


Figure 1: Generated samples on CelebA-HQ 256 × 256 (left) and unconditional CIFAR10 (right)

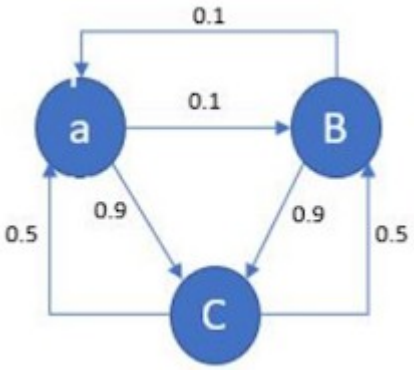
Theory

Let’s now delve into the theory of diffusion models. The idea behind diffusion models is actually very simple. Denoising diffusion models consist of two processes:

- **Forward process to add noise**: This process, in small steps (the fact that the steps are small is important), will add noise to the data.
- **Reverse process to denoise and generate data**: This process, where the learning takes place, denoises the data and generates the data back.



To understand the high-level mathematics behind diffusion models, we need to know what a **Markov Chain** is. A Markov chain or Markov process is a stochastic/probabilistic model describing a sequence of possible discrete events in which the probability of each event (the movement from one state to another) depends only on the state attained in the previous event.



$$P = \begin{bmatrix} 0 & 0.1 & 0.9 \\ 0.1 & 0.0 & 0.9 \\ 0.5 & 0.5 & 0.0 \end{bmatrix}$$

In this case, the probability of staying in the same state is 0.

Essentially, there is a mathematical description of the probability that connects an event with the previous event. So, you have first-order dependencies between what is happening now and what happened in the previous step.

Diffusion models use a Markov Chain to model all the steps that lead the data in the forward process.

The diffusion model assumes that at every step, the process from one state to the following can be described as a Gaussian probability distribution (with its mean and its standard deviation):

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}x_{t-1}, \beta_t \mathbf{I})$$

The mean is centered around the previous step, and the standard deviation, for convenience, is taken as a number (called the noise schedule) multiplied by an Identity Matrix.

In this way, it is possible to define the probability distribution of multiple states given the initial state, by taking the product of the Gaussian probabilities:

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}x_{t-1}, \beta_t \mathbf{I}) \implies q(x_{1:T} | x_0) = \prod_{t=1}^{T} q(x_t | x_{t-1})$$

We can rewrite it with a change of variables in the following way:

$$\alpha_t = (1-\beta_t) \implies \bar{\alpha}_t = \prod_{s=1}^{t} (1-\beta_s) \implies q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I})$$

This allows us to immediately compute the probability associated with a certain step in the sequence from a Gaussian, instead of multiplying multiple Gaussians.

We can then sample directly at the desired timestep t:

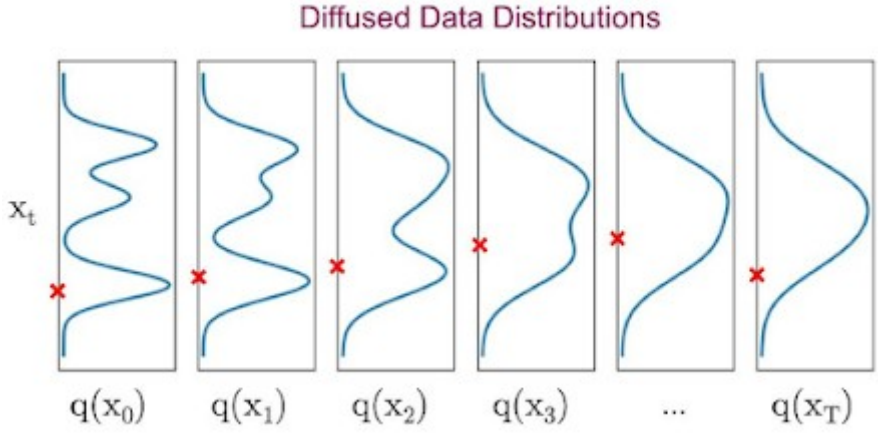
$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{(1-\bar{\alpha}_t)}\epsilon, \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

So the forward process can be simply described by this equation, which gives x_t as a consequence of x_0 given $\underline{a_t}$ by this equation:

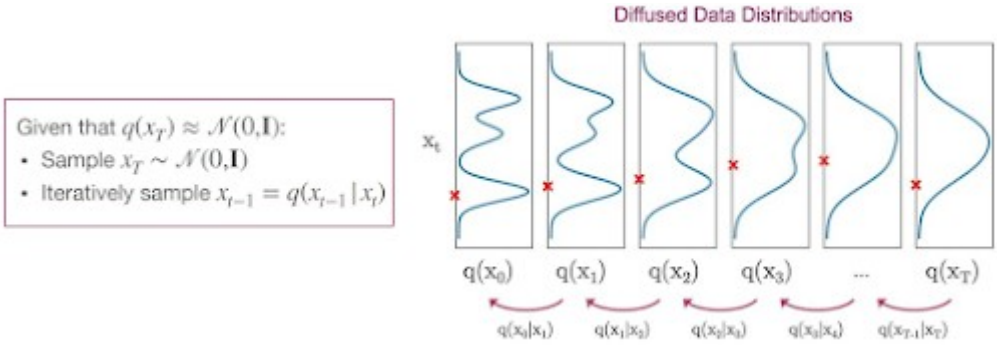
$$\bar{\alpha}_t = \prod_{s=1}^{t} (1 - \beta_s)$$

where β_t is technically called the noise schedule and it is a hyperparameter. It is called a schedule because it changes over time.

In the forward process, at each step, Gaussians are applied. Formally, we are applying a **Gaussian convolution** to the data at each timestep. Practically, we are **smoothing** out the distribution to a Gaussian one.

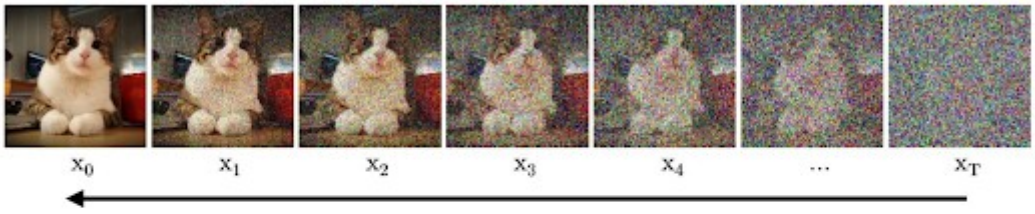


The process to which we pay attention is the reverse process: the generation.



In the generation process, from a Gaussian, we go back to the initial data distribution. This is a bit tricky because this means that we need the probability $q\left(x_{t-1} \mid x_t\right)$, which can be rewritten by Bayes' rule as $q\left(x_{t-1}\right) q\left(x_t \mid x_{t-1}\right)$. This is generally **intractable**, but the original diffusion models paper states that we can **approximate** it with another Gaussian.

So, like VAE and GAN, we invoke the power of neural networks to approximate some Gaussian distribution which describes this reverse process.



$$p(x_T) \sim \mathcal{N}(0, \mathbf{I})$$

$$p_\theta(x_{t-1} \mid x_t) \sim \mathcal{N}\left(\mu_\theta(x_t, t),\ \sigma_t^2 \mathbf{I}\right)$$

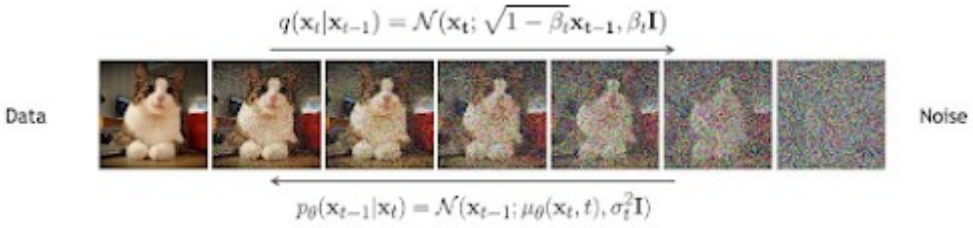
Trainable network

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t)$$

A neural network will be trained to describe the mean of the new Gaussian distributions, while the standard deviation is simply chosen (for the sake of simplicity).

To train this neural network, we need to find a suitable objective function. This was the challenge that took some time to simplify because it is, unfortunately, a complex problem.

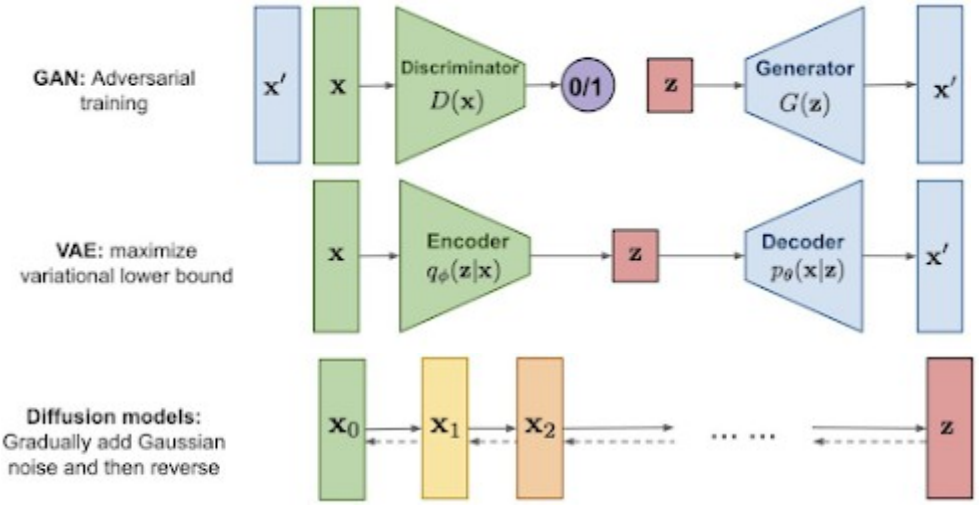
Let's recap. Without delving into the mathematical formulation, a diffusion model can be thought of as a mechanism with a forward process, described by a Gaussian distribution, and a reverse process also described by some Gaussian distribution, which you control through a neural network.



We can control the variance of the forward diffusion and reverse denoising processes, respectively. Often, a linear schedule is used for β_t, and σ_t^2 is set equal to β_t.
Kingma et al. [NeurIPS 2022] introduce a new parameterization of diffusion models using signal-to-noise ratio (SNR), and show how to learn the noise schedule by minimizing the variance of the training objective. Other improvements include: Improved DPM by Nichol and Dhariwal [ICML 2021] and Analytic-DPM by Bao et al. [ICLR 2022].

Overall, the process can be modified with minimal changes, and people have realized that the noise can be treated in a certain way, the schedule can be set in a certain way, and the distribution can be slightly altered, among other things.

Let's address some other important aspects before discussing the objective function of diffusion models. Let's compare VAEs, GANs, and Diffusion Models.



In GANs, the latent space generates samples that the generator uses to create data, which the discriminator then compares to the original data.

In VAEs, you squeeze the data into a latent space with probabilistic guarantees (ensured by the Kullback–Leibler term), and the decoder uses this to produce generated data.

In diffusion models, there isn't a bottleneck caused by the dimensionality of the latent space since latent variables have the **same dimensionality** as the data. In fact, the **same model** is applied across different timesteps. This has significant implications for the type of neural network used, even if it doesn't greatly impact the optimization process.

We're still interested in comparing two distributions. The model is trained by **reweighting** the variational bound. We can train the model similarly to a VAE, using a variational upper bound (to eliminate some unfriendly terms):

$$L = \mathbb{E}_{q(x_0)}\left[-\log p_\theta(x_0)\right] \leq \mathbb{E}_{q(x_0)q(x_{1:T}|x_0)}\left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right]$$

This can be divided into three terms:

$$L = \mathbb{E}_q\left[D_{KL}(q(x_T|x_0)||p(x_T)) + \sum_{t>1} D_{KL}(q(x_{t-1}|x_t,x_0)||p_\theta(x_{t-1}|x_t)) - \log p_\theta(x_0|x_1)\right]$$

So, we have the last term, which looks like the reconstruction term, and multiple Kullback-Leibler divergences:

- One that compares the final distribution obtained from the starting point with the real distribution.
- Multiple ones that compare the distribution p with the associated q at each step of the forward-backward process.

This objective can be optimized to generate images of higher quality than GANs, but it is rarely used in practice because it implies some optimization difficulties.

The KL divergence between Gaussians has a nice closed form, but Ho (with some math) proves the training can be simplified to a noise prediction problem. Skipping some math steps, we obtain a new loss:

$$L_{simple} = \mathbb{E}_{x_0 \sim q(x_0), \epsilon \sim \mathcal{N}(0,\mathbf{I}), t \sim \mathcal{U}(1,T)} \left[|| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{(1-\bar{\alpha}_t)}\epsilon, t) ||^2 \right]$$

So, the problem of matching two distributions can be cast as a problem of having a target noise and trying to have a neural network that predicts this target noise.
In the end, this is a simpler optimization problem where you sample from the distribution q, sample the noise, and sample the steps. The complication of diffusion models is that we don't have a single timestep but multiple timesteps.

The training and the sampling are as follows:

Algorithm 1 Training

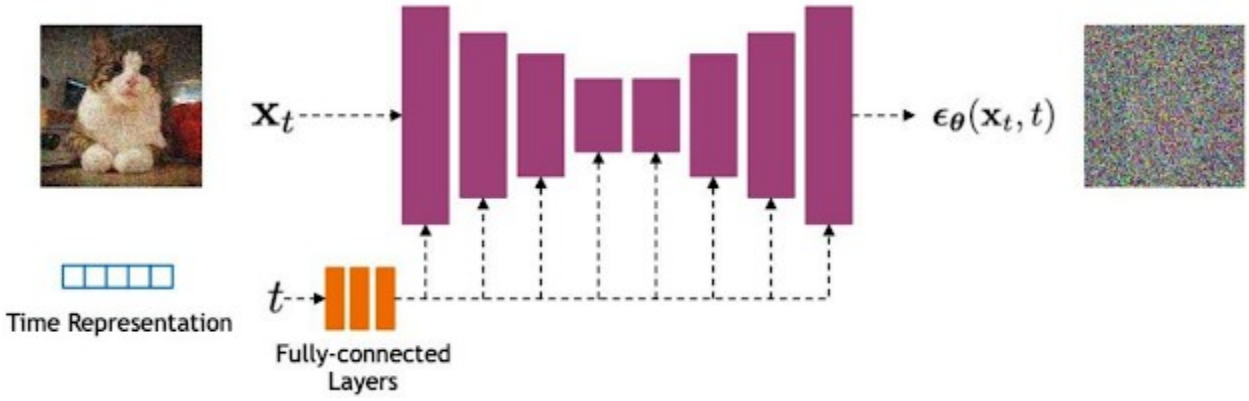
1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$
5: Take gradient descent step on
$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

So, the training is very simple, but the sampling is much more complex than GANs or VAEs because you have to deal with all the different timesteps, which can take some time. The higher the number of timesteps, the better the quality of the results, so we can't have only a few timesteps to make the process faster if we want a good result.

We've mentioned that the neural network predicts the noise, which depends on the input and the time:

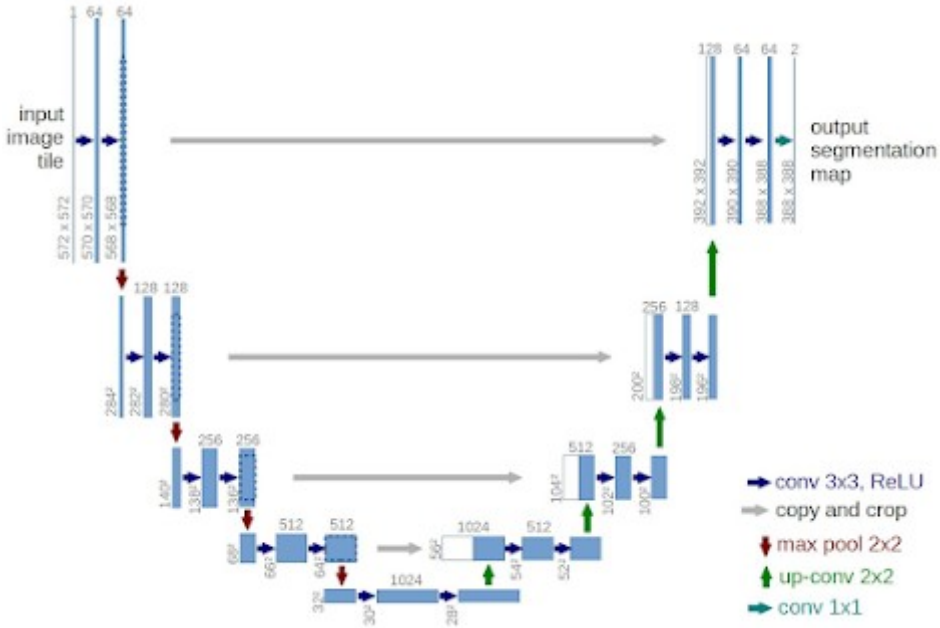


This is straightforward, but unfortunately, you need to make some technical choices to build the neural network. There is a lot of freedom in the network architecture, with no theoretical constraints on this. For images, a **U-Net** (with attention) is often used, as represented in the image above and similar to a VAE (it compresses the representation and then brings it back to the original resolution).

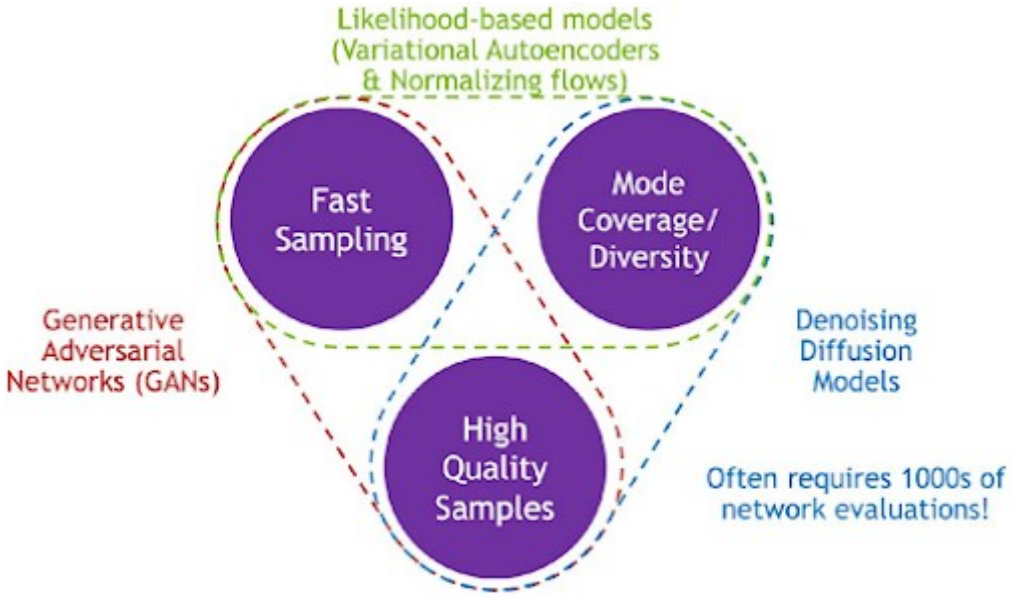
This network has to be fed with time, but it is known that it can't be fed as "1, 2, 3, 4, 5, …" because it will break (because it lacks cyclic representation, causing discontinuity issues). So, time features are usually **sinusoidal** or random **Fourier** features. How to embed them in the network is another free choice (e.g., spatial concatenation, AdaIN, etc.). The short vector of time will be fed into the network and used as extra information.

There is a lot of freedom in the design of the network, the encoding of time, and the schedule, approximating ϵ in different ways.

Let's explore what a **U-Net** is.
The U-Net architecture consists of two paths. The first path is the contraction path (also called the encoder), which is used to capture the context in the image. The encoder is a traditional stack of convolutional and max pooling layers. The second path is the symmetric expanding path (also called the decoder), which is used to enable precise localization using transposed convolutions. It is an end-to-end fully convolutional network (FCN), meaning it only contains convolutional layers and does not contain any dense layers, allowing it to accept images of any size.



In 2022, NVIDIA released a paper comparing GANs, VAEs, and diffusion models:



Generative models are typically evaluated based on the following criteria:

- **Fast Sampling**: This refers to the model's ability to quickly generate new data points. In other words, how efficiently the model can produce samples from the learned distribution.
- **Mode Coverage/Diversity**: This measures the extent to which the model captures the variety in the data. A good generative model should be able to represent all modes (peaks in the probability density function) of the target distribution.
- **High-Quality Samples**: This pertains to the quality of the generated samples. The samples produced by the model should be indistinguishable from the real data, and they should not contain any noticeable artifacts or inconsistencies.

Each model excels in two of these aspects but not in the third.

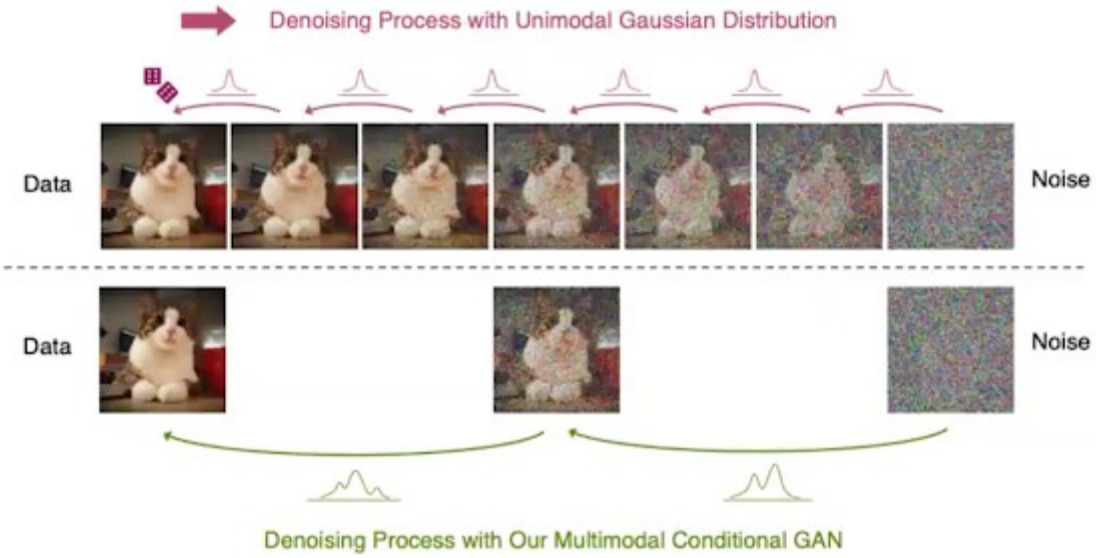
- **GAN**: Good at fast sampling and produces high-quality samples due to the interplay between the generator and the discriminator. However, GANs suffer from mode collapse and do not cover the space well.
- **VAE**: Fast in sampling because the inference phase is just passing through a neural network, and thanks to the Kullback-Leibler divergence, it provides good mode coverage of the data. However, the quality of the data isn't high.
- **Diffusion models**: Good at mode coverage (because they are trained on the variational lower bound similar to VAEs) and produce high-quality samples. However, this comes at the price of requiring several steps.

Thus, we are still searching for the optimal generative model, and researchers are actively trying to make diffusion models achieve fast sampling.

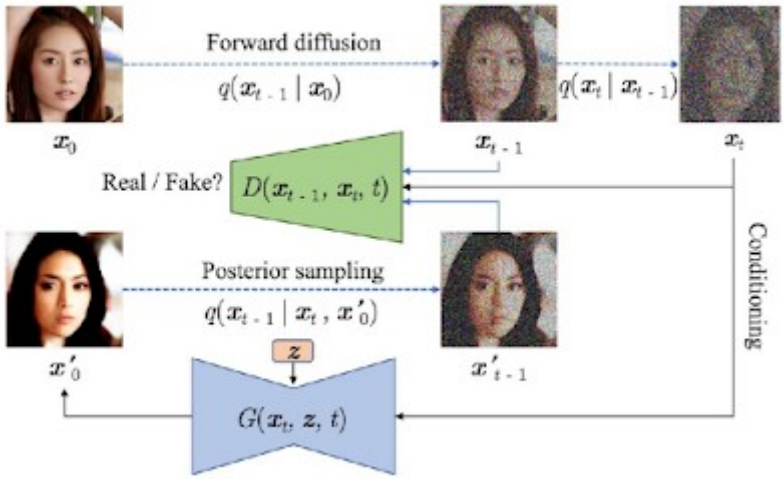
Advanced Tricks

Let’s now explore some methods that make diffusion models the strongest models we have today.

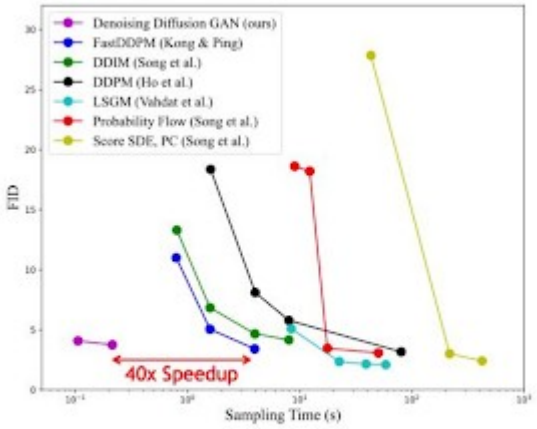
The fact that diffusion models require many steps can be bypassed. NVIDIA noted that the problem arises because, during the backward process, we assume that the denoising distribution can be modeled by a Gaussian distribution. This assumption holds only for small denoising steps, which in practice translates to thousands of denoising steps in the synthesis process.
To address this, they created **diffusion GANs**. In diffusion GANs, the denoising model is represented using multimodal and complex conditional GANs, enabling efficient data generation in a few steps.



Instead of having numerous steps, diffusion GANs reduce the number of steps and include a discriminator to ensure the generated complex distributions are of good quality. Compared to a one-shot GAN generator, both the generator and discriminator in diffusion GANs solve a much **simpler** problem, resulting in stronger **mode coverage** and better training **stability**.



NVIDIA demonstrated that diffusion GANs achieve a massive speedup compared to classical diffusion models while maintaining similar performance in terms of quality.

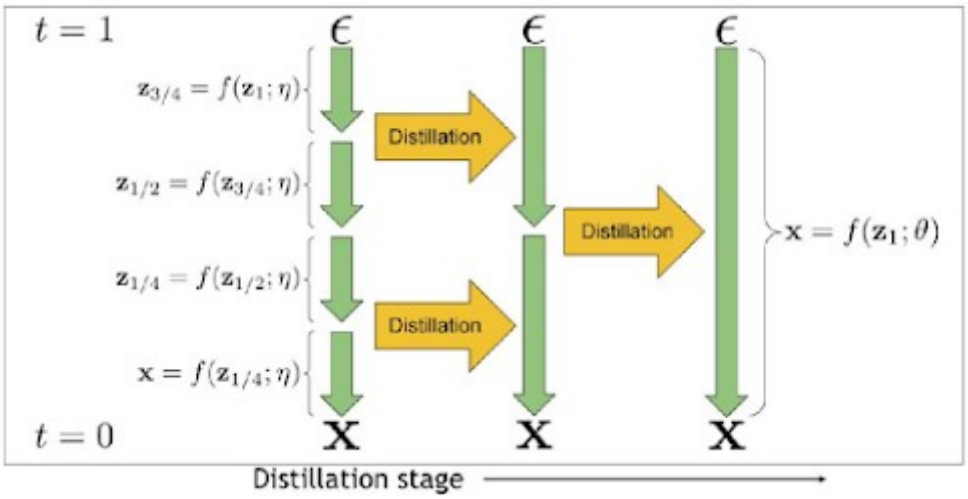


Sample Quality vs. Sampling Time



CIFAR-10 Samples

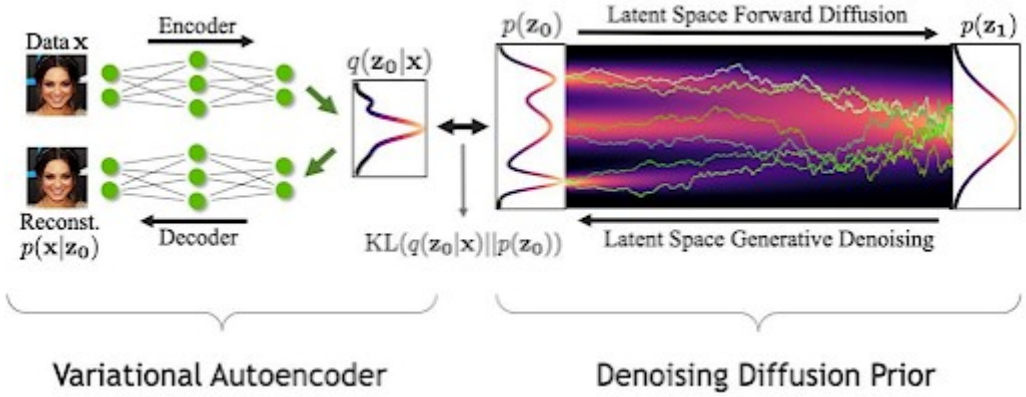
Another method is called **distillation** of diffusion models. This involves distilling a deterministic DDIM (Diffusion Denoising Implicit Models) sampler into the same model architecture. At each stage, a “student” model is trained to **distill two adjacent sampling steps** of the “teacher” model into one sampling step. In the next stage, the “student” model from the previous stage becomes the **new “teacher”** model.



By training diffusion models and learning the functions that map from noise back to the original space, other neural networks can be trained to perform larger computation steps. This approach allows for a trade-off between inference speed and result quality, as larger steps might produce slightly degraded results compared to the original diffusion model. Thus, for each step, a new model is trained that is faster but less precise, and users can choose how much time they wish to spend on inference.

Other research aims to have diffusion models operate in a latent space rather than directly on the pixel space. This involves embedding images with an encoder into another space and using a decoder to revert to the original input. The distribution of **latent embeddings** is close to a

normal distribution, resulting in simpler denoising and faster synthesis. This **augmented** latent space can be **tailored** to specific data types, such as graphs, text, or 3D data.

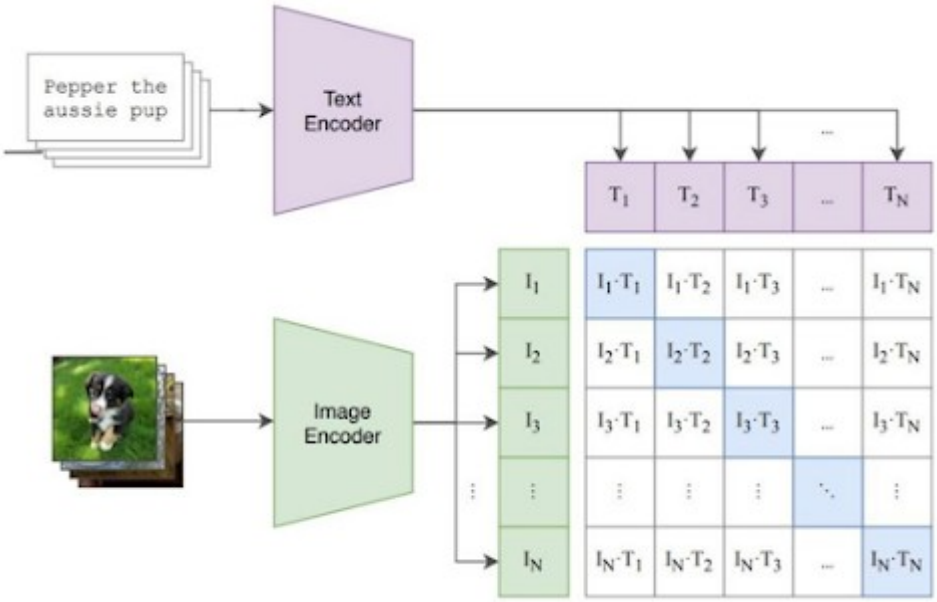


This approach simplifies the work of the diffusion model and speeds up inference.

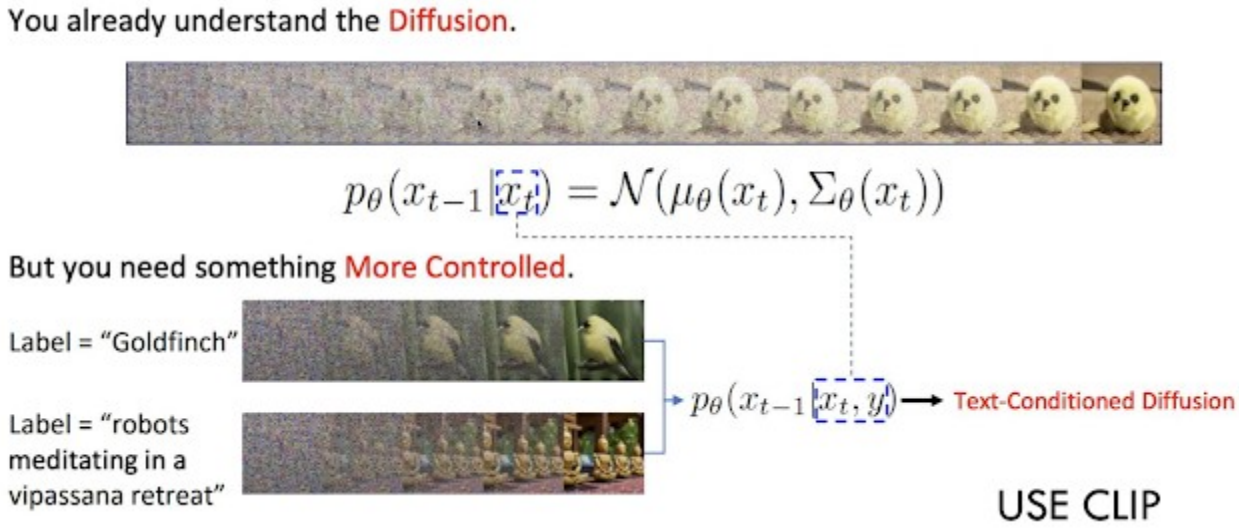
Now, diffusion models can generate images from text, known as **text-to-image generation**.



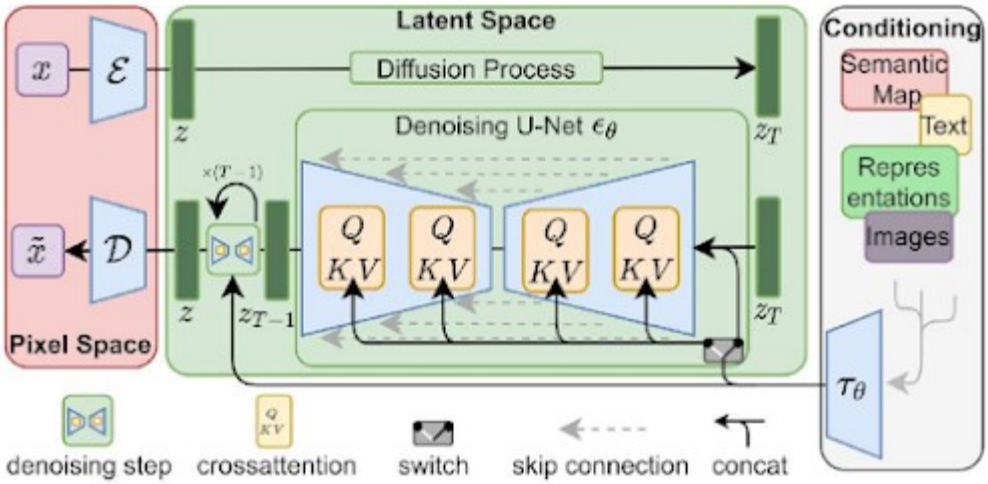
We cannot fully understand the underlying mechanisms (as we haven't covered enough in this course), so let's discuss it at a high level.
Instead of solving the problem of image classification with AlexNet, more complex multimodal networks, like Contrastive Language-Image Pretraining (CLIP), are used. CLIP jointly trains a **text** encoder and an **image** encoder, producing vector representations for both. It trains by maximizing the **similarity** between embeddings of (text, image) pairs using a contrastive loss, which ensures similar representations for an image and its associated text, and very dissimilar ones for images and unrelated text. The resulting space captures the **semantics** of both images and text.



The idea behind GLIDE (Generative Latent Image Disentangling) is to "control" the diffusion process by adding text, making it a text-conditioned diffusion model. Text is converted into vectors using CLIP.



This mechanism is the basis for Stable Diffusion, a text-to-image model. It uses CLIP embeddings as conditioning on **latent diffusion** via cross-attention and is trained on a subset of LAION-5B (the original dataset contains **5 billion** text-image pairs). **Fast sampling** is achieved by performing diffusion in the latent space.



The denoising neural network in this model processes text embeddings. The cross-attention blocks model the relationships between data of different modalities. It is fast in sampling because it doesn't operate directly on the pixel space but in the latent space. This approach is effective because the transformer (the blue part) can model the dependencies among modalities.

This approach allows for various tasks, such as generating consistent images from segmentation masks, upscaling images, and filling in missing parts.

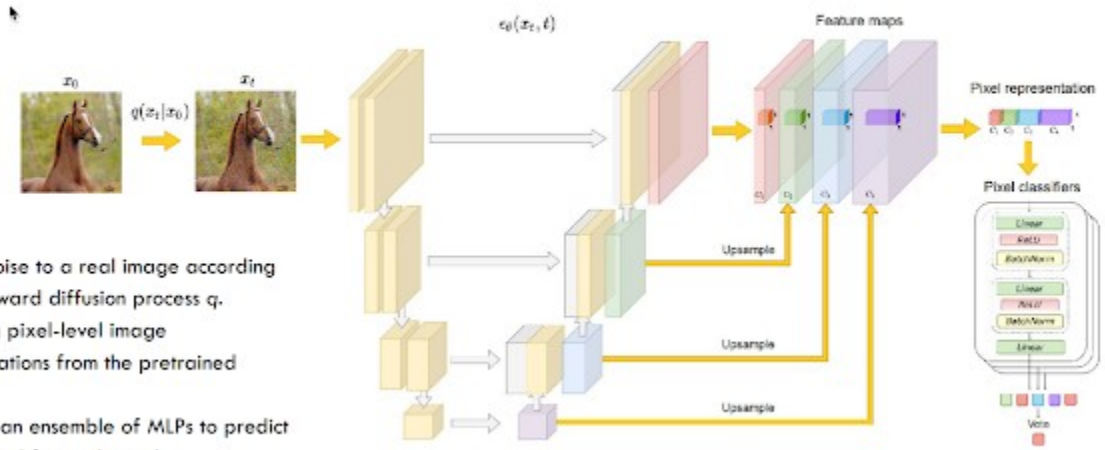
Super Resolution



Image-to-Image



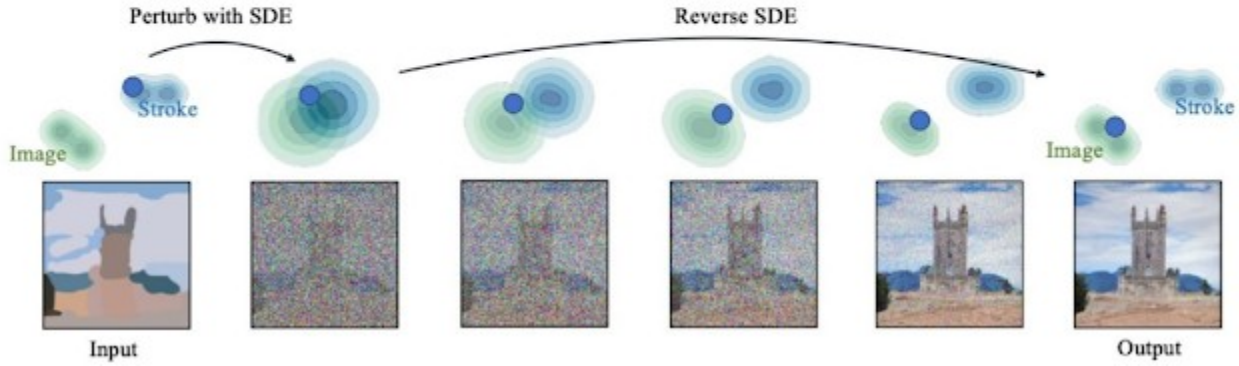
Semantic Segmentation



- Adding noise to a real image according to the forward diffusion process q.
- Extracting pixel-level image representations from the pretrained DDPM.
- Applying an ensemble of MLPs to predict a class label for each pixel representation.

Image Editing

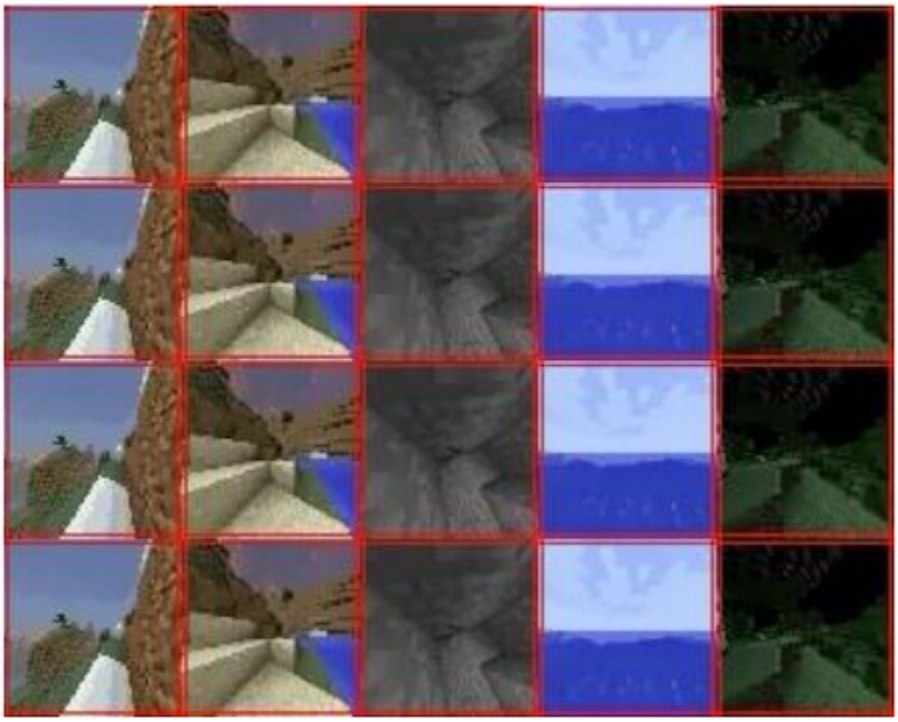
Given a stroke painting, we first perturb it with Gaussian noise and progressively remove the noise by simulating the reverse SDE. This process gradually projects an unrealistic stroke painting to the manifold of natural images.



Video Generation



Video Generation



3D Shape Generation



Here is a simple implementation of a diffusion model:

MINIMAL (SEMI) WORKING CODE



```
class Diffusion:
    def __init__(self):
        betas = linspace(sigma_min, sigma_max, num_timesteps)
        alphas = 1.0 - betas
        alphas_cumprod = cumprod(alphas)
        alphas_cumprod_prev = append(1.0, alphas_cumprod[:-1])
        posterior_variance = betas * (1.0 - alphas_cumprod_prev) / (1.0 - alphas_cumprod)

    def loss(self, x):
        e = randn_like(x)
        t = randint(0, num_timesteps, n)

        selected_alphas = alphas[t]
        x = x * selected_alphas.sqrt() + e * (1 - selected_alphas).sqrt()
        output = model(x, t)
        loss = mse_loss(output, e)

        return loss

    def sample(self, shape):
        x = randn(shape)

        for t in reverse(timesteps):
            predicted_noise = model(x, t)

            weighted_score = betas[t] / (1 - alphas_cumprod[t]).sqrt()
            mean = (1 / alphas[t].sqrt()) * (x - weighted_score * predicted_noise)
            logvar = posterior_variance[t].log()  # not entirely true

            noise = randn_like(x)
            x = mean + (0.5 * logvar).exp() * noise

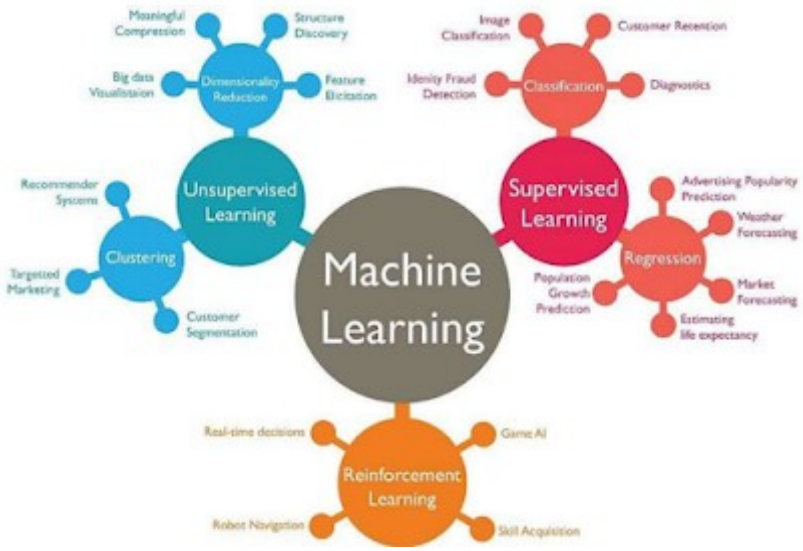
        return x
```

Reinforcement Learning

Reinforcement Learning is the last area of Machine Learning that we have yet to cover. Unlike supervised and unsupervised learning, there is no offline procedure to learn a model in reinforcement learning. Instead, it involves an interaction between an agent and an environment. In this chapter, we will discuss the simplest techniques to address this area.

Reinforcement Learning Idea

So far, we have discussed supervised and unsupervised learning. In this chapter, we will explore Reinforcement Learning (RL).



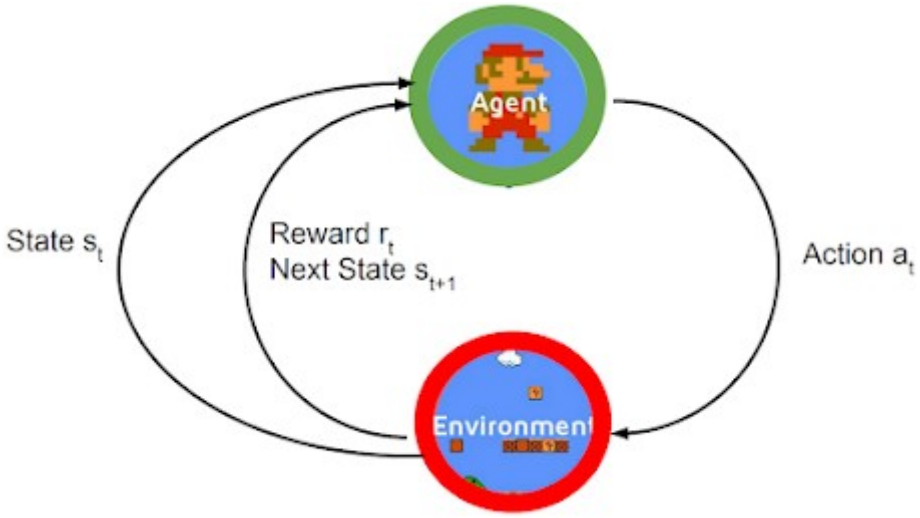
Recently, Reinforcement Learning has gained significant popularity, primarily for two reasons: the success of AlphaGo (an agent that defeated the world champion of the game Go) and the development of ChatGPT (which leverages reinforcement learning with human feedback as one of its core components).

Reinforcement Learning is an area of machine learning inspired by research in psychology and the way animals learn. The problems in RL involve an agent interacting with an environment, which provides numeric reward signals. The goal is to teach the agent how to take actions that maximize its rewards.



In reinforcement learning, we have two key players: the agent and the environment. The agent takes actions that affect the state of the environment and receives rewards based on the state.

Reinforcement learning operates in an episodic manner, introducing the concept of time.
Let's define reinforcement learning more formally using the concept of a policy. A **policy** is a mapping from states to actions, and the goal is to learn a policy that maximizes the expected reward over time.



At time t, the agent performs an action a_t. This action affects the state of the environment, transitioning it from state s_t to state s_{t+1}. The agent also receives a reward r_t from the environment.

Let's explore some practical examples of where Reinforcement Learning is applied.

In Atari games:

- **Objective**: Achieve the highest score.
- **State**: Raw pixel inputs representing the game state, i.e., the values of all the pixels on the screen, which indicate the current situation of the game.
- **Action**: Game controls such as Left, Right, Up, Down.
- **Reward**: Score changes at each time step.



The state being the value of all pixels on the screen explains why reinforcement learning wasn't used for this task until 2015. The state space is large and complex, requiring sophisticated reinforcement learning techniques, such as deep reinforcement learning.

In the game of Go:

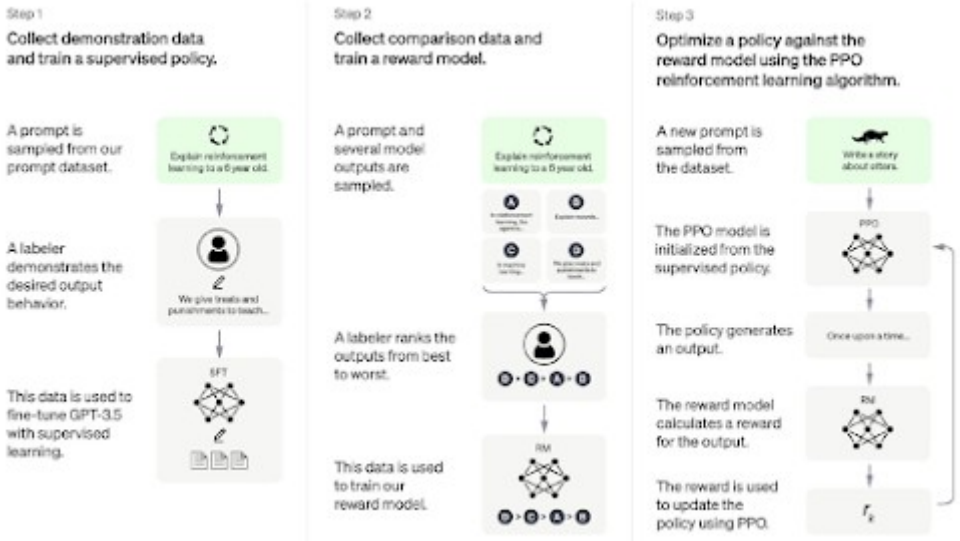
- **Objective**: Win the game.
- **State**: The positions of all pieces on the board.
- **Action**: Deciding where to place the next piece.
- **Reward**: 1 if the agent wins the game, 0 otherwise (unlike Atari games, where rewards are given at every step).





There were already agents that can play chess against humans, but developing an agent for Go was much more challenging due to the vast number of possible board configurations. Google DeepMind's AlphaGo was groundbreaking in this regard, as it was able to defeat a world champion human player.

In the case of ChatGPT, reinforcement learning is used in a more complex manner. This process helps ChatGPT to incorporate feedback from users, allowing it to generate text in a more natural and coherent way.



Markov Decision Process

Let’s delve into a very simple example.

We have already discussed what a Markov model is: a model where the probability of being in a particular state depends only on the previous state.
In a Markov Model, there is a first-order interaction between the state at time t and the state at time $t-1$.

A **Markov Decision Process** (MDP) is a framework used for decision-making in a stochastic environment, where everything is represented in a probabilistic framework. The goal is to find a

policy, which is a mapping that specifies the optimal action for each state in the environment. To solve MDPs and efficiently compute the policy, we use **Dynamic Programming** (DP), particularly the Bellman equation. DP is a method that divides a problem into simpler sub-problems that are easier to solve.

The components of reinforcement learning according to the Markov Decision Process are:

- **States** s, beginning with an initial state s_0.
- **Actions** a.
- **Transition Model** $P(s' \mid s, a)$: A probabilistic model that specifies the probability of transitioning to state s' given the current state s and action a.
  Markov Assumption: The probability of transitioning to state s' from s depends only on s and not on any past states or actions.
- **Reward Function** $r(s)$: Specifies the reward received in each state.

We aim to learn the policy $\pi(s)$, which is the action that an agent takes in any given state s.

An MDP is defined by a tuple of five elements that describe everything needed to model a reinforcement learning problem:

$$(S, A, R, P, \gamma)$$

where:

- S: A set of possible states.
- A: A set of possible actions.
- R: A distribution of rewards given a $(state, action)$ pair.
- P: Transition probabilities, i.e., the distribution over the next state given a $(state, action)$ pair.
- γ: The discount factor, which indicates how much future rewards are valued compared to immediate rewards.

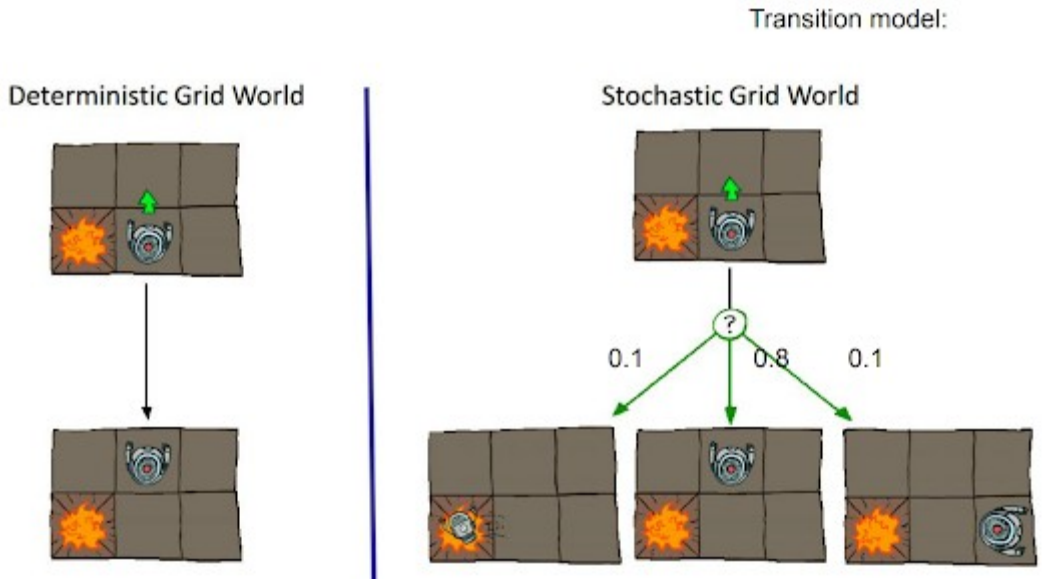
To better understand these concepts, let's consider an example: Grid World.



The objective is to reach the diamond terminal state in the least number of actions.

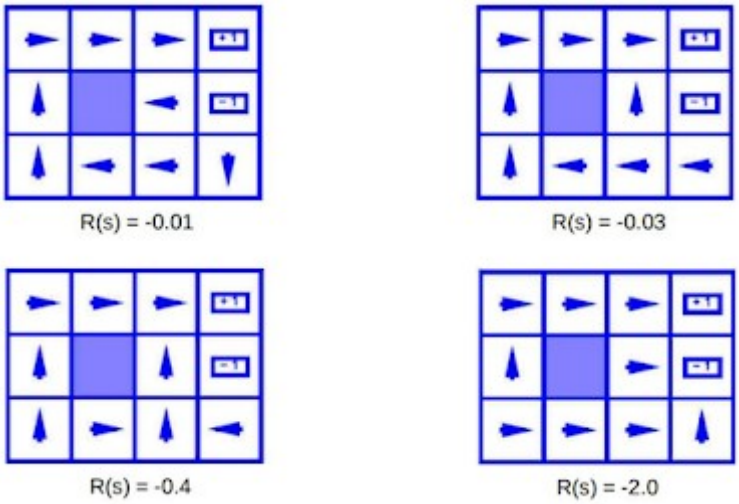
Imagine a robot trying to reach the diamond's position. The robot has three possible actions (left, up, right), and the state is the configuration of the board.
Whether the robot reaches the diamond depends on the reward, which varies by state. In this example, the reward is -0.04 for every non-terminal state, while the diamond's position has a very positive reward $+1$ and the volcano has a reward of -1.

A deterministic approach would imply we are 100% certain of the robot's movement, but here, we use a transition model that provides probabilities for moving in a particular direction from a given state.



The goal is to find the optimal policy. The policy is a map that specifies the optimal action for every state. The optimal policy is one that maximizes the expected reward.

The optimal policy depends on the reward function; different reward functions lead to learning different optimal policies:



- The arrows represent actions.
- The positions represent states.

- Each state has an associated number, the reward.

We have now stated all the components of a Markov Decision Process (MDP).

Let's now examine the MDP loop. Everything is governed by probabilities. There is a prior distribution for sampling the initial state, a distribution for sampling the rewards, and a distribution for sampling the next state.

At time step $t=0$, the environment samples the initial state s_0 from a prior distribution $p(s_0)$. Then, the following steps are repeated:

- The agent selects an action a_t
- The environment samples a reward r_t from the reward function $R(\cdot \mid s_t, a_t)$
- The environment samples the next state s_{t+1} from the transition model $P(\cdot \mid s_t, a_t)$
- The agent receives the reward r_t and the next state s_{t+1}

A policy π is a function from states S to actions A that specifies what action to take in each state. The **objective** is to find the optimal policy π^* that maximizes the cumulative discounted reward.

Cumulative discounted reward means assigning different importance to rewards over time. Suppose that following a policy π starting in state s_0 leads to a sequence $s_0\ s_1\ s_2\ \ldots$. The **cumulative reward** of this sequence is

$$\sum_{t\geq 0}^{\infty} r(s_t)$$

State sequences can vary in length or even be infinite, so we typically define the cumulative reward as the sum of rewards **discounted** by a factor γ:

$$r(s_0)+\gamma r(s_1)+\gamma^2 r(s_2)+\gamma^3 r(s_3)+\ldots$$
$$=\sum_{t\geq 0}^{\infty} \gamma^t r(s_t)$$

with $0<\gamma<1$.

The discount factor γ controls the importance of future rewards compared to immediate ones. The lower the discount factor, the less important future rewards are, leading the agent to focus more on actions that yield immediate rewards. This ensures that the cumulative reward is bounded and helps the algorithm converge.

Reinforcement learning is considerably more complex than supervised learning.

- **In a supervised learning loop**:
    - Get input x_i sampled independently and identically distributed from a data distribution.
    - Use a model with parameters w to predict output y.

    - Observe target output y_i and compute the loss $l(w, x_i, y_i)$.
    - Update w to reduce the loss using stochastic gradient descent (SGD).
      $$w \leftarrow w - \eta \nabla l(w, x_i, y_i)$$
- **In a reinforcement learning loop**:
    - From state s, take action a as determined by policy π.
    - The environment selects the next state s' based on the transition model $P(s' \mid s, a)$.
    - Observe s' and reward $r(s)$, and update the policy.

In supervised learning, the next input does not depend on previous inputs or the agent's predictions, and there is a supervision signal at every step (available offline). Additionally, the loss is differentiable with respect to model parameters.

In reinforcement learning, the agent's actions affect the environment and influence the next observation. The rewards depend on the actions and may be sparse. Rewards are not a function of the model parameters and therefore are not differentiable with respect to them, making the supervision more indirect and complex. Supervision is discovered over time and is not available prior to learning.

Two main approaches in reinforcement learning are:

- **Value-based methods**:
    - Compute the policy indirectly using a proxy function: the value function.
    - The agent's goal is to optimize the value function $V(s)$.
    - The value of each state is the expected total reward an RL agent can collect in the future starting from that state.
- **Policy-based methods**:
    - Define and optimize the policy directly.
    - The policy determines how the agent behaves.
    - A stochastic policy provides a probability distribution over different actions.
      $$\pi_\theta(s, a) \approx P(a|s)$$

Value Based Methods

When people initially started using neural networks, they typically relied on value-based methods for reinforcement learning.

The **value function**, denoted as V, represents the total amount of reward an agent can expect to accumulate from a particular state. It helps in finding an optimal policy by quantifying the expected cumulative reward starting from that state.
Formally, the value function V of a state s with respect to a policy π is defined as the expected cumulative reward of following that policy starting in s:

$$V^{\pi}(s) = \mathbb{E}\left[\sum_{t\geq 0} \gamma^t\, r(s_t) \mid s_0 = s, \pi\right]$$
$$\text{with } a_t = \pi(s_t), s_{t+1} \sim P(\cdot \mid s_t, a_t)$$

where γ is the discount factor and $r(s_t)$ is the reward received at state s_t.

The **optimal value of a state**, $V^*(s)$, is the maximum value achievable by following the best possible policy π^*:

$$V^*(s) = \max_{\pi} \mathbb{E}\left[\sum_{t\geq 0} \gamma^t\, r(s_t) \mid s_0 = s, \pi\right]$$

This function indicates how desirable a state is.

Over time, the concept of the value function evolved to not only consider the value of states but also to quantify the value of state-action pairs. This is known as the **Q-value function** Q, which is more informative as it tells how good a specific action is when taken in a given state:

$$Q^{\pi}(s,a) = \mathbb{E}\left[\sum_{t\geq 0} \gamma^t\, r(s_t) \mid s_0 = s, a_0 = a, \pi\right]$$

The optimal Q-value function $Q^*(s,a)$ provides the maximum expected reward for a state-action pair, considering the best possible actions that can follow:

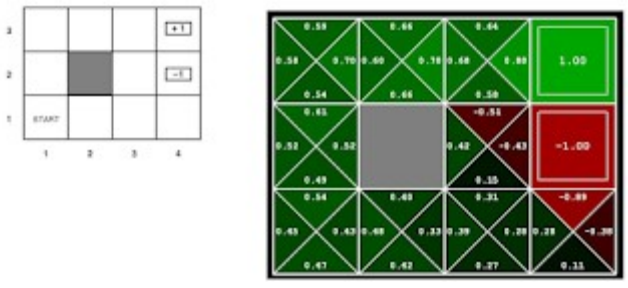
$$Q^*(s,a) = \max_{\pi} \mathbb{E}\left[\sum_{t\geq 0} \gamma^t\, r(s_t) \mid s_0 = s, a_0 = a, \pi\right]$$

Once the optimal Q-value function is determined, it can be used to compute the optimal policy:

$$\pi^*(s) = \arg\max_a Q^*(s,a)$$

In practical problems, both the state space and the action space are often discrete, allowing us to represent the Q-value function as a table where each cell corresponds to the value of the Q-value function for a specific state and action. This table shows the maximum expected future reward for each action at each state. Below is an example of an optimal Q-value function:

$$Q^*(s,a) = \max_{\pi} \mathbb{E}\left[\sum_{t\geq 0} \gamma^t\, r(s_t) \mid s_0 = s, a_0 = a, \pi\right]$$
$$\pi^*(s) = \arg\max_a Q^*(s,a)$$



The challenge, however, is that manipulating the Q-value function directly with standard methods is not feasible due to its complexity. To address this, we use the Bellman Equation, a recursive relationship between the optimal values of successive states and actions:

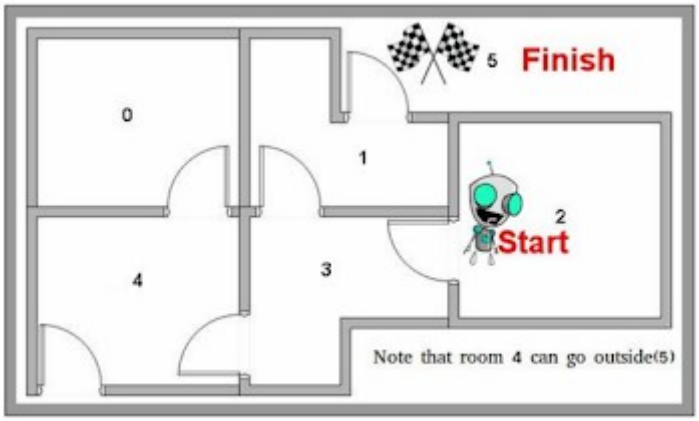
$$Q^*(s,a) = r(s) + \gamma \sum_{s'} P(s'|s,a) \max_{a'} Q^*(s',a')$$
$$= \mathbb{E}_{s' \sim P(\cdot|s,a)} [r(s) + \gamma \max_{a'} Q^*(s',a') | s,a]$$

The Bellman Equation defines how to compute the optimal Q-value function $Q^*(s,a)$ for a given state s and action a in terms of the Q-values of the subsequent state s' and the optimal action a'. Knowing the optimal state-action values $Q^*(s',a')$ for the next time step allows us to determine the action that maximizes the expected value.

The Bellman Equation is crucial because it provides a recursive approach to efficiently compute the optimal Q-value function, which in turn helps in determining the optimal policy.

This approach, known as **Q-learning**, predates the use of neural networks. The main limitation is that the Q-table grows significantly with the size of the state and action spaces. For example, in complex domains like Atari games, where the state space includes all possible configurations of pixels, or in the game of Go, where the state space includes all possible configurations of stones on the board, managing this table becomes impractical. This is why neural networks were later introduced to approximate these tables.

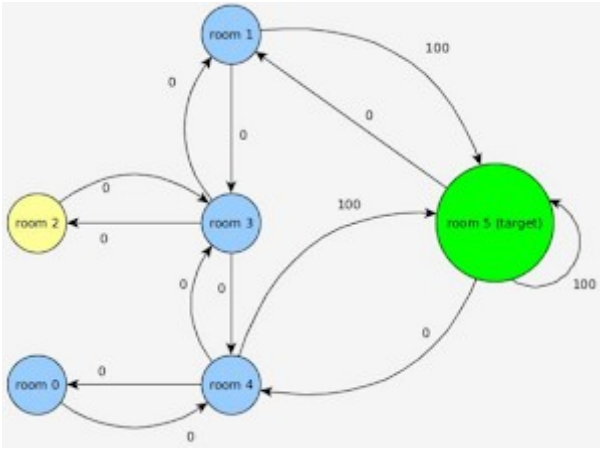
Let's consider an example where the objective is for an agent to reach the goal state (state 5) starting from an initial state (e.g., state 2).



Each time the robot moves to a room that is not the goal state, it receives a reward of 0. Upon reaching the goal state, the reward is 100.

We can represent this problem with a graph where:

- Each node represents a room.
- Each arrow represents an action.
- The values on the arrows represent the immediate reward for taking an action to move from one node to another.



In this example, the components are:

- **Actions**: 0, 1, 2, 3, 4, 5
- **States**: 0, 1, 2, 3, 4, 5
- **Rewards**: 0, 100
- The goal state is 5.

Actions that are not possible can be represented with a negative reward. We create a reward table where a value of -1 indicates that a specific action is not available:

$$R = \begin{array}{c} \\ \text{State} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} & & & \text{Action} & & \end{array} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{array}$$

The key point in Q-learning is that the reward table R is known only to the environment, and the agent must learn R through experience. What the agent will have is a Q matrix that encodes states, actions, and rewards, initialized with zeros. Through experience, this Q matrix gradually becomes similar to the R matrix.

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

$$R = \begin{array}{c} \\ \text{State} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} & & & \text{Action} & & \end{array} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{array}$$

The Q-table is updated using the Bellman Equation. The policy can then be derived from the Q-matrix.

The algorithm for computing the Q-matrix is as follows:

- Initialize the Q matrix with zeros
- Select a random initial state
- For each episode (set of actions that starts on the initial state and ends on the goal state)
- While state is not goal state
  - Select a random possible action for the current state
  - Using this possible action consider going to this next state
  - Get maximum Q value for this next state (All actions from this next state)
  - $Q^*(s,a)=R(s,a)+\gamma' \max_{a'}[Q^*(s',a')]$

Let's consider one episode to better understand the algorithm:

$$Q=\begin{array}{c} \\ 0\\1\\2\\3\\4\\5\end{array}\begin{array}{c}\begin{array}{cccccc}0&1&2&3&4&5\end{array}\\\begin{bmatrix}0&0&0&0&0&0\\0&0&0&0&0&0\\0&0&0&0&0&0\\0&0&0&0&0&0\\0&0&0&0&0&0\\0&0&0&0&0&0\end{bmatrix}\end{array}$$

$$R=\begin{array}{c} \\ 0\\1\\2\\3\\4\\5\end{array}\begin{array}{c}\begin{array}{cccccc}0&1&2&3&4&5\end{array}\\\begin{bmatrix}-1&-1&-1&-1&0&-1\\-1&-1&-1&0&-1&100\\-1&-1&-1&0&-1&-1\\-1&0&0&-1&0&-1\\0&-1&-1&0&-1&100\\-1&0&-1&-1&0&100\end{bmatrix}\end{array}$$

Action / State

$\gamma'=0.8$ $\quad s=1$

Episode 1 **Episode 1**

Randomly choose $a=5$ $\Rightarrow s'=5$

$$R=\begin{array}{c} \\ 0\\1\\2\\3\\4\\5\end{array}\begin{array}{c}\begin{array}{cccccc}0&1&2&3&4&5\end{array}\\\begin{bmatrix}-1&-1&-1&-1&0&-1\\-1&-1&-1&0&-1&100\\-1&-1&-1&0&-1&-1\\-1&0&0&-1&0&-1\\0&-1&-1&0&-1&100\\-1&0&-1&-1&0&100\end{bmatrix}\end{array}$$

$$Q=\begin{array}{c} \\ 0\\1\\2\\3\\4\\5\end{array}\begin{array}{c}\begin{array}{cccccc}0&1&2&3&4&5\end{array}\\\begin{bmatrix}0&0&0&0&0&0\\0&0&0&0&0&100\\0&0&0&0&0&0\\0&0&0&0&0&0\\0&0&0&0&0&0\\0&0&0&0&0&0\end{bmatrix}\end{array}$$

$Q(1,5)=R(1,5)+0.8.max([Q(5,1),Q(5,4),Q(5,5)])$

$\therefore Q(1,5)=100+0.8(0)$

After many episodes, the Q-table will look something like this:

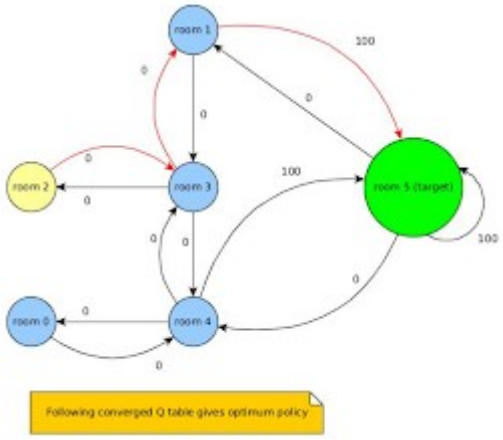
After many episodes

$$Q=\begin{array}{c} \\ 0\\1\\2\\3\\4\\5\end{array}\begin{array}{c}\begin{array}{cccccc}0&1&2&3&4&5\end{array}\\\begin{bmatrix}0&0&0&0&80&0\\0&0&0&64&0&100\\0&0&0&64&0&0\\0&80&51&0&80&0\\64&0&0&64&0&100\\0&80&0&0&80&100\end{bmatrix}\end{array}$$

Once the Q-table is filled, the optimal policy (i.e., the optimal mapping from state to action) can be computed by selecting the action with the highest Q-value for each state:

1. Set current state = initial state.
2. From current state, find the action with the highest Q value
3. Set current state = next state (state from action chosen on 2).
4. Repeat Steps 2 and 3 until current state = goal state.

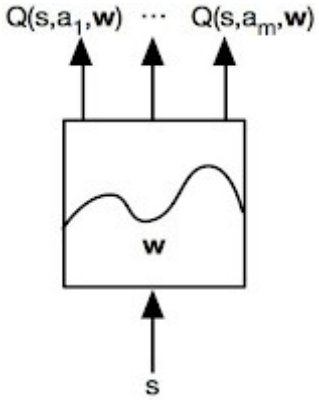
$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix} \end{array}$$



The traditional Q-learning method is not feasible when the number of states and actions is very large, such as in complex environments like Atari games. The Bellman equation, which provides a recursive constraint on the Q-values of successive states, becomes impractical to solve directly due to the vast state and action spaces involved. Therefore, the solution is to approximate the Q-values using a parametric function, typically a **deep neural network**.

$$Q^*(s,a) \approx Q_w(s,a)$$

We train a deep neural network to approximate the Q-values:



This neural network is parameterized by weights w and takes a state s as input, outputting the Q-values for the given state and different actions.

The challenge lies in training this neural network because there's no straightforward notion of loss. However, the Bellman Equation provides a way to compute the loss:

$$Q^*(s,a) = \mathbb{E}_{s' \sim P(\cdot|s,a)}\left[r(s) + \gamma \max_{a'} Q^*(s',a') | s,a\right]$$

The core idea in deep Q-learning is to update the model parameters w at each iteration to bring the Q-values closer to the target values. The target is derived from the previous Q-values we have calculated:

$$y_i(s,a) = \mathbb{E}_{s' \sim P(\cdot|s,a)}\left[r(s) + \gamma \max_{a'} Q_{w_{i-1}}(s',a')|s,a\right]$$

The loss function, which changes at each iteration, is defined as follows (Mean Squared Loss):

$$L_i(w_i) = \mathbb{E}_{s,a\sim\rho}\left[(y_i(s,a) - Q_{w_i}(s,a))^2\right]$$

This is achieved by sampling over ρ, which is a probability distribution over states s and actions a, referred to as the **behavior distribution**.

Therefore, the target is not fixed but varies over time:

$$y_i(s,a) = \mathbb{E}_{s' \sim P(\cdot|s,a)}\left[r(s) + \gamma \max_{a'} Q_{w_{i-1}}(s',a')|s,a\right]$$

The loss function is:

$$L_i(w_i) = \mathbb{E}_{s,a\sim\rho}\left[(y_i(s,a) - Q_{w_i}(s,a))^2\right]$$

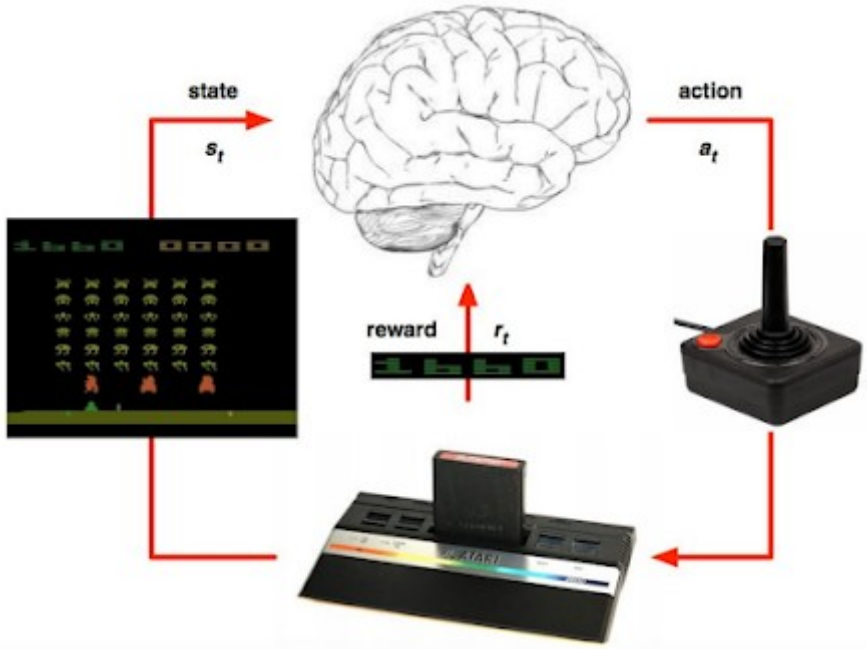
The gradient update is then:

$$\nabla_{w_i} L(w_i) = \mathbb{E}_{s,a\sim\rho}\left[(y_i(s,a) - Q_{w_i}(s,a))\,\nabla_{w_i} Q_{w_i}(s,a)\right]$$
$$= \mathbb{E}_{s,a\sim\rho,s'}\left[(r(s) + \gamma \max_{a'} Q_{w_{i-1}}(s',a') - Q_{w_i}(s,a))\,\nabla_{w_i} Q_{w_i}(s,a)\right]$$

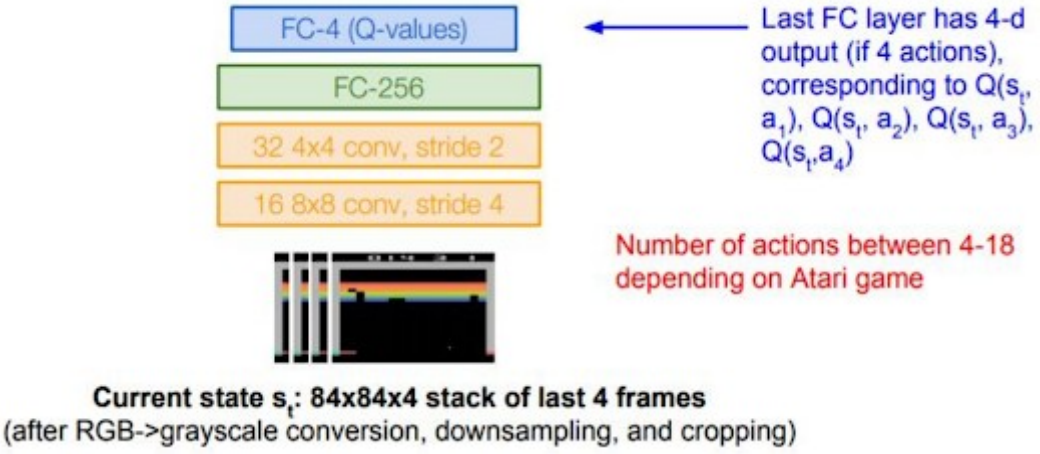
To handle the instability inherent in training, unlike in supervised learning where targets are fixed, we face the challenge of moving targets. Successive experiences are correlated and dependent on the policy, and even slight changes to parameters can lead to significant changes in data distribution. To address these issues, we employ strategies such as freezing the target Q-network for certain periods and using **experience replay**, which involves maintaining a buffer to store experiences and sampling from it.

In Stochastic Gradient Descent (SGD) training, instead of computing the expectation directly, we sample **experiences** (s,a,s') using the behavior distribution and transition model.

In practical applications, like Atari games where states are represented by sets of all possible pixels on the screen, and numerous actions are available, the classical Q-learning approach is inadequate. Instead, a neural network is used to approximate the Q-values of all possible state-action pairs. Given that the states are images, it's beneficial to use a Convolutional Neural Network (CNN).



Effectively, we replace the Q-matrix with a neural network that produces output values corresponding to the Q-values of state-action pairs.



The current state is often represented by the last 4 frames to provide more meaningful input for the network.

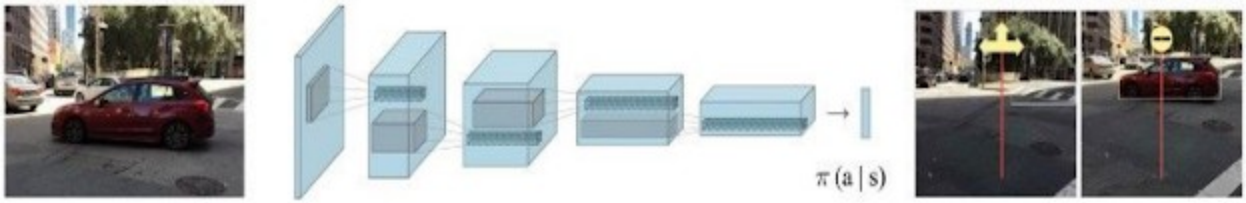
Policy Gradient Methods

The idea behind policy gradient methods is to directly represent the policy rather than indirectly using Q-values. This direct approach is often more efficient, especially in large or continuous action spaces where the Q-value function can become exceedingly complex. For example, in a robot grasping task, the state space is highly dimensional, making it difficult to learn the exact value of every (state, action) pair.

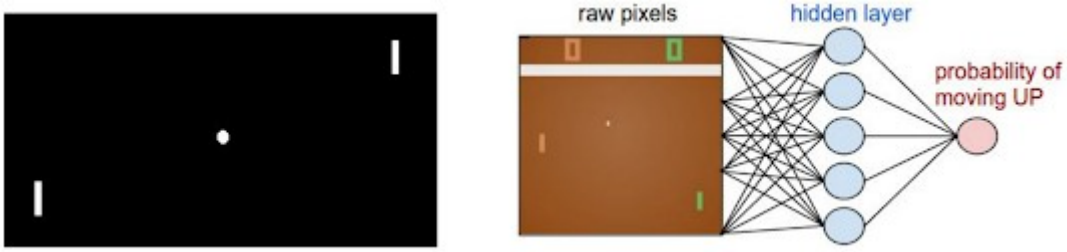


For these reasons, people have adopted a more direct approach: using a neural network to learn a function that provides the probability distribution over actions given the current state.

$$\pi_\theta(s, a) \approx P(a|s)$$



For example, in a game like Pong, the idea behind policy gradient methods is to use a machine learning model that learns a good policy by playing the game and receiving rewards.



The objective of policy gradient methods is to find the best parameters θ for the policy to maximize the expected reward using gradient descent:

$$J(\theta) = \mathbb{E}\left[\sum_{t \geq 0} \gamma^t r_t \mid \pi_\theta\right]$$

$$= \mathbb{E}_\tau[r(\tau)]$$

Expectation of return over *trajectories* $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots)$

$$= \int_\tau r(\tau) p(\tau; \theta) d\tau$$

Probability of trajectory τ under policy with parameters θ

$$p(\tau; \theta) = \prod_{t \geq 0} \pi_\theta(s_t, a_t) P(s_{t+1} | s_t, a_t)$$

Unlike traditional methods, here the notion of sampling is over trajectories: sequences of states, actions, and rewards over time. We aim to maximize the expected value over these trajectories.

The probability of a trajectory is the product of the transition probabilities and the policy's values.

Optimization is achieved by computing the gradient in the following manner:

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau[r(\tau) \nabla_\theta \log p(\tau; \theta)]$$

Probability of trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots)$

Don't know transition probability

$$p(\tau; \theta) = \prod_{t \geq 0} \pi_\theta(s_t, a_t) P(s_{t+1} | s_t, a_t)$$

$$\log p(\tau; \theta) = \sum_{t \geq 0} [\log \pi_\theta(s_t, a_t) + \log P(s_{t+1} | s_t, a_t)]$$

$$\nabla_\theta \log p(\tau; \theta) = \sum_{t \geq 0} \nabla_\theta \log \pi_\theta(s_t, a_t)$$

This approach does not require knowledge of the environment's dynamics P.

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau[r(\tau)\nabla_\theta \log p(\tau;\theta)]$$

$$\nabla_\theta \log p(\tau;\theta) = \sum_{t\geq 0} \nabla_\theta \log \pi_\theta(s_t, a_t)$$

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau \left[\left(\sum_{t\geq 0} \gamma^t r_t \right) \left(\sum_{t\geq 0} \nabla_\theta \log \pi_\theta(s_t, a_t) \right) \right]$$

Stochastic approximation: sample N trajectories

$\tau_1, \ldots, \tau_N$

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T_i} \gamma^t r_{i,t} \right) \left(\sum_{t=0}^{T_i} \nabla_\theta \log \pi_\theta(s_{i,t}, a_{i,t}) \right)$$

One advantage of policy gradient methods is the relatively straightforward computation of the gradient of the objective function, enabling direct minimization. This involves the sum of two terms.

In essence, the gradient is computed by considering sampling over different trajectories (i.e., different possible paths from the initial state to future states). Each trajectory has an associated reward and a term indicating the importance of the reward, which is a function of the policy.

The simplicity of the resulting algorithm (the first one invented) is noteworthy:

1. Sample N trajectories τ_i using current policy π_θ
2. Estimate the policy gradient:
$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} r(\tau_i) \left(\sum_{t=0}^{T_i} \nabla_\theta \log \pi_\theta(s_{i,t}, a_{i,t}) \right)$$
3. Update parameters by gradient ascent:
$$\theta \leftarrow \theta + \eta \nabla_\theta J(\theta)$$

Essentially, we sample trajectories, compute an objective function J that depends on the neural network's parameters, and update these parameters using gradient ascent.

The intuition is that if moving up the hill (i.e., improving rewards) is desirable, we adjust the model parameters and thus the policy to increase the likelihood of trajectories that achieve higher rewards.

